Algoritmen en datastructuren 3, 2026

Gunnar Brinkmann

21 september 2026

Inhoudsopgave

1 Introductie **2**

2 Algoritmen voor slim gebruik van het geheugen **3**
2.1 Hashing . . . 4
2.1.1 Extendible hashing . . . 5
2.1.2 Linear hashing . . . 10
2.1.3 Bloom filters . . . 15
2.2 Sorteren van grote hoeveelheden data – extern sorteren . . . 21

3 Algoritmen voor strings **25**
3.1 Tries (prefix trees) . . . 25
3.1.1 Patricia trees (tries) . . . 28
3.1.2 Ternary trees (tries) . . . 30
3.2 Suffix bomen . . . 32
3.2.1 Het algoritme van Ukkonen . . . 34
3.3 Toepassingen van suffix bomen . . . 48
3.4 Exact string matching . . . 50
3.5 Benaderend string matching . . . 70

4 Compressiealgoritmen **81**
4.1 Huffman codering . . . 84
4.1.1 Adaptive (Dynamic) Huffman Coding . . . 89
4.2 Arithmetisch coderen . . . 98
4.3 LZ77 . . . 104
4.3.1 LZ77 met deelstring bomen . . . 109
4.4 LZW . . . 110
4.5 De Burrows-Wheeler transformatie. . . . 115

5 Metaheuristieken **123**
5.1 Local search . . . 125
5.2 Guided local search . . . 129
5.3 Variable neighbourhood search . . . 134
5.4 Simulated annealing . . . 136
5.5 Genetische algoritmen . . . 140
5.6 Afsluitende opmerkingen . . . 146

6 Parallelle algoritmen **146**
6.1 Branch and bound met verdeelde processoren 148

1 Introductie

De bedoeling van deze tekst is **niet** in plaats van de les te worden gebruikt maar alleen om te helpen de eigen nota's misschien een beetje beter te verstaan als er iets niet echt duidelijk is. Alleen in de les kan je onmiddellijk vragen stellen als je iets niet verstaat en alleen in de oefeningen kan je toetsen of je het geleerde kan toepassen...

2 Algoritmen voor slim gebruik van het geheugen

Een tijdje geleden was er in DA2 een project waar verschillende *AVL-achtige* bomen geïmplementeerd moesten worden. De datastructuur was identiek en de routines die sleutels zochten waren ook dezelfden. Volgens onze ideeën over tijdsverbruik zou dus het aantal bezochte toppen voor een reeks van opzoek-operaties een goede manier zijn om de tijdscomplexiteit te meten. Maar de resultaten waren als volgt (2.000.000 sleutels in de boom en 20.000.000 keer een toevallige sleutel opzoeken):

algoritme	tijd voor opbouwen	aantal bezochte toppen (zoeken)	tijd voor zoeken
1	3.75	426.500.000	33.3
2	8.07	420.300.000	37.6

Hoewel de boom dus beter gebalanceerd was en dezelfde routines de opzoekingen deden, vroeg het meer tijd om de sleutels op te zoeken – er klopte dus iets niet met ons model. Een idee was dat het iets met de cache te maken had. De slechter gebalanceerde boom werd minder herbalanceerd en misschien was de manier waarop het geheugen gealloceerd werd beter omdat toppen die dicht bij elkaar in de boom zitten ook dicht bij elkaar in het geheugen zitten en dus samen in de cache plaats kunnen vinden.
Om dat te testen werden de toppen van de bomen na het toevoegen in het geheugen verplaatst zodat ten minste de toppen die dicht bij de wortel waren allemaal ook dicht bij elkaar zaten. De structuur van de bomen was nog dezelfde (de bomen waren isomorf aan de bomen voor het *verhuizen*). Het resultaat was dat beide algoritmen (nog steeds met dezelfde routines) sneller waren – maar deze keer was de verhouding (ongeveer) zoals verwacht:

algoritme	aantal bezochte toppen (zoeken)	tijd voor zoeken
1	426.500.000	27.7
2	420.300.000	26.8

De modellen die wij gebruikt hebben om te bepalen of een algoritme efficiënt is of niet zijn dus in dit geval niet echt nauwkeurig – en hier gaat het alleen maar om het verschil tussen de cache en het geheugen van de computer. Zodra er zoveel bestanden zijn of de records zo groot zijn dat de boom op de harde schijf wordt bijgehouden, zijn het de leesoperaties op de harde schijf die de snelheid bepalen en niet het aantal vergelijkingen.
De 2-3-bomen die wij in DA2 hebben gezien, gaan al een stukje in deze richting. Als wij het aantal toppen als het aantal leesoperaties zien dan hebben 2-3-bomen een voordeel in vergelijking met alle binaire zoekbomen omdat de diepte het aantal leesoperaties en daardoor de snelheid bepaalt. Door de grotere vertakking hebben 2-3-bomen een kleinere diepte dan binaire zoekbomen (behalve in het ene geval waar een 2-3-boom een complete binaire boom kan zijn) en dat gaat nu ook onze strategie zijn: de vertakking zo groot mogelijk maken zodat zo weinig mogelijk leesoperaties nodig zijn.
Daarbij gaat het niet alleen om zoekbomen – je hebt ook een vertakking als het bv. om recursie gaat. Ons eerste voorbeeld gaat er één zijn waar de vertakking de recursie beschrijft en niet de datastructuur.

2.1 Hashing

Hashing is een efficiënte manier om sleutels op te slaan en op te zoeken. Het probleem is alleen dat er botsingen kunnen zijn – dan kan het gebeuren dat hashing niet meer efficiënt is. Je kan botsingen voorkomen door een hash-tabel te kiezen die voldoende groot is om de kans op botsingen klein te houden. Het probleem is dat je natuurlijk niet te veel ruimte wil verspillen en dat je voor de geziene manieren om met hashtabellen te werken op voorhand moet vastleggen hoe groot de tabel is.
Het zou dus mooi zijn als je de tabel zou kunnen uitbreiden tijdens het gebruik als je ziet dat er te weinig ruimte is. Het onderwerp van dit deel zijn twee manieren om met hashtabellen te werken die dat op een efficiënte manier toelaten.

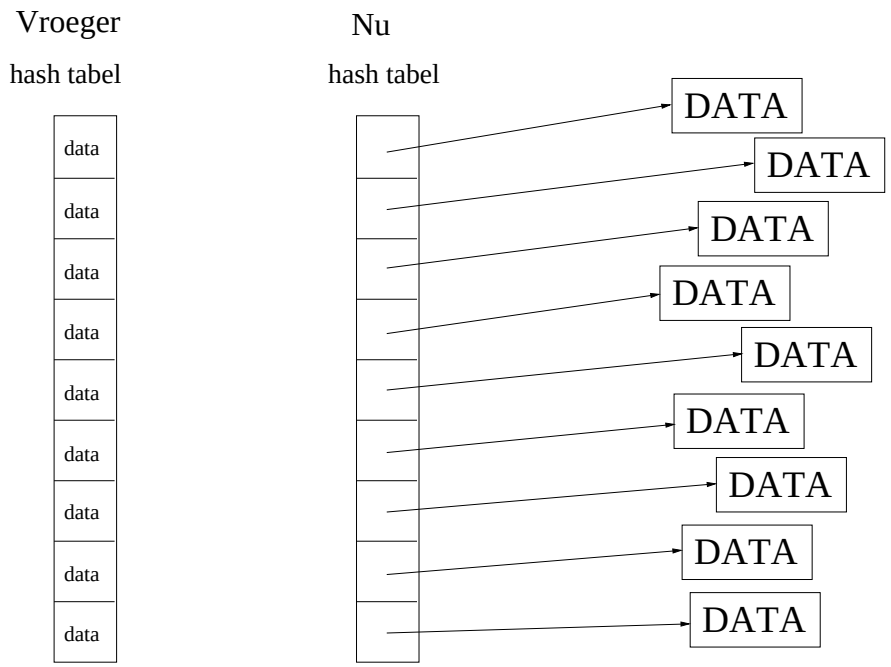
Oefening 1 *Stel dat je closed hashing wil toepassen. Om geen probleem met te veel botsingen te hebben, wordt elke keer dat de laadfactor boven de* 70% *ligt de hashtabel uitgebreid. Wij hebben al in DA2 gezien dat het uitbreiden van een array niet efficiënt kan gebeuren als je elke keer een vast aantal vakken toevoegt maar dat het nodig is de grootte met een getal (in de meeste gevallen* 2*) te vermenigvuldigen. Stel dat je hier (omdat je toch al ruimte verspilt) als factor maar* 1.5 *kiest. Uitbreiden betekent hier wel dat de tabel niet alleen gekopieerd moet worden, maar ook dat voor alle sleutels de hashfunctie opnieuw berekend moet worden.*
Stel voor het volgende dat er geen botsingen zijn:

- *Hoe duur is een toevoegbewerking in het slechtste geval?*
- *Hoe duur is een reeks van n toevoegbewerking op een initieel lege hash-tabel met 100 elementen in het slechtste geval? (Herinnering: dat hebben wij ook de geamortiseerde kost van deze reeks van bewerkingen genoemd.)*

In dit deel zijn we er vooral in geïnteresseerd als de records die je wil opslaan heel groot zijn – zo groot dat je ze niet in het geheugen kan houden maar dat je ze op de harde schijf moet plaatsen. Over hoe dat precies gebeurt gaan wij het niet hebben – dat laten wij gewoon over aan de controller van de harde schijf en aan het besturingssysteem. Bovendien hebben jullie daar al iets over gezien. Wij willen het aantal lees/schrijf-operaties op de harde schijf beperken. Een bestand op de harde schijf kan je – anders dan een array werkt in het geheugen – langer maken zonder het te moeten kopiëren (dat zou heel veel leesoperaties vragen). Wij werken hier gewoon met een model waarvan wij stellen dat je een geïndexeerde lijst van *vakjes* hebt zodat je elk vakje in één stap kan lezen en in één stap er iets in schrijven. Bovendien stellen wij dat je de lijst langer kan maken en dat de kosten om er een vakje aan toe te voegen dezelfden zijn als voor een gewone schrijfoperatie. Hoe het besturingssysteem ervoor zorgt dat dat kan, doet er **in dit geval** niet toe.
Een model is nuttig als je gebaseerd op dit model snelle algoritmen ontwikkelt, waarbij *snel* hier de echte tijd in de praktijk betekent – dat is altijd ons doel. Ook al zijn in ons model sommige dingen zeker te gemakkelijk voorgesteld, blijkt het toch een in de praktijk nuttig model.

2.1.1 Extendible hashing

Extendible hashing werkt met een model dat een beetje verschilt van het gewone model waar je de records (de data) direct in een tabel opslaat. Wij werken met een tabel van pointers die naar de echte data wijzen. In een reële toepassing kan deze *echte data* natuurlijk ook maar een deel van de echte data zijn of alleen een identificatienummer dat nog een pointer naar de hele record bevat – dus de echte data alleen maar representeert.
Als je de records in het geheugen kan houden, kan deze manier van werken ook voordelen hebben als je niet met extendible hashing werkt: het zou kunnen dat je moeilijkheden hebt een samenhangend geheugenblok te krijgen om alle records te indexeren. Als je met pointers werkt, is dat natuurlijk geen probleem. Bovendien zou het ook efficiënter zijn de array groter te maken: Wij zouden alleen de pointers moeten kopiëren en niet de (veel grotere) records. Maar zonder verdere wijzigingen zouden wij de data nog altijd moeten rehashen of tenminste lezen en dat is natuurlijk ook vrij duur – en als het over de harde schijf gaat onaanvaardbaar.
Maar nu gaan wij het over het geval hebben waar de records op de harde schijf zitten.



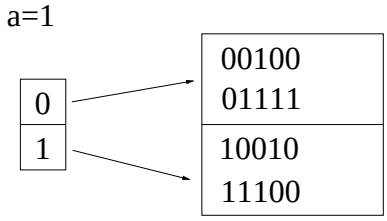
Figuur 1: Een hash tabel en een hash tabel met pointers naar buckets.

De array met de pointers kan zeker nog in het geheugen gehouden worden, dus hebben wij daarvoor geen leesoperaties op de schijf nodig.
Wij werken met buckets (emmers) waarbij in elke bucket een constant en vast aantal records terecht kan komen – bv. 5. De reden is dat de emmers blokken voorstellen die je in één stap kan lezen. Het feit dat je dan misschien de gezochte record nog moet zoeken als er meerdere records per emmer zijn, doet er in ons model niet toe: dat gebeurt in het geheugen en kan dus efficiënt gedaan worden.

Wij werken met een hash-functie $h()$ waarvan wij stellen dat de waarden tussen 0 en $2^n - 1$ voor een zekere n liggen en dat ze zeker voldoende groot zijn om goed te kunnen hashen – bv. waarden tussen 0 en $2^{32} - 1$. Bovendien stellen wij dat de hashfunctie altijd met hetzelfde aantal n bits voorgesteld wordt – dus misschien ook met nullen in het begin. Wij zullen in onze voorbeelden altijd de hash-waarden in de emmers opslaan. In de realiteit zijn dat natuurlijk de records en niet de hashwaarden, maar zo kan je zien wat er precies gebeurt terwijl artificiële data die wij voor de voorbeelden zouden gebruiken niet echt zou bijdragen tot het verstaan van de principes. Je zou in de realiteit bovendien **ook** nog de hashwaarden kunnen opslaan om die niet te moeten herberekenen, maar omdat nooit veel data opnieuw gehasht zal moeten worden en omdat wij alleen de lees- en schrijf-bewerkingen willen minimaliseren, maakt dat geen (groot) verschil.
Arrayindices schrijven wij ook vaak in binaire vorm omdat de methodes de binaire voorstelling van de indices gebruiken. Natuurlijk zijn dat ook gewone getallen – alleen dat ze binair voorgesteld worden om gemakkelijker te verstaan in welk vakje een record terecht moet komen.

- De grootte van onze pointerarray is altijd een macht van 2 – wij gebruiken de variabele a om dat bij te houden, de grootte is 2^a.

Wij evalueren altijd een zeker aantal – dezelfde a – bits van onze hashfunctie. In feite gebruiken wij dus niet $h()$ om de index van het juiste vakje in onze tabel te vinden maar $(h())|_a$. Wij schrijven altijd $x|_a$ voor het getal dat door de eerste a bits in de binaire voorstelling van x voorgesteld wordt, waarbij wij misschien met nullen in het begin werken om ervoor te zorgen dat de voorstelling van elk getal hetzelfde aantal bits heeft. Als $x = 010011$ in binaire voorstelling (dus 19) dan is $x|_3 = 010$ dus gelijk aan 2. Als wij a bits evalueren, hebben wij 2^a mogelijke waarden $0, \ldots, 2^a - 1$ die de indices van onze (pointer) hash-tabel vormen. *In principe* kunnen wij dus pointers naar 2^a buckets bijhouden.
Een voorbeeld met $a = 1$ zien jullie in Figuur 2



Figuur 2: Extendible hashing met $a = 1$ en twee records per emmer.

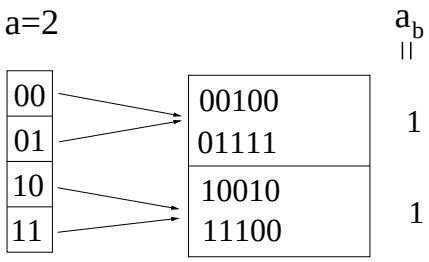
Maar in feite zullen wij niet alle buckets wijzigen als de pointerarray groter gemaakt wordt. Dat is **heel** belangrijk omdat het veel dure leesoperaties zou vragen en de uitbreidingsstap onaanvaardbaar duur zou maken. Dat betekent dat wij voor sommige buckets niet a bits moeten evalueren maar minder. Wij houden dus voor elke bucket b een getal a_b bij dat zegt hoeveel bits voor deze bucket werden geëvalueerd toen hij aangemaakt werd. Het is duidelijk dat als er meer pointers zijn dan buckets dat sommige pointers naar dezelfde bucket moeten wijzen. Figuur 4 toont een situatie waar er één bucket al werd uitgebreid en één niet. In feite zullen er altijd 2^{a-a_b} pointers naar een bucket b wijzen.

Maar nu dat wij al ongeveer weten hoe de situatie er kan uitzien, moeten wij precies beschrijven hoe extendible hashing werkt:
Je begint met een vaste a. In onze voorbeelden beginnen wij met $a = 1$ omdat het anders vrij moeilijk is om te tekenen, maar in de realiteit zal a zeker ten minste 10 zijn. Wij hebben dan een pointerarray met 2^a pointers die indices $0, \ldots, 2^a - 1$ hebben en naar 2^a buckets wijzen waarin wij g records kunnen plaatsen. Voor de voorbeelden kiezen wij $g = 2$. Voor elke emmer b is in het begin $a_b = a$. In het begin – waar voor elke emmer geldt dat $a = a_b$ – wijzen dus inderdaad $2^{a-a_b} = 1$ pointers naar elke emmer.

- Toevoegen en opzoeken gebeurt op de volgende manier: je berekent de hashwaarde $h(r)$ van de record r die je wil toevoegen of opzoeken en kijkt in de tabel waar vakje $(h(r))|_a$ naartoe wijst. Daar moet je record r plaatsen of opzoeken. Opzoeken is nooit een probleem maar als je de record wil toevoegen en de emmer is vol is er wel een probleem...

Als de emmer b waar je iets wil toevoegen volzit, willen wij een nieuwe emmer gebruiken en de inhoud verdelen. Maar voor 2 emmers hebben wij ook 2 pointers nodig. Er zijn twee mogelijkheden:

$a_b = a$**:** In dit geval is er maar één pointer die naar deze emmer wijst. Dat is het slechtste geval. Wij moeten dus eerst onze pointerarray uitbreiden om ervoor te zorgen dat ten minste 2 pointers naar deze emmer wijzen. Wij verdubbelen de pointerarray. De variabele a is nu 1 groter (dus $a = a^{oud} + 1$). Als de nieuwe pointerarray $p_n[]$ is en de oude $p_o[]$ dan vullen wij $p_n[]$ met dezelfde pointers als $p_o[]$: voor $0 \leq i < 2^a$ geldt $p_n[i] = p_o[\lfloor i/2 \rfloor]$. Het resultaat voor de eerste keer verdubbelen zien jullie in Figuur 3. Nu wijzen dus twee keer zo veel pointers naar elke emmer als voor het verdubbelen.



Figuur 3: De pointerarray werd verdubbeld en werkt met $a = 2$ maar de emmers werken nog met $a_b = 1$.

Wij hebben er dus voor gezorgd dat wij altijd in de volgende situatie terechtkomen:

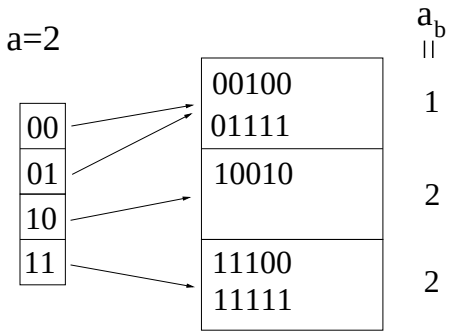
$a_b < a$**:** Dus wijzen nu $2^{a-a_b} \geq 2$ pointers naar de volle emmer b waar wij de nieuwe record willen plaatsen. De eerste a_b bits van de hashwaarde van elke record zijn gelijk – stel dat het $b_1 \ldots b_{a_b}$ zijn. Wij maken nu een nieuwe emmer b' aan en verdelen de records uit b zo dat alle records r met bit nummer $a_b + 1$ gelijk aan 0 (of $2 \cdot (h(r)|_{a_b}) = h(r)|_{a_b+1}$) in de oude emmer b terechtkomen. De eerste $a_b + 1$ bits van de hashwaarden van deze

records zijn dus "$b_1 \dots b_{a_b}0$". De anderen (bit nummer a_b+1 gelijk aan 1, dus $2 \cdot (h(r)|_{a_b}) + 1 = h(r)|_{a_b+1}$ en de eerste a_b+1 bits van de hashwaarde zijn "$b_1 \dots b_{a_b}1$") plaatsen wij in de nieuwe emmer b'. Dan plaatsen wij de nieuwe record volgens dezelfde regel (als dat kan...). De pointers in de pointerarray met een index i waarvoor geldt dat $i|_{a_b+1} = 2 \cdot (h(r)|_{a_b}) + 1$ (de eerste $a_b + 1$ bits van de index zijn "$b_1 \dots b_{a_b}1$") moeten nu gewijzigd worden zodat ze naar b' wijzen. De waarden van a_b en $a_{b'}$ worden de oude waarde van a_b plus 1.

Oefening 2

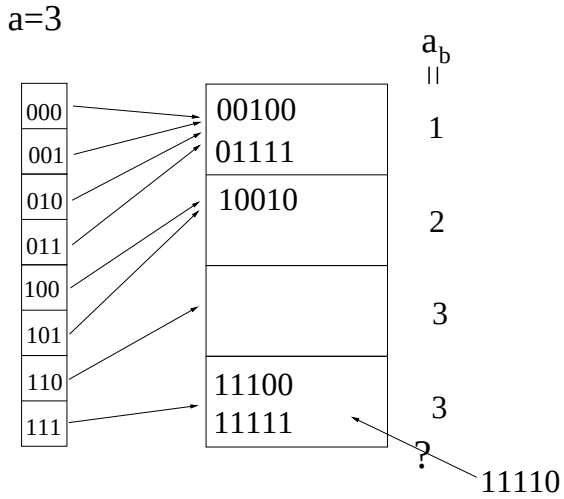
- *Bereken uit a, a_b en $h(r)$ de indices van de pointers die gewijzigd moeten worden. Geef een formule.*
- *Beschrijf expliciet hoe je met bitoperaties de index in de array berekent en hoe je met bitoperaties kan beschrijven welke pointer uit de oude array naar een vakje in de nieuwe gekopieerd moet worden.*

In Figuur 4 zien jullie het resultaat als wij op deze manier 11111 toevoegen: de emmer waarop pointer 11 wees moest herverdeeld worden en omdat $h(r)|_{a_b} = 1$ (alleen het eerste bit wordt geevalueerd van 11111) wordt pointer $2 \cdot 1 + 1 = 3$ gewijzigd en wijst naar de nieuwe emmer.



Figuur 4: Extendible hashing met $a = 2$ maar één emmer werkt nog met $a_b = 1$.

Maar in feite kan er een probleem zijn. Wij hebben de oude records in de emmer en de nieuwe record herverdeeld over twee emmers. Maar als wij pech hebben, kan het gebeuren dat bij het herverdelen één emmer leeg blijft en de andere dan nog altijd geen vrije ruimte voor de nieuwe record heeft. Dat zou bv. gebeuren als je nu sleutel 11110 toevoegde. Figuur 5 toont de datastructuur na het verdubbelen en herverdelen van de volle emmer.
In dit geval moeten wij nog eens verdubbelen en hopen dat het probleem dan opgelost is. Hashing kan natuurlijk altijd problemen veroorzaken als door een slechte hash-functie – of gewoon heel veel pech – de waarden slecht verdeeld zijn. Dit zou ook hier de oorzaak zijn. Met een goede hash-functie en niet te weinig records per emmer zal de load in de praktijk gelijkmatig verdeeld zijn en dus de pointerarray niet overbodig groot zijn. Als in één emmer x records terecht kunnen komen en de hash-functie kent meer dan x keer dezelfde waarde toe, zou het uitbreiden oneindig doorgaan (of precies: het zou doorgaan tot er geen bits meer zijn om te evalueren en zou dan vaststellen dat de methode niet werkt). Een goede hashfunctie is dus heel belangrijk.



Figuur 5: Extendible hashing met $a = 3$ na het verdubbelen maar toch kan een record nog niet toegevoegd worden.

Het belangrijke voordeel qua efficiëntie is dat je alleen maar naar de emmer moet kijken die te vol zit. Dat zorgt ervoor dat je misschien met maar één leesoperatie en twee schrijfoperaties kan werken als de grootte van de emmers zo is dat hij in één stap gelezen kan worden. Naar de andere emmers moet je zelfs niet kijken. Het uitbreiden van de pointerarray gebeurt in het geheugen – dat tellen wij dus niet mee als het om lees/schrijf-operaties gaat. Bovendien moet de array natuurlijk **veel** minder vaak verdubbeld worden dan emmers herverdeeld moeten worden – als er m keer een emmer herverdeeld moet worden dan is het aantal verdubbeloperaties bij een gelijkmatige verdeling van de hashwaarden van de orde $\log m$. Maar het verdubbelen kan ook op een (tenminste geamortiseerd) efficiënte manier gebeuren. Of precies:

Oefening 3 *Deze oefening is niet gemakkelijk (vraag de assistent je tips te geven als je niet weet hoe je moet beginnen) maar het is wel een nuttige oefening om een beetje DA2 te herhalen en vooral om zichzelf duidelijk te maken hoe extendible hashing werkt:*
Het verdubbelen van de pointerarray lijkt natuurlijk sterk op het verdubbelen van een array, dat jullie in DA2 hebben gezien. Maar er zijn toch verschillen: hier wordt de array direct volgeschreven en bovendien worden de records niet gekopieerd en de samenhang tussen het aantal records en de grootte van de pointerarray is ook niet zo duidelijk.

- *Wat is de geamortiseerde kost van een reeks van n bewerkingen op een initiëel lege extendible hashing tabel als je geen voorwaarden aan de hash-functie oplegt (behalve natuurlijk dat ze oneindig veel bits genereert – anders werkt het niet voor arbitrair grote n)?*

- *Gegeven $q > 0$. Wat is de geamortiseerde kost van een reeks van n bewerkingen op een initiëel lege extendible hashing tabel als gegarandeerd is dat altijd als er een tabel met p pointers uitgebreid wordt er ook ten minste $p \cdot q$*

records zijn? De $O()$ notatie is voldoende maar een bewijs is (natuurlijk) vereist. (In feite zou het voldoende zijn dat voor de datastructuur na n stappen te eisen.)

Oefening 4 *Stel dat je met extendible hashing ook identieke records – dus zeker met gelijke sleutels – wilt opslaan. Wanneer kan dat en wanneer is er een probleem ? Hoe kan je het probleem oplossen?*

Oefening 5 *Voeg de sleutels 3, 7, 12, 45, 44 en 1 toe aan een lege extendible hashing tabel met twee emmers in het begin. Er kunnen 2 sleutels (records) per emmer geplaatst worden.*
Wij gebruiken een voorstelling van de hashcodes van $h()$ als binaire getallen met 5 bits en wij hebben
$h(3) = 0$ *dus binair* 00000
$h(7) = 31$ *dus binair* 11111
$h(12) = 9$ *dus binair* 01001
$h(45) = 17$ *dus binair* 10001
$h(44) = 2$ *dus binair* 00010
$h(1) = 1$ *dus binair* 00001

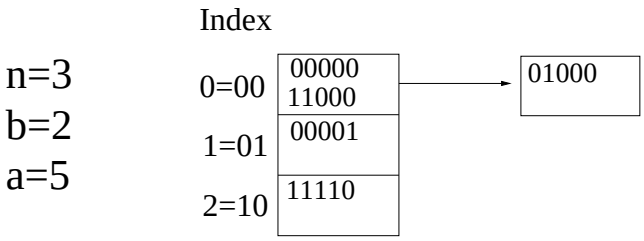
2.1.2 Linear hashing

In het geval van extendible hashing kan een enkele uitbreidingsstap *in principe* wel relatief duur zijn – ook al is de geamortiseerde kost (met een goede hash-functie) heel goed. Het effect van een slechte hashfunctie – of heel veel pech – is dat meerdere keren uitgebreid moet worden of in het ergste geval zelfs de grens van de uitbreidingsmogelijkheden bereikt wordt. Ook het verdubbelen van de pointerarray kan je voor zeer grote arrays niet meer verwaarlozen. Linear hashing gebruikt uitbreidingsstappen waar het slechtste geval goedkoper is. De klemtoon ligt hier op de laadfactor en niet op een te volle emmer. Je moet afhankelijk van de toepassing beslissen of extendible hashing of linear hashing (of gewoon hashing) beter is.
Hier gebruiken wij het voordeel van ons model van de harde schijf dat de tabel zonder te kopiëren uitgebreid kan worden. Het is dus mogelijk een enkele emmer toe te voegen zonder de andere emmers te moeten kopiëren.
De belangrijkste bedoeling is ook hier tijdens het uitbreiden het aantal lees- en schrijf-operaties op de harde schijf te minimaliseren – dus naar zo weinig mogelijk emmers te moeten kijken.
Ook hier zullen wij in de voorbeelden de hash-keys in de emmers plaatsen omdat dat gemakkelijker is om te verstaan. In een toepassing zitten daar natuurlijk de records.
Een groot verschil met extendible hashing is dat wij er hier niet op letten of een emmer te vol zit of niet. Als iets niet meer geplaatst kan worden, werken wij gewoon met overstroomemmers en vormen een emmerketting.

- Je houdt altijd bij hoeveel emmerkettingen je hebt (dat noemen wij altijd n) en hoeveel elementen in de tabel zitten (dat noemen wij a). Het aantal records dat je in één emmer kan plaatsen, noemen wij g (voor grootte).

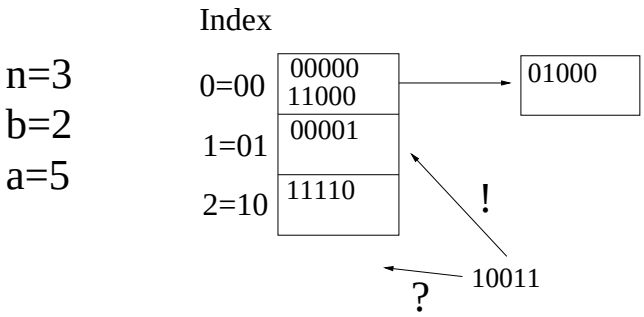
Natuurlijk maken de emmerkettingen het hashen misschien minder efficiënt – vooral als voor de overstroomemmers een extra leesoperatie nodig is – maar als

de hashfunctie goed en de laadfactor $a/(n \cdot g)$ niet te groot is, zullen er normaal niet veel overstroom-emmers zijn. Als wij schrijven dat in een *emmer* gezocht moet worden, bedoelen wij in feite de hele emmerketting – dus de emmer samen met zijn overstroomemmers. Ons doel is dus de laadfactor relatief klein te houden om efficiënt hashen te kunnen garanderen.
Wij gebruiken opnieuw de binaire voorstelling van de hash-waarden. Als er n emmers zijn, hebben wij $b(n) = \lceil \log n \rceil$ bits nodig om de indices van de emmers voor te stellen. Hier gebruiken wij de **laatste** b bits van de hashfunctie $h()$ en schrijven $x|^b$ als wij de laatste b bits van de binaire voorstelling van een getal x willen gebruiken. Wij stellen altijd dat $n \geq 2$ en dus $b \geq 1$.



Figuur 6: Een kleine linear hashing tabel.

Wij proberen een record r in emmer $h(r)|^b$ te plaatsen. Het enige probleem dat zou kunnen opduiken is dat $h(r)|^b \geq n$. Alleen als n een macht van twee is, is $b(n) = \log n$. Dan kan dat niet gebeuren – anders wel!
Als je in ons voorbeeld een record met hashwaarde 10011 wil plaatsen, neem je de laatste $b = 2$ bits, dus 11 – het record moet dus in vakje 3 geplaatst worden – maar dat is er niet. In zulke gevallen plaatsen wij het record in vakje $h(r)|^b$ mod $2^{b-1} = h(r)|^{b-1}$.



Figuur 7: Het plaatsen van een element waar de eerste keuze voor de index te groot is.

De functie $h_n(r)$ die de index bepaalt waar het element r geplaatst moet worden is dus gegeven door (merk op dat b gewoon een functie van n is):

$$h_n(r) = \begin{cases} h(r)|^b & \text{als } h(r)|^b < n \\ h(r)|^{b-1} & \text{als } h(r)|^b \geq n \end{cases}$$

Op deze manier hebben wij een manier van hashing die ook zonder uitbreiden kan werken en door de overstroomemmers ook met een grote laadfactor nog

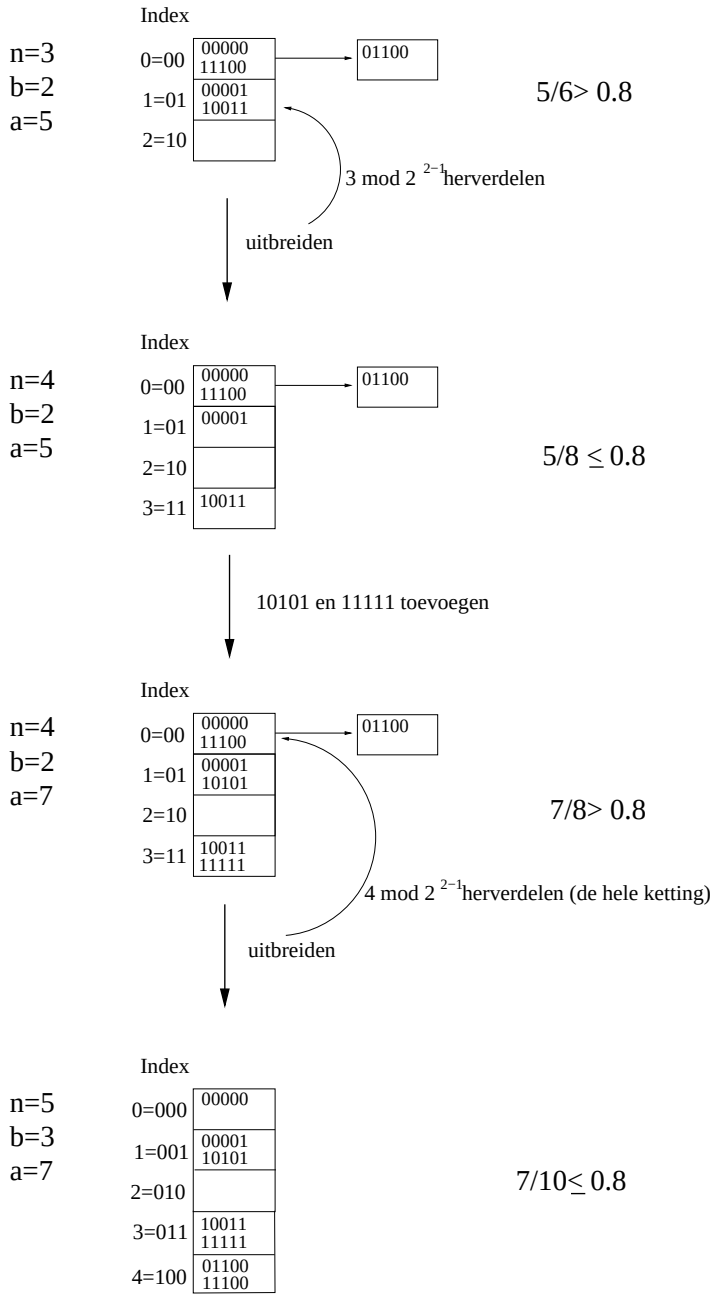
werkt – hoewel de efficiëntie dan slechter wordt. Het lineaire zoeken in de overstroomemmers zou misschien veel leesoperaties vragen.
Maar nu gaan wij beschrijven hoe wij ervoor zorgen dat de laadfactor $a/(n \cdot g)$ nooit groter wordt dan een bovengrens L die wij op voorhand vastleggen. Het getal $n \cdot g$ is het grootste aantal records dat je kan plaatsen zonder overstroomemmers. Hoe dichter a dus bij $n \cdot g$ ligt hoe groter de kans dat je overstroomemmers moet gebruiken. Als $a > n \cdot g$ is het zelfs zeker dat je overstroomemmers nodig hebt! Wij zullen dus normaal $L < 1$ kiezen – bv. $L = 0.7$.
Stel nu dat wij een nieuw element toevoegen en daardoor de laadfactor te groot wordt, dus $a/(n \cdot g) > L$. Dan moeten wij onze tabel uitbreiden. Daardoor wordt n groter en de laadfactor kleiner. Wij nemen er altijd maar één nieuw element bij.
Maar dat betekent natuurlijk dat onze hashfunctie $h_n(r)$ verandert! Wij moeten dus bepalen welke records r nu op een foute plaats zitten dus waarvoor geldt $h_{n_{\text{nieuw}}}(r) \neq h_{n_{\text{oud}}}(r)$ of als n voor de oude n staat: $h_{n+1}(r) \neq h_n(r)$. Er zijn twee gevallen: ofwel b verandert ook – ofwel niet.

$b(n) = b(n+1)$**:** Wij schrijven gewoon b voor $b(n) = b(n+1)$. Als je naar de definitie van $h_n(r)$ kijkt, kan een verschil er alleen door veroorzaakt zijn dat verschillende gevallen van de definitie worden toegepast. Dat gebeurt precies voor $h|^b(r) = n = n_{\text{oud}}$. Vroeger was de waarde $n|^{b-1}$ en nu is hij n. De enige records die op een foute plaats kunnen zitten, zijn dus de records in emmer $n|^{b-1}$. Die moeten herverdeeld worden.

$b(n) = b(n+1) - 1$**:** Dus $n = n_{\text{oud}} = 2^{b_{\text{oud}}}$ Wij schrijven b voor b_{oud}. Nu worden $b+1$ bits van $h(r)$ geëvalueerd (dus $h(r)|^{b+1}$ gebruikt). Als de eerste bit 0 is, geldt $h(r)|^{b+1} = h(r)|^b < n < n+1$. Records met deze hashwaarde werden dus vroeger en nu op dezelfde manier geplaatst. Maar als de eerste bit (die voor 2^b staat) 1 is, geldt $h(r)|^{b+1} \geq 2^b = n$. Als $h(r)|^{b+1} > 2^b = n$ dan geldt $h(r)|^{b+1} \geq n+1$ – dus is $h_{n+1}(r) = h(r)|^b = h_n(r)$ (omdat in dit geval altijd geldt dat $h(r)|^b < n$ – zelfs als alle bits 1 zijn.). Records met deze hashwaarde moeten dus niet herverdeeld worden. Het blijft alleen maar $h(r)|^{b+1} = 2^b = n$ dus $h(r)|^b = 0$ om herverdeeld te worden.

Je hoeft in de twee gevallen dus geen verschillende formules te gebruiken – in elk geval (behalve het uitzonderlijke geval $b_{\text{oud}} = 1$ dat je voor echte toepassingen natuurlijk nooit hebt) is de emmer die herverdeeld moet worden emmer $n_{\text{oud}}|^{b_{\text{oud}}-1}$. De twee gevallen die wij besproken hebben, zijn gewoon bewijzen dat deze formule in beide gevallen geldt.
De emmer die herverdeeld wordt, moet niet bijzonder vol zitten – hij kan (in principe) zelfs helemaal leeg zijn!
Maar belangrijk voor de efficiëntie: maar **één** emmerketting moet herverdeeld worden – en die kan misschien met één enkele leesoperatie ingelezen worden als de hashfunctie en de laadfactor ervoor zorgen dat niet veel overstroomemmers nodig zijn. Als je de schrijfoperaties meetelt zijn er dus maar 2 emmerkettingen die gewijzigd moeten worden.
Een voorbeeld van de manier waarop linear hashing werkt, zien jullie in Figuur 8

Oefening 6 Dit is een bijzonder belangrijke oefening voor linear hashing:



Figuur 8: Deze linear hashing tabel wordt uitgebreid zodra de laadfactor groter is dan $L = 0.8$.

Stel dat je een goede hashfunctie hebt en dat de reeks van hashwaarden goed verdeeld is en elk van de 2^b *mogelijke waarden van* $h()|^b$ *ongeveer dezelfde kans maakt om op te duiken. (Om het iets gemakkelijker te hebben mag je stellen dat het aantal sleutels een macht van twee is die groter dan* 2^b *is en dat elke index* **precies** *even vaak opduikt.) Zitten de emmers dan altijd allemaal even vol?*

Als ja: *bewijs dat.*

Als niet:

- *Wanneer zijn ze wel even vol?*
- *Als ze niet even vol zijn: welke emmers zijn voller en hoe groot is het verschil?*
- *Als ze niet even vol zijn: is de emmer die herverdeeld moet worden voller of minder vol dan anderen?*

Geef uitleg.

Oefening 7 *Geef voldoende voorwaarden op de reeks van sleutels die toegevoegd moeten worden en de hash-functies om te garanderen dat linear hashing voor een reeks van n toevoegbewerkingen gemiddeld constante tijd vraagt!*
Geef uitleg!

Oefening 8 *Voeg de sleutels 3, 7, 12, 45, 44, 11 en 1 toe aan een lege linear hashing tabel met twee emmers in het begin. Er kunnen 2 sleutels (records) per emmer geplaatst worden en de tabel wordt uitgebreid zodra de laadfactor groter is dan* 0.8.
De hashcodes zijn als volgt:
$h(3) = 0$ *dus binair* 0
$h(7) = 31$ *dus binair* 11111
$h(12) = 9$ *dus binair* 1001
$h(45) = 17$ *dus binair* 10001
$h(44) = 2$ *dus binair* 10
$h(11) = 8$ *dus binair* 1000
$h(1) = 1$ *dus binair* 1

2.1.3 Bloom filters

Natuurlijk is het mooi als je een hashing tabel kan uitbreiden als je meer geheugen nodig hebt – maar wat als je gewoon te veel geheugen nodig hebt?
Het volgende algoritme werd in 1970 door Burton Bloom gepubliceerd. Toen was het onmogelijk alle juiste spellingen van woorden in het geheugen op te slaan en het algoritme werd gebruikt om ook met minder geheugen een goede spelling checker te implementeren. Intussen is het beschikbare geheugen zeer veel groter dan toen, maar er zijn nog altijd problemen waar het belangrijk is het geheugengebruik zo klein mogelijk te houden. Soms moet je bv ook informatie over een netwerk sturen en dan is de reductie van de hoeveelheid geheugen dat deze informatie codeert ook essentieel! Wij zullen wel zien dat de reductie van geheugengebruik een prijs heeft...
Stel dat je met een universum $\{0, \ldots, n-1\}$ werkt. Als je een grote verzameling van elementen uit dit universum moet bijhouden, dan hebben wij al in Algoritmen en Datastructuren 2 gezien hoe je dat kan doen: je werkt met een bitvector en zet bit nummer i op 1 als en slechts als element i in de verzameling zit. Bloom filters zijn voor het geval dat je een – in vergelijking met het universum – heel kleine verzameling van elementen wil opslaan.
Bitvectoren vragen maar 1 bit voor elk element uit het universum. Dat werkt goed als het universum niet te groot is en de verzameling in vergelijking met het universum niet te klein. Bloomfilters hebben jammer genoeg een kleine kans soms een fout antwoord te geven, maar voor een vaste (kleine) kans op fouten hangt de grootte van Bloom filters noch van de grootte van het universum af – zoals bij bitvectoren – noch van de grootte van de elementen die je wilt opslaan – zoals bij lijsten – alleen van de grootte van de verzameling.
Bitvectoren werken natuurlijk ook als de elementen niet $\{0, \ldots, n-1\}$ zijn, maar je wel een unieke index $\{0, \ldots, n-1\}$ aan elk element kan toevoegen. Eén manier om (niet unieke) indices aan objecten toe te kennen is een hashfunctie. Stel dat wij een universum U van objecten hebben. Dit kunnen bv. alle mogelijke combinaties van ten hoogste 20 lettertekens zijn, maar het kunnen ook alle strings zijn – dus een oneindige verzameling. Stel bovendien dat wij een hashfunctie $h : U \to \{0, \ldots, n-1\}$ hebben. Als wij nu voor een verzameling $M \subseteq U$ (bv. alle juist gespelde woorden in de Nederlandse taal) willen toetsen of een zeker element x in M zit, kunnen wij als volgt werken:

- Wij nemen een bitvector $b_0, \ldots, b_{n-1}$ met n bits die in het begin allemaal 0 zijn.
- Dan wordt voor elk element $y \in M$ de bit $b_{h(y)}$ op 1 gezet.
- Als wij willen toetsen of een element $x \in U$ in M zit, dan berekenen wij $h(x)$ en zeggen *ja* als bit $b_{h(x)} = 1$ en anders *nee*.

Als wij dit algoritme toepassen en wij testen een element $x \in M$ dan zal het antwoord *ja* zijn – dat klopt dus. Als wij een element $x \notin M$ testen dan kan het antwoord nee zijn, maar er is ook een kans dat toevallig een ander element dat wel in M zit dezelfde hashwaarde heeft en wij dus het (foute) antwoord *ja* krijgen. Er is dus een kans op false positives, dus foute ja-antwoorden. Je zou de antwoorden als *nee* en *misschien* kunnen interpreteren – dan heb je altijd een juist antwoord, maar aan *misschien* heb je vaak niets... Wij zullen zien hoe je

de kans op foute ja-antwoorden kleiner kan maken, maar Bloom filters zijn dus zeker niet geschikt voor gevallen waar een fout ja-antwoord niet achteraf nog door een andere test herkend kan worden (wij zullen een voorbeeld zien waar dat wel kan) en dan misschien rampzalige gevolgen heeft. Als het om een spelling checker gaat dan is een programma dat je een beetje helpt zeker al beter dan helemaal geen hulp – daarvoor waren de Bloom filters dus zeker geschikt. Een andere toepassing waarvoor Bloom-filters voorgesteld werden, was het opslaan van een lijst van onvoldoende veilige passwoorden. Ook hier was een fout ja-antwoord niet erg: je zou gewoon een ander passwoord moeten kiezen, terwijl foute nee-antwoorden wel erg zouden kunnen zijn – maar foute nee antwoorden heb je bij Bloom filters niet.
Het idee om de kans op false positives te reduceren is eenvoudig: gebruik niet één hashfunctie $h()$ maar meerdere hashfuncties $h_1(), \ldots, h_k()$ en geef enkel *ja* als antwoord als voor elke hashfunctie $h_i()$ geldt dat $b_{h_i(x)} = 1$.
Daarbij kan je op twee manieren werken als je n bits ter beschikking hebt:

a.) Als n deelbaar is door k gebruik je voor alle hashfuncties aparte bitvectoren van n/k bits. De functie $h_1()$ gebruikt dus bit $0, \ldots, n/k - 1$, de functie $h_2()$ gebruikt bit $n/k, \ldots, 2(n/k) - 1$, etc.

b.) Je gebruikt voor alle hashfuncties dezelfde n bits waar je bits op 1 zet. Het is dus mogelijk dat dezelfde bit door verschillende hashfuncties gezet wordt.

Het resultaat van b.) is dus hetzelfde alsof je eerst voor $h_1(), \ldots, h_k()$ aparte Bloom filters met n bits aanmaakt en die dan met OR tot één Bloom filter samenvoegt.
Wij zullen hier alleen maar de (standaard) manier b.) analyseren. Daarbij zullen we de kans berekenen dat een toevallig gekozen element het antwoord *ja* krijgt. Deze kans gebruiken wij hier als benadering voor de kans op een fout positief antwoord. Voor universa die groot zijn in vergelijking met M is dat zeker een aanvaardbare benadering en voor andere gevallen zou je Bloom filters vermoedelijk toch al niet toepassen...
Op voorhand is daarbij niet duidelijk dat het goed is met meer dan 1 hashfunctie te werken. Als je bv. met extreem veel hashfuncties zou werken, dan zouden bijna alle bits 1 kunnen zijn en je krijgt zeker bijzonder veel foute positieve antwoorden. Is $k = 1$ misschien zelfs optimaal?
Wij gebruiken dat $e^x = \lim_{i\to\infty}(1 + \frac{x}{i})^i$ en dat voor voldoende grote i geldt dat

$$e^x \approx (1 + \frac{x}{i})^i \tag{1}$$

Bovendien veronderstellen wij dat alle hashfuncties de waarden gelijk verdelen, of precies: wij veronderstellen dat als je een toevallig element $x \in U$ kiest de kans dat $h_i(x) = j$ voor alle $1 \leq j \leq n$ gelijk is aan $\frac{1}{n}$ en dat voor elke verzameling M die we beschouwen de kansen voor alle elementen onafhankelijk van elkaar zijn. Dit zijn veronderstellingen die typisch zijn voor hashfuncties en door realistische hashfuncties (en verzamelingen) goed benaderd worden. Wij veronderstellen ook dat de hashfuncties onafhankelijk zijn – dus dat elementen die van de ene hashfunctie een zekere waarde i krijgen door de andere hashfuncties nog altijd gelijkverdeeld zijn.

Als wij één element met één hashfunctie in onze Bloom filter invullen, dan is de kans dat een zekere bit achteraf nog altijd 0 is dus $1 - \frac{1}{n}$. Nadat we k hashfuncties hebben toegepast, is de kans $(1 - \frac{1}{n})^k$ en nadat wij k hashfuncties op $m = |M|$ elementen hebben toegepast, is de kans $(1 - \frac{1}{n})^{km}$.
Als wij in vergelijking 1 voor x de term $-\frac{km}{n}$ invullen en voor i de term km (waarvan wij hier veronderstellen dat het voor de gekozen x groot genoeg is), dan krijgen wij als kans dat een bit na het invullen van alle elementen nog altijd 0 is

$$(1 - \frac{1}{n})^{km} \approx e^{\frac{-km}{n}} \tag{2}$$

De kans dat een bit achteraf 1 is, is dus (ongeveer) $(1 - e^{\frac{-km}{n}})$.
Omdat wij veronderstellen dat ook de hashfuncties voldoende onafhankelijk zijn, kunnen wij de kansen op een fout positief antwoord schatten: de kans dat een toevallig gekozen element een positief antwoord krijgt (dus dat alle k bits 1 zijn) is ongeveer $(1 - e^{\frac{-km}{n}})^k$.
Als wij de verzameling M (dus ook $m = |M|$) en het ter beschikking staande geheugen (dus n) als gegeven beschouwen, is de vraag hoe we k moeten kiezen om de kans op foute ja-antwoorden zo klein mogelijk te hebben. Dat is *in principe* heel gemakkelijk: pas jouw kennis uit het middelbaar toe en bereken het minimum van $f(k) = (1 - e^{\frac{-km}{n}})^k$ waarbij je n en m als constanten beschouwt.
Met twee trucjes wordt het iets gemakkelijker:
Wij hebben $f(k) = (1 - e^{\frac{-km}{n}})^k = e^{k(\ln(1 - e^{(-km)/n}))}$. Als wij schrijven

$$g(k) = k(\ln(1 - e^{(-km)/n})) \tag{3}$$

dan heeft $f(k)$ een minimum als en slechts als $g(k)$ er één heeft (omdat e^x strict monotoon stijgt). Als wij nu ook nog schrijven $p = e^{(-km)/n}$ dan wordt

$$g(k) = g_1(p) = \frac{-n}{m}(\ln p)(\ln(1 - p)). \tag{4}$$

De functie $g(k)$ heeft dus een minimum als $g_1(p(k))$ dat heeft en die heeft een minimum als $g_2(p) = (\ln p)(\ln(1-p))$ een maximum heeft (omdat wij een negatieve multiplicatieve constante weggelaten hebben).
Daarvoor kunnen wij nu onze kennis uit het middelbaar onderwijs toepassen:

$$\frac{\mathrm{d}g_2(p)}{\mathrm{d}p} = \frac{\ln(1-p)}{p} - \frac{\ln p}{1-p} = \frac{((1-p)\ln(1-p)) - (p \ln p)}{p(1-p)} \tag{5}$$

en dat is 0 voor $p = \frac{1}{2}$. Met een beetje meer rekenen kunnen wij ook nog zien dat $p = \frac{1}{2}$ het enige extremum is en in feite een maximum van $g_2(p)$. Met $p = \frac{1}{2} = e^{(-km)/n}$ krijgen wij dat $f(k)$ een minimum heeft voor $k = \frac{n}{m} \ln 2$.
Als m en n gegeven zijn, kiezen we dus het beste $k = \frac{n}{m} \ln 2$ onafhankelijke hashfuncties om een Bloom filter te implementeren. Natuurlijk moet het aantal een geheel getal zijn, zodat wij dit getal moeten afronden, waarbij wij de voorkeur geven aan $\lfloor \frac{n}{m} \ln 2 \rfloor$ omdat minder hashfuncties natuurlijk ook minder tijd vragen.
Als je dus bv. een verzameling met 1.000.000 elementen wilt opslaan en je hebt 1 MB ter beschikking, dus 8.000.000 bit, dan gebruik je het best $\lfloor \frac{8.000.000}{1.000.000} \ln 2 \rfloor = 5$ hashfuncties. De kans op false positives is dan $(1 - e^{\frac{-5*1}{8}})^5 \approx 0.0217$. Als m

en n vastliggen is dat gewoon het beste wat je kan doen. Als de kans op false positives te groot is, moet je kijken of je n niet misschien toch groter kan kiezen ...

Oefening 9

- *Stel dat je voor een verzameling M met m elementen een Bloom filter wilt maken waar de kans op een fout positief antwoord ten hoogste c is. Natuurlijk werk je met het optimale aantal hashfuncties. Ontwikkel de formule die het aantal bits beschrijft dat je nodig hebt om een Bloom filter te hebben die aan deze eisen voldoet.*

- *Hoeveel bits heb je nodig om een verzameling met $m = 1.000.000$ elementen door middel van een Bloom filter voor te stellen als je ten hoogste een kans van 0.02 (dus 2%) op een fout positief antwoord wil hebben?*

Maar het feit dat $p = \frac{1}{2}$ is ook nog om andere redenen interessant: $p = e^{(-km)/n}$ is de kans dat een bit bij deze optimale keuze achteraf nog 0 is en omdat $p = \frac{1}{2}$, is de kans dat een bit 0 is dus gelijk aan de kans dat de bit 1 is. De structuur van de Bloom filter is dus die van een gelijkverdeelde random string...

Oefening 10 *Veronderstel nu dat je zoals in a.) werkt en de k hashfuncties op disjuncte domeinen met grootte n/k werken. Bereken de kans op foute positieve antwoorden voor gegeven k, n, m en ook de optimale waarde voor het aantal k van hashfuncties dat je moet kiezen om de kans op foute positieve antwoorden zo klein mogelijk te maken als n, m gegeven zijn.*

Oefening 11

- *Als je twee bitmap voorstellingen $b_M, b_{M'}$ van verzamelingen M, M' voor hetzelfde universum hebt, dan kan je door de bitsgewijze `OR`-operator ($|$) de verzameling $M \cup M'$ berekenen: $M \cup M' = b_M | b_{M'}$. Klopt dat ook voor Bloom filters $b_M, b_{M'}$ die twee verzamelingen M, M' voorstellen? Of precies: is het resultaat van $b_M | b_{M'}$ altijd de bloomfilter voor $M \cup M'$ als het gebruikte aantal bits en de gebruikte hashfuncties dezelfden zijn? Geef uitleg.*

- *Als je twee bitmap voorstellingen $b_M, b_{M'}$ van verzamelingen M, M' voor hetzelfde universum hebt, dan kan je door de bitsgewijze `AND`-operator ($\&$) de verzameling $M \cap M'$ berekenen: $M \cap M' = b_M \& b_{M'}$. Klopt dat ook voor Bloom filters $b_M, b_{M'}$ die twee verzamelingen M, M' voorstellen? Of precies: is het resultaat van $b_M \& b_{M'}$ altijd de bloomfilter voor $M \cap M'$ als je hetzelfde aantal bits en dezelfde hashfuncties neemt? Geef uitleg.*

Stel dat de Bloom filters $b_M, b_{M'}$ met een optimaal aantal hashfuncties werken. Hoe zit het dan (in het geval dat $b_M | b_{M'}$ resp. $b_M \& b_{M'}$ een juiste Bloom filter zijn) met de Bloomfilter voor $M \cup M'$, resp. $M \cap M'$?

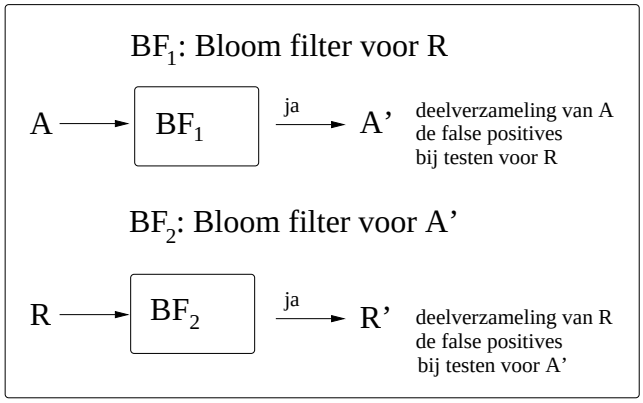
Oefening 12 *Stel dat je met een Bloom filter met $n = 2^i$ bits werkt voor een zekere $i \in \mathbb{N}, i > 1$. Je stelt vast dat je n te groot hebt gekozen en met dit aantal hashfuncties beter met $n/2$ zou werken, waarvoor het aantal fout positieve antwoorden ook nog aanvaardbaar zou zijn. Beschrijf een manier om een Bloom filter voor $n/2$ te berekenen zonder alle elementen te rehashen. Welke hash functies gebruik je?*

Bloom filters zijn niet alleen nuttig als foute positieve antwoorden niet erg zijn – zoals in het voorbeeld met de slechte passwoorden. Soms is het ook mogelijk achteraf foute positieve antwoorden snel te verbeteren. Op die manier kan je ook de snede van twee grote verzamelingen berekenen die op verschillende computers zitten, zonder ooit een van de grote verzamelingen te moeten sturen – zoals in het volgende voorbeeld:
In een netwerk heeft een computer A informatie over alle personen die voor een zeker (groot) bedrijf werken en een andere computer B informatie over alle adressen in Europa waar er in de toekomst gebouwd moet worden. Computer B moet nu weten welke personen van het bedrijf op een plaats wonen waar er in de toekomst gebouwd wordt. Eén mogelijkheid zou zijn de hele lijst van plaatsen door te sturen, maar die zou waarschijnlijk te groot zijn. Dus wordt een Bloom filter naar computer A doorgestuurd waarna computer A de vermoedelijk kleine lijst van mensen die op zo'n plaats wonen terugstuurt. De weinige *false positives* – dus personen die niet op een plaats wonen waar er gebouwd moet worden – kunnen dan snel verwijderd worden. In dit voorbeeld wordt dus de klemtoon gelegd op weinig data die over het netwerk gestuurd moet worden, omdat dat vaak een bottleneck is.
Natuurlijk is het een voor de hand liggend idee dat je compressiealgoritmen gebruikt om Bloom filters te comprimeren voordat je ze stuurt – ten slotte is weinig geheugen het hoofddoel van Bloom filters. Jammer genoeg hebben wij net gezien dat in Bloom filters met een minimale kans op een fout positief antwoord elke bit met kans $p = \frac{1}{2}$ gelijk aan 0 of gelijk aan 1 is – het zijn dus random strings die niet of nauwelijks comprimeerbaar zijn. In 2001 heeft Mitzenbacher iets heel interessants aangetoond: als je jouw Bloom filter niet optimaliseert voor een minimale kans op een fout positief antwoord **voor** het comprimeren, maar **na** het comprimeren, dan kan je soms Bloom filters krijgen die $n' > n$ bits gebruiken en dezelfde kans op een fout positief antwoord hebben als het optimum voor n bits – maar beter comprimeren en na compressie minder dan n bits nodig hebben. Misschien iets om na te lezen...

Bloom filter cascades

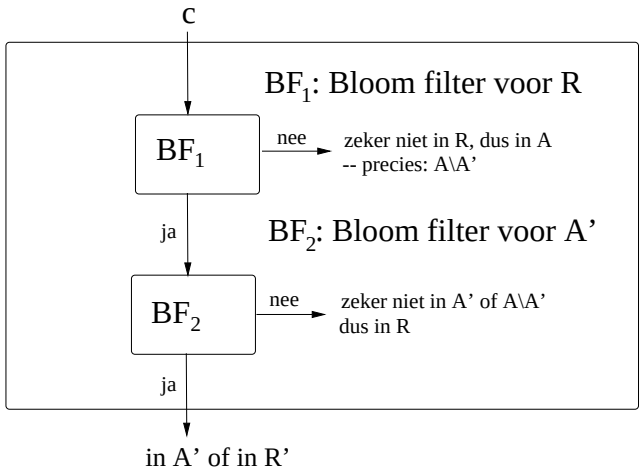
In een typische Bloom filter is het universum zeer groot – vaak zelfs oneindig (bv. alle strings). Je hebt dus zeer veel meer false positives dan elementen die je echt wilt opslaan – vaak zelfs oneindig veel. In een paper van Larisch, Choffnes, Levin, Maggs, Mislove en Wilson van 2017: *CRLite: A Scalable System for Pushing all TLS Revocations to all Browsers* wordt een situatie besproken die een beetje beter is: als de Bloom filter opgesteld wordt, weet je wél voor welke elementen ooit getoetst wordt of die in de filter zitten. Het gaat om certificaten die ingetrokken zijn. Een browser die wil toetsen of certificaten die hij kent nog geldig zijn, zal alleen maar certificaten toetsen die al eens bestonden en geen *random strings*.
Algemeen heb je dus de volgende situatie: je hebt een eindige verzameling $C = R \cup A$ met disjuncte verzamelingen R en A. Wij noemen de delen R voor *revoked* en A voor *active* en de hele verzameling C voor *certificate* om het goed met de toepassing te kunnen vergelijken. Je wil de verzameling R in een datastructuur opslaan zodat iemand die deze datastructuur heeft, kan toetsen of een element uit C in R is of niet – zonder de verzamelingen C, A of R te moeten kennen. Maar hier bedoelen wij met *toetsen* **zeker zijn** – wij willen dus noch false

positives noch false negatives.
Dat zullen wij doen, door meerdere Bloom filters na elkaar te gebruiken. Wij bouwen een *cascade* met meerdere niveau's en in elk niveau wordt voor een deel van de elmenten van C **zeker** beslist of die in R zitten of in A. Eén niveau bouwen we op, zo als het in Figuur 9 beschreven is: je berekent de verzameling A' van false positives voor de Bloom filter van R door alle elementen van A te testen en die waarvoor het antwoord is *"zit in R"* sla je op in A'. Dan maak je een Bloom filter voor A' en toets je alle elementen van R en stopt die, waarvan de filter beweert, dat die in A' zijn (dus de false positives voor deze filter) in R'.



Figuur 9: Het opstellen van één niveau in een cascade van Bloom filters. *Ja* betekent: volgens de Bloom filter zit het getoetste element erin.

Als elke van de twee gebruikte Bloom filters een kans van p op false positives heeft (soms is het ook nuttig verschillende waarden voor p te gebruiken), dan is $|R'| \approx p|R|$ en $|A'| \approx p|A|$ – dus typisch zeer veel kleiner.
Maar nu kunnen wij met $C' = R' \cup A'$ op dezelfde manier een tweede niveau van onze cascade maken met nog kleinere R'' en A'' als resultaat. Als wij de resultaten van niveau i nu R_i en A_i noemen, dan zal er een zekere j zijn, zodat ten minste één van de R_j en A_j leeg is (als er niet iets heel raars is met onze hashfuncties). Als R_i of A_i heel klein zijn, kunnen wij die ook gewoon opslaan als wij dat willen. Als één van de R_j of A_j leeg is, dan ook de andere in de volgende stap (of het volgende niveau), omdat de Bloom filter voor de lege verzameling alleen nullen bevat. Wij gaan er hier vanuit dat we doorgaan met nieuwe niveau's totdat er een lege R_i of A_i is.
Maar hoe testen wij nu of een element uit C in R zit of in A? Dat zien we in Figuur 10. Na de eerste test met BF_1 kunnen we al voor sommige c zeggen, dat ze zeker in A zitten – dat zijn de elementen die de gewone Bloom filter detecteert. Door deze eerste filter raken alleen de elementen uit R en die uit A'. In de volgende test met BF_2 kan je voor sommige elementen uit R zeker zeggen dat die zeker niet in A zitten – dus zeker in R. Na het eerste niveau weet je alleen voor elementen van $R_1 \cup A_1$ nog niet zeker waar die zitten. Die neem je dan als input van het volgende niveau. Elk *nee*-antwoord geeft je een gegarandeerd juiste toekenning aan R of aan A en omdat er op het einde een lege Bloom filter is (die dus altijd nee zegt), krijg je voor elke test van een element uit C een gegarandeerd juist antwoord.



Figuur 10: Het testen van een $c \in C$ in één niveau van een cascade van Bloom filters. *Nee* betekent: volgens de Bloom filter zit het getoetste element er niet in.

Als je dus een verzameling R wilt rondsturen, kan je dat doen, door een Bloom filter cascade rond te sturen. Natuurlijk moeten de gebruikte hashfuncties bij de opbouw en bij het testen dezelfden zijn. Deze cascades zijn wel groter dan een gewone Bloom filter, maar als de data groot is of slecht comprimeerbaar, zeer duidelijk kleiner dan bv. de verzameling R – zelfs na compressie. De grootte van de cascade hangt niet van de grootte van de elementen in R af – elke input wordt toch al naar een hashwaarde omgezet.
Als je alleen de Bloom filter cascade kent, kan je R wel niet reconstrueren, maar in sommige toepassingen zou dat zelfs een voordeel kunnen zijn.

Oefening 13 *Deze filter cascades zijn alleen bedoeld om elementen uit C te testen. Maar als je gewoon een element test dat niet in C is, krijg je ook altijd een antwoord – omdat de laatste filter leeg is. Je kan bv. in de beschreven toepassing iets testen, dat wel een string is, maar nooit als certificaat werd toegekend. Kan het antwoord dan alleen* "zit in A" *zijn, alleen* "zit in R"*, of zijn beide antwoorden mogelijk en misschien zelfs even waarschijnlijk? Geef uitleg.*

2.2 Sorteren van grote hoeveelheden data – extern sorteren

Als je meer data moet sorteren dan je in het geheugen kan houden, kunnen sorteeralgoritmen zoals quicksort die in het geheugen heel goed presteren inefficiënt worden omdat er plotseling een verschil is tussen de verschillende manieren waarop je een element moet lezen – als je een element moet lezen dat al in het geheugen staat is dat veel goedkoper dan een element te lezen dat nog van de harde schijf gelezen moet worden. In dit deel zullen wij een algoritme zien dat het aantal leesoperaties op de harde schijf minimaliseert.
Onze oplossing zal een mergesort algoritme zijn. Maar in plaats van het in elke recursiestap in maar twee delen te splitsen, zullen wij het op elk niveau in

meer delen splitsen waarbij wij rekening zullen houden met de grootte van het geheugen en de grootte van de blokken die je in één keer kan lezen.

basisstap: je splitst het hele bestand op zodat je delen $d_1, \ldots, d_n$ hebt die **net** in het geheugen gesorteerd kunnen worden. Deze delen worden efficiënt in het geheugen gesorteerd (bv. door middel van mergesort of quicksort) en teruggeschreven.

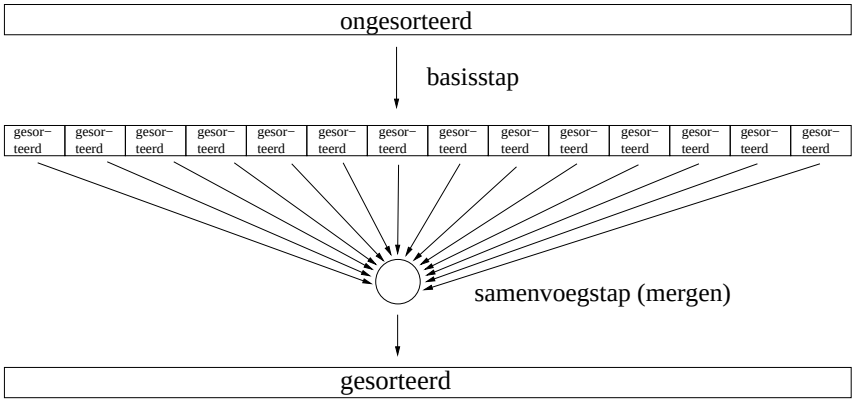
Als je bv. 10GB moet sorteren en je kan 500MB in het geheugen sorteren (waarvoor je natuurlijk meer dan 500MB geheugen nodig hebt) zonder te moeten swappen, zou je 20 delen maken en die in het geheugen lezen, sorteren en dan terugschrijven. Zo wordt elk blok met data één keer gelezen en één keer geschreven.

samenvoegstap: Nu moeten de gesorteerde delen nog gemerged worden. Wij lezen van zoveel mogelijk delen het eerste blok in het geheugen. *Zoveel mogelijk* betekent dat wij maar zoveel delen kiezen dat wij voor elk deel één blok in het geheugen kunnen houden. Dan schrijven wij altijd de kleinste sleutel van één van de delen in een outputblok tot dat volzit en uitgeschreven wordt. Als één van de blokken leeg is, wordt dat door het volgende blok van dat deel vervangen. Dit samenvoegen tot één nieuw gesorteerd deel noemen wij één mergeoperatie.

Als wij t delen tegelijk kunnen mergen en er zijn samen d delen dan passen wij deze mergeoperatie $\lceil d/t \rceil$ keer toe totdat elk deel één keer met andere delen samengemerged werd. Wij hebben dan $\lceil d/t \rceil$ grotere gesorteerde delen.

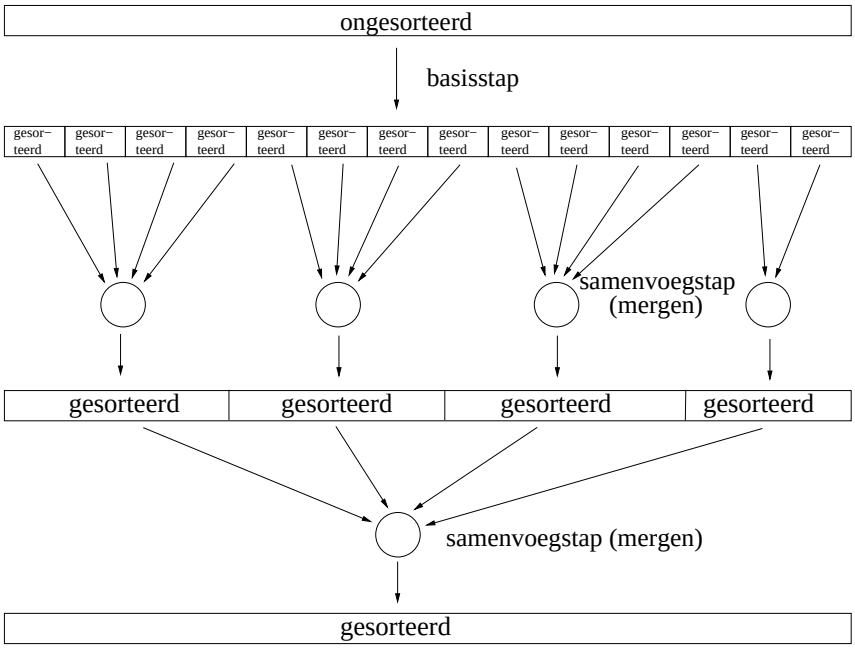
In deze stap wordt elk blok opnieuw één keer gelezen en één keer geschreven.

Deze samenvoegstap wordt herhaald tot er maar één bestand overblijft.
Het principe voor één samenvoegstap zien jullie nog eens in Figuur 11 en in Figuur 12 zien jullie een voorbeeld met een heel kleine vertakking van 4. Daardoor zijn er twee samenvoegstappen nodig. Realistische voorbeelden kan je natuurlijk niet tekenen...



Figuur 11: Het principe van extern sorteren.

Stel dat B de blokgrootte is die je in één stap kan lezen en M de grootte van het geheugen dat je voor het opslaan van de delen ter beschikking kan stellen. Dan kan je dus $\frac{M}{B}$ delen per mergeoperatie mergen, zodat het aantal delen voor de volgende samenvoegstap met de factor $\frac{M}{B}$ daalt. Omdat in het begin de delen grootte M mogen hebben, kan je dus in s samenvoegstappen bestanden van grootte $M * (\frac{M}{B})^s$ sorteren.
Als je er opnieuw van uitgaat dat je 500MB ter beschikking hebt en dat bv. een blok van 4KB in één stap gelezen kan worden, zijn dat al 62.5 Terabyte die je kan sorteren met maar één enkele samenvoegstap. Dat is in feite heel veel en ook al wordt de hoeveelheid data groter – het geheugen groeit ook en de 500 MB waarvan wij hier gesteld hebben dat die ter beschikking staan, zijn zeker een benedengrens...
Wij kunnen dus met 2 lees- en schrijf-operaties per blok 62.5 TB sorteren.



Figuur 12: Extern sorteren in 2 samenvoegstappen. In dit voorbeeld kunnen maar 4 delen tegelijk gemerged worden.

Wat wij hier gezien hebben is dus gewoon mergesort waar wij aan de basis niet alleen enkele sleutels hebben maar al grotere reeds gesorteerde delen. En het belangrijkste: de vertakking is **veel** groter. De vertakking $\frac{M}{B}$ bepaalt de diepte van de recursieboom en de diepte van de recursieboom bepaalt het aantal lees- en schrijf-operaties.
Wij hebben er niet op gelet wat je bv. moet doen als de records die je wil sorteren niet in één blok passen etc. In dergelijke gevallen zijn lichte wijzigingen nodig.

Oefening 14 *Wij hebben geschreven* „Dan schrijven wij altijd de kleinste sleutel van één van de delen in een outputblok...“. *Hoe doe je dat het best op een efficiënte manier? Altijd naar het kleinste element in elk opgeslagen blok kijken*

zou lineair zijn in het aantal blokken. Kan dat efficiënter? Denk aan DA1 en DA2.

Oefening 15 *Gegeven de grootte M van het geheugen dat ter beschikking staat en B de blokgrootte die je in één stap kan lezen. Geef een formule voor het aantal leesoperaties (per blok) op de harde schijf dat nodig is om een bestand met n bytes te sorteren.*

Oefening 16 *De bedoeling van deze oefening is (nog) een beetje meer gevoel te krijgen voor welke betekenis de asymptotische analyse heeft en in welke gevallen ze meer of minder belangrijk is.*
Stel dat een lees/schrijf-operatie op de harde schijf kost C_1 per blok heeft en een operatie in het geheugen (vergelijken, verplaatsen, etc.) kost C_2. De motivatie voor de laatste algoritmen die wij hebben gezien was dat $C_1 >> C_2$ – dus mag je dat ook hier stellen.
Je hebt een vast geheugen van 400 MB ter beschikking en een bestand van n records waarvan elke record een grootte van 400 bytes heeft. Een blok heeft grootte 4 KB.
Onderzoek drie varianten van extern sorteren: één keer sorteer je de deelbestanden die net in het geheugen passen met mergesort, één keer met bubblesort en één keer met quicksort. In elk geval pas je achteraf de mergebewerking op zoveel niveau's toe als er noodzakelijk zijn.

- *Zal er een groot verschil in performantie zijn als je deze 3 varianten in de praktijk op een bestand van bv. 10 GB (dus 25000000 records) toepast?*
- *Bepaal de asymptotische complexiteit van alle drie varianten van extern sorteren.*

3 Algoritmen voor strings

Iedereen van jullie heeft zeker al algoritmen voor strings gebruikt – of het nu *query replace* in een editor is of het zoeken van woorden in een tekst. Maar ook op andere, minder duidelijke plaatsen zijn stringalgoritmen belangrijk – bv. in de bioinformatica waar de strings bv. het DNA voorstellen of in databanken waar de strings een identifier – bv. van een scheikundig molecule kunnen zijn. Wat precies met de strings gedaan wordt – of getest moet worden of twee strings identiek zijn (misschien de gemakkelijkste taak), of of een string deelstring van een andere string is, of of één string een *mutatie* van een andere string kan zijn, etc... verschilt natuurlijk van geval tot geval.
Wij zullen hier verschillende algoritmen zien – soms ook verschillende algoritmen voor dezelfde taak. De hoofdreden om deze algoritmen te leren is om de ideeën te leren kennen en ze achteraf zelfstandig misschien in andere omstandigheden te kunnen toepassen. Wij zullen hier – als een soort bewijs dat het nuttig is ideeën te kennen – bv. ook technieken zien die *in principe* geïnspireerd zijn door technieken die jullie al in DA1 en DA2 hebben gezien!

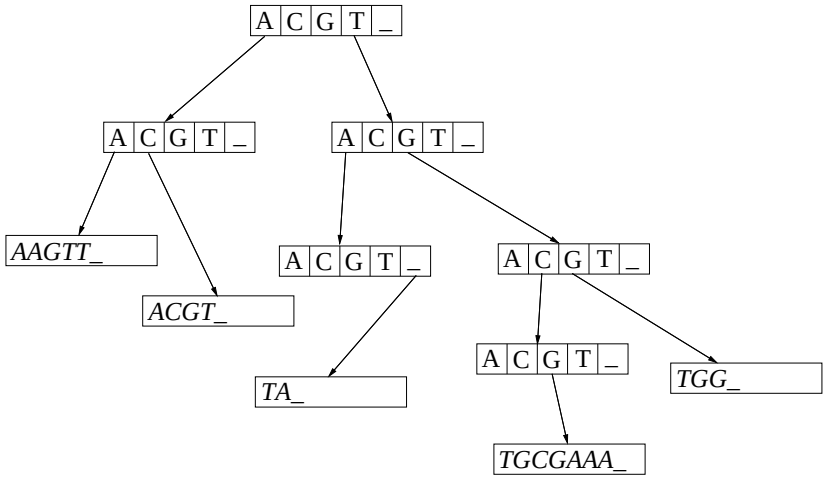
3.1 Tries (prefix trees)

Een opmerking vooraf: de verschillende soorten tries die hier besproken worden, zijn alleen maar *basisprincipes*. Als in het examen gevraagd wordt een *ternary trie* te geven, moet je dat natuurlijk zo doen als in de les gezien, maar in echte toepassingen zijn er vaak goede redenen om van de precieze vorm van een *ternary trie* of een *Patricia trie* (of een *ternary Patricia trie*) een beetje af te wijken. In feite zullen wij zelfs een algoritme zien dat een *een beetje andere vorm* van trie gebruikt – maar duidelijk met ideeën die al gezien zijn.
De naam *trie* is afgeleid van het middengedeelte van *re***trie***val*. Vermoedelijk was het de bedoeling het zoals in *retrieval* uit te spreken – dus zoals *tree* – tenslotte is het een boom. Maar intussen spreekt het iedereen zo uit als *to try*.
Een trie – soms ook prefix tree genoemd – is een boom die ook als zoekboom gebruikt kan worden en vooral veel toepassingen heeft als het om strings gaat – bv. in de computationele biologie. Laten we eerst motiveren waarom er nood aan nog een implemetatie van een zoekboom is:
Stel dat wij geen sleutels, maar strings in een zoekboom willen opslaan. Dat kan natuurlijk heel eenvoudig – je moet alleen een functie hebben die twee strings s, s' vergelijkt en zegt of $s < s'$, $s = s'$ of $s > s'$. Er is maar één probleem: tot nu toe hebben wij nooit rekening gehouden met de kost van het vergelijken. Als je n sleutels in een gebalanceerde zoekboom hebt, is de kost voor het opzoeken $O(\log n)$ – maar als je strings vergelijkt komt er nog een factor $m = |s|$ bij, waarbij s de string is die je opzoekt – het wordt dus $O(m \log n)$. Tries combineren nu het vergelijken met het opzoeken door de vertakking in de boom afhankelijk van de lettertekens in de string te maken!
Jullie zullen (een beetje) verschillende datastructuren in de literatuur vinden die allemaal *trie* genoemd worden. Ze implementeren allemaal hetzelfde idee, maar de details zijn soms een beetje anders. Meestal wordt verondersteld dat voor geen twee strings die je wilt opslaan de ene een prefix van de andere is. Dat zullen wij ook hier in het begin veronderstellen en expliciet zeggen als wij van deze veronderstelling afwijken. Deze eigenschap kan je ook afdwingen door een extra `end-of-string` teken te gebruiken. In de programmeertaal **C** wordt bv.

elke string door een NULL-teken afgesloten, zodat nooit één string een prefix van een andere kan zijn.
Als wij ook strings willen toelaten, die een prefix van een andere string zijn, moeten wij bij interne toppen een flag bijhouden die zegt of de top bij een woord behoort of niet. In feite komt het als je het implementeert bijna op hetzelfde neer: in het geval waar je prefixen toelaat heb je een flag die `FALSE` (of 0) is of `TRUE` (niet 0) en in het geval waar je een extra `end-of-string`-teken gebruikt, heb je de pointer naar het speciale teken die ofwel `NULL` is ofwel niet `NULL`.
Wij kiezen dus voor de volgende definitie:

Definitie 1 *Gegeven een eindig alfabet A. Een (gewone) trie is een wortelboom waar elke interne top voor elke letter $a \in A$ een kind k_a heeft, dat ook leeg kan zijn en elk blad een string $s \in A^*$ als label heeft.*
De wortel stelt de lege string $\emptyset$ voor.
Als een top de string s voorstelt en het kind k_a is niet leeg, dan stelt k_a de string s, a voor.
Als een blad b het label s heeft en de string s' voorstelt, dan is s' een prefix van s.
De labels van de bladeren zijn de strings die in de trie zijn opgeslaan.

Later zullen wij speciale implementaties van tries zien die deze definitie eerder als idee dan letterlijk implementeren.
In deze trie dienen de interne sleutels dus alleen maar voor de navigatie om een sleutel die in een blad is opgeslagen te vinden. Een voorbeeld voor een trie met alfabet $A = \{A, C, G, T, _\}$ zien jullie in Afbeelding 13.



Figuur 13: Een voorbeeld voor een trie.

Het is duidelijk dat het opzoeken van een string van lengte m in tijd $O(m)$ kan – dus een factor van $\log n$ beter dan in een gebalanceerde zoekboom met n elementen en in feite onafhankelijk van het aantal elementen in de boom!
Het idee is hier dus vooral dat je de vertakking in de boom afhankelijk van de tekens in de string doet. Hoe je dat het best precies doet, hangt van het

probleem af. Wij zullen voorbeelden zien (Patricia tries, ternary tries) hoe je sommige van de implementatiedetails kan oplossen, maar dat is vooral om historische redenen – je kan het ook anders doen of alleen maar delen van de in deze voorbeelden beschreven manier van doen gebruiken.
De performantie voor het opzoeken ziet er fantastisch uit, maar jullie als ervaren programmeurs zien al het addertje onder het gras: als het alfabet relatief groot is (bv. alle ASCII tekens), heb je voor elke top vrij veel geheugen nodig – waarbij in de meeste gevallen veel geheugen alleen voor NULL-pointers gebruikt wordt. Bij veel implementaties zijn de kinderen voor elk letterteken dan ook alleen *"impliciet"* aanwezig en is de manier waarop je de pointers in een top opslaat belangrijk – zie bv. het hoofdstuk over *ternary tries*.
Maar laten we eerst kijken hoe je in een trie iets opzoekt, toevoegt en verwijdert. Wij veronderstellen dat de boom altijd ten minste één top heeft. Geïnitialiseerd wordt de boom met één top die de lege string voorstelt.

opzoeken: Stel dat je een string $s = s_0, \ldots, s_k$ wilt opzoeken. Dan ga je – beginnend bij de wortel, dus op diepte $d = 0$ – op diepte d naar kind k_{s_d} totdat

- het kind van een interne top NULL is (string niet gevonden)
- een blad bereikt wordt. Dan vergelijk je de gezochte string met de string die in het blad opgeslagen is.

toevoegen: Stel dat een string $s = s_0, \ldots, s_k$ toegevoegd moet worden. Je begint net zo als bij het opzoeken. Als de string gevonden wordt, hoeft die niet meer toegevoegd te worden. Stel dus dat je in een situatie terechtkomt waar die niet gevonden wordt.

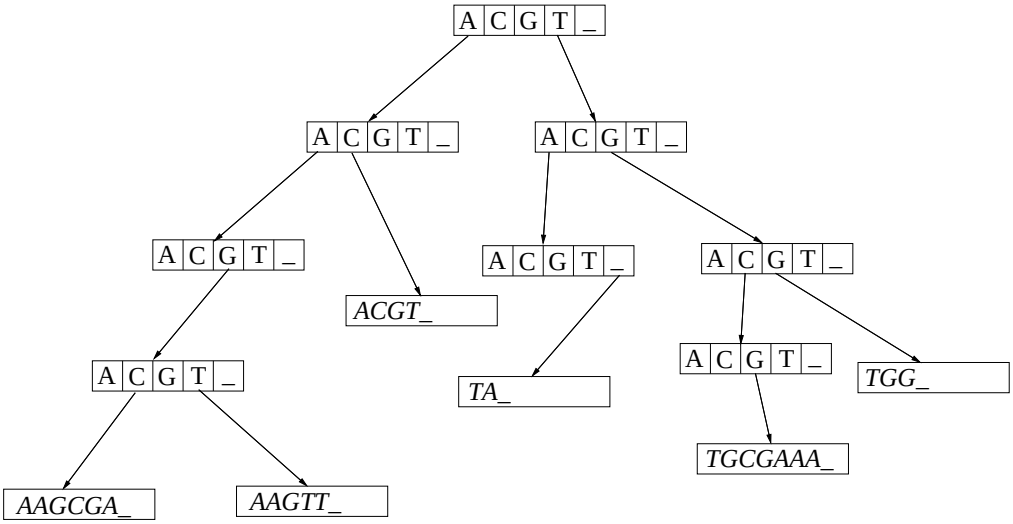
- het kind k_{s_d} van een interne top is NULL: voeg een blad als kind k_{s_d} toe en label het met s.
- een blad b wordt op diepte d bereikt, maar het label $l_0, \ldots, l_m$ is niet gelijk aan $s_0, \ldots, s_k$. Kies $j = \min\{i | i \geq d, l_i \neq s_i\}$. Maak dan b een interne top en vanuit b maak een pad dat langs de kinderen $k_{s_d}, \ldots, k_{s_{i-1}}$ gaat en voeg bij de laatste top twee bladeren als kinderen k_{l_i} en k_{s_i} toe met labels $l_0, \ldots, l_m$ resp. $s_0, \ldots, s_k$.

verwijderen: Je zoekt het blad op dat de string voorstelt. Dan verwijder je alle toppen op dit pad na de laatste top die ten minste twee niet lege kinderen heeft (resp. na de wortel als zo'n top niet bestaat). Als alleen nog de wortel over is, vervang je die door een blad dat de lege string voorstelt.

Als de string `AAGCGA_` aan onze voorbeeld-trie in Afbeelding 13 wordt toegevoegd, is het resultaat de trie in Afbeelding 14.

Oefening 17 *Voeg* `TGCGAAG_` *aan de trie in Afbeelding 14 toe. Verwijder dan* `TA_`.

Oefening 18 *Stel dat jouw alfabet A alle ASCII-tekens zijn – dus alle tekens van 8 bit. Natuurlijk kan je een string van lengte m met tekens uit A ook interpreteren als een string van lengte* $8 * m$ *met tekens uit* $\{0, 1\}$.



Figuur 14: De voorbeeld-trie uitgebreid met een nieuwe string.

Wij implementeren de trie door voor de kinderen een array van pointers (64 bit) van lengte $|A| = 256$, resp. lengte $|\{0,1\}| = 2$ aan te maken.

- *Stel dat je (bijna) alle strings van lengte l wilt opslaan. Heb je meer geheugen nodig als je de trie volgens A vertakt of als je een trie maakt die volgens $\{0,1\}$ vertakt?*
- *Stel dat je – in verhouding met het totale aantal strings – relatief weinig strings van lengte l wilt opslaan en dat jouw trie veel lange paden zonder vertakking heeft (de paden in de trie die volgens $\{0,1\}$ vertakt, zijn natuurlijk nog langer dan die in de trie die volgens A vertakt). Heb je meer geheugen nodig als je de trie volgens A vertakt of als je een trie maakt die volgens $\{0,1\}$ vertakt?*

Het grootste probleem bij de implementatie van een trie is dus het opslaan van de pointers naar de kinderen. Als die in de vorm van een array worden opgeslaan, kan je sleutels bijzonder snel opzoeken, maar vooral als er in vergelijking met de lengte van de strings weinig strings opgeslaan zijn (dat noemen wij soms een dunne trie (Engels: sparse)), verspil je in het geval van een groot alfabet heel veel geheugen. Alternatieven zouden bv. hashmaps zijn, die in het geval van veel NULL-pointers geheugen kunnen besparen, maar trager zijn. Wij zullen nu speciale tries zien om dunne tries efficiënt te implementeren.

3.1.1 Patricia trees (tries)

Onze definitie van een trie laat het al toe, geen lange paden direct voor een blad te hebben – die kunnen verwijderd worden. Paden zonder vertakking tussen twee interne toppen kunnen wij volgens de definitie niet gewoon verwijderen. Patricia trees of Patricia tries reduceren het geheugenverbruik door lange paden zonder vertakking te vermijden. Zij werden in 1968 door Donald R. Morrison geïntroduceerd. *Patricia* staat voor **P**ractical **A**lgorithm **t**o **R**etrieve **I**nformation **C**oded **i**n **A**lphanumeric.

Het idee is om lange paden zonder vertakking te vermijden en altijd alleen interne toppen met vertakking te hebben door altijd alleen rekening te houden met de toppen waar wel vertakt wordt. Patricia tries worden soms ook gecomprimeerde tries genoemd. Daarvoor heeft elke interne top een extra variabele `skip` die zegt hoeveel lettertekens zonder vertakking er zijn – die sla je gewoon over. Je zoekt dus altijd alleen maar volgens die tekens waar er verschillende mogelijkheden zijn. Daarbij kan het natuurlijk gebeuren dat op pad naar een blad de string die je zoekt al vroeg een verschil heeft met de string die in het blad opgeslagen is – en je merkt het niet omdat het een teken is dat je overslaat. Als een string in de trie bv. `AAAAABAAAAABC_` is en er zijn alleen maar vertakkingen bij de `B`'s, dan zou het opzoeken van `BBBBBBBBBBBB_` tot het blad gaan dat `AAAAABAAAAABC_` bevat en pas dan zou je merken dat het niet dezelfde string is – ook al verschilt al het eerste letterteken. Belangrijk is natuurlijk dat **als** de string in de trie zit het juiste blad gevonden wordt.
Als pseudocode ziet dat er zo uit:

Algoritme 1 *(Zoek Patricia)*

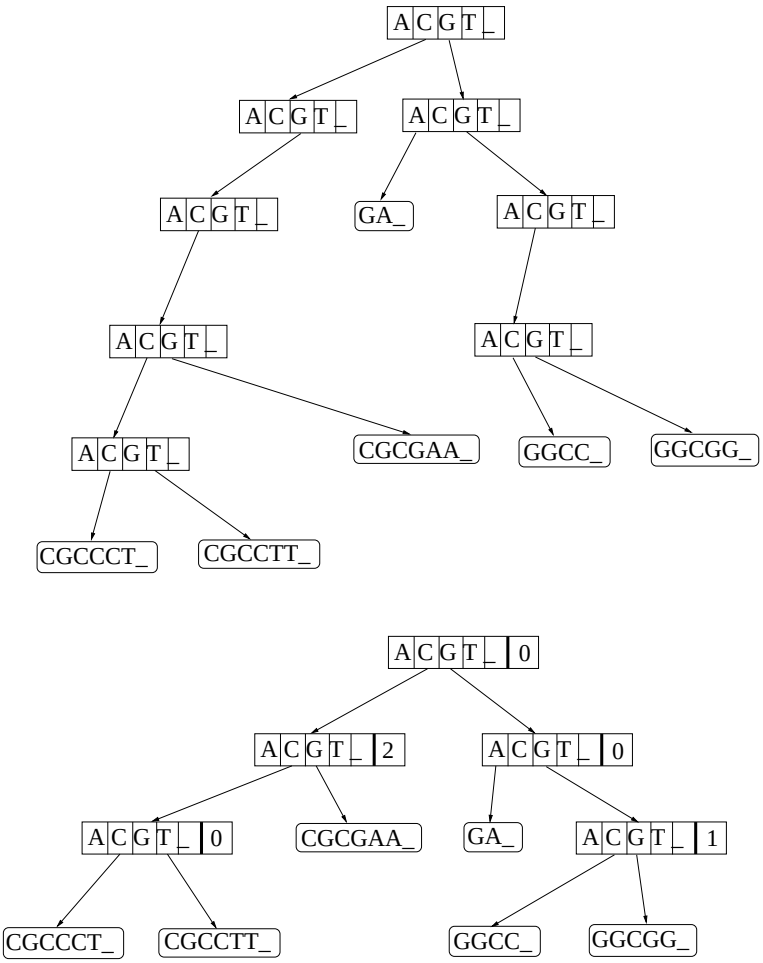
```
zoek_Patricia(s,m,top)
// zoekt de string s[]=s[0],...,s[m] in de Patricia tree
// met wortel top
{

next=0

while (top geen blad)
{ if (next+top.skip > m) return ‘‘niet gevonden’’
  // de string s is korter dan alle strings die vanaf
  // hier bereikt worden
  next = next+top.skip+1
  top = top.kind[s[next-1]]
  if (top = NULL) return ‘‘niet gevonden’’
}
// nu zit je in een blad
if (top.string = s) return ‘‘gevonden’’
else return ‘‘niet gevonden’’
```

De complexiteit is – net zo als voor gewone tries – $O(m)$ als een string van lengte m opgezocht wordt.
Toevoegen bij een Patricia tree is duidelijk ingewikkelder, omdat je zolang je geen blad bereikt hebt niet weet met welke prefix je bezig bent. Zoals in ons voorbeeld `AAAAABAAAAABC_` kan het gebeuren dat je heel laat merkt dat je al vroeger een vertakking zou hebben moeten introduceren. Je moet dan dus eerst een string vinden om de tekens die je hebt overgeslaan te reconstrueren. Als de klemtoon niet alleen op het efficiënte opzoeken ligt en je ook efficiënt wilt toevoegen, is het misschien zinvol de deelstrings met de overgeslagen tekens bij de interne toppen bij te houden. Dat vraagt wel een beetje extra geheugen voor de interne toppen, maar je kan dan ook al tijdens het zoeken de prefixen vergelijken. Omdat je de hele strings dan in principe in de bladeren niet hoeft bij

te houden (alleen het laatste deel) – je kan het eerste deel tijdens het opzoeken opbouwen – zou je in het geheel zelfs ruimte kunnen besparen.



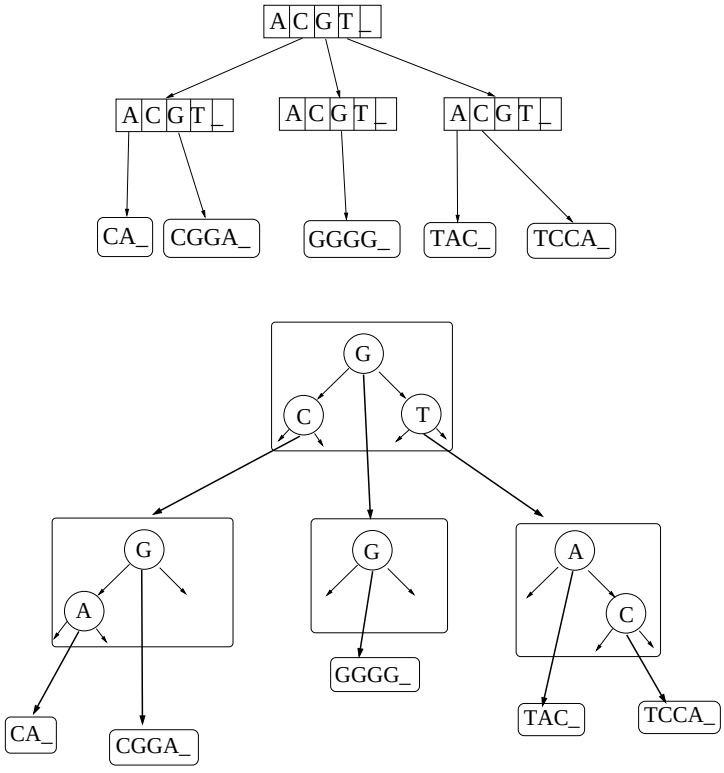
Figuur 15: Een trie voorgesteld op de *gewone* manier en als Patricia trie.

In Figuur 15 zien jullie een trie die één keer op de gewone manier voorgesteld is en één keer als Patricia trie. Hier zien jullie al dat je geheugen kan besparen. In realistisch grote en dunne tries (die je zeker niet meer wilt tekenen) is dat natuurlijk nog veel meer.

3.1.2 Ternary trees (tries)

Ternary trees of ternary tries – beschreven door J. Bentley and B. Sedgewick in 1988 zijn in feite alleen maar een manier om de toppen in een trie voor te stellen en die kan met alle soorten tries gecombineerd worden. Wij zullen gewone tries als voorbeeld kiezen. Ternary tries kan je op verschillende manieren interpreteren – aan de ene kant als bomen met drie vertakkingen, waarbij de middenvertakking een bijzondere eigenschap heeft: dan wordt overgeschakeld naar het volgende teken in de string – en aan de andere kant als een implemen-

tatie van het datatype trie waar elke top door een kleine boom is voorgesteld. Wij beschrijven het op de tweede manier. Als je het implementeert komt het in principe op hetzelfde neer. Het probleem dat hier opgelost wordt, is dat ook als je wel een vertakking hebt, maar een groot alfabet en een kleine vertakking – dus veel geheugen voor NULL-pointers gebruikt – veel geheugen verspild wordt. In ternary tries wordt het probleem met het geheugen voor de pointers naar de kinderen opgelost door die pointers in kleine zoekboompjes op te slaan. Elke top in de trie heeft dus een pointerboom voor de kinderen. Elke top in de pointerboom heeft naast een sleutel uit het alfabet nog drie pointers: `kleiner`, `gelijk` en `groter` de `kleiner`- en `groter`-pointers gaan naar andere toppen in de pointer-boom. De `gelijk`-pointer gaat naar de volgende top in de trie – dat is dus een pointer naar het kind. Op deze manier geïnterpreteerd zijn de ternaire bomen dus in feite binair: de pointers naar de kinderen zijn gewoon de data die bij de sleutel behoort en die je wilt opslaan. De bomen bevatten dus alleen de lettertekens die ook echt gebruikt worden en omdat het aantal lettertekens dat met die prefix en op die positie gebruikt wordt normaal relatief klein is, wordt de boom bij het toevoegen ook niet geherbalanceerd.



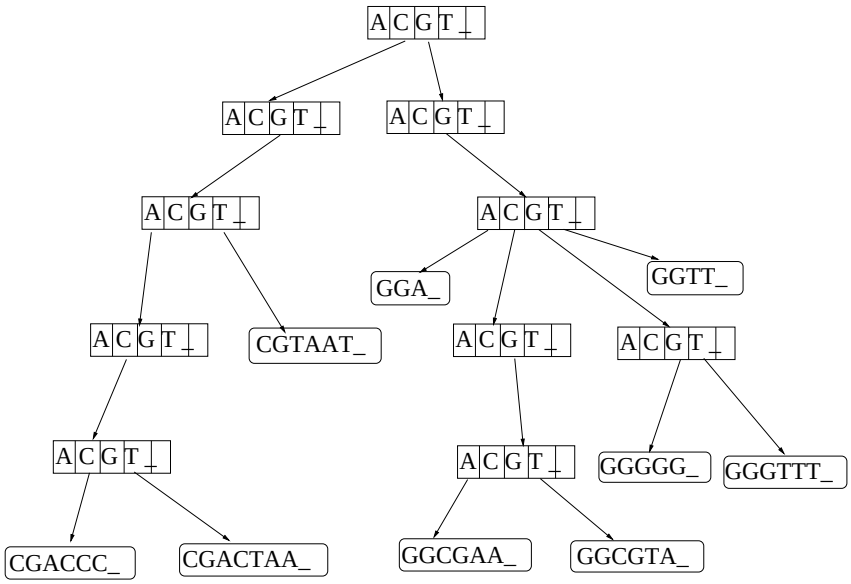
Figuur 16: Een trie voorgesteld op de *gewone* manier en als ternary trie.

In Figuur 16 zien jullie een trie die één keer op de gewone manier voorgesteld is en één keer als ternary trie. Hier lijkt het er niet op dat ternary tries een voordeel hebben. Als daarentegen het alfabet bv. alle ASCII tekens zou bevatten, zou de gewone manier veel meer ruimte eisen, terwijl de ternary trie niet zou veranderen. Deze manier van doen is dus bijzonder interessant bij grotere alfa-

betten waar het vaak voorkomt dat er in elke interne top veel NULL-pointers zitten.

Oefening 19

- *Stel de trie in Figuur 17 voor als Patricia trie.*
- *Stel de trie in Figuur 17 voor als ternary trie.*
- *Stel de trie in Figuur 17 voor als Patricia trie waar je de toppen op de manier van een ternary trie voorstelt.*



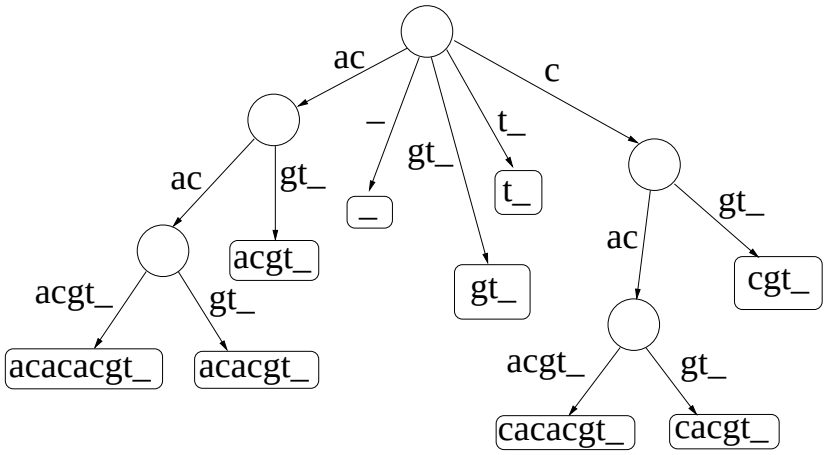
Figuur 17: Een trie voorgesteld op de *gewone* manier.

Oefening 20 *Stel dat jouw alfabet $A = \{a, c, g, t, _\}$ is en je strings in een trie opslaat waarbij de interne toppen gemiddeld maar 2 kinderen hebben. Stel je die trie op de manier voor waar je een array van pointers bijhoudt of op de manier van een ternary trie? Geef voldoende uitleg waarom jouw keuze goed is.*

3.2 Suffix bomen

Suffix trees of suffix bomen zijn een soort trie die sommige verrassend snelle algoritmen voor strings mogelijk maakt. Omdat het tries zijn, zijn het dus *prefix suffix bomen*: je slaat de suffixen op en zoekt die volgens de prefixen van de suffixen. Daarbij is het opstellen van de suffix boom een preprocessing dat een heel krachtige datastructuur ter beschikking stelt die dan uitgebuit kan worden – zie bv. de Burrows-Wheeler transformatie. Anders dan gewone tries die een dynamische datastructuur vormen en die bedoeld zijn om ook strings te kunnen toevoegen en verwijderen, wordt een suffix boom dus normaal één keer opgebouwd en dan blijft die zo.
Stel dat een tekst t gegeven is. Wij stellen ook hier dat die door een uniek teken afgesloten is – wij zullen opnieuw _ gebruiken. Als het laatste teken niet uniek zou zijn, zou het laatste teken alleen al een prefix van andere suffixen zijn. De

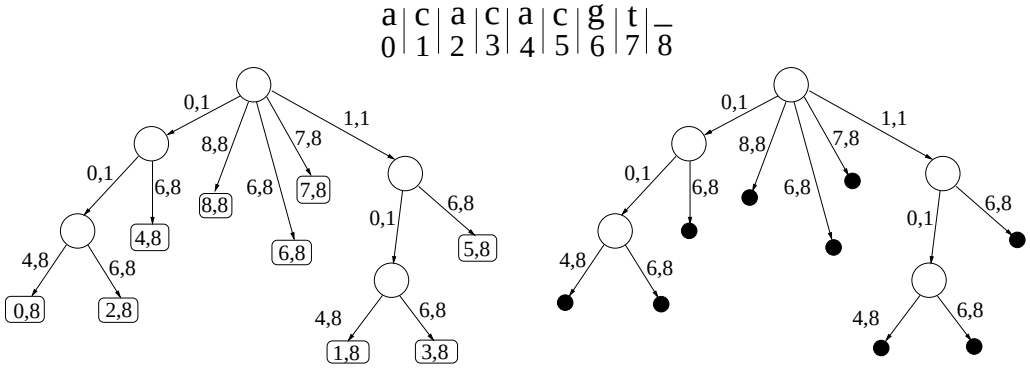
suffix boom voor t is een trie die alle suffixes van t bevat – en geen andere strings. Voor de tekst `acacacgt_` bevat de trie dus `acacacgt_`, `cacacgt_`, `acacgt_`, `cacgt_`, `acgt_`, `cgt_`, `gt_`, `t_` en `_`.
Het soort trie dat wij kiezen, lijkt het meest op een Patricia trie: er zijn geen toppen met maar één kind. Al voor Patricia tries hebben wij gezegd dat je de strings die je overslaat ook bij de toppen kan bijhouden en dat is wat wij hier doen. Je kan deze strings in de top bijhouden waar een boog naartoe gaat of in de boog zelf – dat komt op hetzelfde neer. Als wij de suffix boom tekenen, schrijven wij deze strings samen met het eerste teken dat het verschil maakt altijd aan de bogen – dat is gemakkelijker om te verstaan en dan hoeven wij de tekens van het alfabet ook niet meer in de top te tekenen. Het voorbeeld voor `acacacgt_` zien jullie in Figuur 18.



Figuur 18: Een suffix boom.

Als de tekst lengte n heeft, heeft de suffix boom voor de tekst ten hoogste $2n-1$ toppen en $2n-2$ bogen. Het aantal toppen en bogen is dus $\Theta(n)$. Jammer genoeg vraagt alleen het opslaan van alle prefixen in de bladeren $\Theta(n^2)$ geheugen en dus normaal ook ten minste $\Theta(n^2)$ tijd. De oplossing om het misschien toch in tijd $O(n)$ te kunnen doen: hier – anders dan in algemene Patricia tries – is het niet nodig de prefixen allemaal expliciet op te slaan. Hier gebruiken wij dat het geen arbitraire trie is, maar dat alle opgeslagen strings de suffixen van dezelfde tekst zijn – alle strings die wij opslaan zijn dus deelstrings van deze tekst en wij kunnen gewoon de tekst opslaan (dat eist $\Theta(n)$) en elke keer dat wij een string willen opslaan – om het even of het in een blad is of in een boog – slaan wij gewoon de pointers naar het begin en het einde van een kopie van deze string in de tekst op. Dat vraagt constant geheugen voor elke string en omdat het aantal bladeren en bogen maar $\Theta(n)$ is, vraagt deze manier de suffix boom op te slaan ook maar geheugen $\Theta(n)$. Natuurlijk kan je hier de labels van de bladeren ook nog verwijderen – ten slotte kan je de hele string uit de labels van de bogen reconstrueren. Dat bespaart geheugen, maar natuurlijk blijft het geheugenverbruik $\Theta(n)$.
In principe kunnen de indices naar arbitraire posities in de tekst refereren, waar de string zit. Als de string dus bv. `ied` is, dan kunnen de indices van elke positie in de tekst waar `ied` staat, gebruikt worden. Maar het algoritme van Ukkonen

doet het gelukkig anders: de posities die gekozen worden, zijn altijd zo dat als de top waar de boog vertrekt een string s voorstelt – bv. `geb` – dan zijn de posities ook zo dat ze een deel van de tekst beschrijven die direct na s – in het voorbeeld dus na `geb` – in de tekst komt.



Figuur 19: Een suffix boom waar de strings als pointers naar de tekst zijn opgeslagen.

De voor de hand liggende manier de suffix boom op te bouwen – ook al eist die nu maar $\Theta(n)$ geheugen – vraagt wel tijd $\Theta(n^2)$ voor een tekst van lengte n. Maar het volgende is een heel belangrijk resultaat, dus zullen wij het een stelling noemen en een hoofdstuk aan een algoritme dat deze grens verwezenlijkt besteden:

3.2.1 Het algoritme van Ukkonen

Stelling 2 *De suffix boom voor een tekst van lengte n kan in tijd $O(n)$ opgebouwd worden.*

Het eerste algoritme dat in lineaire tijd werkte, was van Weiner (1973). Later heeft Mc Creight (1976) een verbeterd lineair algoritme gegeven dat minder geheugen eist. Het tot nu toe beste algoritme is van Ukkonen (1995). Dit algoritme heeft alle voordelen van de vroegere algoritmen en is beter verstaanbaar. Het algoritme is ook zorgvuldig beschreven in het boek *Algorithms on strings, trees, and sequences* van Dan Gusfield – het is dus zeker een goed idee niet alleen in de lesnota's, maar ook daar eens te kijken.
De meest eenvoudige manier om een suffix boom voor een tekst van lengte n op te bouwen, zou zijn gewoon alle suffixen na elkaar toe te voegen. Dat zijn $n+1$ suffixen en de tijd die nodig is zal $O(n^2)$ zijn. Het algoritme van Ukkonen ligt niet echt voor de hand en gebruikt zelfs stappen die eerst geen goed idee lijken. Wij zullen bv. ermee beginnen veel meer strings toe te voegen – namelijk $\Theta(n^2)$ strings, waaronder de $n+1$ suffixen van de tekst – wat op het eerste gezicht geen goed idee lijkt als je een sneller en bovendien lineair algoritme wilt hebben. Pas laat zullen wij zien dat deze strings in een zekere volgorde wel bijzonder gemakkelijk toegevoegd kunnen worden en vaak zelfs meerdere strings tegelijk in één stap...

Wij zullen de suffix-boom opbouwen door met de boom voor alleen maar het eerste letterteken te beginnen en altijd één letterteken meer toe te voegen. Als

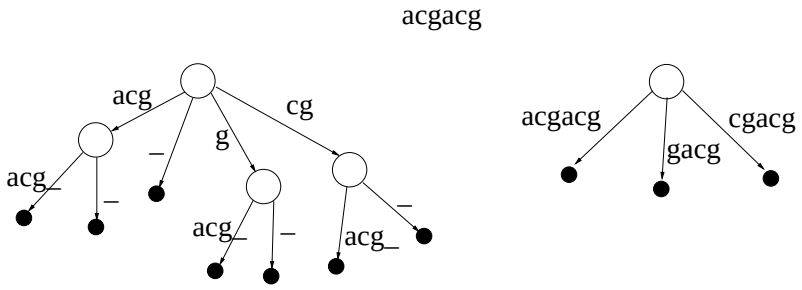
de string $s_1, s_2, \ldots, s_n, s_{n+1}$ is (waarbij $s_{n+1} = _$), schrijven wij $S(1,i)$ voor de *"suffix"* boom van de prefix $s_1, \ldots, s_i$. De bomen $S(1,i)$ kunnen natuurlijk geen echte suffix bomen zijn: wij hadden het teken _ toegevoegd om ervoor te zorgen, dat geen suffix een prefix van een andere suffix is – maar dat teken staat er pas op het einde. Wij zouden natuurlijk eerst de suffix boom voor $s_1, _$, dan die voor $s_1, s_2, _$, etc. kunnen opbouwen, maar het blijkt dat het beter is dat niet te doen en het expliciet toe te laten dat een string een prefix van een andere is. Dat noemen wij hier een deelstring boom:

Definitie 3 *Een string boom is een wortelboom waar elke boog e met een string $s_{e,0}, \ldots, s_{e,n(e)}$ gelabeld is. Elke interne top (dat betekent: die niet de wortel en geen blad is) heeft ten minste twee kinderen en de eerste lettertekens van de labels van de bogen naar die kinderen zijn paarsgewijs verschillend.*
Een pad dat bij de wortel begint en bogen $e_1, e_2, \ldots, e_k$ bevat, beschrijft de string $s_{e_1,0}, \ldots, s_{e_1,n(e_1)}, s_{e_2,0}, \ldots, s_{e_2,n(e_2)}, \ldots, s_{e_k,0}, \ldots, s_{e_k,n(e_k)}$. Elke prefix van een in de boom beschreven string wordt als in de boom aanwezig beschouwd.
Een deelstring boom voor een string $s = s_1, s_2, \ldots, s_n$ is een stringboom, waar alle deelstrings van s – en geen andere strings – aanwezig zijn. In deelstring bomen worden de labels van een boog e door twee getallen $1 \leq s(e) \leq f(e) \leq n$ voorgesteld die de eerste en de laatste positie van een voorkomen van de string in s beschrijven.

Het is niet onmiddelijk duidelijk, dat er voor elke string s een deelstring boom bestaat. Het zou *in principe* kunnen gebeuren dat als je ervoor zorgt dat alle deelstrings bevat zijn, automatisch ook strings bevat zijn die geen deelstring van s zijn. De constructie die wij nu zullen beschrijven, zal aantonen dat er voor elke string een deelstring boom bestaat.
Uit de definitie volgt onmiddelijk dat in een deelstring boom alle paden naar bladeren suffixen beschrijven: als een pad naar een blad een deelstring $s_i, \ldots, s_k$ met $k < n$ zou beschrijven, dan zou $s_i, \ldots, s_{k+1}$ niet in de boom zitten. Omgekeerd geldt dat niet noodzakelijk: een suffix wordt niet altijd door een pad naar een blad beschreven, maar kan daarvan een prefix zijn die zelfs in het midden van de label van een boog kan eindigen.
Als s een string is waar geen suffix een prefix van een andere deelstring is – bv. omdat *"toevallig"* het laatste teken uniek is – zoals onze afsluitende "_" – dan is onze deelstring boom inderdaad een suffix boom: alle suffixen zitten in de boom (ze zijn ten slotte deelstrings) en ze worden door een pad naar een blad beschreven (anders zou de suffix een prefix van een langere deelstring zijn).

Oefening 21 *Bewijs: Gegeven een string s met een uniek teken als afsluiting. Dan is de suffix boom van s ook de deelstring boom van s. Daarbij gaat het alleen om de datastructuur die je opbouwt – de interpretatie welke strings je als aanwezig in de boom beschouwt, is natuurlijk wel verschillend.*

Wij bouwen dus de deelstring bomen $S(1,1)$, $S(1,2)$,...,$S(1,n)$ in deze volgorde op en daarna de deelstring boom $S(1,n+1)$ – waarbij het $(n+1)$-de teken "_" is. $S(1,n+1)$ is dan niet alleen een deelstring boom, maar ook een echte suffix boom. In de afbeeldingen – en alleen daar – zullen wij altijd de strings aan de bogen schrijven omdat dan beter te zien is wat er gebeurt, maar in de implementatie zijn de bogen natuurlijk gelabeld met de pointers naar de posities in de string.



Figuur 20: Een voorbeeld waar een suffix boom (met afsluitend "_") en een deelstring boom (zonder afsluitend "_") sterk verschillen. Om gemakkelijker gelezen te kunnen worden, staan er wel de strings in plaats van de indices – bv. "acg" is dus $0, 2$ of $3, 5$.

Het uitbreiden van een boom $S(1, i)$ naar een boom $S(1, i+1)$ noemen we een iteratie – de iteratie $i+1$. In het midden van een iteratie bevat onze boom niet noodzakelijk alle deelstrings – het is dus maar een string boom en wordt pas op het einde van de iteratie terug een deelstring boom. Een suffix in iteratie i (dus een suffix van $s_1, s_2, \ldots, s_i$) die bij j begint (dus de deelstring $s_j, \ldots, s_i$) noemen wij $\underline{\sigma_{j,i}}$. In de iteratie $i+1$ moet je er dus voor zorgen dat er alle suffixen $\sigma_{j,i+1}$ van $\underline{s_1}, s_2, \ldots, s_{i+1}$ aanwezig zijn. Alle al aanwezige suffixen van $s_1, s_2, \ldots, s_i$ moeten dus met het teken s_{i+1} verlengd worden en je moet ervoor zorgen dat de suffix die alleen uit s_{i+1} bestaat ook aanwezig is. Wij zullen eerst $\sigma_{1,i+1}$ toevoegen, dan $\sigma_{2,i+1}$, etc. en ten slotte $\sigma_{i+1,i+1}$. De boom $S(1, i)$ uitgebreid met de suffixen $\sigma_{1,i+1}, \ldots, \sigma_{j,i+1}$ noemen wij $\underline{S(1, i)_j}$. Natuurlijk is $S(1, i)_j$ niet noodzakelijk een deelstring boom als $j < i + \overline{1}$, maar alleen een stringboom.
De suffixen die we in de laatste iteratie $n+1$ toevoegen, zijn net de suffixen die de naïeve manier van doen onmiddellijk zou toevoegen.
Een belangrijk inzicht is hier dat de suffixen die bij bladeren horen niet over de hele rij $\sigma_{1,i}, \ldots, \sigma_{i,i}$ verdeeld zijn, maar dat die allemaal in het begin staan – na iteratie i bv. tot voor een positie $b(i)$ – en vanaf deze positie $b(i)$ komen alleen nog suffixen die niet bij bladeren horen.

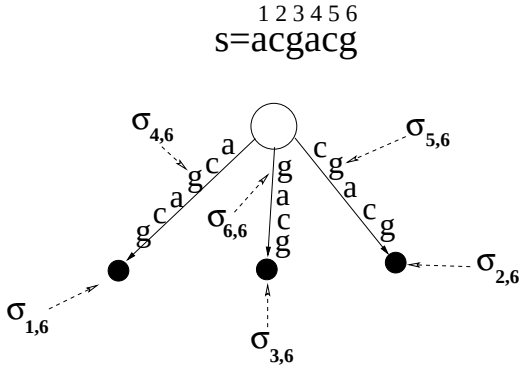
Lemma 4 *Als $S(1, i)$ een deelstring boom is voor de string $s_1, s_2, \ldots, s_i$ dan bestaat er een $1 \leq b \leq i+1$ zodat $\sigma_{1,i}, \ldots, \sigma_{b-1,i}$ in bladeren van $S(1, i)$ eindigen en $\sigma_{b,i}, \ldots, \sigma_{i,i}$ in interne toppen of in het midden van een label van een boog eindigen. Wij schrijven voor deze index b die afhangt van i ook $\underline{b(i)}$.*

Hierbij betekent *in een blad eindigen*, dat het blad precies deze string voorstelt.

Bewijs: De suffix $\sigma_{1,i}$ is de langste suffix en eindigt dus zeker in een blad. Als alle suffixen in bladeren eindigen, klopt het lemma zeker. Stel dus dat er een suffix is die niet in een blad eindigt en dat $\sigma_{b,i} = s_b, \ldots, s_i$ van deze suffixen die met de kleinste eerste index is. Stel bovendien dat $s_b, \ldots, s_i, u_1, \ldots, u_m$ door een blad l_b voorgesteld wordt dat vanuit het einde van $\sigma_{b,i}$ bereikt kan worden. Omdat l_b een blad in $S(1, i)$ is, is $s_b, \ldots, s_i, u_1, \ldots, u_m$ een suffix van $s_1, \ldots, s_i$. Maar dan is voor elke $b' \geq b$ ook $s_{b'}, \ldots, s_i, u_1, \ldots, u_m$ een suffix van $s_1, \ldots, s_i$ en al in de boom aanwezig – $s_{b'}, \ldots, s_i$ kan dus niet naar een blad gaan.

■

Een voorbeeld van een deelstring boom en waar de posities van de verschillende suffixen in de boom zijn, zien jullie in afbeelding 21



Figuur 21: Een voorbeeld van een deelstring boom waarin de posities van de verschillende suffixes getoond zijn.

De rij $\sigma_{1,i}, \ldots, \sigma_{i,i}$ heeft zelfs nog meer structuur! Wij zullen zien dat twee soorten suffixen die vanaf $b(i)$ in de rij staan, mooi van elkaar gescheiden zijn. Daarbij is het nuttig om te zien dat als een suffix $\sigma_{j+1,i+1}$ in $S(1,i)_j$ zit, die inderdaad al in $S(1,i)$ zat (en natuurlijk omgekeerd): in iteratie $i+1$ worden de aanwezige strings maar met één letterteken verlengd. De nieuwe strings zijn dus alleen in deze iteratie al toegevoegde strings (en geen strings die impliciet mee toegevoegd werden omdat je een aanwezige string met meer dan één letterteken verlengt) – maar die in iteratie $i+1$ al aan $S(1,i)$ toegevoegde strings zijn allemaal langer dan $\sigma_{j+1,i+1}$, dus moet die al in $S(1,i)$ gezeten hebben.

Lemma 5 *Gegeven een string $s_1, s_2, \ldots, s_n$ en $i < n$. Als $S(1,i)$ een deelstring boom is voor de prefix $s_1, s_2, \ldots, s_i$ dan bestaat er een $b(i) \leq b' \leq i+2$ met de volgende eigenschap:*
Voor $k < b'$ geldt dat $\sigma_{k,i+1}$ nog niet bevat is in $S(1,i)_{k-1}$ (en equivalent $S(1,i)$) en voor $i+1 \geq k \geq b'$ dat $\sigma_{k,i+1}$ al bevat is in $S(1,i)_{k-1}$ en $S(1,i)$.
Wij schrijven voor deze index b' die afhangt van i ook b'(i).

Bewijs: Voor $k < b(i)$ is $\sigma_{k,i+1}$ zeker nog niet bevat in $S(1,i)$ – en dus ook niet in $S(1,i)_{k-1}$ – omdat $\sigma_{k,i}$ naar een blad gaat.

Als voor geen k de suffix $\sigma_{k,i+1}$ al bevat is in $S(1,i)_{k-1}$ geldt het lemma zeker met $b' = i+2$. Veronderstel dus dat de suffix $\sigma_{b',i+1}$ die met de kleinste eerste index is die al bevat is in $S(1,i)_{b'-1}$ en dus $S(1,i)$. Omdat $S(1,i)$ een deelstring boom is, zitten dus ook alle $\sigma_{k,i+1}$ met $k \geq b'$ al in $S(1,i)$.

■

De situatie voor $\sigma_{1,i+1}, \sigma_{2,i+1}, \ldots \sigma_{i+1,i+1}$ kan dus als volgt geschetst worden:

$$\underbrace{\sigma_{1,i+1}, \ldots, \sigma_{b-1,i+1},}_{\text{toevoegen aan blad}} \quad \underbrace{\sigma_{b,i+1}, \ldots, \sigma_{b'-1,i+1},}_{\text{toevoegen op plaats die geen blad is}} \quad \underbrace{\sigma_{b',i+1}, \ldots, \sigma_{i+1,i+1}}_{\text{zit al in de boom}}$$

Oefening 22 *Geef een voorbeeld van een string $s_1, s_2, \ldots, s_n$ (n kan je vast kiezen) zodat bij het uitbreiden van $S(1, n-1)$ naar $S(1, n)$ alle drie soorten suffixen van de schets aanwezig zijn.*

Als wij ernaar kijken hoeveel toevoegoperaties in één iteratie kunnen gebeuren, dan kunnen dat jammer genoeg $\Theta(n)$ zijn bij een tekst van lengte n. Zelfs als wij de toevoegingen in bladeren niet meetellen, waarvan wij zullen zien dat wij die heel gemakkelijk kunnen oplossen, kunnen er nog altijd in een enkele iteratie $\Theta(n)$ operaties zijn, waar wij nieuwe toppen moeten aanmaken. Dat zou dan voor alle iteraties samen alleen maar een grens van $O(n^2)$ operaties opleveren. Wij tellen dus beter op een andere manier, die jullie zeker aan de aggregaatmethode uit *Algoritmen en datastructuren 2* doet denken.

Oefening 23 *Geef een voorbeeldtekst van lengte n, waar er bij het toepassen van het Ukkonen algoritme in één iteratie $\Theta(n)$ nieuwe toppen worden aangemaakt.*

Stel nu dat $s_j, \ldots, s_i, s_{i+1}$ een suffix is die wij willen toevoegen. Wij zoeken dus $s_j, \ldots, s_i$ en willen er s_{i+1} aan toevoegen. Er zijn 5 mogelijkheden hoe $s_j, \ldots, s_i$ in de boom $S(1, i)$ kan eindigen – drie mogelijkheden (i),(ii),(iii) waar wij de boom echt moeten uitbreiden en twee mogelijkheden (a),(b) waar de suffix er al inzit:

(i): De suffix $s_j, \ldots, s_i$ eindigt in een blad. Dan moet voor de laatste boog alleen de eindpositie van de string waarmee de boog gelabeld is van i naar $i+1$ aangepast worden.

(a): De suffix $s_j, \ldots, s_i$ eindigt in een interne top, maar er is een tak die met s_{i+1} begint. Dan hoeven wij niets te doen: de string zit er al in.

(ii): De suffix $s_j, \ldots, s_i$ eindigt in een interne top, maar er is geen tak die met s_{i+1} begint. Dan voegen wij een nieuwe tak naar een nieuw blad toe die wij met s_{i+1} labelen – of precies: die $i+1$ als begin- en eind-label krijgt.

(b): De suffix $s_j, \ldots, s_i$ eindigt in het midden van een label van een boog en het volgende teken van dat label is s_{i+1}. Dan hoeven wij – net zoals in (a) – niets te doen.

(iii): De suffix $s_j, \ldots, s_i$ eindigt in het midden van een label van een boog, maar het volgende teken van die label is niet s_{i+1}. Dan moeten wij deze boog splitsen in een deel dat met s_i in een nieuwe top eindigt van waar er twee takken vertrekken: de ene tak is een boog naar een nieuw blad met $i+1$ als begin- en eind-label en de andere bevat de rest van de string van het oorspronkelijke label.

Wij zullen nu eerst kijken hoe vaak wij deze uitbreidingsstappen tegenkomen en hoeveel tijd wij eraan besteden voordat wij erover nadenken hoe wij zo snel mogelijk de plaatsen vinden waar onze $\sigma_{.,i}$ in $S(1, i)$ eindigen.

(i): De situatie (i) is speciaal: als wij in iteratie $i+1$ een blad aanmaken, dat bij een suffix $\sigma_{j,i+1}$ behoort (dat gebeurt in de situaties (ii) en (iii)), dan weten wij al dat in iteratie $i+2$ de suffix $\sigma_{j,i+1}$ daar zal eindigen en dat

wij de eindpointer naar $i+2$ moeten aanpassen – en in iteratie $i+3$ naar $i+3$, enzovoort. Deze stappen kunnen dus allemaal samengevat worden: als wij een blad aanmaken, dan gebruiken wij als label dat het einde van de string aangeeft een pointer naar een variabele *it* (voor iteratie). In elke iteratie wordt deze variabele met 1 verhoogd – zo wordt de waarde voor alle bladeren tegelijk in maar één stap aangepast. Dat heeft dus kost $n+1$ voor alle $n+1$ iteraties samen. Omdat de bladeren allemaal in het begin van $\sigma_{1,i}, \ldots, \sigma_{i,i}$ staan, moeten wij dus – als wij deze lijst gemakkelijk kunnen overlopen en het eerste element $\sigma_{b(i),i}$, dat niet in een blad eindigt, kennen – de bladeren gewoon overslaan.

(a),(b): Stel dat wij het geval (a) of (b) tegenkomen – bv. bij het toevoegen van $\sigma_{j,i+1}$ Dat betekent dat de string $\sigma_{j,i+1}$ al in $S(1,i)$ zit – en dus (volgens Lemma 5) ook alle nog toe te voegen strings $\sigma_{j+1,i+1}, \ldots, \sigma_{i+1,i+1}$. Als wij in een iteratie dus de eerste keer (a) of (b) tegenkomen, kunnen wij de hele iteratie stoppen: alles wat wij nog willen toevoegen zit er al in! In elke iteratie is er dus ten hoogste één keer geval (a) of (b) – in alle iteraties samen is dat dus ten hoogste $n+1$ keer.

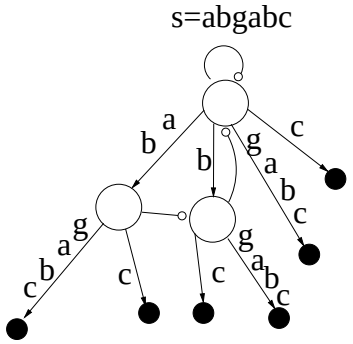
(ii),(iii): In de situaties (ii) en (iii) wordt een nieuwe tak naar een blad aangemaakt. Als wij met $\sigma_{b(i),i}$ beginnen, en in deze situatie terechtkomen, dan eindigt $\sigma_{b(i),i+1}$ dus in een blad – en wij kunnen het in toekomst overslaan, dus b verhogen. Omdat wij in het geval (a) en (b) onmiddelijk stoppen, is elke suffix $\sigma_{j,i}$ die wij opzoeken dus de kleinste die niet in een blad eindigt. Wij bepalen dus $b(i+1)$ gewoon als de eerste index waar wij (a) of (b) tegenkomen (en stoppen) of als wij dat nooit tegenkomen $b(i+1) = i+2$. Omdat wij al weten dat in de hele boom op het einde $n+1$ bladeren zijn, komen wij in deze situaties in alle iteraties samen maar n keer terecht (de eerste iteratie niet meegerekend) – ook dat vraagt dus tijd $O(n)$.

Als wij nu tellen, hoe vaak we in al de situaties samen terechtkomen, dan is dat $O(n)$:
In situaties (a) en (b) samen komen we ten hoogste $n+1$ keer terecht – ten hoogste één keer per iteratie. In situatie (i) komen we nooit terecht: zodra een prefix een blad is, wordt die overgeslaan en alleen *impliciet* uitgebreid door de variabele *it* voor alle bladeren tegelijk te verhogen. De situaties (ii) en (iii) maken een suffix die op een positie j begint tot een suffix die in een blad eindigt – ook dat kan alleen maar één keer voor elke positie, dus $n+1$ keer voor (ii) en (iii) samen. Omdat elke bewerking die in deze situaties moet gebeuren een constante kost heeft, is de som van alle kosten om de bewerkingen te doen dus $O(n)$.

Oefening 24 *Bouw de suffix boom voor* `aabbabc_` *op door de strings in de volgorde van het algoritme toe te voegen – maar nog zonder erop te letten dat je de plaatsen waar de strings toegevoegd worden snel kan vinden.*

Het grote probleem is nu nog onze veronderstelling te rechtvaardigen dat wij gewoon bij een suffix $\sigma_{b(i),i}$ kunnen starten en met lineaire kost voor alle stappen in alle iteraties **samen** van de ene suffix naar de volgende kunnen gaan. Om

dat te verwezenlijken werken wij met suffix links. Een suffix link is een pointer van een interne top t van de boom die de string $s_i, s_{i+1}, \ldots, s_j$ voorstelt naar een andere top $l(t)$ die de string $s_{i+1}, \ldots, s_j$ voorstelt. De string $s_{i+1}, \ldots, s_j$ kan ook leeg zijn en omdat de wortel de lege string voorstelt, gaat de pointer dan naar de wortel. Wij definieren ook nog dat de wortel een suffix link naar zichzelf heeft.



Figuur 22: Een voorbeeld van een deelstring boom met suffix links. De suffix links zijn de pijlen met een bolletje op het einde.

Opmerking 6 *Als t een interne top van een deelstring boom voor een string s is en t stelt de string $s_i, s_{i+1}, \ldots, s_j$ voor, dan is er ook een top die $s_{i+1}, \ldots, s_j$ voorstelt.*

Dat ligt niet voor de hand omdat sommige strings in het midden van booglabels eindigen, maar als t een interne top is, dan betekent dat, dat er ten minste twee lettertekens $a \neq b$ zijn, zodat $s_i, s_{i+1}, \ldots, s_j, a$ en $s_i, s_{i+1}, \ldots, s_j, b$ deelstrings van s zijn, dus ook $s_{i+1}, \ldots, s_j, a$ en $s_{i+1}, \ldots, s_j, b$ – de string $s_{i+1}, \ldots, s_j$ kan dus niet in het midden van een boog eindigen, omdat na het laatste teken een vertakking moet zijn.
Dit geldt dus voor deelstringbomen, dus op het einde van een iteratie. Als wij nieuwe interne toppen toevoegen, dan zullen we de suffix links in de volgende stap zetten. Dan gaan we naar de top die de suffix voorstelt of maken die aan. Deze opmerking heeft nog een belangrijk gevolg: stel dat je een pad met toppen $t_0, t_1, \ldots, t_n$ naar een interne top $t_n = t$ hebt. Dan zijn $t_1, \ldots, t_{n-1}$ ook interne toppen en er zijn dus links naar $l(t_1), \ldots, l(t_{n-1})$. Daarbij stellen $l(t_1), \ldots, l(t_{n-1})$ allemaal prefixen van de door $l(t)$ voorgestelde string voor en liggen dus op het pad van de wortel naar $l(t)$. Dat geeft een benedengrens voor de lengte van het pad van de wortel naar $l(t)$! Als wij nu voor een top t in de deelstring boom $a(t)$ als de afstand van de wortel definieren, dan krijgen wij dus

Opmerking 7 *Voor een interne top t in een deelstring boom geldt*
$a(l(t)) \geq a(t) - 1$

De positie van de plaats in de boom die de string $\sigma_{j,i}$ voorstelt die wij bewerken, kunnen wij bv. coderen als een 2-tal (p, x), waarbij p (voorgesteld door een pointer) naar een interne top wijst en x een getal is dat zegt hoeveel tekens je nog verder moet gaan vanuit deze interne top. Dit 2-tal noemen wij de *wijzer*

naar de positie in de boom. Een *geldige* wijzer is gemakkelijk om te vinden: laat p naar de wortel wijzen en kies x als de lengte van $\sigma_{j,i}$ (dus $i+1-j$). Maar wij willen natuurlijk een wijzer waarmee wij zo efficiënt mogelijk kunnen werken. Als de string niet door een blad wordt voorgesteld, kiezen wij de pointer p altijd naar de laatste top op het pad naar het eindpunt van de string in de boom. Voor een blad wijst de pointer naar de laatste top voor dit blad. Dan beschrijft het getal x hoeveel lettertekens je vanuit p nog van de uitgaande boog moet nemen. Omdat we met een eindig alfabet werken, kan je op deze manier in constante tijd de juiste boog vanuit p en ook de plaats vinden waar $\sigma_{j,i}$ eindigt. Tussendoor zullen wij wel ook (p, x) hebben, waar p nog niet naar de laatste top voor de positie wijst – dan moeten wij die zoeken en (p, x) wijzigen.
Stel dat (p, x) de positie van $\sigma_{j,i}$ voorstelt. Die string gebruik je om $\sigma_{j,i+1}$ toe te voegen. Wij schrijven altijd p_w voor de pointer naar de wortel van de boom. Dan zijn er twee mogelijkheden, wat de volgende $\sigma_{.,.}$ is die je moet toevoegen: ofwel ga je door in dezelfde iteratie – dus de i blijft dezelfde – ofwel ben je klaar met deze iteratie en ga je naar de volgende (omdat de string er al inzat of omdat je $\sigma_{i,i}$ hebt bereikt).

Dezelfde iteratie: Je moet vanuit (p, x) het volgende 2-tal (p_v, x_v) vinden dat de positie van $\sigma_{j+1,i}$ voorstelt – dus i verandert niet. Eerst ga je naar $\bar{p} = l(p_j)$.

Als $p = p_w$ (en dus ook $l(p_j) = p_w$) zijn nog $\bar{x} = x - 1$ tekens over en kiezen wij $(\bar{p}, \bar{x})$ als volgende pointer. De pointer p stelde de lege string voor, het teken s_j dat wegvalt, zit dus in het door x beschreven gedeelte – in de string $s_{i-\bar{x}+1}, \ldots, s_i$. Omdat de suffix pointer bij de wortel geen teken verwijdert, gebeurt dat bij de $\bar{x}$. Wij schrijven $\bar{p}$ en $\bar{x}$ in plaats van p_v en x_v omdat op deze manier wel de juiste positie in de boom gevonden kan worden, maar $\bar{p}$ voldoet nog niet noodzakelijk aan onze eis dat de pointer echt naar de **laatste** top voor de positie in de boom gaat.

Als $p \neq p_w$ en dus een niet lege string voorstelt, ben je door naar $\bar{p} = l(p)$ te gaan al s_j kwijt. Dan stelt met $\bar{x} = x$ dus $(\bar{p}, \bar{x})$ de string $\sigma_{j+1,i}$ voor. Ook die voldoet (nog) niet noodzakelijk aan onze eis met de laatste top.

De volgende iteratie: Hier heb je twee mogelijkheden. Stel eerst dat je de iteratie helemaal tot het einde hebt afgewerkt en daadwerkelijk $\sigma_{i+1,i+1}$ (dat maar 1 teken is) hebt toegevoegd. Dat is dan een nieuw blad. Dan moet in de volgende iteratie alleen $\sigma_{i+2,i+2}$ toegevoegd worden (de anderen worden impliciet toegevoegd in de bladeren). De prefix is dus leeg en je vertrekt altijd van de lege string die door $(p_v, x_v) = (p_w, 0)$ voorgesteld wordt.

Als je de iteratie niet tot het einde moet afwerken omdat je vaststelt dat $\sigma_{j,i+1}$ al in de boom zit (dat kan ook $\sigma_{i+1,i+1}$ zijn), moet je dus de positie van $\sigma_{j,i+1}$ bepalen om op die positie nog s_{i+2} toe te voegen als die er niet al is. Omdat je eerste positie niet opschuift (de j), moet je dus niet de suffix link nemen, maar neem je gewoon $\bar{p} = p$. De x wordt wel $\bar{x} = x + 1$ omdat je één teken verder moet gaan – het teken s_{i+1}.

Vanuit deze $(\bar{p}, \bar{x})$ moet je nu het 2-tal (p_v, x_v) vinden dat de positie van de gezochte string (dus $\sigma_{j+1,i}$ of $\sigma_{j,i+1}$) voorstelt en wel aan onze eisen voldoet. De pointer p_v moet naar de laatste interne top wijzen op het pad naar de positie van de string die we zoeken. Als $\bar{x}$ kleiner is dan de lengte x_l van de string die het label is van de boog waarlangs je moet lopen of die naar een blad leidt, ben

je klaar. Anders laat je $\bar{p}$ naar de top wijzen, waar de boog naartoe gaat en vervang je $\bar{x}$ door $\bar{x} - x_l$ totdat je de laatste interne top voor de positie van de gezochte string bereikt hebt.
Hierbij is heel belangrijk, dat je altijd alleen naar het eerste teken van de label van de boog en naar de lengte van de label moet kijken om de juiste boog te vinden. De andere tekens hoef je niet te vergelijken – omdat je weet dat de string die je zoekt in de boom zit, kloppen die gegarandeerd! De kost om vanuit $(\bar{p}, \bar{x})$ uiteindelijk (p_v, x_v) te bepalen is dus niet direct afhankelijk van $\bar{x}$, maar van het aantal m van toppen dat je tijdens deze zoektocht moet overlopen. Omdat het alfabet eindig is en alle stappen in constante tijd kunnen gebeuren, is de kost dus – op een constante factor na – m.

Oefening 25 *Geef een voorbeeld van een situatie bij het opbouwen van een deelstring boom, waar je nadat je naar het einde van een suffix link bent gegaan nog ten minste twee toppen moet overlopen voordat je de juiste top voor de volgende suffix hebt gevonden.*

Ook hier kan de kost voor het opzoeken van een enkele volgende suffix niet door een constante begrensd worden, wij moeten dus opnieuw de geamortiseerde kost van de hele reeks van opzoekingen bepalen. Wij gebruiken opnieuw de aggregaatmethode.
Wat is nu de afstand van de toppen waarnaar de p_v wijzen van de wortel? Als wij in stap j precies m_j toppen op het pad hebben, dat wij overlopen, dan gaan wij langs $m_j - 1$ bogen. De eerste keer gaan wij misschien een stap terug naar $l(p)$, maar de volgende $m_j - 2$ bogen gaan wij altijd naar een grotere diepte. Omdat $a(l(p)) \geq a(p) - 1$ geldt dus dat $a(p_{j+1}) \geq a(p) - 1 + (m_j - 2) = a(p) + m_j - 3$. Als de tekst lengte n heeft (dus $n + 1$ met het teken "_"), dan is de diepte van de boom begrensd door $n + 1$, dus $a(p_{n+1}) \leq n$, maar

$$a(p_{n+1}) \geq a(p_n) + m_n - 3 \geq a(p_{n-1}) + m_{n-1} + m_n - 2*3 \geq \cdots \geq a(p_1) + \sum_{j=1}^{n} m_j - 3n$$

Omdat $a(p_1) = 0$ krijgen wij:

$$n \geq a(p_{n+1}) \geq \sum_{j=1}^{n} m_j - 3n$$

en dus

$$\sum_{j=1}^{n} m_j \leq 4n$$

.
Maar $\sum_{j=1}^{n} m_j$ is de kost voor alle zoekstappen samen – en die is dus door een lineaire functie in n begrensd!

Het lijkt bijna alsof we nu klaar zijn, maar er zit nog een laatste addertje onder het gras: de suffix links zijn heel nuttig om te gebruiken, maar dat betekent ook dat wij tijdens het uitbreiden van de boom de suffix links moeten onderhouden. Al bestaande suffix links blijven natuurlijk geldig, maar als wij een nieuwe interne top aanmaken moet die ook een suffix link krijgen en wij moeten natuurlijk garanderen dat dat op een efficiënte manier kan gebeuren.

Een nieuwe interne top wordt alleen in situatie (iii) aangemaakt. Stel dat wij $S(1,i)$ naar $S(1,i+1)$ willen uitbreiden en $\sigma_{j,i+1}$ willen toevoegen. Die zat er dus in deze situatie (iii) nog niet in. Wij hebben dus de positie van $\sigma_{j,i} = s_j, \ldots, s_i$ gegeven door (p,x) en x is kleiner dan de lengte van de label van de boog die wij volgen. Wij willen voor de nieuwe interne top die $s_j, \ldots, s_i$ voorstelt dan de suffix link naar $s_{j+1}, \ldots, s_i$ leggen. Maar dat is precies de positie die wij in de volgende stap zoeken. Er zijn twee mogelijkheden:

- Als dat al een top is, dan vinden wij die in de volgende stap dus toch al en leggen dan de link.
- Als dat nog geen top is, dan maken wij die in de volgende stap aan en leggen dan de link: in situatie (iii) zat er al een string $s_j, \ldots, s_i, a$ met $a \neq s_{i+1}$ in de boom. Als die pas in deze iteratie werd toegevoegd, zou a het laatste teken geweest zijn – maar $a \neq s_{i+1}$. Dus zit die string al in $S(1,i)$, maar omdat dat een deelstring boom is, zit ook $s_{j+1}, \ldots, s_i, a$ in de boom. In de volgende stap wordt dus op de plaats $s_{j+1}, \ldots, s_i$ een nieuwe top – de gezochte eindtop van onze suffix link – toegevoegd.

Wij kunnen onze suffix links dus onderhouden met een extra kost die alleen maar een constante factor aan sommige al bestaande operaties toevoegt. Alle stappen samen geven dus een algoritme dat een suffix boom voor een string $s = s_1, \ldots, s_n, s_{n+1}$ (met $s_{n+1} =$ "_") in tijd $O(n)$ opbouwt.

Hoewel de klemtoon op suffix bomen ligt en de deelstring bomen die geen suffix bomen zijn alleen maar een tussenstap zijn, zullen wij later zien dat die ook nuttig kunnen zijn. Daarbij hebben wij gezien, dat wij de deelstring bomen heel efficiënt online kunnen opbouwen door altijd het laatste teken toe te voegen. In feite hebben we bij de argumenten voor de complexiteit nooit gebruikt dat op het einde een uniek teken is en dat het op het einde een suffix boom is. Dus:

Stelling 8 *Als s_1; s_1, s_2; $\ldots s_1, s_2, \ldots, s_n$ een rij van n strings is, dan kunnen de bijhorende deelstring bomen $T_1, T_2, \ldots, T_n$ in tijd $O(n)$ opgebouwd worden (voor alle bomen samen). De uitbreidstappen van T_i naar T_{i+1} hebben dus voor $1 \leq i < n$ geamortiseerd constante kost per uitbreiding.*

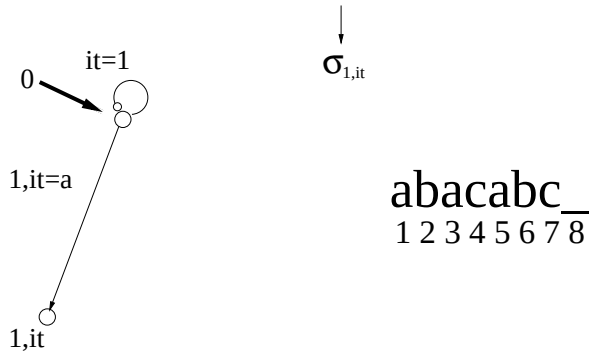
Oefening 26 *Bij het tellen van de toppen die we overlopen tijdens het zoeken van de volgende prefix van de toe te voegen suffix, hebben wij bewezen.*

$$\sum_{j=1}^{n} m_j \leq 4n$$

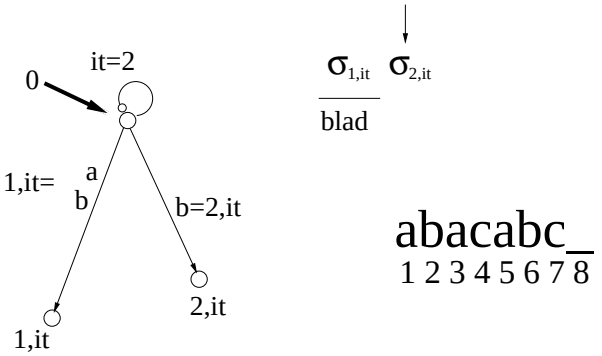
.
Kan je als je nu wel veronderstelt dat het laatste teken uniek is iets beters bewijzen?

Het is moeilijk een volledig voorbeeld met de hand uit te werken, omdat de bomen al heel groot zijn voordat er echt iets interessants gebeurt. Maar het helpt toch eens *"Ukkonen at work"* te zien. Wij bouwen de prefixboom voor de string `abacabc_` op. In de tekeningen labelen wij de bogen voor de betere leesbaarheid met de strings – maar er staan natuurlijk alleen de grenzen! Het is artificieel te zeggen wat tot het einde van de ene stap behoort en wat tot het

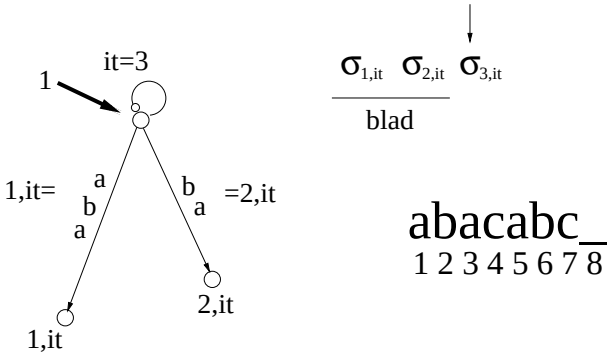
begin van de volgende. In de tekeningen toont de dikke pijl met het getal het 2-tal $(\bar{p}, \bar{x})$, dus de pointer **voordat** naar de wijzer (p_v, x_v) met de laatste top voor de positie van de prefix voor de volgende toevoeging wordt gezocht. De suffix links zijn opnieuw de pijlen met bolletjes op het einde. In het begin is het alleen maar één boog van de wortel naar één blad. De pijl naar de $\sigma_{.,.}$ toont met welke suffix wij bezig zijn.



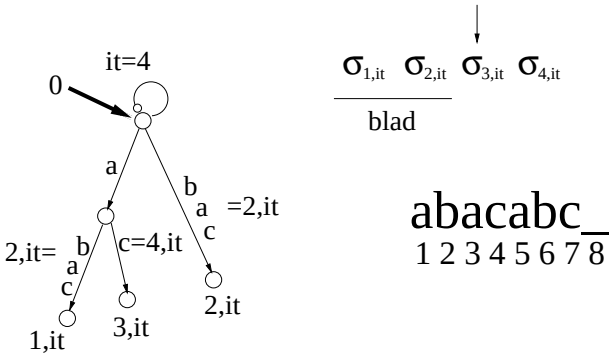
Als de eerste suffix de nieuwe suffix met maar één teken is, is de wijzer naar de prefix $(p_w, 0)$. Nu moeten alle suffixen van `ab` toegevoegd worden en omdat de iteraie volledig afgewerkt werd, beginnen wij de volgende toevoeging opnieuw met $(p_w, 0)$, wat door de dikke pijl met label 0 wordt aangeduid.



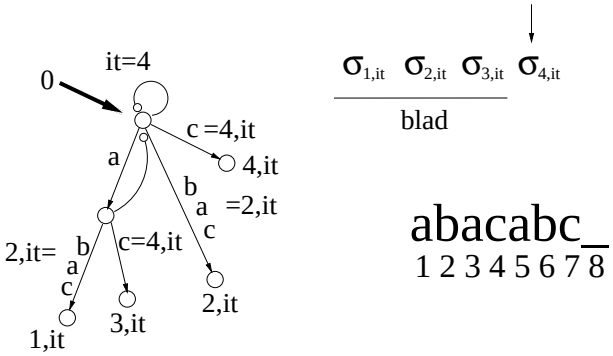
De suffix $\sigma_{1,it}$ werd impliciet geüpdatet door it te verhogen – de suffix $\sigma_{2,it}$ werd nieuw toegevoegd en wij beginnen dus opnieuw met $(p_w, 0)$. Nu moeten alle suffixen van `aba` toegevoegd worden:



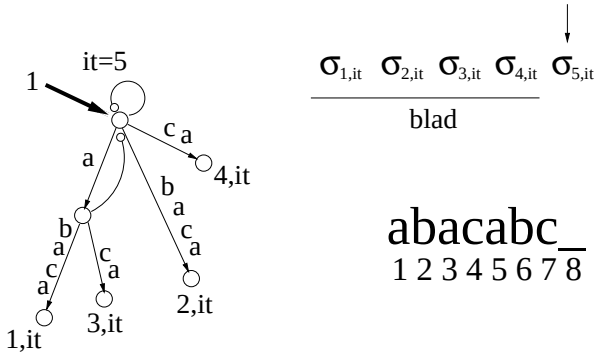
De suffixen $\sigma_{1,it}$ en $\sigma_{2,it}$ werden impliciet geüpdatet door it te verhogen. Suffix $\sigma_{3,it}$ zit al in de boom en voor de volgende stap verhogen wij de x in (p, x). Nu moeten alle suffixen van `abac` toegevoegd worden:



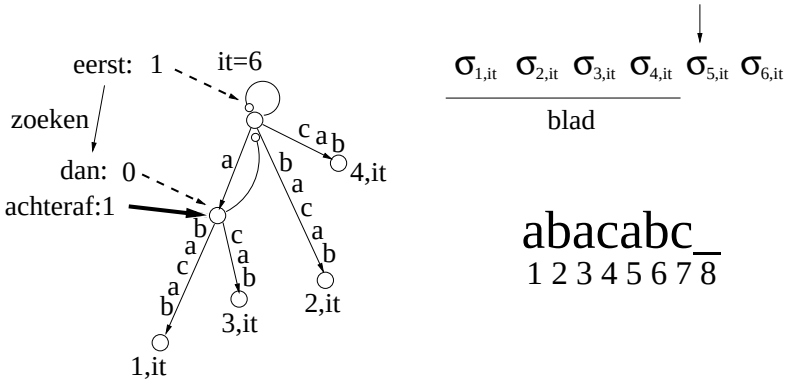
De suffix $\sigma_{3,it-1}$ – dus `a` eindigt in het midden van een boog. Hier wordt dus een nieuwe interne top aangemaakt en $\sigma_{3,it}$ eindigt in een blad. Omdat $p = p_w$ verlagen wij voor de volgende stap x.



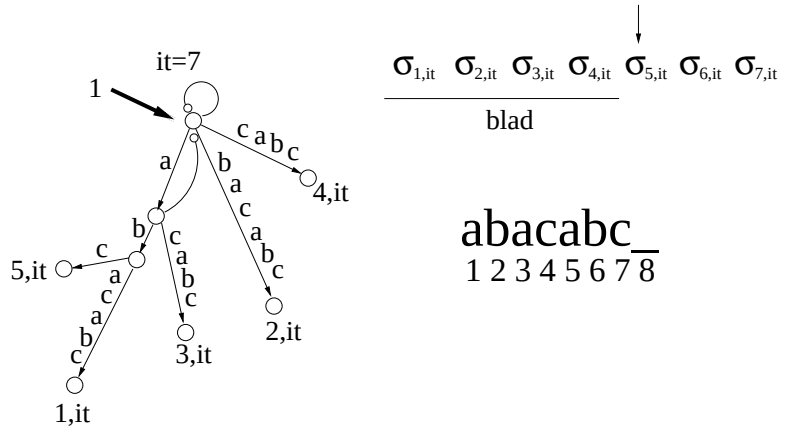
De suffix link voor de nieuwe interne top wordt toegevoegd en een boog vanuit de wortel voor de suffix $\sigma_{4,4}$ aangemaakt – die eindigt onmiddelijk in een blad. Onze wijzer voor de volgende stap is dus opnieuw $(p_w, 0)$.
Nu moeten alle suffixen van `abaca` toegevoegd worden. Om niet te veel tekst te hebben, schrijven wij ook voor bogen naar bladeren niet meer de grenzen i, it – maar natuurlijk is elke boog gelabeld met grenzen en wij schrijven de lettertekens alleen om het beter leesbaar te maken.



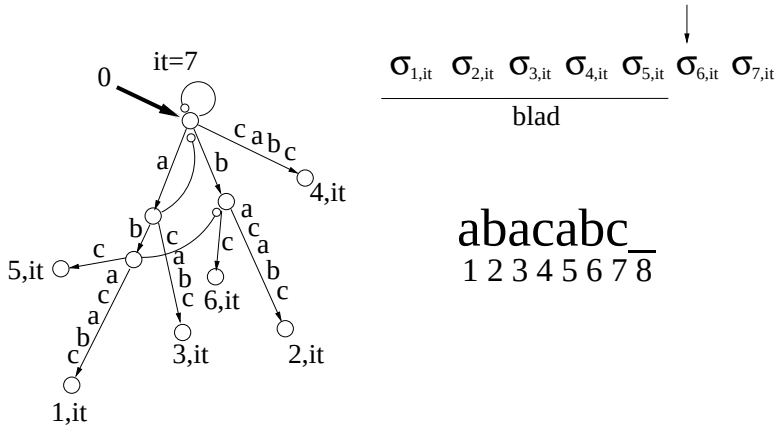
De eerste suffix $\sigma_{5,it}$ – dus `a` – die aangepast moet worden, zit er al in, wij kunnen dus onmiddelijk stoppen en x met 1 verhogen. De beginpointer werd wel aangepast.
Nu moeten alle suffixen van `abacab` toegevoegd worden:



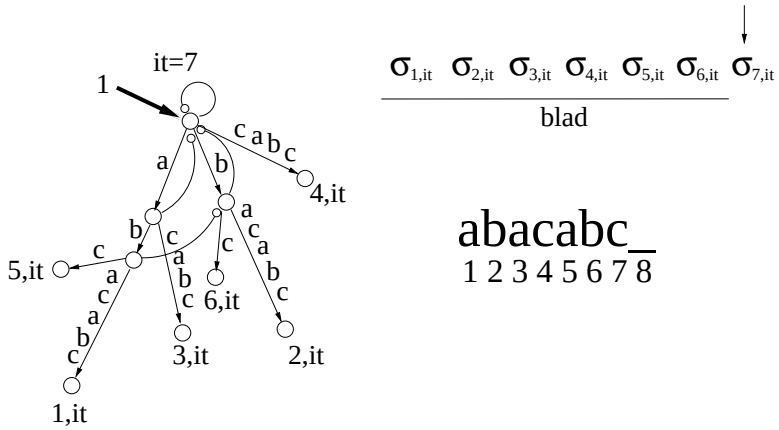
Hier hebben we de $(\bar{p}, \bar{x})$ eerst vervangen door de juiste positie te zoeken. Opnieuw zit de suffix $\sigma_{5,it}$ – dus nu `ab` – er al in, en wij kunnen onmiddelijk stoppen en x verhogen.
Nu moeten alle suffixen van `abacabc` toegevoegd worden:



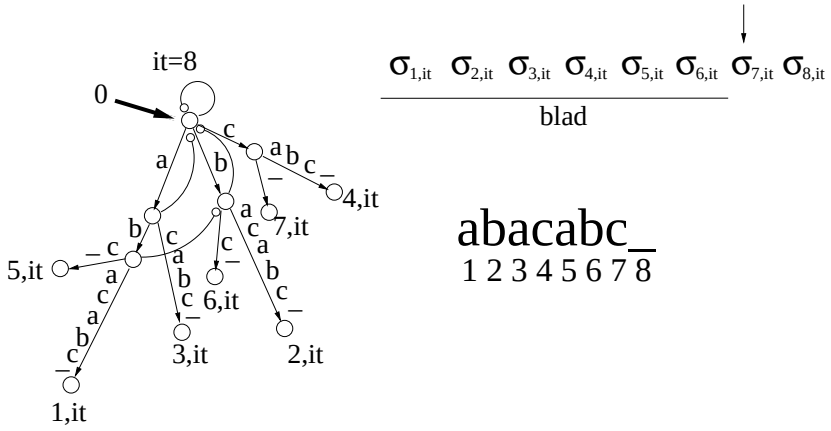
Nu zit $\sigma_{5,it}$ – dus in iteratie 7 `abc` – er niet in en wij moeten een nieuwe interne top aanmaken, waardoor $\sigma_{5,it}$ naar een blad gaat. Wij volgen de suffix link voor de volgende stap en $p \neq p_w$, wij verlagen x dus niet.



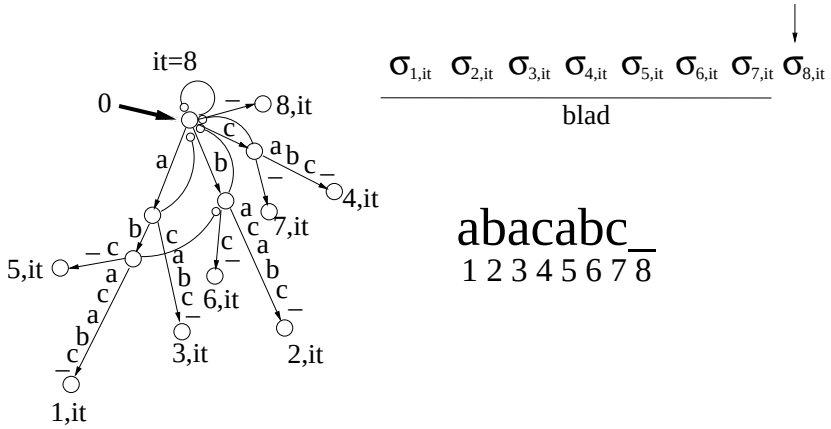
Ook voor $\sigma_{6,it}$ wordt een nieuwe top aangemaakt en de suffix link van de vorige nieuwe top gaat hier naartoe. Omdat $p = p_w$ verlagen wij x.



De suffix $\sigma_{7,it}$ zit er al in en wij moeten alleen nog de suffix link van de vorige nieuwe top leggen en x verhogen.
Uiteindelijk moeten alle suffixen van `abacabc_` toegevoegd worden:



Hier wordt dus eerst een nieuwe interne top aangemaakt, waarna $\sigma_{7,it}$ naar een blad gaat en wij x met 1 verlagen.



Dan wordt de suffix link vanuit de nieuwe top toegevoegd, een nieuw blad voor $\sigma_{8,it}$ aangemaakt en wij zijn klaar.

Oefening 27 *Gebruik het algoritme van Ukkonen om de suffix boom voor* `aabbabc_` *op te bouwen.*

3.3 Toepassingen van suffix bomen

Suffix bomen hebben bijzonder veel toepassingen. Hier zullen wij maar sommige kleine voorbeelden zien.
Stel dat je een string $z[]$ van lengte m in een tekst $t[]$ met lengte n zoekt. Dan kan je in tijd $O(n)$ een suffix boom voor $t[]$ opbouwen en dan in tijd $O(m)$ vanuit de wortel de tekens van $z[]$ volgen om te zien of de deelstring in de boom zit of niet. Wij kunnen dus in tijd $O(m+n)$ beslissen of $z[]$ in $t[]$ zit – of niet. Als je heel vaak in dezelfde tekst $t[]$ zoekt en de suffix boom is al opgebouwd, dan kan je een string $z[]$ van lengte m zelfs in tijd $O(m)$ zoeken – om het even hoe lang de tekst is!
De bogen in een deelstring boom hebben twee labels die de begin- en eind-positie van de label van de boog in de originele tekst beschrijven. Als de tekst die string meerdere keren bevat, kan dat in principe natuurlijk op meerdere manieren. Als de tekst bv. `aababbab` is dan zou voor de label `ab` zowel $2, 3$ (wij zijn altijd met positie 1 begonnen) als ook $4, 5$ of $7, 8$ in orde zijn. Maar bij het algoritme van Ukkonen is het helemaal niet toevallig welke posities gekozen worden. Als een nieuwe boog aangemaakt wordt, die aan een top t vertrekt die een string s_t van lengte l_t voorstelt, dan krijgt die altijd een beginlabel b zodat op positie $b - l_t$ een deelstring begint, die met s_t begint en dan met de lettertekens van de nieuwe boog doorgaat. Dit beginlabel wordt dan nooit meer gewijzigd – de boog wordt ten hoogste later onderverdeeld. Dat betekent dat wij als wij een string zoeken niet alleen kunnen bepalen **of** die in de tekst zit, maar ook een positie kunnen geven **waar** wij die kunnen vinden: als wij de boog nemen waar het laatste teken van onze string in zit en t als de top waar die boog vertrekt, dan is met de notatie van boven $b - l_t$ de positie waar een kopie van de string in de tekst begint.

Oefening 28

- *Is de positie die je op de net beschreven manier met $b - l_t$ krijgt altijd de eerste positie van de string in de tekst?*
- *Stel dat de suffix boom voor een tekst $t[]$ van lengte n (die je kent) al opgebouwd is en je wilt niet alleen weten of een zoekstring $z[]$ van lengte m bevat is, maar je wilt* **alle** *startposities van kopieën van $z[]$ in $t[]$ hebben. Natuurlijk kan dat niet in tijd $O(m)$ – er kunnen ten slotte zeer veel meer dan maar $c * m$ startposities zijn die uitgevoerd moeten worden. Maar als er k startposities zijn, kan dat in tijd $O(m + k)$.*

  Geef een algoritme dat deze grens verwezenlijkt.

Oefening 29 *Gegeven een tekst $t[]$ van lengte n. Beschrijf een datastructuur die in tijd $O(n)$ opgebouwd kan worden en die het toelaat in tijd $O(m)$ te beslissen hoe vaak een gegeven string van lengte m in $t[]$ voorkomt.*

Soms wil je niet weten, of een string $z[]$ met lengte m in een zekere tekst $t[]$ bevat is, maar heb je een hele hoop van teksten $t_1[], \ldots, t_i[]$ (met lengte $n_1, \ldots, n_i$) en wil je weten of $z[]$ in een van deze teksten bevat is en als ja in welke. Een voorbeeld is als je een databank van DNA van veel mensen hebt en dan wordt ergens een stukje DNA gevonden en je wilt weten of die van één van de mensen is. Een ander voorbeeld is dat je een databank met veel boeken hebt en je wilt weten in welk boek een zeker citaat staat. Stel dat je met één boek waarin het staat als antwoord tevreden bent. Natuurlijk kan je dat doen door in elke tekst $t_j[]$ apart te zoeken – dat zou dan kost $O(i * m)$ hebben – maar wij zijn gretig: wij willen het in tijd $O(m + k)$ doen, waarbij k het aantal keren is dat $z[]$ in de teksten voorkomt.
Daarvoor gebruiken wij een veralgemeende suffix boom. Een veralgemeende suffix boom is een trie waarin de suffixen van niet alleen één , maar meerdere teksten samen opgeslaan zijn. Om nog te kunnen zien bij welke tekst een suffix behoort, werken wij met meerdere unieke afsluitingstekens $__1, __2, __3, \ldots$ (die je natuurlijk met jouw eindig alfabet kan simuleren, bv. als _1_, _2_, _3_, ...). Tekst $t_i[]$ wordt door $__i$ afgesloten.
Je kan ook met alleen maar 2 afsluitingstekens werken – één voor het einde van een tekst en één voor het einde van alle teksten. Dan zou je wel moeten bijhouden op welke positie van de aan elkaar geschakelde teksten elke tekst begint om te kunnen zien in welke tekst je iets hebt gevonden.
Het opbouwen van een veralgemeende suffix boom voor teksten $t_1[], t_2[], \ldots, t_k[]$ met lengtes $n_1, n_2, \ldots, n_k$ kan natuurlijk niet in tijd $o(n_1 + \cdots + n_k)$ omdat je de teksten ten minste moet lezen – maar je kan (met $n = n_1 + \cdots + n_k$) een boom met dezelfde functionaliteit als een veralgemeende suffix boom (een soort *onechte veralgemeende suffix boom*) zeker in tijd $O(n)$ opbouwen: schrijf de teksten gewoon achter elkaar en bereken de suffix boom voor de tekst $t_1[]t_2[] \ldots t_k[]$. In die tekst kan je dan gewoon zoeken of een string aanwezig is. In die boom zijn ook suffixen opgeslaan die in geen van de teksten $t_1[], t_2[], \ldots, t_k[]$ zitten – namelijk suffixen die een afsluitingsteken in het midden hebben – maar omdat wij nooit naar strings met een afsluitingsteken in het midden zoeken, zal dat niet tot foute matches leiden.

Oefening 30

- *Bewijs: als σ een suffix in een dergelijke* onechte veralgemeende suffix boom *is die een afsluitingsteken in het midden heeft, dan*

is het hele gedeelte van dit afsluitingsteken tot het einde deel van het label van één boog naar een blad.

- *Beschrijf hoe je het algoritme van Ukkonen moet wijzigen om de echte veralgemeende suffix boom op te bouwen. In deze suffix boom – waar je dus niet één maar meerdere teksten hebt – moeten de labels van de bogen natuurlijk naast de begin- en eind-positie ook nog de index van een tekst bevatten waarin je de deelstring kan vinden.*

  Als je ook in alleen maar één van de teksten wilt kunnen zoeken en dezelfde performantiegaranties als voor een boom voor een specifieke tekst wilt, moeten de bogen zelfs indicaties hebben van elke tekst waarin ze zitten. Anders kan je soms lang zoeken en merk je pas bij de bladeren dat de strings in alleen maar andere teksten zitten.

- *In de tekst staat dat wij in tijd $O(m+k)$ (met k het aantal keren dat $z[]$ in de teksten voorkomt) een tekst willen vinden waarin het staat. Werkt jouw manier misschien zelfs in tijd $O(m)$ (verondersteld dat elke boog maar de label van één tekst heeft)? Geef uitleg.*

Veel algoritmen werken ook met suffix arrays. Dat is een array waarin alle suffixen in lexicografische volgorde staan. Je kan de suffixen natuurlijk gewoon voorstellen door de beginpositie in de tekst, zodat het geheugengebruik niet alleen lineair in de lengte van de tekst is – dat geldt ook voor suffix bomen – maar de constante in de lineaire functie is ook relatief klein – je hebt één integer voor elk teken in de tekst nodig.

Oefening 31 *Geef een algoritme dat een suffix array gebruikmakend van een suffix boom in lineaire tijd opbouwt.*

In hoofdstuk 4 zullen wij nog meer toepassingen van suffix bomen en deelstring bomen zien.

3.4 Exact string matching

In dit hoofdstuk zoeken wij een string $z[]$ in een andere string $t[]$. Dit is dus de typische toepassing van zoekmachines of van een editor die een woord in een tekst zoekt. Wij hebben al gezien hoe je dat in lineaire tijd met suffixbomen kan doen, maar als studenten die al veel programma's hebben geschreven, kunnen jullie inschatten dat de constante in de $O()$ veel te groot is als je maar één keer in de tekst zoekt en dat het geheugenverbruik voor grote teksten ook een probleem zal zijn. Wij willen dus eenvoudige algoritmen die snel zijn in de praktijk, ook geschikt zijn voor streams, en niet te veel geheugen verbruiken.
Wij zullen de string die gezocht wordt altijd z noemen (z voor zoeken) en de lengte van deze string m (de string is dus $z[0]z[1]z[2]\ldots z[m-1]$). De string waarin gezocht wordt (vaak ook de tekst genoemd) heet altijd t (t voor tekst) en de lengte wordt als n genoteerd (deze string is dus $t[0]t[1]\ldots t[n-1]$).
Natuurlijk heeft iedereen onmiddellijk een idee hoe je een string kan zoeken:

Algoritme 2 *(Brute kracht)*

```
strings_gelijk(z,t,start)
// test of de eerste |z| tekens van t -- beginnend met
// positie start -- gelijk zijn aan die van z
// het wordt gesteld dat in t na start nog ten minste even
// veel tekens staan als in s en dat |z| gekend is
{
for (i=0; i<|z|; i++) if (z[i]!=t[start+i]) return FALSE;

return TRUE;
}

is_contained(z,t)
// test of de string z in de string t voorkomt
{
for (i=0; i<= |t|-|z|; i++)
  if strings_gelijk(z,t,i) return TRUE;

return FALSE;

}
```

Hier worden de strings vanaf $n-m+1$ startposities vergeleken en elke vergelijking vraagt ten hoogste m stappen, de totale complexiteit is dus $O((n-m+1)*m)$.

Opmerking 9 *Algoritme 2 toegepast op een tekst van lengte n en een zoekstring van lengte m vraagt in het slechtste geval* $O((n-m+1)*m)$ *stappen.*

Als wij m als constant beschouwen (normaal zal de zoekstring heel klein zijn in vergelijking met de tekst) is dat $O(n)$ en dus zeker niet slecht. Maar er zijn twee redenen om toch betere algoritmen te zoeken. Ten eerste is het wel een beetje gesjoemeld als wij m verwaarlozen – de zoekstring **kan** in principe ook $\Theta(n)$ zijn – bv. $n/2$ – en dan mag m natuurlijk niet als constant beschouwd worden en wordt de complexiteit $O(n^2)$. En ten tweede is voor praktische doeleinden een grote constante factor voor de complexiteit ook niet echt goed en vooral in DA3 ligt de klemtoon toch sterk op de praktische kant van algoritmen!

Oefening 32 *Als de binnenste lus van algoritme 2 echt m stappen nodig heeft, hebben wij het woord gevonden – dus zal het algoritme stoppen. En als de lus* bijna *m stappen doet, zouden wij verwachten dat misschien de volgende keren dat de functie wordt gebruikt al heel snel een tegenstrijdigheid gevonden wordt. Het is dus niet onmiddellijk duidelijk dat het algoritme echt soms* $\Theta(n^2)$ *stappen nodig heeft. Daarom moet dat aangetoond worden:*
Geef een geparametriseerde reeks van voorbeelden waarvoor dit algoritme $\Theta(n^2)$ *stappen nodig heeft. Natuurlijk moet je ook aantonen dat* $\Theta(n^2)$ *stappen nodig zijn.*

Oefening 33

- *Herschrijf Algoritme 2 zo dat de zoekstring ook één of meerdere wildcards* `?` *mag bevatten. Wij stellen dat* `?` *geen deel uitmaakt van*

de tekst en dat elk letterteken erdoor gematcht wordt. Voor `?uizen` *zou dus bv.* `huizen` *maar ook* `buizen` *een match in de tekst zijn.*

Wat is de complexiteit van jouw algoritme?

- *Geef een algoritme zodat de zoekstring ook één of meerdere wildcards* `*` *mag bevatten waarbij* `*` *alle deelstrings matcht – ook de lege string. Voor* `ver*uizen` *zouden dus bv.* `verhuizen` *en* `vergeet de muizen` *een match in de tekst zijn. Jouw algoritme moet een arbitraire string in de tekst teruggeven die de zoekstring matcht als die bestaat en anders de lege string.*

  De complexiteit van jouw algoritme mag ten hoogste de complexiteit van Algoritme 2 zijn.

In DA1 en DA2 werd gezegd dat één van de bedoelingen van deze lessen is om technieken en ideeën te zien die je achteraf in een andere context kan toepassen om snelle algoritmen te ontwikkelen. Het volgende algoritme is een voorbeeld daarvan.

Het algoritme van Rabin-Karp

Als wij de kost van de functie `strings_gelijk()` in Algoritme 2 constant zouden kunnen maken of de functie alleen maar in heel zeldzame gevallen zouden moeten toepassen, zou de factor m in de complexiteit verdwijnen en wij zouden een complexiteit van $O(n)$ hebben. Het algoritme van Rabin-Karp doet dat niet **gegarandeerd** – de complexiteit van het **slechtste geval** blijft dezelfde – maar in de praktijk is de kost van de met de functie `strings_gelijk()` corresponderende functie wel constant. Misschien doet jullie dat al aan een techniek uit DA1 denken waar het slechtste geval ook geen verbetering ten opzichte van de standaardtechnieken was, maar die in de praktijk wel heel nuttig is: hashing. En het idee van hashing – dat ook al voor Bloom-filters toegepast werd – is wat wij ook hier zullen gebruiken.
Voordat wij de zoekstring en een deelstring van de tekst vergelijken, vergelijken wij eerst de waarde van een hashfunctie van deze strings. Als die niet gelijk is, zijn de strings zeker verschillend en moeten wij ze ook niet expliciet vergelijken. Dat betekent wel dat wij misschien een hashfunctie moeten berekenen voor alle deelstrings van lengte m in de tekst t. Als deze berekeningen even duur of duurder zouden zijn dan het vergelijken van de twee strings zou het natuurlijk geen voordeel zijn. Het trucje is dus dat wij om de hashfunctie voor de string die op positie $i+1$ begint te berekenen de hashwaarde voor de string die op positie i begint gewoon updaten.
Wij zullen als voorbeeld een hashfunctie voor het alfabet $\Sigma = \{0, 1\}$ geven die al door Rabin en Karp werd gesuggereerd – wij werken dus met bitstrings. Maar het is duidelijk (zie oefeningen) dat het voor elk alfabet op een gelijkaardige manier gedaan kan worden.
Kies een natuurlijk getal q (bv. een priemgetal) en interpreteer een bitstring s (in ons geval kan dat ofwel $z[]$ ofwel een deelstring van $t[]$ zijn) gewoon als voorstelling van een getal $g(s)$. De hashwaarde $h(s)$ van de string is dan gewoon $h(s) = g(s) \mod q$. Als de string $s = s_0 s_1 s_2 \dots s_{m-1}$ is dat als formule

$$h(s) = (\sum_{i=0}^{m-1} (s_i 2^{(m-1)-i})) \mod q$$

De som ziet er misschien ingewikkeld uit, maar het is alleen maar het getal dat je krijgt als je de bitstring interpreteert als de voorstelling van een binair getal. Als wij dus de hashfunctie willen berekenen van de string van lengte m die op positie p in de tekst $t_0 t_1 \dots t_n$ begint, dan is dat

$$h(t_p \dots t_{p+m-1}) = (\sum_{i=0}^{m-1} (t_{p+i} 2^{(m-1)-i})) \mod q$$

Hierbij hebben wij de posities als index geschreven in plaats van als arrayindex (dus t_i in plaats van $t[i]$) omdat dat formules beter leesbaar maakt.
Als wij gebruiken dat
$(a + b) \mod q = (a + (b \mod q)) \mod q$
en
$(a * b) \mod q = (a * (b \mod q)) \mod q$
dan krijgen wij voor de string die op positie $p + 1$ begint:

$$\begin{aligned}
h(t_{p+1} \dots t_{p+m}) &= (\textstyle\sum_{i=0}^{m-1} (t_{p+1+i} 2^{(m-1)-i})) \mod q \\
&= (\textstyle\sum_{i=0}^{m-2} (t_{p+1+i} 2^{(m-1)-i}) + t_{p+m}) \mod q \\
&= (-t_p 2^m + \textstyle\sum_{i=-1}^{m-2} (t_{p+1+i} 2^{(m-1)-i}) + t_{p+m}) \mod q \\
&= (-t_p 2^m + \textstyle\sum_{i=0}^{m-1} (t_{p+i} 2^{m-i}) + t_{p+m}) \mod q \\
&= (2 \textstyle\sum_{i=0}^{m-1} (t_{p+i} 2^{(m-1)-i}) - t_p 2^m + t_{p+m}) \mod q \\
&= ((2 \textstyle\sum_{i=0}^{m-1} (t_{p+i} 2^{(m-1)-i}) \mod q) - t_p 2^m + t_{p+m}) \mod q \\
&= (2h(t_p \dots t_{p+m-1}) - t_p 2^m + t_{p+m}) \mod q
\end{aligned}$$

Deze berekening geeft ons een heel snelle en gemakkelijke manier om de hashwaarde $h(t_{p+1} \dots t_{p+m})$ in *constante tijd* te berekenen als wij de hashwaarde $h(t_p \dots t_{p+m-1})$ al kennen. Alleen voor het berekenen van de hashwaarde van de zoekstring $z[]$ en van $h(t_0 \dots t_{m-1})$ hebben wij dus tijd $O(m)$ nodig – alle andere hashwaarden die wij nodig hebben, kunnen in constante tijd berekend worden.
Maar er lijkt wel een addertje onder het gras: wij veronderstellen hier dat wij $t_p 2^m$ of ten minste $t_p 2^m \mod q$ in constante tijd kunnen berekenen, wat voor zeer grote m zeker niet klopt. Maar $t_p 2^m \mod q$ heeft maar twee mogelijke waarden – afhankelijk ervan of $t_p = 0$ waarvoor het zeker 0 is of $t_p = 1$. De waarde voor $t_p = 1$ kunnen wij op voorhand één keer berekenen (bij zeer grote m bv. met de square en multiply methode).
De volgende pseudocode beschrijft het algoritme van Rabin-Karp

Algoritme 3 *(Rabin-Karp)*

```
strings_gelijk(z,t,start)
// test of de eerste |z| tekens van t -- beginnend met positie
// start -- gelijk zijn aan die van z
// het wordt gesteld dat in t na start nog ten minste even veel
// tekens staan als in z en dat |z| gekend is
{
for (i=0; i<|z|; i++) if (z[i]!=t[start+i]) return FALSE;
```

```
return TRUE;
}

is_contained_Rabin_Karp(z,t)
// test of de string s in de string t voorkomt
{
bereken h(z)

for (i=0; i<=|t|-|z|; i++)
{ bereken h(t_i...t_{i+m-1})
  // als i>0 kan dat gewoon door de oude waarde van h()
  // te updaten
  if (h(t_i...t_{i+m-1})==h(z))
    { if strings_gelijk(z,t,i) return TRUE; }
}
return FALSE;

}
```

De extra tijd in vergelijking met Algoritme 2 voor het berekenen van de hashfuncties is $O(m)$ voor het berekenen van $h(z)$, $O(m)$ voor het berekenen van $h(t_0...t_{m-1})$ en dan nog $O(n-m)$ voor het updaten van $h()$ – samen dus $O(n)$. De functie `strings_gelijk()` vraagt nog altijd tijd $O(m)$, in het slechtste geval is de complexiteit dus nog altijd $O((n-m+1)*m)$. Maar als wij ervan uitgaan dat de hashfunctie goed is en dus de waarden van de deelstrings ongeveer gelijk zijn verdeeld over de q mogelijkheden, dan wordt de functie maar in $(n-m+1)/q$ gevallen toegepast. De totale tijd die ervoor nodig is, is dus $O(m*(n-m+1)/q)$ en als wij $q \geq m$ kiezen dus $O(n-m+1)$. De buitenste lus vraagt ook tijd $O(n-m+1)$ – alles samen vraagt als wij deze veronderstellingen doen dus een tijd van $O(n)$.

Opmerking 10 *Het algoritme van Rabin Karp vraagt in het slechtste geval (als de waarden van de hashfunctie heel slecht verdeeld zijn) – net zoals het Algoritme 2 – $O((n-m+1)*m)$ stappen.*
In de praktijk – en veronderstellend dat de waarden van de hashfunctie gelijkmatig verdeeld zijn en q voldoende groot is – vraagt het algoritme tijd $O(n)$.

Oefening 34 *Je zoekt de string z =*`00101` *en past het algoritme van Rabin Karp toe met $q = 13$.*

- *Zoek z in t =*`1101110010001010`*.*
- *Geef een voorbeeld van een tekst t van lengte n waarvoor het algoritme de functie* `strings_gelijk()` *bijzonder vaak oproept als je naar z zoekt.*

Oefening 35

- *Beschrijf precies hoe je de hashwaarden berekent en update om te garanderen dat je geen problemen met overflow van integers krijgt.*

- *Het argument dat bewijst dat het updaten van de hashwaarde in constante tijd kan gebeuren telt alleen het aantal* integer operaties. *Maar voor arbitrair grote waarden van q passen de hashwaarden niet noodzakelijk in een integer van beperkte grootte. Wat is de invloed op de complexiteit als je voor de hashwaarden de arbitrair grote integers door integers van beperkte grootte (bv. 64 bit) moet simuleren? Zal dat in de praktijk gebeuren? Waarom (of waarom niet)?*

- *Stel dat je met q werkt dat groter is dan de maximale som in de formule – dat komt dus neer op het schrappen van de modulo-berekening. Dan hoef je als de hashwaarden overeenstemmen de strings zelfs niet meer vergelijken omdat ze zeker gelijk zijn. Is dat een goed idee?*

Oefening 36 *Stel dat je als alfabet niet alleen $\Sigma = \{0, 1\}$ hebt maar een algemeen (eindig) alfabet Σ. Definieer voor Σ een hashfunctie die in constante tijd geüpdatet kan worden.*
Denk daarbij erover na, of het belangrijk is dat in de som als basis voor de exponenten een 2 wordt genomen. Werken de argumenten ook met een andere basis?

Oefening 37

- *Wijzig het algoritme van Oefening 36 zo dat de zoekstring ook één wildcard* `?` *mag bevatten. Wij stellen dat* `?` *geen deel uitmaakt van de tekst en dat elk letterteken erdoor gematcht wordt (zoals in oefening 33). Ook hier moet het updaten van de hashfunctie (behalve in het begin) in constante tijd mogelijk zijn.*

- *Wijzig het algoritme van Oefening 36 zo dat de zoekstring ook één of meerdere wildcards* `*` *mag bevatten (zoals in oefening 33).*

Jouw algoritme moet een arbitraire string in de tekst teruggeven die de zoekstring matcht als die bestaat en anders de lege string.
De complexiteit van jouw algoritme mag ten hoogste de complexiteit van het algoritme van Rabin-Karp zijn.

Oefening 38 *Wijzig het algoritme van Rabin-Karp zo dat je efficiënt naar een langste match kan zoeken.*

Het algoritme van Knuth-Morris-Pratt

Als jullie naar het brute kracht algoritme 2 kijken dan komt de factor m van de functie `strings_gelijk()`. Maar deze functie is alleen maar duur als het deel in de tekst waarmee wij het vergelijken in het begin gelijk is aan de string die wij zoeken. In principe duikt het probleem dus altijd voor een deelstring op die zo begint als de zoekstring – die wij dus al op voorhand heel goed kennen. En dat betekent dat met een beetje preprocessing voor deze string zeker een voordeel te halen is. En dat is het principe van het door D. Knuth, J.H. Morris en V. Pratt in 1977 ontwikkelde algoritme.
Het principe wordt duidelijk als je naar een voorbeeld kijkt: Wij zoeken de string `zoek_me` in een tekst. Stel dat je probeert of die in de tekst t vanaf

positie i voorkomt. Als al na 1 stap de vergelijking fout teruggeeft – dus $t[i] \neq z$ moet je de volgende keer vanaf positie $i+1$ zoeken. Die heb je ten slotte nog niet gezien en het is dus mogelijk dat daar een match staat. Maar stel nu bv. dat de posities $0, \ldots, 4$ van de zoekstring nog overeenkomen en de vergelijking pas op positie 5 mislukt. Dat betekent dat vanaf positie i de tekst `zoek_` in de tekst staat maar niet de tekst `zoek_m`. Nu heb je de volgende posities wel al gezien en daaruit kan je een voordeel halen: je weet dat bv. tot positie $t[i+4]$ geen z voorkomt en je dus de string ten vroegste vanaf positie $i+5$ terug kan vinden. En je kan dat al op voorhand weten: **altijd** als ik `zoek_me` opzoek en de vergelijking op positie 5 voor de eerste keer mislukt, kan ik de volgende 4 posities als mogelijke startposities overslaan.

Het basisidee:

Ik wil op voorhand voor mijn string een verschuivingstabel $V[]$ opstellen. Deze tabel bevat voor elke positie van de zoekstring het aantal vakjes waarmee ik mijn startpositie in de volgende iteratie zal mogen opschuiven als er de eerste mismatch optreedt op die positie in de zoekstring.

In het geval dat het eerste letterteken maar één keer in de zoekstring $z[]$ opduikt (zoals in `zoek_me`) is de tabel $V[]$ gemakkelijk te berekenen: $V[0] = 1$ en $V[j] = j$ voor $0 < j < |s|$. Je weet namelijk dat het eerste letterteken verschilt van de volgende j lettertekens – een vergelijking die vroeger in de tekst start dan j tekens later zou dus al voor het eerste letterteken van $z[]$ een fout opleveren!
Voor andere teksten, als de zoekstring bijvoorbeeld `barbados` is, is het al iets moeilijker...
We zullen nu eerst een precieze definitie van $V[]$ ontwikkelen waarmee we verder kunnen werken.
Stel dat wij de posities $t[i+0], \ldots, t[i+j]$ al met $z[0], \ldots, z[j]$ vergeleken hebben en dat $z[j] \neq t[i+j]$ de eerste mismatch is. Hierbij is $j < |z|$ aangezien we anders het zoekwoord al volledig gematcht zouden hebben. Een verdere potentiële match zal dus zeker na positie $j-1$ eindigen aangezien tot deze positie nog geen match gevonden werd. Nieuwe startposities die nog geen mismatch voor positie $j-1$ geven, zijn dus posities s waarbij $z[0] = z[s], \ldots, z[j-1-s] = z[j-1]$. Als zo'n positie s niet bestaat, m.a.w. er is geen interne match binnen de zoekstring tot positie $j-1$, dan is j de eerst mogelijke startpositie waarop een verdere match mogelijk is. Aangezien we de eerst mogelijke startpositie zoeken, nemen we de kleinst mogelijke s die aan de voorwaarden voldoet. Of formeel:

Definitie 11 *Voor een gegeven zoekstring z is de verschuivingstabel $V[]$ gedefinieerd als*
$V[0] = 1$ *en voor* $j > 0$
$V[j] = \min\{s | (0 < s < j$ *en* $z[0] = z[s], \ldots, z[j-1-s] = z[j-1])$
of $s = j\}$

Als je op een positie i met $0 < i < V[j]$ begint dan weet je dat er ten laatste op positie $j-i$ een botsing is en dus de string vanuit deze positie niet gevonden kan worden.
Voor het voorbeeld `barbados` betekent dat

	b	a	r	b	a	d	o	s
positie j	0	1	2	3	4	5	6	7
V[j]	1	1	2	3	3	3	6	7

Wij weten nu hoeveel lettertekens je kan overslaan als je een match tot positie $j-1$ hebt maar geen tot positie j – maar er is nog een observatie die nodig is om ervoor te zorgen dat het algoritme echt lineair is:

Observatie: Stel dat er een match van $z[]$ tot positie $j-1$ in $t[]$ vanaf positie i staat. Dan kan een complete match ten vroegste op positie $i+V[j]$ beginnen (dat hadden wij al). **Maar:** als $V[j] < j$ dan weten wij van de posities $i+V[j], \ldots, i+j-1$ al dat ze overeenkomen met $z[V[j]], \ldots, z[j-1]$ – ze werden al vergeleken. Maar volgens de definitie van $V[]$ komen die dan ook overeen met $z[0], \ldots, z[j-1-V[j]]$. Als wij de string vanaf positie $i+V[j]$ zoeken, moeten wij dus de eerste $j-V[j]$ tekens niet opnieuw vergelijken. Wij moeten pas beginnen op de positie waar de eerste mismatch opdook.

Er kan dus op twee manieren bespaard worden: aan de ene kant wordt `vergelijk_strings()` minder vaak opgeroepen (door sommige startposities over te slaan) en aan de andere kant zijn de oproepen goedkoper omdat niet altijd vanaf het begin vergeleken moet worden.

Stel op dit moment dat wij de tabel $V[]$ efficiënt kunnen berekenen. Dan is het algoritme van Knuth-Morris-Pratt als volgt:

Algoritme 4 *(Knuth-Morris-Pratt)*

```
vergelijk_strings(z,t,start,gelijke_tekens)
// zoekt of de string z in de tekst t vanaf positie start voorkomt.
// Het wordt verondersteld dat de tekens 0...(gelijke_tekens-1)
// in z en t (vanaf start) gelijk zijn.
// Het wordt bovendien verondersteld dat in t na start nog ten
// minste even veel tekens staan als in z en dat |z| gekend is.
// De functie geeft de index van de eerste mismatch terug
{
for (i=gelijke_tekens; i<|z|; i++)
         if (z[i]!=t[start+i]) return i;

return |z|;
}

is_contained_Knuth_Morris_Pratt(z,t)
// test of de string z in de string t voorkomt
{
bereken_V(z)

start=0;
gelijke_tekens=0;

while (start<=|t|-|z|)
{
  eerste_mismatch=vergelijk_strings(z,t,start,gelijke_tekens);
  if (eerste_mismatch=|z|) return TRUE;
```

```
  start=start+V[eerste_mismatch];
  if (eerste_mismatch=0) gelijke_tekens=0; // speciaal geval
    else gelijke_tekens=eerste_mismatch-V[eerste_mismatch];
}

return FALSE;
}
```

Het geval waar de mismatch al op positie 0 opduikt is in zekere zin speciaal omdat daar ook de argumenten waarom wij kunnen opschuiven anders zijn. Het is het enige geval waar wij opschuiven **na** de positie van de eerste mismatch. Het beste is natuurlijk de werking van het algoritme eens te illustreren aan de hand van een voorbeeld. Wij zoeken de string `barbados` in `barbaarse_barbecue_in_barbados`. De eerste positie die vergeleken moet worden wordt met een v gemarkeerd.

```
v
barbaarse_barbecue_in_barbados
barbados
     5
```

De eerste mismatch is op positie 5. Omdat $V[5] = 3$ kunnen wij met 3 posities opschuiven en moeten de eerste $5 - 3 = 2$ tekens niet meer vergeleken worden:

```
     v
barbaarse_barbecue_in_barbados
   barbados
     2
```

De eerste mismatch is nu op positie 2. Omdat $V[2] = 2$ kunnen wij met 2 posities opschuiven en moeten de eerste $2 - 2 = 0$ tekens niet meer vergeleken worden – dat helpt in dit geval dus niet.

```
     v
barbaarse_barbecue_in_barbados
     barbados
     0
```

Wij hebben onmiddellijk een mismatch, wij kunnen dus met $V[0] = 1$ positie opschuiven en moeten 0 posities (speciaal geval) niet vergelijken:

```
      v
barbaarse_barbecue_in_barbados
      barbados
      0
```

Wij hebben opnieuw onmiddellijk een mismatch en kunnen dus met $V[0] = 1$ positie opschuiven. Dat gaat door totdat wij de `b` van `barbecue` bereikt hebben:

```
          v
barbaarse_barbecue_in_barbados
          barbados
              4
```

De eerste mismatch is nu op positie 4. Omdat $V[4] = 3$ kunnen wij met 3 posities opschuiven en moeten de eerste $4 - 3 = 1$ tekens niet meer vergeleken worden:

```
              v
barbaarse_barbecue_in_barbados
             barbados
              1
```

De eerste mismatch is nu op positie 1. Nu zal dat zo doorgaan en er zal altijd onmiddellijk een mismatch gedetecteerd worden totdat de volgende `b` bereikt wordt – en dan wordt de match ook gevonden:

```
                       v
barbaarse_barbecue_in_barbados
                       barbados
                              8
```

Maar wat is de complexiteit van dit algoritme? Op dit moment stellen wij dat het berekenen van de tabel in tijd $O(m)$ kan gebeuren – dat zullen wij later natuurlijk nog bewijzen – en kijken wij alleen naar de routine die de string in de tekst echt zoekt.

Stelling 12 *Het algoritme van Knuth, Morris en Pratt vraagt tijd $O(n)$ als in een tekst $t[]$ van lengte n een zoekstring $z[]$ van lengte $m \leq n$ gezocht wordt.*

Bewijs: Wij veronderstellen hier dat "`bereken_V()`" in tijd $O(m)$ (dus ook $O(n)$) kan gebeuren. Dat zullen wij later nog zien.

De lus in `is_contained_Knuth_Morris_Pratt()` wordt duidelijk ten hoogste $n - m$ keer doorlopen en de kost in de lus is constant als wij de kost van `vergelijk_strings()` niet meerekenen. Wij moeten dus alleen nog bewijzen dat alle oproepen van
`vergelijk_strings()` samen ten hoogste tijd $O(n)$ vragen.

De complexiteit van `vergelijk_strings()` wordt bepaald door het aantal vergelijkingen die in de routine gebeuren en daar is een belangrijk verschil tussen succesvolle vergelijkingen en vergelijkingen die een mismatch opleveren: je kan hetzelfde teken uit de tekst meerdere keren *zonder succes* vergelijken, maar (dat zullen wij tonen) maar één keer succesvol. Wij tellen hier de vergelijkingen met en zonder succes dus apart. Omdat `vergelijk_strings()` ten hoogste $n - m$ keer wordt opgeroepen en omdat in elke oproep ten hoogste één vergelijking zonder succes gebeurt (bij een mismatch wordt onmiddellijk een waarde teruggegeven) zijn ten hoogste $n - m$, dus $O(n)$ vergelijkingen zonder succes.

Voor de andere oproepen is het nuttig de ontwikkeling te zien van de absolute positie in de tekst die met de string vergeleken wordt. In het algoritme wordt altijd de positie relatief met de startpositie bijgehouden, maar als je naar de absolute positie in de tekst kijkt dan hebben jullie misschien al in het voorbeeld gemerkt dat die nooit kleiner wordt. Op het moment dat een teken één keer succesvol vergeleken wordt, wordt dit teken nooit meer met de zoekstring vergeleken – en omdat er maar n lettertekens

zijn, volgt uit deze observatie natuurlijk dat ook deze bewerkingen samen maar tijd $O(n)$ vragen.

Wij moeten dus alleen nog de observatie bewijzen:

Stel dat `vergelijk_strings()` wordt opgeroepen met de variabelen `start` en `gelijke_tekens`. Dan wordt vergeleken vanaf positie `start+gelijke_tekens`.

Wij moeten nu bewijzen dat als de volgende oproep met `start'` en `gelijke_tekens'` is dat dan
`start'+gelijke_tekens'`$\geq$
`start+eerste_mismatch`

omdat `start+eerste_mismatch` net de eerste positie **na** de succesvolle vergelijkingen is en dat garandeert dat al succesvol vergeleken tekens niet nog eens getest worden.

Als `eerste_mismatch`$= 0$ dan zijn er geen succesvolle vergelijkingen en `gelijke_tekens` en `gelijke_tekens'` zijn beide 0 dus `start'`=`start`$+1$ – de bewering klopt dus. Anders geldt

`start'+gelijke_tekens'`=
`start+V[eerste_mismatch]+eerste_mismatch-V[eerste_mismatch]`=
`start+eerste_mismatch`

En precies dat wouden wij bewijzen.

■

In typische toepassingen die jullie kennen, zijn de zoekstrings veel korter dan de tekst – zelfs de brute kracht manier om $V[]$ te berekenen (die in $O(m^2)$ of zelfs $O(m^3)$ draait afhankelijk van hoe je het implementeert) zou dan geen probleem zijn. Maar voor toepassingen waar het verschil tussen m en n niet zo groot is, moet de preprocessing wel efficiënt zijn – en het best in tijd $O(m)$ gebeuren.
Wij zoeken niet voor elke index i van $V[]$ waar de vroegste startpositie is, maar voor elke mogelijke startpositie – beginnend met de kleinste, dus 1 – kijken wij hoe ver we raken zonder mismatch. Voor de nog niet ingevulde indexen i tot en met de eerste mismatch is de startpositie dan de vroegste startpositie, dus de juiste waarde voor $V[i]$. In principe zoeken wij dus alleen maar een match van $z[]$ in zichzelf. Daarvoor kan je **bijna** hetzelfde algoritme toepassen als voor het zoeken van de tekst. Je zoekt de tekst in zichzelf – dus zijn $z[]$ en $t[]$ dezelfde string. Omdat je nu naar matches van verschillende lengten zoekt, moet je wel tot het einde doorgaan. Belangrijk daarbij is dat de waarden van $V[]$ die je gebruikt altijd waarden zijn die je al op voorhand hebt berekend. $V[0] = V[1] = 1$ is voor elk zoekwoord hetzelfde – daarmee kan je dus beginnen.
Om de pseudocode eenvoudiger te houden gaan we ervanuit dat op het einde van $z[]$ nog een speciaal `end_of_line` teken volgt dat verschilt van alle tekens in de string en dat je in $V[]$ één teken meer kan invullen dan de lengte van $z[]$ (ook al gebruik je dat achteraf niet). Op deze manier moeten wij geen testen in `vergelijk_strings()` invullen die garanderen dat je niet over de grenzen van de array leest, wat de pseudocode minder leesbaar zou maken.

Algoritme 5 *(Berekenen van $V[]$)*

```
bereken_V(z)
// bereken de verschuivingstabel van de string z
{
V[0]=1;
V[1]=1;

start=1; // voor 0 zou je gewoon dezelfde string terugvinden
gelijke_tekens=0;

while (start+gelijke_tekens<|z|-1)
// anders zouden alleen posities groter |Z|-1 ingevuld worden
{
  eerste_mismatch=vergelijk_strings(z,z,start,gelijke_tekens);

  if (eerste_mismatch=0) V[start+1]=start+1;
    else
     for (i=gelijke_tekens+1; i<=eerste_mismatch; i++)
                                              V[start+i]=start;
    //  de andere tekens werden al ingevuld

  start=start+V[eerste_mismatch]; // deze waarde is
                                  // gegarandeerd al ingevuld

  if (eerste_mismatch=0) gelijke_tekens=0; // speciaal geval
    else gelijke_tekens=eerste_mismatch-V[eerste_mismatch];
}
}
```

Het is duidelijk dat `bereken_V()` $O(m)$ tijdsbegrensd is omdat het extra werk in vergelijking met `is_contained_Knuth_Morris_Pratt()` (het invullen van $V[]$) zeker $O(m)$ tijdsbegrensd is. De `while()` lus loopt wel tot het einde, maar wij hebben bij de analyse van `is_contained_Knuth_Morris_Pratt()` ook gewoon gesteld dat het ten hoogste tot het einde loopt. En het deel dat `bereken_V()` vervangt is hier het invullen van maar 2 getallen in een array – dat is dus zeker constant.
De werking van `bereken_V()` zie je het best opnieuw aan een voorbeeld. Hoe berekent deze functie de tabel voor `barbados`? Wij beginnen:

	b	a	r	b	a	d	o	s
positie j	0	1	2	3	4	5	6	7
V[j]	1	1						

```
 1
barbados
 barbados
 0
```

Wij hebben onmiddellijk een mismatch (`eerste_mismatch=0`), kunnen dus $V[1+1] = 2$ invullen en $V[0] = 1$ opschuiven.

	b	a	r	b	a	d	o	s
positie j	0	1	2	3	4	5	6	7
V[j]	1	1	2					

```
  2
barbados
  barbados
  0
```

Wij hebben opnieuw onmiddellijk een mismatch, kunnen dus $V[2 + 1] = 3$ invullen en $V[0] = 1$ opschuiven.

	b	a	r	b	a	d	o	s
positie j	0	1	2	3	4	5	6	7
V[j]	1	1	2	3				

```
   3
barbados
   barbados
     2
```

Nu hebben wij de eerste mismatch op positie 2. Wij kunnen dus $V[3 + 1] = 3$ en $V[3 + 2] = 3$ invullen en $V[2] = 2$ posities opschuiven.

	b	a	r	b	a	d	o	s
positie j	0	1	2	3	4	5	6	7
V[j]	1	1	2	3	3	3		

```
     5
barbados
     barbados
     0
```

Nu krijgen wij nog twee keer onmiddellijk een mismatch en hebben dan

	b	a	r	b	a	d	o	s
positie j	0	1	2	3	4	5	6	7
V[j]	1	1	2	3	3	3	6	7

Oefening 39 *Pas het algoritme van Knuth-Morris-Pratt toe op de zoekstring* `z[]= volvoert` *en de tekst* `t[]= volvo_vol_vogels_volvoert_kunststukje`. *Bereken de verschuivingstabel en pas het zoekalgoritme toe. Werk voldoende tussenstappen uit om te laten zien hoe het algoritme werkt.*

Oefening 40 *Pas het algoritme van Knuth-Morris-Pratt toe op de zoekstring* `z[]= ananas` *en de tekst* `t[]= bananen_in_ra'ananas'_centrum`.
Bereken de verschuivingstabel en pas het zoekalgoritme toe. Werk voldoende tussenstappen uit om te zien hoe het algoritme werkt.

Oefening 41 *Wat moet aan de pseudocode gewijzigd worden om* **alle** *matches van de zoekstring te vinden?*
Werkt het algoritme van Knuth-Morris-Pratt ook in tijd $O(n)$ als in een tekst $t[]$ van lengte n alle voorkomens van een zoekstring $z[]$ van lengte $m \leq n$ gezocht worden?

Oefening 42 *Stel dat $m >> n$. Geef argumenten waarom het algoritme van Knuth-Morris-Pratt zoals beschreven dan niet $O(n)$ tijdsbegrensd is en beschrijf precies hoe je het moet wijzigen dat het dat wel is.*

Oefening 43 *Bewijs: als het algoritme zonder de extra "observatie" wordt geïmplementeerd (dat is hetzelfde alsof je* `gelijke_tekens` *altijd gelijk aan* 0 *zet) zijn er gevallen waarvoor het algoritme $\Theta(n^2)$ is.*

Oefening 44 *Het algoritme houdt alleen rekening met de informatie waar je een match hebt en niet met het letterteken op de mismatch – hoewel dat ook al gelezen werd. Zou het helpen ook daarmee rekening voor de verschuivingstabel te houden?*

Oefening 45 *Wijzig het algoritme van Knuth-Morris-Pratt zo dat het voor twee strings $z[]$ en $t[]$ de positie en de lengte van een langste match van een prefix van $z[]$ in $t[]$ teruggeeft.*
Gezocht is dus een match van $z[0]z[1]\ldots z[i]$ waarvoor i zo groot mogelijk is.

Oefening 46 *Stel dat je met de wildcard* `?` *wil werken en het algoritme van Knuth-Morris-Pratt gewoon wijzigt door een vergelijking waarin een* `?` *betrokken is altijd te aanvaarden. Toon aan dat het gewijzigde algoritme niet juist werkt.*

Oefening 47 *Wijzig het algoritme van Knuth-Morris-Pratt zo, dat het met wildcards* `*` *in de zoekstring $z[]$ kan werken, maar om het even hoeveel wildcards gebruikt worden nog altijd gegarandeerd in tijd $O(n)$ draait met n de lengte van de tekst $t[]$.*
Bewijs expliciet dat jouw algoritme de gegeven complexiteit heeft.

Het algoritme van Boyer-Moore-Horspool

Het algoritme van Boyer-Moore-Horspool (1980) is een vereenvoudiging van het algoritme van Boyer and Moore (1977). Het algoritme van Boyer and Moore in zijn oorspronkelijke vorm was *bijna* even efficiënt als het algoritme van Knuth-Morris-Pratt en met een kleine wijziging van Galil in 1979 zelfs even efficiënt (dat is: $O(n+m)$ voor elke toepassing). Maar het bijzondere aan het algoritme van Boyer-Moore-Horspool is dat het heel eenvoudig en gemakkelijk toe te passen is. Dat maakt het in de praktijk – waar je ook rekening moet houden met de constanten die in de $O()$-notatie verwaarloosd zijn – bijzonder snel.
Voor ons is het hier interessant te zien dat een eenvoudig idee soms tot een heel goed algoritme kan leiden – ook al is de prestatie in het slechtste geval misschien niet zo goed als die van andere – ingewikkeldere – algoritmen.
Het volgende algoritme is bijna identiek aan het brute kracht algoritme. Het enige verschil is dat de strings niet van positie 0 tot $|z|-1$ maar van $|z|-1$ tot 0 worden vergeleken. Daardoor kan soms vroeger en soms later gestopt worden – een echt voordeel is dat niet.

```
strings_gelijk_invers(z,t,start)
// test of t[start]...t[start-|z|-1] = z[|z|-1]...z[0]
// hierbij wordt vanaf het einde vergeleken
```

```
{
for (i=0; i<=|z|-1; i++) if (z[|z|-1-i]!=t[start-i]) return FALSE;

return TRUE;
}


is_contained(z,t)
// test of de string z in de string t voorkomt
{
for (i=|z|-1; i<|t|; i++)
  if strings_gelijk_invers(z,t,i) return TRUE;

return FALSE;

}
```

Stel nu dat je bijvoorbeeld de string `barbara` in `barbados_zoekt_barbara` zoekt. De eerste keer dat de lus in `is_contained()` wordt doorlopen, vergelijk je

```
barbados_zoekt_barbara
barbara
```

en het eerste teken dat je vergelijkt is het laatste van `barbara`. Je vergelijkt dus `a` met `o`. Daarbij zie je dat de vergelijking niet alleen een fout oplevert – het is zelfs zo dat er in de zoekstring `barbara` helemaal geen `o` zit ...
Het is dus zinloos vanaf een startpositie te vergelijken waar de `o` nog in het deel zit dat vergeleken wordt – de volgende startpositie met kans op succes is dus de positie na de `o`.

```
barbados_zoekt_barbara
       barbara
```

Nu vergelijken wij eerst een `t` met de laatste `a` – en zijn in dezelfde situatie: er is geen `t` in `barbara`, wij kunnen dus opschuiven tot na de `t`.

```
barbados_zoekt_barbara
              barbara
```

Nu vergelijken wij een `r` met de `a` en ook al is dat een mismatch, kunnen wij ons argument hier niet meer toepassen: `barbara` bevat wel degelijk een `r` (zelfs twee). Hoe ver kunnen wij dus opschuiven? Wij weten dat er alleen maar een kans is als onze startpositie zo is dat er een `r` op de positie van de `r` in de tekst terechtkomt. Als je van achteraan telt dan zie je dat je ofwel 1 ofwel 4 posities moet opschuiven. Als je 4 posities opschuift, toets je niet of er een mogelijke match is als je maar 1 positie opschuift. Voor lettertekens uit de zoekstring kiezen wij dus altijd voor het kleinste aantal posities (maar wel ten minste 1 positie) dat je kan opschuiven om een match van het laatste teken te hebben – in ons voorbeeld dus 1:

```
barbados_zoekt_barbara
               barbara
```

Deze ideeën zullen wij nu gebruiken om een verschuivingstabel op te stellen. Een groot verschil met Knuth-Morris-Pratt is daarbij dat je de verschuiving opzoekt gebaseerd op een teken in de tekst – en niet gebaseerd op een positie in de zoekstring zoals bij Knuth-Morris-Pratt. Als wij een zoekstring $z[]$ hebben en de tabel V_H noemen, dan kunnen wij voor een letterteken x definiëren:

$$V_H(x) = \begin{cases} |z| & \text{als } x \text{ niet in } z[] \text{ voorkomt} \\ & \text{of alleen op positie } |z|-1 \\ p-1 & \text{als } |z|-p \text{ de laatste positie voor } |z|-1 \\ & \text{is, waar } x \text{ in } z[] \text{ voorkomt} \end{cases}$$

Deze tabel kan je met de volgende pseudocode gemakkelijk en snel berekenen:

```
vul_VH(z)

{
// eerst voor alle bytes (ASCII tekens) de waarde |z| invullen:
for (i=0; i<256; i++) VH[i]=|z|;

// dan voor tekens die in z[] voorkomen de waarden overschrijven
// de laatste keer dat een teken voorkomt bepaalt de waarde:

for (i=0; i<=|z|-2; i++) VH[z[i]]=|z|-i-1;
}
```

Deze tabel kan duidelijk heel gemakkelijk en snel ingevuld worden. Ook in de $O()$-notatie is dat lineair in $m = |z|$, dus $O(m)$.

Voor `barbara` is dat

$x =$	`a`	`b`	`r`
$V_H[x]$	2	3	1

en $V_H[x] = 7$ voor alle andere tekens.

Daarmee kunnen wij nu het algoritme van Boyer-Moore-Horspool beschrijven:

```
is_contained_BMH(z,t)

{
vul_VH(z);

for (i=|z|-1; i< |t|; )
  { if strings_gelijk_invers(z,t,i) return TRUE;
    i= i+ VH[t[i]];
```

```
  }

return FALSE;

}
```

Omdat wij de werking al voor het zoeken van `barbara` in `barbados_zoekt_barbara` gezien hebben, zullen wij hier niet nog een voorbeeld uitwerken.

Oefening 48 *Pas het algoritme van Boyer-Moore-Horspool toe op de zoekstring* `z[]= ananas` *en de tekst* `t[]= de_spa_in_ra'ananas'`.
Bereken de verschuivingstabel en pas het zoekalgoritme toe. Werk voldoende tussenstappen uit om te zien hoe het algoritme werkt.

Natuurlijk zien jullie onmiddelijk, dat hier – vergelijkbaar met Knuth-Morris-Pratt – informatie die je in `strings_gelijk_invers()` verwerft ook gebruikt zou kunnen worden om soms verder op te schuiven of vergelijkingen te besparen. Dan gaan jullie al in de richting van het (iets ingewikkeldere) algoritme van Boyer-Moore, resp. Boyer-Moore-Galil .

Oefening 49 *In de pseudocode wordt de functie* `strings_gelijk_invers(z,t,i)` *gebruikt. Zou de code ook juist werken als je dat door* `strings_gelijk(z,t,i)` *vervangt?*

Oefening 50 *Gegeven een zoekstring $z[]$ met $|z| = m$ en een tekst $t[]$ met $|t| = n$.*

- *Geef voorbeelden van $z[]$ en $t[]$ (die voor alle $n \geq m > 0$ werken) die voor het algoritme van Boyer-Moore-Horspool het beste geval voorstellen als $z[]$ geen deelstring is van $t[]$. Wat is de complexiteit van het algoritme van Boyer-Moore-Horspool in dit beste geval ($\Theta()$-notatie).*
- *Geef voorbeelden van $z[]$ en $t[]$ (die voor alle $n \geq m > 0$ werken) die voor het algoritme van Boyer-Moore-Horspool het slechtste geval voorstellen. Wat is de complexiteit van het algoritme van Boyer-Moore-Horspool in dit slechtste geval ($\Theta()$-notatie).*

Oefening 51 *Het algoritme van Boyer-Moore-Horspool werkt natuurlijk duidelijk minder goed als je een kleiner alfabet hebt – bv. alleen $\{a, c, g, t\}$. Stel expliciet een manier voor om de efficiëntie te verbeteren. Ga ervanuit dat ook voor deze 4 tekens de string uit bytes bestaat die de gewone ASCII-waarden van a, c, g, t voorstellen.*

Het shift-AND algoritme

Het algoritme van Knuth, Morris, and Pratt kan gezien worden als een optimalisatie van het brute kracht algoritme dat wij misschien als eerste zouden proberen. Het volgende algoritme van R. Baeza-Yates en G. Gonnet is om meerdere redenen interessant: aan de ene kant is al de aanzet een beetje verrassend en ligt de klemtoon hier sterk op de praktijk – waardoor het natuurlijk heel goed bij DA3 past – en aan de andere kant zouden de technieken voor jullie

ook een motivatie kunnen zijn in andere toepassingen waar jullie zelf algoritmen moeten ontwerpen eens na te denken of een vertaling naar bitvectoren misschien een snel algoritme zou opleveren! De theoretische slechtste-geval complexiteit is niet alleen $O(n * m)$ – net zoals het brute-kracht algoritme – maar in dit geval heb je zelfs $\Theta(n * m)$ stappen in het beste geval nodig (als je de hele tekst moet doorlopen) terwijl het brute-kracht algoritme in dat geval maar tijd $\Theta(n)$ vraagt. Maar als wij rekening houden met toepassingen waar m relatief klein is, wordt het niet alleen lineair (dat wordt het brute-kracht algoritme ook), maar presteert het zelfs bijzonder goed omdat de constante in de lineaire functie heel klein is! Dit algoritme is vooral een goede keuze als je alle matches zoekt en er zijn er veel in de tekst waarin je zoekt.
In sommige boeken kan je dit algoritme als shift-AND terugvinden en in anderen als shift-OR – de redenen daarvoor zullen jullie in de oefeningen zien.

Definitie 13 *Gegeven een zoekstring $z[]$ van lengte m en een tekst $t[]$ van lengte n.*
Dan definiëren wij de $m \times n$ matrix M als $M[i][j] = 1$ als de prefix uit de tekens t.e.m. positie i van $z[]$ met de deelstring die in $t[]$ op positie j eindigt gematcht kan worden en anders 0. *Of formeel:*

$$M[i][j] = \begin{cases} 1 & \textit{als } z[0] = t[j-i], \ldots, z[i] = t[j] \\ 0 & \textit{anders} \end{cases}$$

Hierbij hebben wij de rijen en kolommen nummers vanaf 0 gegeven in plaats van vanaf 1.

Voor het geval $t[]$ =ananas en $z[]$ =ana krijgen wij

	a	n	a	n	a	s
a	1	0	1	0	1	0
n	0	1	0	1	0	0
a	0	0	1	0	1	0

In de laatste rij staat dus een 1 op positie j ($M[m-1][j] = 1$) als en slechts als een match van het hele zoekwoord $z[]$ op positie j van de tekst eindigt. Deze matrix heeft $n * m$ elementen, die alle n te berekenen en in te vullen vraagt dus ten minste tijd $\Theta(m * n)$.
Maar hoe berekenen wij deze matrix?
Wij zullen een dynamisch programmeren algoritme gebruiken dat bovendien – door bitvectoren te gebruiken – parallel werkt: meerdere van de $M[i][j]$ worden tegelijk met hulp van vroegere waarden berekend.
Daarvoor definiëren (en berekenen) wij eerst de binaire karakteristieke vector C_x voor elke letter x in het alfabet. Deze vector heeft lengte m en

$$C_x[i] = \begin{cases} 1 & \text{als } z[i] = x \\ 0 & \text{anders} \end{cases}$$

C_x houdt dus bij waar het teken x in $z[]$ staat. In ons voorbeeld $z[]$ =ana krijgen wij $C_a = (101)$, $C_n = (010)$ en $C_x = (000)$ voor alle anderen tekens x in het alfabet.
De eerste kolom is gemakkelijk: $M[i][0] = 0$ voor $1 \leq i < n$ omdat op positie 0 natuurlijk geen match van lengte groter dan 1 kan eindigen. En $M[0][0] = 1$ als en slechts als $z[0] = t[0]$

Stel nu dat wij kolom j al hebben en kolom $j+1$ willen berekenen. $M[0][j+1] = 1$ als en slechts als $t[j+1] = z[0]$ – of equivalent $M[0][j+1] = C_{t[j+1]}[0]$. Voor $i > 0$ hebben wij $M[i][j+1] = 1$ als en slechts als er een match van lengte i (de posities $0, \ldots, i-1$) tot positie j is **en** $t[j+1] = z[i]$. Of anders geschreven: $M[i][j+1] = M[i-1][j] \quad \& \quad C_{t[j+1]}[i]$ – en nu zien jullie misschien waar de naam van dit algoritme vandaan komt: als de $M[][j]$ en de karakteristieke vector $C_{t[j+1]}[]$ bitvectoren zijn dan is $M[][j+1] = \text{shift}^1(M[][j]) \quad \& \quad C_{t[j+1]}[]$ als de shift-operatie shift^1 bit $i-1$ op positie i schuift en het nieuwe bit 0 op 1 zet. Als pseudocode ziet dat er als volgt uit – waarbij wij altijd alleen maar één kolom van de matrix bijhouden – zodra wij de nieuwe kolom berekend hebben, hebben wij de oude ten slotte niet meer nodig:

Algoritme 6 *(Het shift-AND algoritme)*

```
shiftAND(z,t)
{
// test of de string z in de string t voorkomt
// geeft TRUE terug als ja anders FALSE
// alle karakteristieke vectoren C[x][] en ook de vector kolom[]
// zijn bitvectoren -- dus bv. een unsigned long int als m<=64

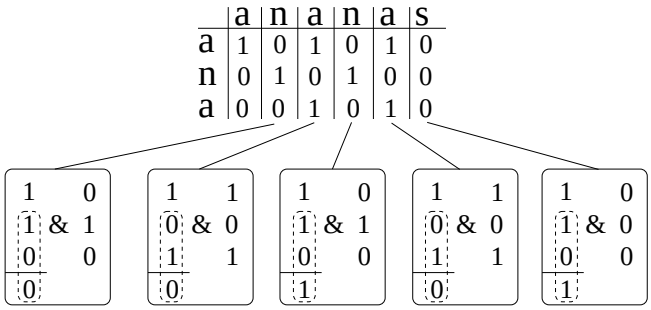
  bereken_karakteristieke_vectoren(); // dat is straightforward

  if (z[0]=t[0])
      { if (|z|=1) return TRUE;
       kolom=(1,0,0,...,0) // lengte |z|
      }
    else  kolom=(0,0,0,...,0)

  for (j=1; j<|t|;j++)
  {
    kolom[]= SHIFT^1(kolom[]) & C[t[j]][];
    if (kolom[|z|-1]=1) return TRUE;
  }

  return FALSE;
}
```

De pseudocode weerspiegelt – op het feit na dat je met maar 2 kolommen werkt – de beschrijving van het algoritme. Je kan de eerste kolom ook berekenen door `kolom` op nul te initialiseren en dan de lus met `j=0` te beginnen. In de beschrijving zou dat op een kolom -1 neerkomen waar je natuurlijk geen match hebt.
Hoe deze operaties tot de tabel voor ons voorbeeld leiden, zien jullie in Figuur 23. Als je deze operaties algemeen wil implementeren dan vraagt het berekenen van de volgende kolom – zoals verwacht – tijd $O(m)$. Als m naar boven door een constante begrensd is, is dat dus constant – maar ook dat is niet echt



Figuur 23: De stappen van het shift-AND algoritme toegepast op $z[] = ana$ en $t[]$ =ananas (waarbij na de eerste match niet gestopt wordt).

een voordeel in vergelijking met andere algoritmen. Maar als de lengte van de zoekstring ten hoogste het aantal bits van een woord in de computer is – en voor heel veel toepassingen mag je zeker stellen dat de zoekstring niet meer dan 64 tekens bevat – dan kan je de bitvectoren in één computerwoord voorstellen en de bitmanipulaties gebruiken (<<,>>,&,|) die de processor kan uitvoeren. Dan zijn de bewerkingen niet alleen *constant* maar *echt heel goedkoop.*
In gevallen waar je met heel lange woorden bezig bent, kan je eerst naar een prefix zoeken die nog in één computerwoord past. De lengte van een computerwoord is op dit moment meestal 64 bits. Als het om *gewone teksten* gaat, zal deze prefix vermoedelijk maar voor een heel klein deel van de mogelijke startposities gematcht kunnen worden. Voor de zeldzame gevallen waar die gematcht wordt, pas je voor de rest dan de brute kracht manier toe om te zien of de rest van de zoekstring hier aansluit.
Een andere mogelijkheid is met bitvectoren te werken die meer dan één computerwoord gebruiken.

Oefening 52 *Geef een algoritme in pseudocode dat de verzameling van alle C_x in tijd $O(m)$ kan berekenen waarbij m de lengte van het gezochte woord $z[]$ is.*

Oefening 53 *Schrijf de C-code die de shift-AND operaties implementeert voor een woordlengte op de computer van WORDSIZE bits.*

- *voor een zoekwoord dat gegarandeerd ten hoogste WORDSIZE letters bevat*
- *voor een zoekwoord met onbeperkte lengte (maar werk hier wel met kolommen van dezelfde lengte als het woord!)*

Oefening 54 *Pas het shift-AND algoritme toe op het zoekwoord $z[]$ =*`zoeken` *en de tekst $t[]$ =*`iets_zoets_zoeken`

Oefening 55 *Wijzig het shift-AND algoritme zo dat het ook met wildcards* `?` *en* `*` *kan werken en nog altijd even goed presteert.*

Oefening 56 *Wij zijn eraan gewend dat* 1 *voor* `TRUE` *staat en* 0 *voor $FALSE$ en zo is het shift-AND algoritme ook beter uit te leggen en te verstaan. Maar nu dat je het algoritme* **hebt** *verstaan: wissel in Definitie 13 de betekenis van* 0 *en* 1*.*

- *Werk het algoritme uit voor deze gewijzigde definitie en geef de pseudocode.*
- *Schrijf de belangrijkste lijn van de C-code voor het algoritme dat het algoritme volgens deze nieuwe definitie implementeert.*
- *Welke versie zou je implementeren? Die met de interpretatie van* 1 *zoals in de tekst of zoals in de oefening? Waarom?*

Oefening 57 *Wijzig Algoritme 6 zo dat de lengte van een langste match beginnend met* $z[0]$ *terug wordt gegeven.*

3.5 Benaderend string matching

shift-AND voor matches met fouten

Maar het shift-AND algoritme laat het ook toe matches nog meer te veralgemenen dan met wildcards: matches met een beperkt aantal mismatches – zonder op voorhand te specifiëren waar die moeten zitten. Wij zullen hier het algoritme zien dat het programma *agrep* te gronde ligt en dat door S. Wu en U. Manber in 1992 ontwikkeld werd. Ook hier ligt de klemtoon op de praktijk – dus het geval waar het toegelaten aantal fouten begrensd en klein is.
Wij zullen het hier op dit moment alleen maar hebben over *mismatches* – dus waar een letterteken met een **ander** letterteken gematcht moet worden. Je kan het ook over *fouten* in de betekenis van ontbrekende lettertekens (in de tekst of de zoekstring) hebben.

Definitie 14 *Gegeven een zoekstring* $z[]$ *van lengte* m*, een tekst* $t[]$ *van lengte* n *en een (klein) getal* k*.*
Dan definiëren wij de $m \times n$ *matrix* M^k *als* $M^k[i][j] = 1$ *als de prefix uit de tekens t.e.m. positie* i *van* $z[]$ *met de deelstring die in* $t[]$ *op positie* j *eindigt gematcht kan worden waarbij er ten hoogste* k *mismatches optreden en anders* 0*. Of formeel (waarbij wij gebruiken dat* ten hoogste k mismatches *betekent dat er ten minste* $i + 1 - k$ *goede matches zijn):*

$$M^k[i][j] = \begin{cases} 1 & \text{als er indices } 0 \leq l_0 < l_1, < \cdots < l_{i-k} \leq i \text{ bestaan zodat} \\ & z[l_0] = t[j - i + l_0], \ldots, z[l_{i-k}] = t[j - i + l_{i-k}] \\ 0 & \text{anders} \end{cases}$$

Ook hierbij hebben wij de rijen en kolommen nummers vanaf 0 gegeven in plaats van vanaf 1.

De matrix $M^0[][]$ is dus de matrix $M[][]$ die wij al kennen – hier laten wij 0 fouten toe en wij weten hoe wij die moeten berekenen.
In de laatste rij van $M^k[][]$ staat dus een 1 op positie j ($M^k[m-1][j] = 1$) als en slechts als een match met ten hoogste k fouten van het hele zoekwoord $z[]$ op positie j van de tekst eindigt. Het idee lijkt dus heel sterk op het gewone shift-AND algoritme en de vraag is alleen maar hoe je $M^k[][]$ het best berekent.
De eerste kolom is gemakkelijk voor alle $k > 0$: $M^k[][0] = (1, 0, \ldots, 0)$
Er zijn 2 mogelijkheden waarom voor een $0 < l \leq k$ de waarde van $M^l[i][j] = 1$ is (wat betekent: er zijn ten hoogste l mismatches van een prefix van lengte $i+1$ met een string die op positie j eindigt). Wij veronderstellen dat $j > 0$ en $l > 0$

omdat wij voor deze waarden al weten hoe we $M^l[][j]$ berekenen. Bovendien kunnen wij ook stellen dat $i > 0$ omdat $M^l[0][j] = 1$ voor alle $l > 0$.

- als er ten hoogste $l-1$ mismatches van de prefix met lengte i tot positie $j-1$ zijn (dus $M^{l-1}[i-1][j-1] = 1$) dan is het zonder belang of het letterteken op positie j gematcht wordt – er zijn zeker ten hoogste l fouten tot positie j.
- als er precies l mismatches van de prefix met lengte i tot positie $j-1$ zijn (dan is $M^l[i-1][j-1] = 1$) **en** $t[j] = z[i]$ dan zijn er ten hoogste l mismatches tot positie j.

In andere gevallen is er duidelijk geen match met de vereiste eigenschappen en moet $M^l[i][j] = 0$ zijn. Wij moeten de matrices dus in een volgorde berekenen zodat de waarden voor $l-1$ en $j-1$ al bekend zijn.
De kolom $M^l[.][j]$ kunnen wij dus berekenen als

$$M^l[][j] = (1, 0, \ldots, 0) \quad | \quad \text{shift}(M^{l-1}[][j-1]) \quad | \quad (\text{shift}(M^l[][j-1]) \quad \& \quad C[t[j]][])$$

Hierbij kan shift() bv. dezelfde operatie zijn als $\text{shift}^1()$ of het kan een shift zijn die de 0-bit op positie 0 schuift. Door de operatie $(1, 0, \ldots, 0) \quad |$ wordt deze bit toch al op 1 gezet.

Oefening 58 *Werk ook dit algoritme uit met de betekenis van* 0 *en* 1 *in Definitie 14 gewisseld.*

Oefening 59 *Schrijf de pseudocode voor dit algoritme op.*

Oefening 60 *Pas het algoritme toe op* $z[]$ =`dank` *en* $t[]$ =`een_dansfeest` *met* 1 *toegelaten mismatch.*

Oefening 61 *Wij kunnen de definitie van* toegelaten fouten *in Definitie 14 ook zo beschrijven, dat er een perfecte match is met een woord dat wij uit het zoekwoord kunnen bouwen als wij ten hoogste k lettertekens door een ander letterteken vervangen. Stel dat het nu ook toegelaten is een letterteken uit de tekst te verwijderen (deletion) en dat dat ook als* 1 fout *telt (als de tekst dus bv.* `ik_ben_peter` *is dan zou dit algoritme voor de zoekstring* `peer` *zeggen dat er een match met 1 fout gevonden is). Geef de* **exacte** *definitie van* $\bar{M}^k[][]$ *voor dit concept en geef het algoritme om* $\bar{M}^k[][]$ *te berekenen.*

De editeerafstand

Wij hebben al verschillende stappen in de richting van benaderend stringmatching gedaan door wildcards of mismatches toe te laten. In dit deel willen wij het **eerst** over een speciaal geval hebben: het geval waar de rol van $z[]$ en $t[]$ symmetrisch is. Wij hebben hier dus niet één string die typisch kort is en een andere die lang is, maar ze zijn beide van ongeveer dezelfde lengte en wij willen weten *hoe verschillend* ze zijn. Je hebt bv. twee bestanden waarbij het ene bestand een gewijzigde versie van het andere bestand is en wilt weten hoe sterk het bestand gewijzigd werd. Dat wordt bv. door het Unix-commando *diff* berekend (waarbij *diff* ook nog zegt wat de wijzigingen zijn).

Als wij zeggen *hoe verschillend* dan hebben wij het over een maat – of een afstand. Hoe je de definitie van een afstand kiest, hangt er sterk van af wat de context is. In de biologie zijn de strings bv. vaak DNA structuren en de afstand hangt ervan af hoeveel mutaties nodig zijn om van het ene DNA naar het andere te gaan. Maar je kan er ook best rekening mee houden hoe waarschijnlijk de mutaties zijn. Dat geldt natuurlijk ook voor teksten: je kan tellen hoeveel *wijzigingen* nodig zijn om de ene tekst in de andere te veranderen, maar dan is nog altijd niet duidelijk welk soort *wijzigingen* bedoeld zijn. Als de afstand moet beschrijven hoe waarschijnlijk het is dat het woord $t[]$ in feite hetzelfde woord zou moeten zijn als $t'[]$ dan zouden toegelaten wijzigingen bv. het weglaten van een letterteken, het toevoegen van een letterteken of het verwisselen van een letterteken zijn. Maar je zou ook hier waarschijnlijkheden kunnen gebruiken. Misschien is het waarschijnlijker dat lettertekens verwisseld worden die naast elkaar op het toetsenbord staan dan lettertekens die niet naast elkaar staan – of is het waarschijnlijker dat de volgorde van twee lettertekens fout is (bv. *volgrode* in plaats van *volgorde*) dan dat twee lettertekens verwisseld zijn (bv. *vilgorde* in plaats van *volgorde*), etc.
De afstand die wij hier gebruiken, is dus gewoon één van vele mogelijke afstanden die allemaal in zekere omstandigheden zinvol kunnen zijn.

Definitie 15 *Voor de editeerafstand zijn 3 bewerkingen toegelaten:*

- *Het vervangen van een letterteken door een ander letterteken. Deze bewerking hangt nauw samen met de mismatches die wij in verband met het shift-AND algoritme al besproken hebben.*
- *Het verwijderen van een letterteken. Als het verwijderde letterteken positie i had, houden alle tekens die voor het verwijderen een positie $j < i$ hadden de positie en alle tekens die een positie $j > i$ hadden, hebben achteraf positie $j - 1$.*
- *Het toevoegen van een letterteken. Als het nieuwe letterteken na het toevoegen positie i heeft, houden alle tekens die voor het toevoegen een positie $j < i$ hadden de positie en alle tekens die een positie $j \geq i$ hadden, hebben achteraf positie $j + 1$.*

De editeerafstand $d(t, t')$ tussen twee strings $t[]$,$t'[]$ is het kleinst mogelijke aantal bewerkingen van dit soort toegepast op $t[]$ zodat het resultaat gelijk aan $t'[]$ is. Deze afstand wordt soms ook de Levenshtein afstand *genoemd.*

Wij zullen dit aan een voorbeeld illustreren. Wij gebruiken de strings $t =$`lessen` en $t' =$`feesten`

```
l e s s e n        (1) vervang l door f

f e s s e n        (2) vervang s op positie 3 door t
0 1 2 3 4 5

f e s t e n        (3) voeg nieuwe e op positie 1 toe
0 1 2 3 4 5

f e e s t e n
0 1 2 3 4 5 6
```

Er bestaat dus een reeks met 3 stappen van $t =$`lessen` naar $t' =$`feesten`. En omdat er geen kortere reeks bestaat (gemakkelijk om te zien), geldt $d(t, t') = 3$.

Opmerking 16 *Voor strings $t[], t'[], t''[]$ geldt:*

(i) $d(t, t') \geq 0$

(ii) $d(t, t') = 0$ *als en slechts als* $t = t'$

(iii) $d(t, t') = d(t', t)$

(iv) $d(t, t'') \leq d(t', t) + d(t', t'')$

(v) $d(t, t') \leq \max\{|t|, |t'|\}$

Eigenschap (v) zegt dat de editeerafstand goed gedefinieerd is (hij kan niet oneindig zijn) en de eigenschappen (i) t.e.m. (iv) rechtvaardigen het over een *afstand* of een *metriek* te spreken.

Bewijs: De meeste punten volgen onmiddellijk uit de definitie. Het enige punt waarover wij misschien een beetje meer moeten nadenken is (iii) en dat punt zullen wij hier expliciet bewijzen:

Als $d(t, t') = 0$ dan is $t = t'$ en dan geldt zeker dat $d(t, t') = d(t', t)$. Maar je kan elke bewerking b zodat b toegepast op t de string t' oplevert omkeren: als de bewerking is dat $t[i]$ vervangen wordt door letterteken x dan is $t'[i] = x$ en kunnen wij als omgekeerde bewerking $t'[i]$ vervangen door $t[i]$. Als een nieuw teken op positie i in $t[]$ wordt toegevoegd dan kunnen wij als omgekeerde bewerking teken $t'[i]$ verwijderen. Als teken $t[i]$ wordt verwijderd dan kunnen wij omgekeerd in t' het teken $t[i]$ op positie i toevoegen.

Stel dus dat $d(t, t') = n \geq 1$. Dan is er een reeks $t = t_0, t_1, \ldots t_{n-1}, t_n = t'$ met $d(t_i, t_{i+1}) = 1$ voor $0 \leq i < n$. Als wij de bewerkingen tussen twee opeenvolgende strings omkeren, hebben wij een reeks die t' in t omvormt. Dus $d(t', t) \leq n$. Maar als $d(t', t) < n$ zou op dezelfde manier volgen $d(t, t') < n$ – een tegenstrijdigheid. Dus geldt $d(t', t) = d(t, t')$.

■

Om de editeerafstand efficiënt te berekenen hebben wij nog een lemma nodig:

Lemma 17 *Gegeven $t[]$ en $t'[]$. Dan is er een reeks van $k = d(t, t')$ bewerkingen $b_1, \ldots, b_k$ die $t[]$ naar $t'[]$ wijzigt zodat als i_j de bij bewerking b_j betrokken index is, geldt $i_1 \leq i_2 \leq \cdots \leq i_k$.*

Dit betekent dat wij beginnend aan de linkerkant van de string de string kunnen wijzigen en dan doorgaan naar de rechterkant waarbij wij nooit reeds vroeger bezochte delen opnieuw moeten wijzigen. Dat is natuurlijk heel nuttig voor een algoritme! Het betekent dat wij de prefixen in volgorde van hun lengte wijzigen!

Bewijs: Dit kan je bewijzen door gewoon naar alle combinaties van bewerkingen te kijken waar eerst index i betrokken is en daarna index j waarbij $j < i$. In elk geval kan je ofwel bewijzen dat er een kortere reeks van bewerkingen is – wat een tegenstrijdigheid zou zijn – (bv. als je eerst een

teken op positie i toevoegt en het dan door een ander teken vervangt) of dat je de bewerkingen kan vervangen door 2 bewerkingen met indices in volgorde.

Maar dit bewijs bevat noch interessante ideeën noch is het moeilijk – wij zullen het hier dus niet geven.

■

Eén van de doelen van de lessen over algoritmen is ook in te kunnen schatten welke technieken het best toegepast kunnen worden om een probleem op te lossen. Soms heb je dan zo'n gevoel dat het ene probleem op een zekere manier op het andere lijkt en dus misschien dezelfde technieken toegepast kunnen worden. Misschien hebben sommigen van jullie de indruk dat dit probleem een beetje op het probleem lijkt waar wij volgorden van matrices moesten bepalen om zo efficiënt mogelijk te kunnen vermenigvuldigen. Beide hebben een gelijkaardige *lineaire* structuur...
Wij zullen – net zoals voor het matrixvermenigvuldigingsprobleem en het shift-AND algoritme – ook hier dynamisch programmeren toepassen. Wij berekenen in feite niet alleen de editeerafstand tussen de strings $t[]$ en $t'[]$, maar tussen alle prefixen van $t[]$ en $t'[]$. Hier wordt dan heel duidelijk hoe dicht dit algoritme bij het shift-AND algoritme staat. Het is alleen jammer dat hier niet alleen de waarden 0 en 1 kunnen voorkomen die het mogelijk gemaakt hebben de bewerkingen door bitvectoren te paralleliseren. Als $n = |t|$ en $m = |t'|$ dan schrijven wij voor $0 \leq i \leq n$ en $0 \leq j \leq m$ de notatie $d[i][j]$ voor de afstand tussen de prefix van lengte i van $t[]$ (dus $t[0], t[1], \ldots, t[i-1]$) en de prefix van lengte j van $t'[]$ (dus $t'[0], t'[1], \ldots, t'[j-1]$). De notatie duidt er al op dat wij dat in een 2-dimensionale array zullen bijhouden. De waarden $d[i][0] = i$ en $d[0][j] = j$ voor alle i, j zijn duidelijk omdat als één van de strings leeg is alle lettertekens ofwel verwijderd moeten worden ofwel toegevoegd moeten worden. Als wij veronderstellen dat wij op het moment dat wij $d[i][j]$ willen berekenen de waarden van $d[i'][j']$ voor alle tweetallen (i', j') die lexicografisch kleiner zijn dan (i, j) al kennen, dan kunnen wij als volgt argumenteren:

- als $t[i-1] = t'[j-1]$ dan kan het laatste letterteken ongewijzigd blijven en dus $d[i][j] \leq d[i-1][j-1]$. Zie Oefening 63.

Als $t[i-1] \neq t'[j-1]$ dan zijn er 3 bewerkingen die de laatste bewerking in een kortst mogelijke reeks kunnen zijn. Wij gebruiken hier de notatie $t[k]$ om het letterteken te beschrijven dat voor de wijzigingen op positie k staat, maar merk op dat het voor de laatste wijziging door toevoegingen of verwijderingen op een helemaal andere positie kan staan.

- $t[i-1]$ vervangen door $t'[j-1]$. Dan is $d[i][j] = d[i-1][j-1] + 1$.
- $t'[j-1]$ aan de (misschien gewijzigde) $t[0], t[1], ..., t[i-1]$ toevoegen. Dan was de gewijzigde string ervoor $t'[0], t'[1], \ldots, t'[j-2]$ en die werd bereikt door $t[0], t[1], \ldots, t[i-1]$ te wijzigen. Wij hebben dus $d[i][j] = d[i][j-1]+1$.
- $t[i-1]$ verwijderen. Dan wordt $t[0], t[1], \ldots, t[i-2]$ veranderd naar $t'[0], t'[1], \ldots, t'[j-1]$ en achteraf het overbodige letterteken $t[i-1]$ verwijderd. Dus geldt $d[i][j] = d[i-1][j] + 1$.

Deze observaties kunnen nu gebruikt worden om een algoritme in pseudocode te formuleren. Daarbij zijn sommige dingen voor de betere verstaanbaarheid anders opgeschreven dan in een efficiënt programma – soms weet je bv. al voordat je het minimum berekent wat het zal zijn (zie oefeningen).

Algoritme 7 *(Dynamisch programmeren om de editeerafstand te berekenen)*

```
editeerafstand(t[],t'[])
{
// het wordt verondersteld dat t,t' beide niet leeg zijn en dat de
// lengte |t|, resp. |t'| is. De editeerafstand tussen t en t' wordt
// teruggegeven.


for (i=0;i<=|t|;i++) d[i][0]=i;
for (j=0;j<=|t'|;j++) d[0][j]=j;

for (i=1;i<=|t|;i++)
{
  for (j=1;j<=|t'|;j++)
      {  // als k lettertekens beschouwd worden heeft het laatste
         // index k-1
         if (t[i-1]=t'[j-1]) wisselkost=0;
            else wisselkost=1;
         d[i][j] = min{d[i-1][j-1]+wisselkost,
                       d[i-1][j]+1, d[i][j-1]+1};
      }
}

return d[|t|][|t'|];
}
```

Als wij de werking van dit algoritme eens beschouwen aan de hand van ons voorbeeld $t[]$ =lessen en $t'[] = feesten$ dan krijgen wij de volgende tabel. De lijn toont aan hoe de waarden verkregen zijn en ook één van de minst kostelijke mogelijkheden om de string `lessen` te wijzigen om de string `feesten` te verkrijgen.

		f	e	e	s	t	e	n
	0	1	2	3	4	5	6	7
l	1	1	2	3	4	5	6	7
e	2	2	1	2	3	4	5	6
s	3	3	2	2	2	3	4	5
s	4	4	3	3	2	3	4	5
e	5	5	4	3	3	3	3	4
n	6	6	5	4	4	4	4	3

Omdat de tijd voor één doorloop van de binnenste lus duidelijk constant is, hebben wij het volgende resultaat:

Lemma 18 *De editeerafstand tussen twee strings van lengte n en m kan in tijd $O(n * m)$ berekend worden.*

Voor de echte implementatie is het natuurlijk belangrijk geheugen te besparen en misschien niet de hele matrix in het geheugen te houden. Los zeker Oefening 64 op!
Het is niet alleen interessant te weten wat de editafstand is, maar ook de bewerkingen te kennen om de ene string naar de andere te wijzigen. Als je bv. een heel groot bestand hebt die op de twee computers zit dan is het misschien beter als die op de ene computer gewijzigd wordt alleen de nodige bewerkingen naar de andere computer te sturen dan het hele bestand.
De bewerkingen moeten ook niet altijd alleen met lettertekens gebeuren. In het Unix-commando *diff* worden bv. lijnen als characters geïnterpreteerd en de bewerkingen zijn dan verwijderen, toevoegen of vervangen van lijnen... Er zijn dus talrijke variaties en toepassingen van deze algoritmen...

Oefening 62 *Gebruik het beschreven algoritme om de editeerafstand tussen* `t=roestig` *en* `t'=oesters` *te berekenen. Toon de tabel en duid aan hoe de getallen in de tabel werden berekend. Beschrijf ook de mogelijke kortste reeksen van bewerkingen.*

Oefening 63 *Stel dat* `t[]=t[0],t[1],. . . ,t[i]`, `t'[]=t'[0],t'[1],. . . ,t'[j]` *en* `t[i]=t'[j]`. *Schrijf* t_{i-1} *voor* `t[0],t[1],. . . ,t[i-1]` *en analoog* t'_{j-1} *voor* `t'[0],t'[1],. . . ,t'[j-1]`
Is dan altijd $d(t, t') = d(t_{i-1}, t'_{j-1})$*?*
Bewijs dat of geef een tegenvoorbeeld.

Oefening 64 *Herschrijf de pseudocode van Algoritme 7.* **Stel** *daarbij dat het antwoord op de vraag in Oefening 63* ja *is en let erop zo weinig mogelijk geheugen te gebruiken. De strings kunnen natuurlijk te lang zijn om de hele matrix* `d[][]` *efficiënt in het geheugen te houden!*

Oefening 65 *Bewijs gedetailleerd dat voor de volgende gevallen van twee opeenvolgende bewerkingen b_i, b_{i+1} die vervangen kunnen worden door minder bewerkingen of twee bewerkingen waarvan de tweede een teken met een groter index gebruikt dan de eerste.*

- *b_i verwijdert een teken op positie j, b_{i+1} voegt een teken op positie j toe.*
- *b_i voegt een teken op positie j toe, b_{i+1} verwijdert een teken op positie $j' < j$.*
- *b_i verwijdert een teken op positie j, b_{i+1} vervangt een teken op positie $j' < j$ door een ander teken.*

Bepaal precies de paren van bewerkingen die in een reeks van minimale lengte zoals in Lemma 17 – dus met stijgende indices – gelijke indices moeten hebben en niet door even veel of minder bewerkingen met strict stijgende indices vervangen kunnen worden. Hier is een redenering die niet alle details geeft (dus de indices, etc) voldoende.

Oefening 66 *Definieer een nieuwe afstand $d'()$ door behalve de drie al gekende bewerkingen ook nog een nieuwe bewerking toe te laten: het verwisselen van twee op elkaar volgende lettertekens. Met* `t=lui` *en* `t'=liu` *is dus $d(t,t') = 2$ en $d'(t,t') = 1$. Beschrijf een algoritme om $d'()$ te berekenen in pseudocode en geef uitleg waarom het algoritme juist is.*

Oefening 67 *Wijzig de pseudocode van het algoritme zo dat ook de nodige bewerkingen om $t[]$ naar $t'[]$ te wijzigen worden bijgehouden.*

Oefening 68 *Gegeven 2 woorden $z_1[]$ en $z_2[]$. Het doel is deze twee woorden zo te wijzigen dat ze gelijk zijn. Maar de enige bewerking is dat je lettertekens tot $z_1[]$ en $z_2[]$ mag toevoegen. Beschrijf een efficiënt algoritme dat de minimale lengte van een woord $z'[]$ bepaalt zodat $z_1[]$ en $z_2[]$ omgevormd kunnen worden naar $z'[]$.*

De beste benaderende match

Nu zullen wij het terug over het probleem hebben matches van een korte string in teksten te vinden – alleen dat wij het hier over de beste benaderende match hebben en geen exacte matches. Hoe wij *benaderende matches* in de betekenis van *een gegeven aantal mismatches* efficiënt kunnen berekenen, hebben wij al gezien. Hier zullen wij het er nu over hebben hoe je voor een gegeven string $z[]$ en een gegeven tekst $t[]$ de deelstring van $t[]$ met de kleinste editeerafstand kan vinden.
Het eerste idee zou misschien zijn het algoritme voor de editeerafstand tussen twee strings op alle deelstrings van $t[]$ toe te passen. Maar met $m = |z|$ en $n = |t|$ zijn er $(\sum_{i=0}^{n-1}(n-i))+1 = n(n+1)/2+1$ deelstrings van $t[]$ (op positie i starten $n-i$ mogelijke niet lege strings en dan nog de ene lege string 1 keer geteld). Om $z[]$ met een string van lengte l te vergelijken hebben wij $O(m*l)$ stappen nodig en als wij dat voor alle mogelijke lengten l opsommen, krijgen wij een totale kost van $O(m*n^3)$. En – veronderstellend dat het een goede bovengrens is – betekent dat natuurlijk dat een hierop gebaseerd programma vrij traag zal zijn voor grote teksten.

Oefening 69 *Als je echt naar* **alle** *deelstrings van $t[]$ kijkt, is dat natuurlijk niet bijzonder slim. Je ziet onmiddellijk dat je naar heel lange deelstrings niet moet kijken (waarom?). Maar wat is "*heel lang*"?*
Geef een functie $f()$ zodat voor alle $z[]$ en $t[]$ een deelstring in $t[]$ bestaat met lengte ten hoogste $f(|z|)$ en minimale editeerafstand van $z[]$. Kies $f()$ minimaal – dus zo dat er in feite $z[]$ en $t[]$ bestaan waarin geen kortere beste benaderende matches zijn.

Maar aan de andere kant zien jullie ook dat bij deze manier van doen veel deelresultaten altijd opnieuw berekend worden – de afstand voor dezelfde prefix wordt bv. voor elk woord berekend waarvan het een prefix is. En dan is natuurlijk duidelijk dat het veel sneller kan door opnieuw het principe van dynamisch programmeren toe te passen. Maar omdat er dan nog altijd $O(m * n^2)$ editeerafstanden berekend en opgeslagen worden zal de complexiteit nog altijd ten minste $O(m * n^2)$ zijn.
Wij gaan dus nog 1 stap verder: voor de j-de letter in de tekst (dus $t[j-1]$) en een lengte i van een prefix slaan wij niet de afstanden van $z[0], \ldots, z[i-1]$ van alle deelwoorden van $t[]$ op die op positie $j-1$ eindigen, maar alleen de minimale editeerafstand van een deelwoord dat er eindigt.
Of precies:

$$D[i][j] = \min\{d((z[0], \ldots, z[i-1]), (t[l], \ldots, t[j-1]))|0 \leq l \leq j\}$$

waarbij $(z[0], \ldots, z[-1])$ en $(t[j], \ldots, t[j-1])$ lege woorden zijn. Zie ook hier Oefening 63.
Ook hier zijn de eerste rij en de eerste kolom van $D[][]$ gemakkelijk om te berekenen:
$D[i][0] = i$ voor alle i omdat alle i lettertekens verwijderd moeten worden.
$D[0][j] = 0$ voor alle j omdat de beste match van de lege string natuurlijk met de lege string is en de beste afstand dus 0 is.
Volledig analoog met het berekenen van de editeerafstand tussen twee strings kunnen wij nu voor $i > 0$ en $j > 0$ argumenteren:

$$D[i][j] = \min\{D[i-1][j] + 1, D[i][j-1] + 1, D[i-1][j-1] + g(i,j)\}$$

waarbij $g(i,j) = 0$ als $z[i-1] = t[j-1]$ en anders 1.
Als wij deze tabel ingevuld hebben, moeten wij alleen nog de kleinste waarde in de laatste rij zoeken (dus de kleinste afstand die het hele woord $z[]$ heeft). Dat is de gezochte minimale afstand.

Algoritme 8 *(Dynamisch programmeren om de kleinste editeerafstand van een deelwoord te berekenen)*

```
shortest_editeerafstand_deelwoord(z[],t[])
{
// het wordt verondersteld dat z,t beide niet leeg zijn en dat de
// lengte |z|, resp. |t| is. De kleinste editeerafstand van z met
// een deelwoord van t wordt teruggegeven.
```

```
for (i=0;i<=|z|;i++) D[i][0]=i;
for (j=0;j<=|t|;j++) D[0][j]=0;

for (i=1;i<=|z|;i++)
{
  for (j=1;j<=|t|;j++)
      {  // als k lettertekens beschouwd worden, heeft het laatste
         // index k-1
         if (z[i-1]=t[j-1]) wisselkost=0;
            else wisselkost=1;
         D[i][j] = min{D[i-1][j-1]+wisselkost,
                       D[i-1][j]+1, D[i][j-1]+1};
      }
}

best=oneindig;
for (i=0;i<=|t|;i++) if (D[|z|][i]<best) best=D[|z|][i];
return best;
}
```

Merk op dat dit algoritme bijna identiek is aan Algoritme 7. Alleen de initialisatie van de eerste rij en eerste kolom verschillen en op het einde wordt nog het kleinste getal in de laatste rij gezocht. Wij hebben dus onmiddellijk:

Lemma 19 *De kleinste editeerafstand tussen een string $z[]$ van lengte n en een deelwoord van een string $t[]$ van lengte m kan in tijd $O(n*m)$ berekend worden.*

Maar ook hier is de implementatie en het besparen van geheugen belangrijk. Zie Oefening 75.
De werking van het algoritme voor het voorbeeld $z[]$ =`examen` en
$t[]$ =`de_armen_vouwen` kan in de volgende tabel gezien worden.

		d	e	_	a	r	m	e	n	_	v	o	u	w	e	n
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
e	1	1	0	1	1	1	1	0	1	1	1	1	1	1	0	1
x	2	2	1	1	2	2	2	1	1	2	2	2	2	2	1	1
a	3	3	2	2	1	2	3	2	2	2	3	3	3	3	2	2
m	4	4	3	3	2	2	2	3	3	3	3	4	4	4	3	3
e	5	5	4	4	3	3	3	2	3	4	4	4	5	5	4	4
n	6	6	5	5	4	4	4	3	2	3	4	5	5	6	5	4

Oefening 70 *Pas Algoritme 8 toe op het woord $z[]$ =`zoeken` en*

$t[]$ =`koekjes_kweken`. *Bouw de tabel op en toon aan wat de beste match is en welke wijzigingen nodig zijn om* `zoeken` *in deze match te wijzigen.*

Oefening 71 *Pas Algoritme 8 toe op het woord* $z[]$ =`test` *en* $t[]$ =`beste_tekst`. *Bouw de tabel op en toon aan wat de beste matches zijn en welke wijzigingen nodig zijn om* `test` *in deze matches te wijzigen.*

Oefening 72 *Beschrijf een algoritme in pseudocode dat als* $z[]$, $t[]$ *en de tabel die door Algoritme 8 opgebouwd wordt ingevoerd worden de beste match teruggeeft.*

Oefening 73 *Beschrijf een algoritme in pseudocode dat voor een woord* $z[]$ *van lengte* m *en een tekst* $t[]$ *van lengte* n *alle editeerafstanden van* $z[]$ *met deelwoorden uit* $t[]$ *berekent in tijd* $O(m * n^2)$. *Geef uitleg over de complexiteit van jouw algoritme.*

Oefening 74 *Stel dat de kost van een verwijder-, toevoeg- en wisselbewerking om de ene string in de andere om te vormen niet constant 1 is maar dat elke van de 3 bewerkingen een individuele (positieve) kost heeft. Welke resultaten zijn nog altijd geldig en hoe kan je voor dit probleem een efficiënt algoritme schrijven?*

Oefening 75 *Herschrijf de pseudocode van Algoritme 8.* **Stel** *daarbij dat het antwoord op de vraag in Oefening 63* ja *is en let erop zo weinig mogelijk geheugen te gebruiken. Vooral hier kan de tekst natuurlijk veel te lang zijn om de hele matrix* `D[][]` *in het geheugen te houden! Welke van de twee for-lussen is beter de buitenste lus?*

4 Compressiealgoritmen

Stel dat jullie de string
3, 14159265358979323846264338327950288419716939937510582097494459 23
willen comprimeren. Dan zouden jullie misschien zeggen dat dat een benadering van Pi is en dus voor de meeste toepassingen zeker 3, 141592653 voldoende precies is (en gelijk hebben). Wat jullie dan doen, is compressie *met verlies*. Het resultaat van de compressie laat het niet toe de oorspronkelijke data te herstellen. Om te beslissen wat belangrijk is en wat niet moet je weten wat de data voorstelt en hoe hij geïnterpreteerd wordt. Als iemand bv. alleen in elk vijfde cijfer geïnteresseerd was, zou deze manier van compressie heel slecht zijn: je houdt overbodige informatie bij en gooit belangrijke informatie weg. Belangrijke formaten zoals jpg of mp3 zijn ook compressie met verlies: je kan de oorspronkelijke foto of het oorspronkelijke geluid niet reconstrueren. De formaten zijn erop gebaseerd dat geweten is hoe ons zicht en gehoor functioneren. Omdat het voor dergelijke formaten ook vooral belangrijk is te verstaan hoe de waarneming werkt (wat niet ons onderwerp is) zullen wij die niet bespreken maar ons alleen bezig houden met compressie zonder verlies – dus met compressiealgoritmen die het toelaten elke bit van het originele bestand te reconstrueren. Een eerste vraag zou natuurlijk zijn of er een compressiealgoritme is dat elk bestand kan comprimeren en het antwoord is duidelijk *nee*: Omdat wij elk bestand willen reconstrueren moet een compressiealgoritme een bijectie definiëren tussen de gecomprimeerde bestanden en de bestanden die gecomprimeerd worden. Als twee bestanden op hetzelfde bestand afgebeeld zouden worden, zou je het oorspronkelijke bestand niet kunnen reconstrueren! Maar hoe minder bytes hoe minder mogelijke bestanden – dus kan dat zeker niet. En even gemakkelijk kunnen wij zien dat als er ook maar één bestand is dat echt gecomprimeerd wordt (dus waar het resultaat korter is) dat er dan ook tenminste één bestand moet zijn dat langer wordt.

Oefening 76

- *Bewijs expliciet dat als er een routine voor het comprimeren van bestanden is die een bestand echt comprimeert dat er dan ook ten minste één bestand is dat door deze routine langer gemaakt wordt.*
- *Een een beetje vage uitspraak: elk bestand met ten minste 3 bytes kan gecomprimeerd worden. Schrijf deze uitspraak op een* **precieze** *manier met kwantoren (voor elk bestand met ten minste 3 bytes bestaat...) en bewijs de precieze uitspraak of vind een tegenvoorbeeld.*

Maar hoeveel bestanden van een gegeven lengte kan je met een gegeven compressiealgoritme comprimeren? Er zijn

$$\sum_{i=0}^{n} 256^i = \frac{1}{255} 255 \sum_{i=0}^{n} 256^i = \frac{1}{255}(256^{n+1} - 1)$$

bestanden met ten hoogste n bytes. Als wij nu bestanden met $n+1$ bytes willen comprimeren (waarbij wij altijd ten minste één byte willen winnen) dan is het beeld van een gecomprimeerd bestand een bestand met ten hoogste n bytes. Omdat er 256^{n+1} bestanden met $n+1$ bytes maar alleen maar ongeveer $256^{n+1}/255$ bestanden met ten hoogste n bytes zijn, kan dus **ten hoogste**

1/255de van de bestanden gecomprimeerd worden – en daarvoor moeten wij dan nog *betalen* omdat andere bestanden langer worden! Dat klinkt niet goed...
Maar onze ervaring is helemaal anders: de meeste bestanden waarop wij compressiealgoritmen zoals gzip, zip, bzip2, etc. toepassen, kunnen gecomprimeerd worden – in sommige gevallen zelfs heel sterk. De reden lijkt op de reden waarom metaheuristieken werken: als wij het maximum van een toevallige functie zouden moeten vinden, zouden metaheuristieken nutteloos zijn. Zij werken goed omdat de problemen waarmee mensen meestal bezig zijn structuur vertonen – ze zijn helemaal niet toevallig! En hier is het hetzelfde: de bestanden die wij normaal willen comprimeren zijn niet toevallig. Het zijn bv. tekstbestanden met woorden uit een zekere taal. In dergelijke bestanden komen sommige letters veel vaker voor dan anderen – en dat is een structuur die toevallige bestanden niet tonen. Dergelijke structuren maken het mogelijk bestanden te comprimeren. Alle technieken die wij zullen zien, zullen slecht presteren op bestanden die gewoon toevallige bits bevatten. En wij hebben net getoond dat dat ook niet anders kan!

Oefening 77 *Wij hebben aangetoond dat je met een gegeven compressiealgoritme ten hoogste ongeveer* 1/255 *van alle bestanden kan comprimeren. Maar dan hebben wij aanvaard dat het misschien een compressie van maar 1 byte is.*

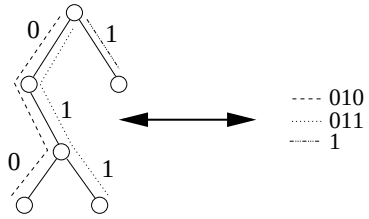
- *Bereken hoe groot het aandeel is van bestanden die door een vast compressiealgoritme met ten minste* 100 *bytes gecomprimeerd kunnen worden.*
- *Geef een bovengrens voor de kans dat een vast compressiealgoritme een toevallig bestand van* $5MB$ *of meer met ten minste de helft kan comprimeren*

Oefening 78 *Wij hebben net gezien dat je maar* 1/255*ste van alle bestanden kan comprimeren. Stel nu dat elke computer ongelofelijk veel ruimte ter beschikking heeft en op voorhand al grote delen kan opslaan die in bestanden kunnen voorkomen (dus heel grote voorgëimplementeerde woordenboeken – bv. alle bestanden t.e.m. 500 bytes) zodat je op de volgende manier kan comprimeren: je slaat gewoon de index van de onderdelen op en stuurt die door. De index van de woorden kan je natuurlijk nog eens met andere compressiealgoritmen bewerken. Bepaal voor deze manier van comprimeren een bovengrens voor het aantal bestanden dat je kan comprimeren. Resultaten en bewijzen uit de les mogen geciteerd worden en hoeven niet herhaald te worden.*

Een eerste aanzet:
Stel dat je niet meer een vaste lengte voor alle codewoorden (bv. bytes) wil hebben, maar variabele lengte van de codewoorden toelaat. Je kan bv. bytes coderen door nullen in het begin weg te laten. Dus bv. `00011111` als `11111`. Dat lijkt al ruimte te besparen, maar het probleem is natuurlijk dat je op deze manier niet meer kan herkennen waar het ene woord stopt en het volgende begint als je ze in een bestand achter elkaar schrijft. Waar zijn bv. de grenzen in `1111101101010111010111`?
Eén manier om dit op te lossen is het gebruik van prefix-codes:

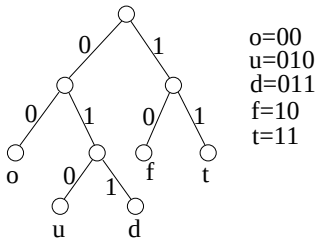
Definitie 20 *Een verzameling M van woorden (met tekens uit een zeker alfabet) heet een prefix-code als voor elk woord $w = w_1, w_2, \ldots, w_m$ geen woord $w' = w'_1, w'_2, \ldots, w'_n$ bestaat met $n > m$ en $w_i = w'_i$ voor alle $1 \leq i \leq m$.*

Je kan dus gemakkelijk herkennen wanneer je het einde van een woord hebt bereikt – gewoon omdat als je doorgaat er geen element in de verzameling zit dat door de langere code gecodeerd zou kunnen zijn! In de toekomst zal het alfabet waarover wij het hebben gewoon $\{0, 1\}$ zijn – wij hebben het dus over woorden van bits.
Prefixcodes kan je altijd in de vorm van een boom voorstellen waar de bogen gelabeld zijn met letters uit het alfabet. De codewoorden corresponderen dan met de labellingen langs de paden van de wortel naar de bladeren.



Figuur 24: Een kleine prefixboom en de bijbehorende codewoorden.

De prefixeigenschap correspondeert dan met het bereiken van een blad: er is geen langer woord met hetzelfde begin omdat het blad anders een kind had (en dan zou het geen blad zijn).
De rijen van nullen en enen staan normaal voor tekens – zoals bv. de ASCII tekens beschreven worden door rijen van nullen en enen met lengte 8. In een boom kunnen wij dat aanduiden door de letters aan de bladeren te schrijven.



Figuur 25: Een kleine prefixboom, de bijbehorende codewoorden en tekens.

Wat zou bv. de string 00010011 coderen als wij de code nemen die in Figuur 25 beschreven is?
Je begint met het eerste bit en volgt de takken in de boom. Als je een blad bereikt, schrijf je het teken dat bij dit blad behoort op. Dan is één codewoord gedecodeerd en beginnen wij met de volgende bit opnieuw bij de wortel om het volgende teken op dezelfde manier te decoderen. Als je dat toepast tot alle bits gelezen zijn, heb je het hele woord gedecodeerd.
In het voorbeeld lezen wij eerst 00 en hebben het blad o bereikt. Dan beginnen wij opnieuw met de wortel en bereiken het blad u na 010, beginnen opnieuw en bereiken d na 011. De string die er gecodeerd was was dus het woord *oud*.

Oefening 79 *Wat is het juiste antwoord op de vraag wat de string* 100001011 *codeert als wij de code nemen die in Figuur 25 beschreven is?*

Als wij deze manier van coderen toepassen, kunnen de codes voor verschillende tekens ook een verschillende lengte hebben. Als wij een code willen hebben die zo kort mogelijk is, is het dus verstandig korte codes voor tekens te gebruiken die vaak opduiken en de langere codes voor tekens die minder vaak gebruikt worden. Om een voordeel aan deze techniek te hebben, moet je wel bestanden hebben waar de tekens niet *toevallig met gelijke kans* verdeeld zijn, maar die structuur vertonen – waar sommige tekens dus veel vaker opduiken dan anderen.
Als wij een alfabet A hebben en voor $x \in A$ het aantal keren dat x in het bestand zit dat wij willen comprimeren $a(x)$ is en de lengte van zijn code $l(x)$ dan is de lengte van het gecodeerde bestand $\sum_{x \in A}(a(x) * l(x))$ bits. Wij hebben geen invloed op de functie $a()$ – het bestand is gegeven – maar wij kunnen de codering kiezen en dus invloed op de $l(x)$ uitoefenen.
Ons doel is dus de codering op een manier te kiezen dat $\sum_{x \in A}(a(x) * l(x))$ minimaal is.

4.1 Huffman codering

Huffman codering is een gretige manier om een code te construeren. Maar terwijl gretige algoritmen vaak alleen maar benaderende resultaten opleveren kunnen wij in dit geval aantonen dat het resultaat optimaal is.
Huffman codering werkt op de volgende manier:

- Doorloop eerst het bestand en bepaal voor elk teken x de frequentie $a(x)$.

Nu zullen wij de coderingsboom stap voor stap opbouwen. De boom zal niet uniek bepaald zijn, maar alle bomen die je op deze manier krijgt, zijn even goed. Wij veronderstellen dat er ten minste twee verschillende lettertekens in de tekst staan:

Algoritme 9 Huffman codering:

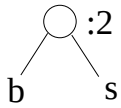
- *maak een verzameling van bomen aan waarin elke letter met een strikt positieve frequentie een boom met één top vormt.*
- *herhaal de volgende stap tot er nog maar één enkele boom in de verzameling zit:*
  - *Kies de wortel X van een boom met de kleinste frequentie en dan de wortel Y van een boom met de kleinste frequentie van de bomen die niet X zijn. Maak één nieuwe boom Z door van de wortels van X en Y de twee kinderen van één nieuwe wortel te maken. De frequentie $a(Z)$ wordt gedefinieerd als $a(Z) = a(X) + a(Y)$.*

De lettertekens zijn achteraf de bladeren van deze boom.

Als voorbeeld zullen wij nu de korte tekst **boosdoeer** coderen waar de lesgever een n is vergeten:

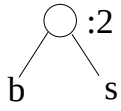
b:1 o:3 e:2 s:1 d:1 r:1

eerste stap:

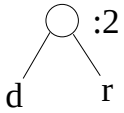


o:3 e:2 d:1 r:1

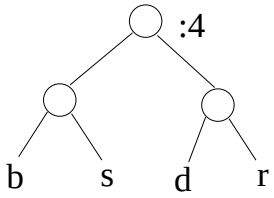
tweede stap:



o:3 e:2

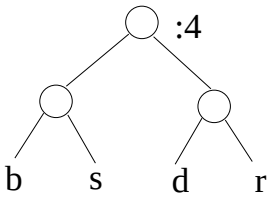


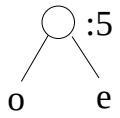
derde stap:
(niet uniek!)



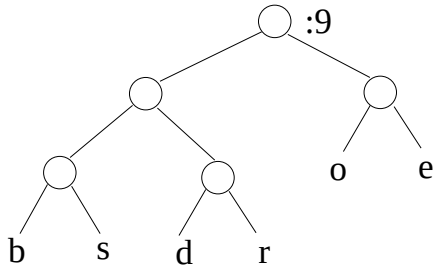
o:3 e:2

vierde stap:





vijfde stap:



Deze laatste boom die alle lettertekens bevat is de Huffman-boom van de code.
De codes van de lettertekens zijn dus:

b: 000

o: 10

e: 11

s: 001

d: 010

r: 011

De code van **boosdoeer** is de Huffman-boom samen met de string 000101000101010111011.

Oefening 80 *Bereken de Huffman codering voor* bananenboom.

In feite is dat een gretig algoritme, en wij hebben gezien dat gretige algoritmen soms alleen benaderingen berekenen – en in sommige gevallen zelfs slechte benaderingen. Maar in dit geval krijgen wij inderdaad de optimale oplossing:

Stelling 21 *Voor een gegeven alfabet $x_1, \ldots, x_k$, $k \geq 2$ en frequenties $a(x_i)$ berekent het Huffman algoritme een prefixcode (boom) T waarvoor*

$$Q(T) = \sum_{i=1}^{k} l(x_i) * a(x_i)$$

minimaal is, waarbij $l(x_i)$ de lengte van het codewoord voor x_i is.

Bewijs: Let op: voor het Huffman algoritme zijn de letters in het alfabet natuurlijk kleine boompjes. Wij zullen het dus altijd over bomen hebben.

Opmerking: In een optimale boom met ten minste 3 toppen heeft elke top ofwel 0 ofwel 2 kinderen. Toppen met maar 1 kind zou je gewoon kunnen verwijderen (en de ouder en het kind met elkaar verbinden of de top alleen maar verwijderen als het de wortel was) en het resultaat zou een betere boom zijn: voor alle bladeren x blijft $l(x)$ gelijk of wordt kleiner. Dat betekent ook dat elke top (behalve de wortel) een broer heeft (een andere top met dezelfde ouder).

Wij gebruiken inductie in de grootte s van het alfabet. Als er maar twee lettertekens zijn, is het resultaat uniek (er is maar één mogelijke optimale boom) – dat is dus triviaal.

Stel nu dat bewezen is dat het Huffman algoritme voor s lettertekens altijd een optimale boom bouwt en dat er nu $s+1$ lettertekens zijn. Stel bovendien dat het algoritme als de boom T_{s+1} voor $s+1$ lettertekens opgebouwd wordt eerst de laatste 2 letters in een boompje $T_{x_s,x_{s+1}}$ samenvoegt (anders moeten de letters gewoon hernoemd worden) en er de frequentie $a(T_{x_s,x_{s+1}}) = a(x_s) + a(x_{s+1})$ aan toekent. Deze twee hebben dus de kleinste frequenties (maar ze zijn misschien niet de enigen met deze eigenschap).

Dan bouwt het algoritme een optimale boom T_s voor de verzameling van bomen $x_1, \ldots, x_{s-1}, T_{x_s,x_{s+1}}$ als wij $T_{x_s,x_{s+1}}$ als één letterteken beschouwen. Stel nu dat de boom T_{s+1} die voor $x_1, \ldots, x_{s+1}$ gebouwd wordt – dat is dezelfde boom maar met $T_{x_s,x_{s+1}}$ als boom van diepte 1 in plaats van een enkele top – niet optimaal is maar dat er een betere optimale boom T_{opt} bestaat.

Opmerking: Wij mogen stellen dat x_s, x_{s+1} broers zijn in T_{opt} en op het laagste niveau zitten:

Neem twee broers y,y' op het laagste niveau. Stel dat $y \notin \{x_s, x_{s+1}\}$ en $x \in \{x_s, x_{s+1}\}$, $x \notin \{y, y'\}$. Dan kunnen wij x met y wisselen en krijgen en ten minste even goede boom: Wij hebben $l(y) \geq l(x)$ (omdat y op het

laagste niveau zit) en $a(y) \geq a(x)$ (omdat x één van de twee toppen met laagste frequentie is).

Voor de nieuwe boom T'_{opt} na het wisselen zou dus gelden:

$Q(T'_{opt}) = Q(T_{opt}) - ((a(x)l(x)) + a(y)l(y)) + ((a(x)l(y)) + a(y)l(x)) = Q(T_{opt}) - (a(y) - a(x))(l(y) - l(x)) \leq Q(T_{opt})$

De nieuwe boom zou dus ten minste even goed zijn. Wij kunnen dus x_s, x_{s+1} met andere toppen wisselen zodat aan de voorwaarden uit de opmerking voldaan is.

Als wij in deze boom nu x_s, x_{s+1} en de ouder door één top z met frequentie $a(x_s) + a(x_{s+1})$ vervangen dan geldt voor de zo ontstane boom $T_{opt,s}$:

$Q(T_{opt,s}) = Q(T_{opt}) - (a(x_s) + a(x_{s+1})) < Q(T_{s+1}) - (a(x_s) + a(x_{s+1})) = Q(T_s)$

Maar dan zou T_s niet optimaal zijn wat een tegenstrijdigheid met de inductiehypothese is.

■

Je kan een tekst dus coderen door eerst de boom op te slaan die de code beschrijft en daardoor vastlegt hoe je de tekst moet decoderen, en achteraf de code volgens deze boom. Over de manier waarop je de boom het best opslaat, zullen wij het hier niet hebben – compressie is alleen maar nuttig voor grote bestanden en dan is het kleine en voor een gegeven alfabet constante deel voor de boom zonder veel belang.
Wat wij Huffman codering genoemd hebben, kan je ook *statische Huffman codering* noemen. Wij gaan ervan uit dat je de frequenties van elk letterteken in het bestand kent. Aan de ene kant betekent dat natuurlijk dat je het bestand twee keer moet lezen – één keer om de frequenties te bepalen en één keer om het te comprimeren. Dat is natuurlijk niet ideaal. Maar soms is het zelfs onmogelijk: als je geen bestand maar een stream van data wil comprimeren dan moet dat *online* gebeuren – je **kan** de stream niet helemaal lezen en dan nog een keer opvragen. Daarvoor kan je adaptieve Huffman codering gebruiken. Die zullen wij in het volgende hoofdstuk zien.

Oefening 81 *Construeer de Huffman code van de volgende tekst:*

`zaken_nemen_geen_keer`

Oefening 82 *Stel een manier voor om statische Huffman codering voor streams te gebruiken. Wat zijn de voordelen en nadelen in vergelijking met statische Huffman codering als je die op de hele verzameling van data kan toepassen?*

Oefening 83 *Een geautomatiseerd systeem om goederen in een opslagplaats op te slaan kan ook onderafdelingen van het magazijn samenvoegen – maar nooit meer dan 2 tegelijk.*
De tijd die het nodig heeft om deze onderafdelingen samen te vatten is $c*(d_1+d_2)$ als d_1 het aantal goederen in afdeling 1 is en d_2 het aantal goederen in afdeling 2.
Een groot bedrijf wil nu al zijn n onderafdelingen tot één grote afdeling samenvatten. De hoeveelheden $d_1, \ldots, d_n$ van goederen in de afdelingen zijn gekend.

Geef een $O(n \log n)$ algoritme dat een volgorde bepaalt waarop de onderafdelingen zo snel mogelijk samengelegd kunnen worden.

Oefening 84 *Gegeven een vast compressiealgoritme A. De taak is een bestand van n MByte te vinden dat door het algoritme* **niet** *gecomprimeerd kan worden. Geef een algoritme dat zo'n bestand construeert. Welk soort algoritme zou je voorstellen? Stel dat het algoritme A in tijd $O(n)$ werkt. Hoeveel tijd zou je verwachten dat jouw algoritme vraagt – kan het even snel?*

Oefening 85 *Bediscussieer de manier waarop je tijdens de Huffman codering altijd de twee boompjes met de minimale frequentie vindt. Welke datastructuren zou je kunnen toepassen?*

Stel dat je op de volgende manier werkt:
Je houdt twee lijsten bij: lijst (a) bevat in het begin de tekens samen met hun frequenties – dat zijn dus ook kleine boompjes. Je moet de lijst in stijgende volgorde van de frequentie sorteren. De lijst (b) gaat de gebouwde boompjes bevatten en is in het begin leeg.
Als je nu moet beslissen welke twee boompjes je moet samenvoegen om de volgende boom te bouwen, kies je 2 keer het kleinste van de elementen die in de lijsten vooraan staan en verwijder je het uit zijn lijst. Dan vorm je daarvan de nieuwe boom en schrijf je hem op het einde van lijst (b).
Toon aan dat dit algoritme correct werkt – dus: dat je op deze manier altijd de 2 bomen met de kleinste frequenties samenvoegt.
Hoeveel tijd vraagt dit algoritme voor n tekens met gegeven frequenties ?

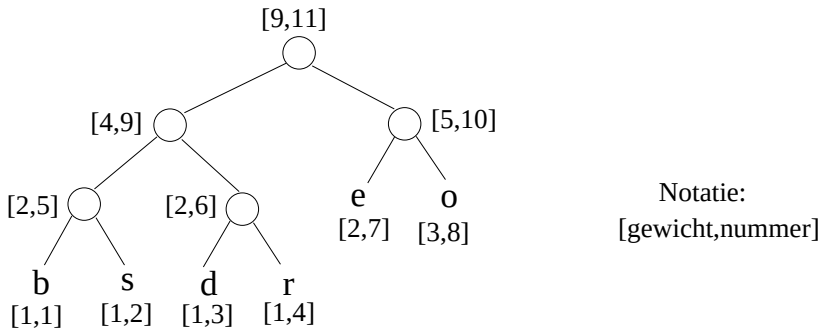
Oefening 86 *Stel dat je niet één groot bestand moet sorteren, maar n al bestaande gesorteerde bestanden $b_1, \ldots, b_n$ tot één groot gesorteerd bestand moet samenvoegen. Voor $1 \leq i \leq n$ bevat bestand b_i l_i sleutels en je kan de bestanden niet tegelijk mergen maar je kan altijd alleen maar 2 bestanden mergen tot een nieuw bestand. Als de oude bestanden l_i en l_j sleutels bevatten, bevat het nieuwe bestand dus $l_i + l_j$ sleutels en de kost van deze mergebewerking is $l_i + l_j$.*

- *Geef een voorbeeld dat toont dat de kost om alle bestanden te mergen afhankelijk is van de volgorde waarop je de bestanden mergt.*
- *Beschrijf een efficiënt algoritme om de optimale (goedkoopste) volgorde te bepalen.*
- *Wat is de complexiteit van dit algoritme?*
- *Toon aan dat jouw algoritme inderdaad de optimale volgorde vindt.*

Wij hebben *in zekere zin* bewezen dat *Huffman codering optimaal is.* Maar wat betekent dat precies? Is er geen betere prefixcode om iets op te slaan? Daarvoor moeten wij precies kijken wat de voorwaarden van de stelling waren: één van de voorwaarden was dat de code **vast** is, dat dus dezelfde code voor de hele te coderen tekst gebruikt wordt.

4.1.1 Adaptive (Dynamic) Huffman Coding

Een nadeel van Huffman coding is dat je de tekst twee keer moet lezen – eerst om de boom te berekenen en achteraf nog eens om de tekst te coderen. Het wordt soms ook two-pass Huffman algoritme genoemd. Het streamen van data kan op deze manier duidelijk niet. Het feit dat je de Huffman-boom moet doorsturen wordt soms ook als nadeel genoemd, maar voor grote hoeveelheden data is het aantal bytes dat voor de Huffman boom nodig is natuurlijk verwaarloosbaar.
Adaptive Huffman Coding overloopt de tekst maar één keer en kan dus ook voor streamen gebruikt worden. Als de tekst $t_1, \ldots, t_n$ is dan wordt voor $i > 1$ het teken t_i gecodeerd door middel van een optimale Huffman boom voor de tekst $nng, t_1, \ldots, t_{i-1}$, waarbij nng een speciaal teken is dat voor *"nog niet gezien"* staat en de mogelijkheid biedt nieuwe tekens toe te voegen. Als t_i gecodeerd of gedecodeerd is, wordt het gewicht van teken t_i in de boom met 1 verhoogd en de boom als nodig gewijzigd tot een optimale boom voor de tekens $nng, t_1, \ldots, t_i$. De boom is dan klaar om teken t_{i+1} te coderen of te decoderen. Als de structuur van de te coderen tekst sterk verandert – bv. als een tar-bestand met veel verschillende types bestanden gecomprimeerd moet worden – kan deze code dus in in het begin zelfs een efficiëntere codering gebruiken dan de statische Huffman coding die altijd ook rekening houdt met tekens die later vaak voorkomen.
De basis van adaptive Huffman coding werd door Faller (1973) en Gallager (1978) gelegd. Wij zullen het algoritme van Knuth (1985) zien, dat sommige verbeteringen bevat en FGK-algoritme (Faller-Gallager-Knuth) genoemd wordt. In zijn artikel bespreekt Knuth ook variaties, zoals een *sliding window* waarbij niet alleen het gewicht van het nieuwe teken verhoogd wordt, maar ook het gewicht van oude tekens verlaagd wordt, zodat de boom altijd optimaal is voor de laatste s tekens, waarbij s een op voorhand vastgelegd getal is. Vitter heeft later (1987) een nog beter algoritme voorgesteld dat zeker de moeite is om te bestuderen. Het FGK-algoritme is intuïtief en een goede basis om dan ook naar de verschillen met het Vitter-algoritme te kijken – maar in deze les zullen wij dat niet doen.
Als wij de boom wijzigen, moeten wij natuurlijk zeker zijn dat de nieuwe boom optimaal is. Daarvoor gebruiken wij iets dat wij al bewezen hebben (Stelling 21): een boom die het resultaat van het statische Huffman algoritme kan zijn, is optimaal.
In de volgende afbeelding zien we nog eens ons voorbeeld voor *boosdoeer* (hier gebruiken wij op dit moment nog niet het teken `nng`). Bij elke top staat er $[g, o]$. Daarbij geeft g het gewicht van de top en het tweede nummer is o als de top de wortel van de o-de boom is die door het algoritme gekozen wordt. Wij noemen o het ordenummer van de top. Voor een top t schrijven wij $a(t)$ en $o(t)$ voor het gewicht (het aantal keren dat die voorkomt), resp. het ordenummer van t. Om de boom te kunnen aanpassen, heeft adaptive Huffman coding ook de gewichten van elke top nodig – die zullen wij dus tijdens de uitvoering van het algoritme voor elke top bijhouden – net zoals de ordenummers.



Daarbij zijn drie eigenschappen van een Huffman boom duidelijk:

(a) het gewicht van een interne top is altijd de som van de gewichten van zijn twee kinderen (zo hebben wij het gewicht ten slotte vastgelegd).

(b) als voor twee toppen t, t' geldt dat $o(t) < o(t')$ dan geldt $a(t) \leq a(t')$ (wij kiezen de toe te voegen bomen met gewichten in stijgende volgorde).

(c) als er n bladeren zijn, dan zijn er $2n-1$ toppen en voor $1 \leq j < n$ zijn de toppen met ordenummers $2j-1$ en $2j$ broers – ze hebben dezelfde ouder (wij kiezen altijd twee toppen in elke stap en voegen die aan dezelfde ouder toe).

Het mooie is nu dat dat ook omgekeerd juist is: als een boom (met gewichten en ordenummers) deze eigenschappen heeft, dan kan die ook het resultaat van het Huffman algoritme zijn – en is dus optimaal voor de gewichten. Omdat het zo belangrijk is, zullen wij dat als Lemma opschrijven:

Lemma 22 *Stel dat een binaire boom T met $2n-1$ toppen en de volgende eigenschappen gegeven is: van de $2n-1$ toppen zijn n toppen $b_1, \ldots, b_n$ bladeren en elk blad b_i is gelabeld met een letterteken. Alle toppen t – dus alle bladeren en alle interne toppen – hebben paarsgewijs verschillende ordenummers $o(t)$ uit $1, \ldots, 2n-1$ en (niet negatieve) gewichten $a(t)$. Dan geldt:*
Als T aan (a), (b) en (c) voldoet en $l(b_i)$ de diepte van het blad b_i is, dan is T een boom zodat

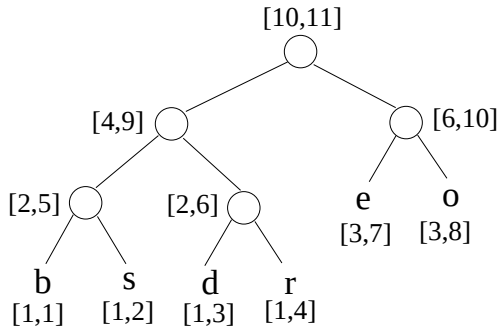
$$Q(T) = \sum_{i=1}^{n} l(b_i) * a(b_i)$$

minimaal is voor alle binaire bomen met bladeren $b_1, \ldots, b_n$.

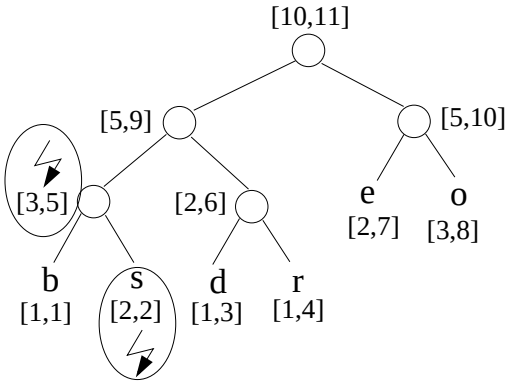
Het is dus in zekere zin *een optimale prefixboom – als je aanvaardt dat toppen met gewicht* 0 *ook in de boom moeten zitten.*

Oefening 87 *Geef een expliciet bewijs van Lemma 22. De beste manier om dit te doen, is te bewijzen dat de boom met het Huffman-algoritme opgebouwd kan worden. Dit kan* "gemakkelijk" *met inductie (zie ook het bewijs van Stelling 21) – maar het moet wel nauwkeurig opgeschreven worden.*

Als wij een nieuw letterteken lezen dat al in de boom zit, wordt het gewicht van dat teken en de toppen op het pad naar de wortel één groter. Als wij na `boosdoeer` dus bv. nog een `e` lezen, wordt de boom:



Deze boom voldoet aan (a)-(c) en hoeft dus niet gewijzigd te worden – maar dat was geluk! De reden was dat elke top die gewijzigd werd het hoogste ordenummer had van alle toppen met hetzelfde gewicht. Als wij na `boosdoeer` bv. nog een `s` zouden lezen, wordt de boom:



– en die boom voldoet niet aan (b)!
Het idee is nu de situatie waar het wel lukt af te dwingen: als je het gewicht van een top t één groter wilt maken, wissel je die (en de hele boom die eraan hangt) eerst met de top die het grootste ordenummer heeft van alle toppen met gewicht $a(t)$. Je wisselt daarbij de positie in de boom en de ordenummers. Dan verhoog je het gewicht met 1 en ga je naar de nieuwe ouder. Zo ga je door totdat je de wortel hebt bereikt. Of als pseudocode:

Algoritme 10 *Adaptive Huffman coding voor strikt positieve gewichten*

```
adjust_Huffman_tree_pos(z)
// z is het volgende teken dat gelezen werd en wij veronderstellen
// hier dat het al in de boom zit

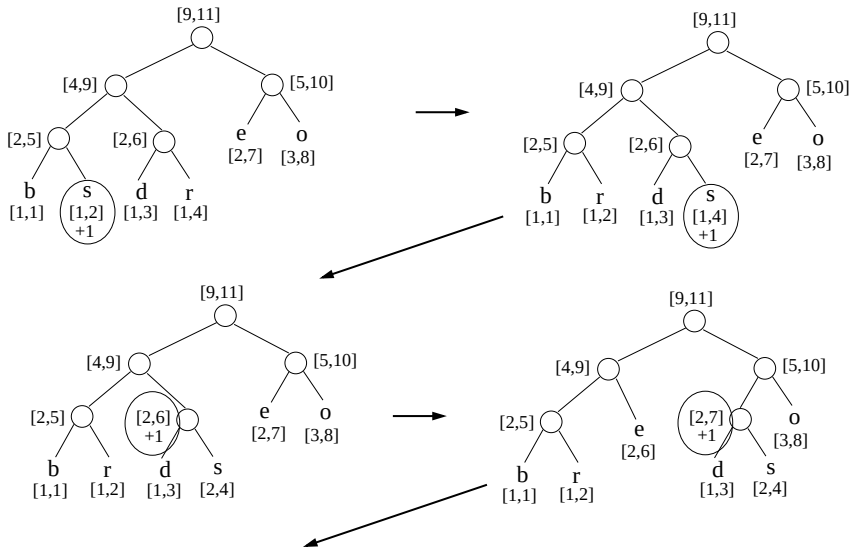
t = de top met label z
```

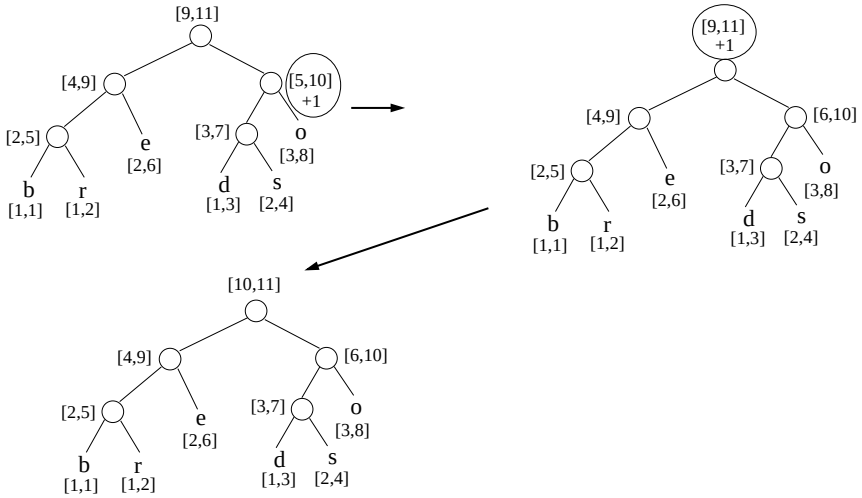
```
while (t is niet de wortel)
{
  t'= de top x met a(x)=a(t) en o(x) maximaal;
  wissel t en t'in de boom
  wissel de ordenummers van t en t'
  verhoog a(t) met 1
  t = (nieuwe) ouder van t
}

// nu is t de wortel
verhoog a(t) met 1

}
```

In de titel staat hier *voor strikt positieve gewichten* en dat is belangrijk om te garanderen dat je `t` en `t'` ook echt kan wisselen. Als je gewichten hebt die 0 zijn, zou het kunnen gebeuren dat de ene top in een deelboom zit waarvan de andere de wortel is – ook al zijn ze niet gelijk – en dan kan je natuurlijk niet wisselen.
Hierbij hebben we het alleen over het wijzigen van de boom gehad. Als je een tekst codeert moet je natuurlijk ook altijd het teken uitschrijven – gecodeerd door de boom voor de wijziging op dezelfde manier als bij statisch Huffman coderen.
Het helpt zeker een voorbeeld te zien. Wij passen het algoritme toe op het lezen van een `s` na `boosdoeer`, waarvan wij al gezien hebben, dat het een probleem vormde als de boom niet gewijzigd wordt:





Wij zouden graag zeggen dat de kost voor het wijzigen van de boom net zoals bij het wijzigen van zoekbomen lineair is in de lengte van het pad na de top die gewijzigd wordt – maar doordat de toppen altijd op andere plaatsen in de boom terechtkomen, is dat – ten minste niet onmiddelijk – duidelijk. Het is zelfs zo, dat een top bij het vervangen op een grotere diepte geplaatst kan worden.

Oefening 88 *Geef een voorbeeld van een met ordenummers en gewichten gelabelde boom die aan (a),(b),(c) voldoet en een blad bevat dat als het gewicht met 1 verhoogd wordt door Algoritme 10 in de eerste iteratie op een grotere diepte geplaatst wordt.*

Lemma 23 *Als Algoritme 10 een blad t in de Huffman-boom op diepte d met 1 verhoogt, dan wordt de wortel na ten hoogste d iteraties bereikt.*

Bewijs: Stel dat $o(t) = x_1 < \cdots < x_d$ de ordenummers van de toppen op het pad van t naar de wortel zijn (in de volgorde van het pad) en $o(t') = x'_1 < \cdots < x'_{d'}$ de ordenummers van de wisselende toppen t' die in Algoritme 10 gebruikt worden – in de volgorde waarop het algoritme ze gebruikt. Denk goed na waarom de "$<$" tekens kloppen ...
Wij willen aantonen dat $d' \leq d$. Wij hebben $x_d = x'_{d'}$ omdat wij in beide gevallen bij de wortel stoppen. Als wij dus voor alle i met $1 \leq i \leq \min\{d, d'\}$ bewijzen dat $x_i \leq x'_i$ is, dan kunnen wij als volgt argumenteren:

Stel dat $d' > d$. Dan is $x'_{d'} > x'_d \geq x_d = x'_{d'}$ wat natuurlijk een tegenstrijdigheid is.

Wij willen dus bewijzen: (*) $x_i \leq x'_i$ voor alle $1 \leq i \leq d, d'$.

Wij bewijzen eerst dat als voor twee (verschillende) toppen u, u' geldt dat $o(u) < o(u')$, de ouders p_u en $p_{u'}$ voldoen aan $a(p_u) \leq a(p_{u'})$. De broer u_b van u heeft ordenummer $o(u)-1$ of $o(u)+1$. Omdat $o(u)-1 < o(u)+1 \leq o(u')$ geldt dus dat $a(u_b) \leq a(u')$ (b) en (c). Voor de ouder p_u geldt dus $a(p_u) \leq a(u) + a(u')$ (a). Analoog heeft de broer u'_b van u' ten minste gewicht $a(u)$, dus $a(p_{u'}) \geq a(u) + a(u')$.

Nu bewijzen we (*) (en dus het lemma) met inductie. In het begin hebben t en t' hetzelfde gewicht en t' wordt met het grootste ordenummer gekozen

– voor $i = 1$ geldt (*) dus. Stel nu dat (*) voor $i - 1 < \min\{d, d'\}$ geldt. Dan is $x_{i-1} \leq x'_{i-1}$ en stel dat de bijhorende toppen $t_x, t_{x'}$ zijn. Dus geldt voor de ouders $a(p_{t_x}) \leq a(p_{t_{x'}})$. Dan wordt t' als de top met gewicht $a(p_{t_{x'}})$ en maximale ordenummer gekozen. Als $a(p_{t_x}) < a(p_{t_{x'}})$, dan geldt zeker dat $x_i = o(p_{t_x}) < o(p_{t_{x'}}) \leq o(t') = x'_i$. Door de keuze van t' als de top met het maximale ordenummer en gewicht $a(p_{t_{x'}})$ geldt ook als de gewichten van p_{t_x} en $p_{t_{x'}}$ gelijk zijn $x_i = o(p_{t_x}) \leq o(t') = x'_i$ – en dat wouden wij bewijzen.

■

Het aantal iteraties van Algoritme 10 is dus de diepte van het letterteken dat gelezen wordt. Omdat voor het coderen de code van het letterteken toch al geschreven moet worden, zijn dat op een constante na even veel stappen als voor het algoritme dat de uit te schrijven code berekent en schrijft. Als wij het over een vast alfabet hebben en alleen maar naar de $O()$-notatie kijken, zijn deze stappen natuurlijk toch al allemaal constant. Maar wij zullen nog zien dat je hier niet noodzakelijk met een vast alfabet moet werken. Maar wat ons echt interesseert, is natuurlijk altijd de tijd. Het is mogelijk het algoritme zo te implementeren dat de tijd voor het wijzigen in feite ook $O(d)$ is als het net gelezen teken op diepte d staat. Op een constante na is het wijzigen van de boom dus even duur als het opzoeken van het letterteken! Daarvoor is wel een slimme boekhouding nodig, zodat je altijd snel t' kan vinden – en de boekhouding ook snel kan updaten. Deze details van het algoritme zijn belangrijk en kunnen best uitgewerkt worden door het algoritme echt te implementeren. In zijn artikel *Dynamic Huffman coding* (Journal of Algorithms 6, 1985, pp.163–180) geeft Knuth alle details die nodig zijn om het te kunnen implementeren – als het je alleen niet lukt.

Op dit moment zijn we er altijd vanuitgegaan dat alle tekens die we tegenkomen vanaf het begin in de boom zitten. Je kan natuurlijk het hele alfabet in het begin in een op voorhand vastgelegde boom stoppen – bv. in een boom met minimale diepte. Omdat de gewichten allemaal 0 zijn, is elke boom optimaal. Het algoritme zou wel een beetje aangepast moeten worden omdat we veronderstellen dat alle gewichten strikt positief zijn, of je zou met gewicht 1 voor alle toppen moeten starten en de echte gewichten alleen benaderen (wat nadat een groot aantal tekenen gelezen is natuurlijk een zeer goede benadering zou zijn). Inderdaad stelt ook Knuth voor met een dergelijke optimaal gebalanceerde boom te werken – maar niet als deel van de Huffmanboom die gewijzigd wordt, maar als extra boom. Hij stelt voor één extra teken `nng` (nog niet gecodeerd) met gewicht 0 bij te houden dat betekent *"een nieuw teken wordt toegevoegd"*. Als het alfabet alle tekens van k bytes zijn, komt de manier met de optimaal gebalanceerde boom erop neer dat de $8k$ op een gecodeerd `nng`-teken volgende bits het nieuwe teken direct geven. Hij stelt ook voor de boom als er een teken *opgehaald* wordt te wijzigen, maar omdat elk teken in de boom ten hoogste één keer opgehaald wordt, gaat het erbij maar om heel weinige bits die bespaard kunnen worden – wij zullen de details hier dus niet geven. Voor de code die wij voor een nog niet gezien teken uitschrijven gebruiken wij dus de Huffman code van het teken `nng` gevolgd door het teken voorgesteld als een op voorhand vastgelegd aantal bits (bv. één byte).

Je kan het `nng`-teken natuurlijk ook gebruiken om hele woorden die misschien vaak voorkomen als één code te coderen – je moet alleen vastleggen op welke manier het einde van het volgende nieuwe teken herkend kan worden.
Wij hebben dan wel nog een nieuwe operatie nodig: de boom uitbreiden met een nieuw teken. Dat doen we door de `nng`-top te vervangen door een boom met 3 toppen: één ouder o met gewicht 1 die de `nng`-top (gewicht 0) en de nieuwe top (gewicht 1) als kinderen heeft. Dan wordt de top o net zo behandeld alsof die er al zat en van gewicht 0 naar gewicht 1 zou veranderd zijn. Behalve in het begin als o al de wortel is, begin je de gewone recursie dus met de ouder van o. De `nng`-top houdt ordenummer 1, de nieuwe top krijgt ordenummer 2 en o ordenummer 3. Voor alle andere toppen moeten de ordenummers met 2 verhoogd worden. Dat kan efficiënt door de ordenummers als $o(t) + b$ op te slaan, waarbij $o(t)$ bij de top hoort en b een *verschuiving* voorstelt die voor alle toppen dezelfde is (en die dan om 2 verhoogd wordt) ofwel door gewoon af te stappen van ordenummers die bij 1 beginnen. In feite zijn de ordenummers vooral belangrijk om het algoritme goed te verstaan – voor de implementatie moet je zelfs niet altijd alle ordenummers van alle toppen bijhouden...
Er zit wel een addertje onder het gras: omdat er een top met gewicht 0 is, zal zijn broer – en alleen maar die – een ouder hebben met hetzelfde gewicht. Als die ouder nu de top met dit gewicht en het grootste ordenummer is, zou het mislopen: je kan de top niet vervangen door een boom waar die zelf inzit. In dit speciaal geval kan je wel gewoon het gewicht van de top met 1 verhogen en dan met de ouder doorgaan zonder dat op het einde (a),(b) of (c) geschonden zijn.
De licht gewijzigde pseudocode voor tekens die al in de boom zitten, is dus:

Algoritme 11 *Adaptive Huffman coding voor ten hoogste één gewicht 0*

```
adjust_Huffman_tree(z)
// z is het volgende teken dat gelezen werd

als z in de boom zit
    t = de top met label z
anders
    maak de kleine boom met nng, teken z en ouder o
    verhoog alle andere ordenummers met 2
    als o geen ouder heeft: return
    anders t = ouder van o

while (t is niet de wortel)
{
  t'= de top x met a(x)=a(t) en o(x) maximaal;
  als t' niet de ouder is van t
   { wissel t en t'in de boom
     wissel de ordenummers van t en t'
   }
  verhoog a(t) met 1
  t = (nieuwe) ouder van t
}
```

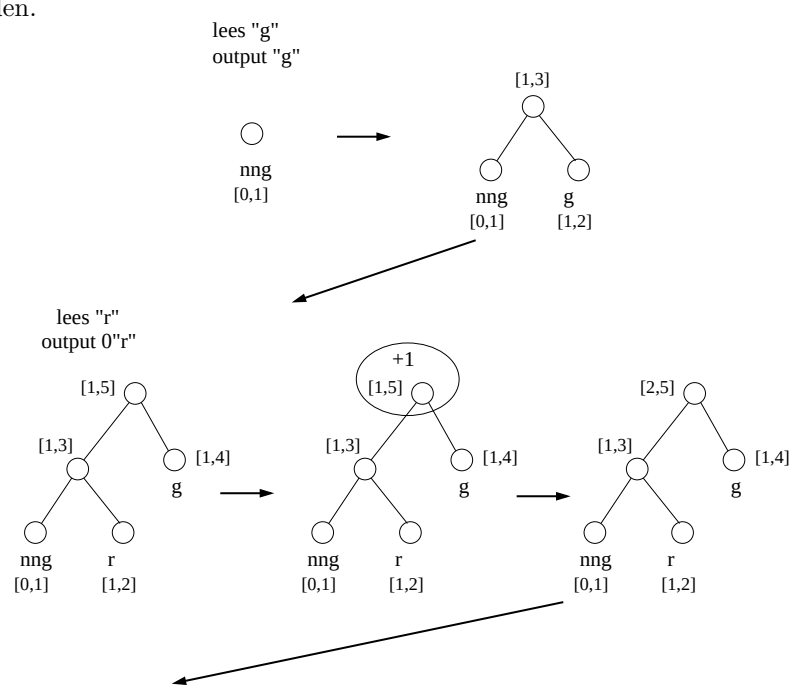
```
// nu is t de wortel
verhoog a(t) met 1

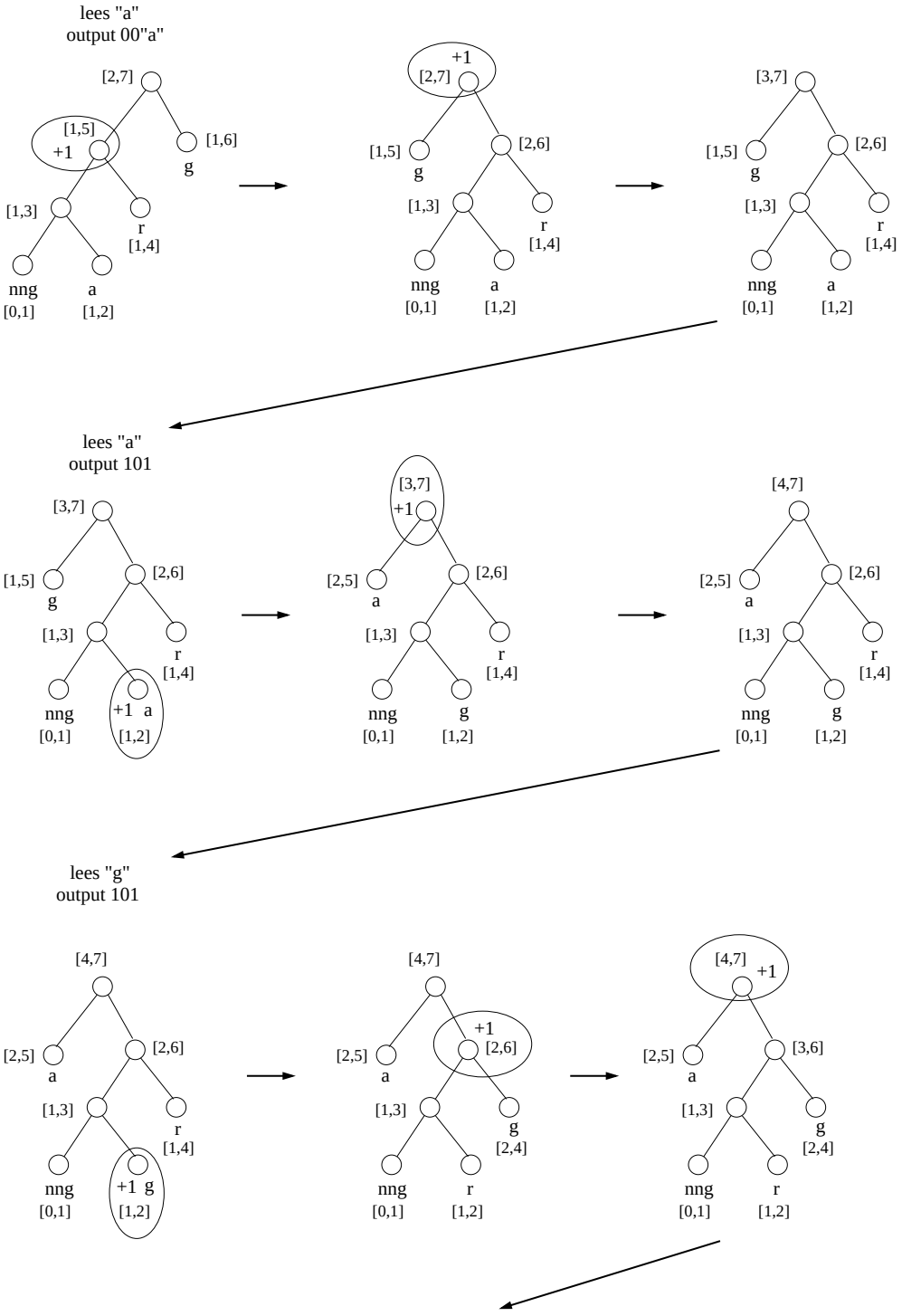
}
```

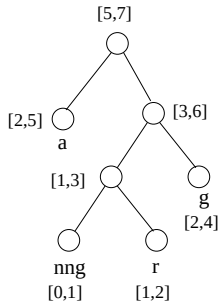
Oefening 89

- *Toon aan dat er in een Huffman boom met maximaal één top met gewicht* 0 *ten hoogste één top* t *zit, zodat de top* t' *met hetzelfde gewicht en het grootste ordenummer de wortel is van een boom die* t *bevat.*
- *Toon aan dat de wijziging in de pseudocode ervoor zorgt, dat de boom op het einde aan (a),(b) en (c) voldoet. Je mag gebruiken dat het algoritme voor strikt positieve gewichten juist werkt.*

Maar opnieuw is het het beste een voorbeeld te zien. Wij zullen eens het woord `graag` coderen. Het wordt afgesloten door een `EOF` dat natuurlijk ook doorgestuurd wordt, maar de boom moet daarvoor natuurlijk niet meer gewijzigd worden.




lees "a"
output 00"a"
[2,7]
[1,5]
+1
[1,6]
g
[1,3]
r
[1,4]
nng
[0,1]
a
[1,2]
+1
[2,7]
[1,5]
g
[2,6]
[1,3]
r
[1,4]
nng
[0,1]
a
[1,2]
[3,7]
[1,5]
g
[2,6]
[1,3]
r
[1,4]
nng
[0,1]
a
[1,2]
lees "a"
output 101
[3,7]
[1,5]
g
[2,6]
[1,3]
r
[1,4]
nng
[0,1]
+1 a
[1,2]
[3,7]
+1
[2,5]
a
[2,6]
[1,3]
r
[1,4]
nng
[0,1]
g
[1,2]
[4,7]
[2,5]
a
[2,6]
[1,3]
r
[1,4]
nng
[0,1]
g
[1,2]
lees "g"
output 101
[4,7]
[2,5]
a
[2,6]
[1,3]
r
[1,4]
nng
[0,1]
+1 g
[1,2]
[4,7]
[2,5]
a
+1
[2,6]
[1,3]
g
[2,4]
nng
[0,1]
r
[1,2]
[4,7]
+1
[2,5]
a
[3,6]
[1,3]
g
[2,4]
nng
[0,1]
r
[1,2]



Als wij nu nog een `r` zouden lezen, dan zouden wij het speciale geval tegenkomen, waar wij zonder de wijziging in het algoritme de top met een deelboom zouden moeten wisselen waar die zelf inzit.
Het decoderen gaat nu eenvoudig op dezelfde manier: het eerste teken en de tekens na het `nng`-teken worden direct gelezen (bv. door het aantal op voorhand vastgelegde bits te lezen of zoals door Knuth beschreven) en anders lees je de Huffman code en wijzig je achteraf de boom.

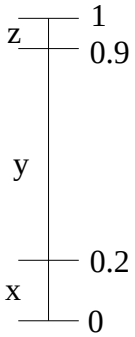
Oefening 90
- *Codeer de tekst* `voordeel`*.*
- *Decodeer de door jou opgestelde code om te toetsen of die juist was.*

Oefening 91 *Decodeer de adaptive Huffman code*
"n"0"o"0100"d"001100"a"100111
Tekens tussen aanhalingstekens staan – net zoals in de tekst – voor de 8 bits die dit letterteken voorstellen. Het resultaat is wel niet echt een woord uit de woordenlijst van de Nederlandse taalunie...

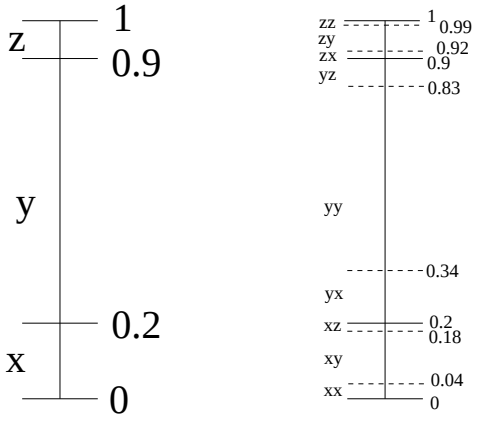
4.2 Arithmetisch coderen

Net zoals Huffman coding buit ook arithmetisch coderen (arithmetic coding) uit dat de lettertekens die gecodeerd worden verschillend vaak voorkomen – of anders gezegd: een verschillend grote kans hebben om voor te komen. Het is niet alleen zo dat arithmetisch coderen de basis is van de algoritmen met de hoogste compressiefactor, het idee van arithmetisch coderen is ook bijzonder interessant, dus moeten wij dat zeker zien. Wij gaan uit van wat in de teksten over arithmetisch coderen altijd een model genoemd wordt. Dit model is niets anders dan een verzameling van tekens (bv. alle bytes) en een functie $p(t)$ die beschrijft hoe groot de kans is dat teken t gebruikt wordt (bv. de relatieve hoeveelheid van dit teken in de Nederlandse taal). Als je het model voor een vaste tekst wilt optimaliseren en het met Huffman coding vergelijkt, is $p(t)$ dus $a(t)/n$, waarbij $a(t)$ het gewicht van teken t is en n de lengte van de tekst. Voor het coderen en decoderen moet (natuurlijk) hetzelfde model gebruikt worden – het moet dus ofwel meegestuurd worden of (gebaseerd op het type tekst) bij het coderen en decoderen gekend zijn. Als je bv. een Nederlandstalige tekst wilt coderen, hoef je niet noodzakelijk de probabiliteiten in deze tekst te kennen, maar zal het model dat de gekende probabiliteiten van lettertekens voor de Nederlandse taal beschrijft een goede benadering zijn. In dit hoofdstuk gaan we uit van een vast model.

Voor de voorbeelden zullen wij altijd met hetzelfde, **heel** eenvoudige, model werken: wij hebben maar drie lettertekens: x, y en z. De probabiliteiten zijn $p(x) = 0.2$, $p(y) = 0.7$ en $p(z) = 0.1$.
Wij gebruiken de notatie $[a, b)$ voor een interval dat bij a gesloten is (a bevat) en bij b open (b niet bevat). Het idee is nu aan elke string s een interval $i(s) = [a, b)$ met $0 \leq a < b \leq 1$ toe te kennen en dat zo te doen dat voor alle vaste n de verzameling van alle intervallen die aan strings s van lengte n toegekend zijn een partitie van het interval $[0, 1)$ vormen – dus:
$\bigcup_{\{s|\ |s|=n\}} i(s) = [0, 1)$ en
$\forall s, s'$ met $|s| = |s'| = n$ en $s \neq s'$ geldt $i(s) \cap i(s') = \emptyset$.
Omdat de intervallen voor een vaste lengte n een partitie vormen, zit dus elk reëel getal r met $0 \leq r < 1$ voor elke n in precies één interval $i_n(r)$ die bij een string $s_n(r)$ van lengte n hoort.
Als er k verschillende lettertekens zijn en de string heeft lengte n, dan zijn er k^n intervallen in de partitie – dat zijn dus heel snel veel te veel om te construeren. Gelukkig is dat geen probleem, omdat ze nooit allemaal geconstrueerd hoeven te worden. Wij moeten altijd alleen maar het interval voor één string construeren – de string die wij willen comprimeren.
Laten we eerst een triviaal voorbeeld zien: de partitie voor strings van lengte 1. Elk teken krijgt een interval dat even lang is als zijn probabiliteit. In ons model definiëren wij voor het eerste teken x (met $p(x) = 0,2$) $i(x) = [0, 0.2)$. Aan het tweede teken y (met $p(y) = 0.7$) kennen wij het volgende interval $i(y) = [0.2, 0.9)$ toe en uiteindelijk aan het derde teken z (met $p(z) = 0.1$) het daarop volgende interval $i(z) = [0.9, 1)$. Omdat alle tekens samen natuurlijk kans 1 hebben, zal op die manier een partitie van het interval $[0, 1)$ gedefinieerd zijn. Als afbeelding ziet dat er zo uit (waarbij uit de afbeelding niet duidelijk wordt, waar de intervallen open en gesloten zijn, maar dat weten wij natuurlijk):



Als wij de partitie voor k tekens al gedefinieerd hebben, dan krijgen wij de partitie voor $k + 1$ tekens door elk interval van de partitie met k tekens net zo onder te verdelen als wij het in het begin met het interval $[0, 1)$ gedaan hebben. Voor een string s van lengte k zullen de intervallen van de strings van lengte $k + 1$ die s als prefix hebben een partitie van $i(s)$ vormen. Voor 2 tekens ziet ons voorbeeld er dan zo uit:



Nu kunnen wij een string voorstellen door eender welk getal in het interval dat bij de string behoort. Wij stellen getallen natuurlijk binair voor. Omdat wij alleen getallen moeten coderen die in $[0, 1)$ liggen, leggen wij vast dat de bitstring $b_1, \ldots, b_k$ het getal $\sum_{i=1}^{k}(b_i * 2^{-i})$ voorstelt. Het getal 0.5 (binair 1) zit bv. in $i(yy)$ en dus is $s_2(0.5) = yy$. Voor het getal 0.8125 (binair 1101) geldt ook dat $s_2(0.8125) = yy$. Voor 0.9 krijgen wij $s_2(0.9) = zx$, etc. De lengte van een interval is altijd het product van de probabiliteiten van de tekens die erin zitten. Hoe langer een interval is, hoe zekerder je kan zijn dat het een getal bevat dat met relatief weinig bits voorgesteld kan worden. Het kan ook dat een heel kort interval een getal bevat dat toevallig een korte voorstelling heeft in de gekozen (bv. binaire) representatie – bv. het interval $[0.5, 0.5000000000000000000000001)$, maar dat is dan toeval. Wij kunnen uit de lengte van een interval wel een bovengrens voor de lengte van onze representatie afleiden:

Lemma 24 *Als een interval $[a, b)$ met $0 \leq a < b \leq 1$ lengte $l < 1$ heeft, dan bevat het een getal waarvan de voorstelling ten hoogste lengte $\lceil \log \frac{1}{l} \rceil$ heeft.*

Bewijs: Als $a = 0$ voldoet de bitstring 0 – stel dus dat $a > 0$. Stel dat $n = \lceil \log \frac{1}{l} \rceil$ en dat $g < a$ maximaal is met de eigenschap dat $g \notin [a, b)$ en dat g met ten hoogste n bits voorgesteld kan worden. Het getal $g + 2^{-n}$ is ofwel 1 of kan ook met maximaal n bits voorgesteld worden. Als het interval geen getal bevat dat met n bits voorgesteld kan worden, geldt $b \leq g + 2^{-n}$ (voor $g + 2^{-n} = 1$ geldt dat ook) – dus $l = b - a < g + 2^{-n} - g = 2^{-n}$, dus $\log l < -n$ en uiteindelijk $\log \frac{1}{l} = -\log l > n$, wat de keuze van n tegenspreekt.

■

Alle compressiemethoden hebben hun voor- en nadelen – dus gevallen waar ze bijzonder geschikt (of niet zo goed) geschikt zijn. Wij moeten ook – soms meer en soms minder – rekening houden met de tijd die nodig is om iets te comprimeren. Wij zullen dus zeker niet zeggen, dat arithmetisch coderen in elk opzicht beter is dan Huffman coderen – maar wij kunnen hier al zien, dat het soms verrassend korte codes toelaat. In feite kan bewezen worden dat als letterteken met de gegeven probabiliteiten onafhankelijk verdeeld zijn (het volgende letterteken dus niet afhangt van het vorige), de lengte van de code (in zekere zin) zelfs optimaal is! Dit is sterker dan Stelling 21, waar wij bewijzen dat het

een optimale **prefixcode** is, waar dus duidelijke voorwaarden zijn aan het soort code. Om de optimaliteit te bewijzen, zouden wij de entropie moeten introduceren – maar omdat de klemtoon van AD3 meer toepassings- en ideeën-gericht is, zullen wij dat hier niet doen. Een gespecialiseerde les over compressie waar al deze dingen aan bod kunnen komen, zou zeker zeer interessant zijn...
Huffman codes zouden voor elke string van lengte n natuurlijk ten minste n bits nodig hebben – de string `yyyxzxxzyyyxyyyyxyyyzxyxyyyyxxyyyyyyxyyy` dus ten minste 40 bits (in feite zelfs meer). Als je bv. in ons voorbeeld het interval voor deze string wilt berekenen, dan is dat **ongeveer**
$[0.49999999999999639178, 0.50000000000000310862)$.
In dit interval ligt $0.5 = 2^{-1}$ – de string kan dus met 1 bit voorgesteld worden: $s_{40}(0.5) = yyyxzxxzyyyxyyyyxyyyzxyxyyyyxxyyyyyyxyyy$. In feite codeert 0.5 niet alleen de strings $s_1(0.5) = y$, $s_2(0.5) = yy$, ..., $s_{40}(0.5) = yyyxzxxzyyyxyyyyxyyyzxyxyyyyxxyyyyyyxyyy$, maar is $s_n(r)$ voor alle $0 \leq r < 1$ en alle $n \in \mathbb{N}$ gedefinieerd. Door onze manier om de partitie te definiëren geldt ook dat als $k < k'$ de string $s_k(r)$ de prefix van lengte k van $s_{k'}(r)$ is.
Wij zullen nu nog gedetailleerd beschrijven hoe het algoritme werkt en hoe wij het interval voor een string *in principe* berekenen. Dus: gegeven een verzameling $A = \{t_1, \ldots, t_k\}$ van tekens en een probabiliteitsfunctie $p : A \to \mathbb{R}$, zodat de som van alle probabiliteiten 1 is. Wij definiëren dan voor $1 \leq i \leq k$ de functie $f : A \to \mathbb{R}$ met $f(t_i) = 0 + \sum_{j<i} p(t_j)$ (dus het *startpunt* van het interval $i(t_i)$).
Stel nu dat een string $s[1], \ldots, s[n]$ met tekens uit A gegeven is. Dan berekenen wij het interval $i(s[])$ als volgt:

Algoritme 12 *Berekening van het interval voor arithmetisch coderen*

```
get_interval(s[],n)
{
links=0.0;
lengte=1.0;

for (i=1; i<=n; i++)
{ links = links+(f(s[i])*lengte);
  lengte= lengte * p(s[i]);
}

return: [links, links+lengte);
}
```

Het is dus – **in principe** – heel eenvoudig om te programmeren. Als je het interval hebt, is ook een getal om het voor te stellen eenvoudig te vinden – het is gewoon logaritmisch zoeken, maar je kan het ook zo zien, dat wij het interval van beneden benaderen – totdat wij er voor de eerste keer inzitten. Wij letten alleen op, dat wij nooit een te groot getal kiezen:

Algoritme 13 *Berekening van het getal (de bitstring) dat (die) een interval voorstelt*

```
get_bitstring(low,up)
// output een bitstring die het interval [low,up) voorstelt
```

```
{
getal=0.0;
next=0.5;
if (low=0) { found=true; output 0; }
else found=FALSE;

while (found=FALSE)
  {
  if ((getal+next)< up)
     { output 1;
       getal=getal+next;
       if (getal>=low) found=TRUE;
      }
  else output 0;
  next=next/2;
  }

}
```

In feite berekent dit algoritme niet alleen **een** bitstring die een getal in het interval voorstelt, maar een kortst mogelijke bitstring met deze eigenschap!

Oefening 92 *Bewijs dat Algoritme 13 een kortst mogelijke voorstelling van een getal in het interval berekent.*

Oefening 93 *Gegeven het alfabet* $A = \{a, c, g, t\}$ *met tekens in deze volgorde en probabiliteiten* $p(a) = 0.3$, $p(c) = 0.2$, $p(g) = 0.4$ *en* $p(t) = 0.1$. *Bereken de kortste bitstring die de string* `cag` *voorstelt.*

Oefening 94 *Gegeven: een vast model A,* $p()$, *een bitstring* $b[]$ *en een getal* $n \in \mathbb{N}$ *dat de lengte van de string geeft. De bitstring* $b[]$ *codeert een string* $s[]$. *Geef de pseudocode van een algoritme dat* $s[]$ *berekent. Ga in jouw pseudocode ervanuit dat reële getallen met oneindige precisie berekend worden. Misschien helpt het daarbij het interval soms te herschalen...*

Oefening 95 *Gegeven het alfabet* $A = \{a, c, g, t\}$ *met tekens in deze volgorde en probabiliteiten* $p(a) = 0.3$, $p(c) = 0.2$, $p(g) = 0.4$ *en* $p(t) = 0.1$.
Geef de string van lengte 3 *die door de bitstring* `0010011` *gecodeerd wordt.*

Coderen en decoderen is *in principe* heel eenvoudig, maar als je het algoritme zoals geschetst efficiënt en nauwkeurig zou willen implementeren, zijn er wel nog sommige kleine en grote problemen die opgelost moeten worden:

(a) Omdat elk getal in feite een oneindige string codeert, moeten wij behalve de bitstring ook het getal n doorsturen dat zegt hoe lang de prefix is die gecodeerd wordt. Een alternatief is een extra *einde*-teken mee te coderen. In dat geval zou het decoderen gestopt worden zodra de prefix met op het einde het einde-teken berekend is. Gezien het feit dat bv. het aantal bits om de lengte n door te sturen in vergelijking met de data zeer klein is, is dat snel opgelost.

(b) Zelfs in ons klein voorbeeldje met weinig tekens en grote probabiliteiten worden de intervallen al snel zo klein, dat bij gewone datatypes al niet te verwaarlozen afrondingsfouten gebeuren. Als je bv. alle tekens met 8 bits wilt voorstellen, heb je al voor 7 tekens niet voldoende bits in het IEEE 754 double-precision binary floating-point formaat om een verschil tussen alle intervallen te kunnen maken – om het even wat de probabiliteiten zijn. Je moet dus met arbitrary precision datatypes werken die **extreem** lang worden (wat niet echt snel is...). Dit zou een groot probleem zijn, als je het idee echt zou willen implementeren zoals het geschetst is – wat je beter niet doet. Er zijn oplossingen die – op kost van een klein verlies aan efficiëntie – met een vaste precisie kunnen werken en dus echt toepasbaar zijn. Ze werken door continu de intervallen te herschalen. Daarbij worden de echte intervallen maar benaderd, maar als bij het coderen en decoderen dezelfde afrondingen gebeuren, werkt het algoritme nog altijd juist. In feite kan je dan ook met integers werken – maar dat is natuurlijk niet verrassend als het met een vaste precisie werkt. De details van deze oplossingen zijn echt mooi en jullie zouden die best eens in een boek of online (er is een goed leesbare en gedetailleerde beschrijving van Amir Said, HP Laboratories) opzoeken. Hier is er niet de tijd voor – en als jullie later niet in een richting gaan waar jullie dat nodig hebben, zijn dat te veel details en is het voldoende het schitterende idee van arithmetisch coderen te kennen.

Oefening 96 *Stel dat je tijdens het coderen het interval al op* $[l, u)$ *kon beperken en* $(u - l) \leq 2^{-i}$*. Geef een eis die voldoende is om bits nummer* $1, \ldots, i - 1$ *al te kunnen uitschrijven.*

PPM – Prediction by Partial Matching

Bij PPM wordt *in principe* gewoon arithmetisch coderen toegepast, maar de probabiliteiten, die in de verschillende stappen gebruikt worden, zijn niet meer gebaseerd op één model, maar er worden afhankelijk van de context (dat is: de vorige tekens) bij het coderen **en** decoderen in verschillende stappen verschillende modellen toegepast. Als de laatste vier tekens in een Nederlandstalige tekst bv. `_thu` zijn, dan is de kans dat het volgende teken een `i` is zeer groot – veel groter dan zonder de context van de vorige lettertekens. Je hebt dus op het einde grotere intervallen en dus kortere voorstellingen van de intervallen als je in dit geval een speciaal model afhankelijk van de laatste vier tekens gebruikt. Ook hier heb je een hele boel problemen waarover veel artikels gepubliceerd werden:

- Met hoeveel vroegere tekens houdt jouw model het best rekening?
- Hoe sla je de modellen het best op en hoe zoek je efficiënt in deze datastructuur?
- Hoe houdt je rekening met probabiliteiten die 0 zijn – arithmetisch coderen kan daarmee niet werken. Als je bv. met een model voor de Nederlandse taal werkt, waar er misschien geen woord is dat met `thur` begint, kan het toch dat er een naam is die zo begint of gewoon het Engelse *thursday* gebruikt wordt – en dan moet compressie ook werken.

- Wat doe je als je geen voorkennis hebt over het soort data dat gecomprimeerd wordt en je dus niet weet welk model je moet toepassen?

Het hoofdidee, dat wij net geschetst hebben, werd in verschillende algoritmes geïmplementeerd: PPMC, PPM*, BOA, RK.... Deze algoritmen hebben twee dingen gemeen:

(a) Ze comprimeren **heel** goed. PPM-gebaseerde algoritmen hebben de beste compressiefactoren die er zijn en zijn vooral als het om teksten gaat – waar de modellen op voorhand berekend kunnen worden – en als snelheid minder belangrijk is de beste keuze!

(b) Ze zijn vrij traag, zodat voor de meeste toepassingen toch die algoritmen gebruikt worden, die wij in de volgende hoofdstukken zullen zien, ook al is de compressiefactor iets slechter.

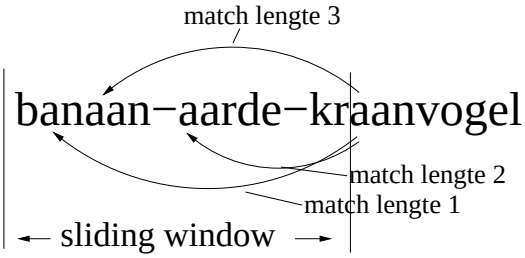
4.3 LZ77

Wij hebben met Huffman coding en arithmetic coding manieren die rekening houden met de relatieve hoeveelheden (of met andere woorden kansen) van de tekens in de tekst, maar die algoritmen hebben nog altijd de voorwaarde dat het alfabet op voorhand vastligt. Stel bv. dat in de tekst het letterteken "o" altijd gevolgd wordt door "e". Dan zou het toch verstandig zijn niet "o" en "e" voor de code te gebruiken maar "oe"! En zelfs in sommige gevallen waar er "o", "e" en "oe" in de code zitten, zou het kunnen, dat het nuttig is alle drie te coderen. PPM gaat al in die richting door de kansen afhankelijk van de laatste tekens te kiezen, maar toch zijn het kansen van één teken uit een vast alfabet.
Je kan dus het alfabet uitbreiden door kleine deelstrings als lettertekens te beschouwen. In het extreme geval zou je natuurlijk de hele tekst als één letterteken kunnen beschouwen en de code is alleen maar – bv. – "1". Maar hoe kan je dan aan de decoder mededelen waarvor de 1 staat? De ruimte die daarvoor nodig is, is – anders dan bij Huffman coding, waar ook extra informatie gestuurd wordt – zeker niet verwaarloosbaar...
Basisideeën om algoritmen met een betere compressie dan Huffman codering te ontwikkelen die toch al heel snel zijn (vooral: sneller dan PPM), zijn

- Gebruik een groter alfabet dan alleen maar lettertekens – gebruik ook strings!
- Gebruik een woordenboek dat dynamisch kan veranderen.

Het LZ77 (Abraham Lempel en Jacob Ziv 1977) algoritme is de basis van sommige algoritmen die in de praktijk goed of zelfs heel goed presteren en vaak gebruikt worden – zoals bv. zip, gzip, 7-zip and xzip. De laatste twee zijn wel alleen op het idee gebaseerd en worden niet als puur LZ77 beschreven maar als *Lempel–Ziv–Markov chain* algoritmen, kort LZMA en LZMA2. Omdat je deze zip-algoritmen ook voor directories gebruikt – dus gevallen waar er heel verschillende soorten data samen kunnen staan (tekst, binaire data, etc.) heb je dus een algoritme nodig dat niet met één vast model werkt, maar zich aanpast aan de verschillende soorten data.
Het algoritme kan gezien worden als een veralgemening van run length encoding, waar je voor elk letterteken opschrijft hoe vaak het herhaald wordt in plaats van

het te herhalen – dus bv. `aaaabbbrrrrrrrd` codeert als `4a3b7r1d`. LZ77 zoekt ook naar herhalingen die niet alleen identieke lettertekens zijn.
Het idee: Het idee is dat je de tekst zelf als woordenboek gebruikt waarin je zoekt. Een woordenboek is natuurlijk alleen maar nuttig als het ook bij de context past (bv. zou een Nederlands woordenboek nauwelijks nuttig zijn voor een Engelse tekst), maar de tekst als woordenboek past natuurlijk ideaal bij zijn eigen context! Dus: aan de ene kant heb je de tekst toch al nodig en aan de andere kant heb je dan ook automatisch de woorden die je nodig hebt. Jammer genoeg kan je niet altijd de hele tekst bijhouden (als het bv. een heel lange stream van data is) – dus gebruik je altijd alleen maar een eindig deel van de tekst – bv. de laatste 32 KB die je gezien hebt. Dit eindige deel noemen wij het *sliding window*. Dit sliding window is in het begin leeg en bevat tijdens het algoritme altijd de laatste G bytes waarbij G een getal is dat je op voorhand vastlegt. Zo heb je wel minder woorden ter beschikking maar als de tekst qua structuur verandert heb je de actuele woorden toch al ter beschikking! In de code houd je dan altijd bij waar het gecodeerde woord in het sliding window begint en hoe lang het is. Wat *het woord* is waarnaar je verwijst, hangt ervan af wat de langste match in het sliding window is. Maar je moet er natuurlijk ook voor zorgen dat in het geval dat er helemaal geen match in het sliding window is het volgende letterteken gecodeerd kan worden.
Het algoritme werkt als volgt:



neem langste match: positie 3 lengte 3

Algoritme 14 LZ77:
Stel dat het bestand dat gecodeerd moet worden de rij van lettertekens $x_0, x_1 \ldots$ is. Als wij het over een positie in het bestand hebben dan bedoelen wij de index. Analoog gebruiken wij positie voor de lettertekens die op dat moment in het sliding window staan – daar beginnen wij met het eerste teken in het sliding window (om het even wat de index van dit teken in het bestand is) en geven die het nummer 0.

- *Houd altijd een sliding window van de laatste G bytes bij waarbij G een op voorhand vastgelegd getal is.*
- *Als de tekst vanaf x_k gecodeerd moet worden dan wordt de langste match gezocht voor een woord dat op positie k begint. De match moet met een letterteken in het sliding window beginnen – maar het einde van de match mag er wel buiten liggen.*

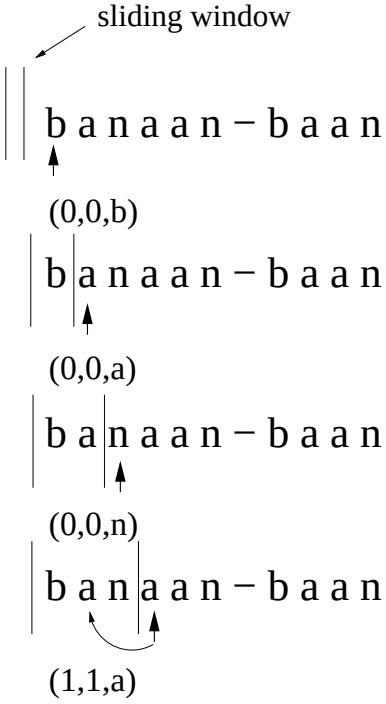
  Stel dat de index in het sliding window van het begin p is, de lengte l en het letterteken dat op de langste match volgt x. Als er geen match is, neem

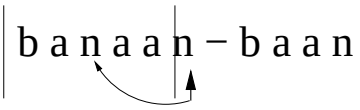
gewoon $p = 0$ maar omdat $l = 0$ doet het er niet echt toe – ook andere waarden voor p zouden dan werken.

Dan wordt het 3-tal (p, l, x) als code geschreven.

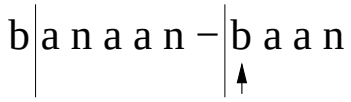
- *Het volgende teken vanaf waar gecodeerd moet worden, is op positie $k+l+1$ in het bestand.*

Daarbij kan je natuurlijk de positie op verschillende manieren beschrijven – bv. door het aantal tekens vanaf het begin van het sliding window of vanaf het einde van het sliding window, etc.
Je kan ook getallen en lettertekens als aparte lettertekens voor een alfabet beschouwen en dan ofwel de positie en de lengte opslaan ofwel het niet gevonden teken (dus (p, l) ofwel x). Omdat het verschil tussen getallen en lettertekens herkend kan worden is dat geen probleem. Of je kan alleen maar het letterteken schrijven als de lengte 0 is ofwel...
Al deze manieren van doen zijn mogelijk en verwezenlijken het idee van LZ77.
Wij zullen nu een voorbeeld zien waar wij de codering met 3-tallen gebruiken.
In de les gebruiken wij alleen deze codering waarbij we de positie vanaf de linkerkant tellen en het eerste teken de positie 0 krijgt.
Stel dat de *tekst* banaan-baan gecodeerd moet worden en dat de grootte van het sliding window 6 is (om iets te tonen moet het natuurlijk zo klein zijn...)

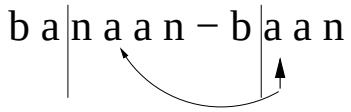




(2,1,−)



(0,0,b)



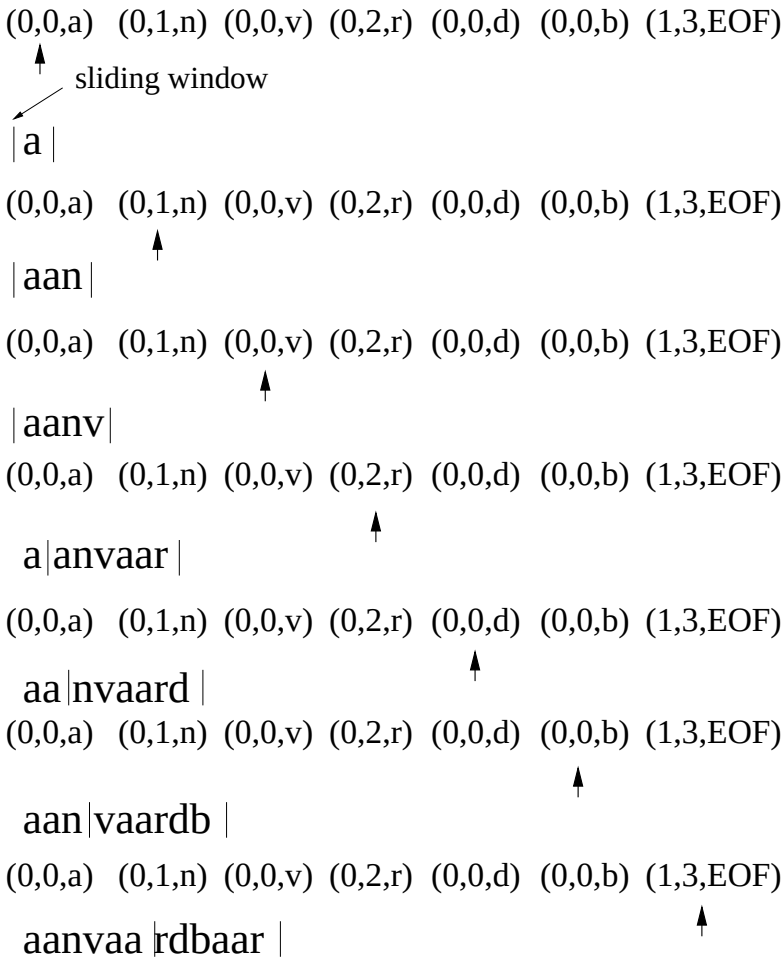
(1,3, EOF)

Deze output kan dan nog Huffman gecodeerd worden zodat er bv. rekening mee gehouden kan worden dat sommige lengten vaker voorkomen dan andere. Daarbij heb je verschillende mogelijkheden: je kan de output als een reeks van bytes zien – dus ook de lengten gewoon als strings van cijfers. Maar de symbolen in jouw Huffman-boom zouden ook de hele drietallen kunnen zijn. Je kan ook niet alleen één Huffman boom gebruiken maar bv. twee – één voor de getallen die posities en lengten beschrijven en één voor de lettertekens – of zelfs drie – één voor de posities, één voor de lengten en één voor de lettertekens. Omdat je elke keer als je een nieuw symbool wil lezen, weet of het een lengte, een positie of een letterteken moet zijn, weet je welke boom je moet gebruiken. In het geval van de interpretatie van de output als gewone tekst van bytes heeft dat zeker voordelen. Je kan dan bv. dezelfde korte bitstring voor verschillende tekens gebruiken in de verschillende omstandigheden. In het geval dat je de drietallen als tekens beschouwt, heb je dan wel kortere codes voor elk teken – maar er zijn ook twee of drie codes die gelezen moeten worden terwijl anders een heel drietal door één code beschreven wordt. Het is dus niet op voorhand duidelijk wat in dit geval beter is. Een voordeel zou misschien kunnen zijn dat meer rekening wordt gehouden met de individuele hoeveelheden van de verschillende symbolen in het drietal. Terwijl zekere posities of lengtes misschien vaker voorkomen dan andere, is dat misschien minder het geval als zich hele drietallen moeten herhalen. Of dat echt zo is en voldoende winst oplevert om meer codes te rechtvaardigen is een goede programmeeroefening! De programma's *DEFLATE* en *gzip* gebruiken bv. twee bomen: één voor de posities van de matches en één voor de rest (lengtes en het volgende teken) (waarbij nog verschillende optimalisaties aan bod komen) – dat is een mogelijke oplossing die goede resultaten oplevert.

LZMA en LZMA2 gebruiken wel zowel voor de eerste stap als ook de tweede nog bijzondere trucjes...

Tijdens het decoderen wordt hetzelfde sliding window opgebouwd – altijd als een code naar het woordenboek verwijst, staat het gezochte woord dus ook ter beschikking.

Ook dit kan het gemakkelijkst met een voorbeeld duidelijk gemaakt worden. Wij stellen ook hier dat de lengte van het sliding window maar 6 is:



Natuurlijk zijn deze voorbeelden weinig overtuigend – de codes zijn normaal veel langer dan het door ons gecodeerde woord en ook het sliding window maakt natuurlijk weinig kans overeenkomsten te vinden als het zo klein is! Maar realistische voorbeelden kan je natuurlijk niet met de hand uitwerken...

Oefening 97 *Codeer de teksten*

- `swiss_miss_missing`
- `een_benen_been`
- `ananas-kanaal`
- `een_been_beet_hebben`

met LZ77 (zonder het resultaat achteraf nog met het Huffman algoritme te coderen). Gebruik een sliding window van grootte 8*. Denk eraan dat matches in de sliding window moeten beginnen maar niet noodzakelijk eindigen!*

Oefening 98 *In dit voorbeeld hebben wij de code achteraf niet nog Huffman gecodeerd. Decodeer de volgende twee codes met een sliding window met grootte 6:*

- $(0,0,e)(0,1,n)(0,0,d)(1,2,b)(0,4,EOF)$
- *Als je het volgende woord decodeert, gebeurt er iets raars – maar dat kan:* $(0,0,b)(0,0,l)(0,0,a)(0,9,EOF)$

Oefening 99 *Welk algoritme voor string matching zou je gebruiken om een langste match te vinden. Geef redenen voor jouw keuze.*

Oefening 100

- *Comprimeer de volgende tekst met het LZ77 algoritme en pas achteraf Huffman met twee bomen toe. Gebruik een sliding window dat duidelijk groter is dan deze kleine tekst.*

  `abracadabra_simsalabim`

- *decodeer achteraf jouw code om te zien of je inderdaad hetzelfde resultaat krijgt.*

In de praktijk presteren op LZ77 gebaseerde algoritmen heel goed qua compressiefactor. Maar een probleem is de tijdscomplexiteit als je met grote sliding windows werkt. Natuurlijk verwacht je dat hoe groter het sliding window is, hoe meer tijd het vinden van de langste match vraagt. Een sliding window heeft geen structuur – zoals een boom – die het je gemakkelijker maakt om te zoeken. In het volgende hoofdstuk zullen wij zien hoe je voor oneindig grote windows wel zo'n structuur kan opbouwen en uitbuiten. In de $O()$-notatie zal de tijdcomplexiteit schitterend zijn, maar in de praktijk mag je de constanten die in de $O()$-notatie niet expliciet opduiken natuurlijk niet verwaarlozen.
Het decoderen is altijd efficiënt – om het even hoe groot het sliding window is. Als je de posities op de manier van de voorbeelden beschrijft, moet het sliding window tijdens het decoderen even groot zijn als tijdens het coderen. Als je de posities vanaf de rechterkant van het sliding window gebruikt, moet het niet even groot zijn maar ten minste even groot.

4.3.1 LZ77 met deelstring bomen

In dit hoofdstuk zullen wij zien hoe je LZ77 zonder sliding window (of equivalent: met een oneindig grote sliding window) in lineaire tijd kan implementeren. Een tekst met lengte n wordt dus in tijd $O(n)$ gecodeerd.
In hoofdstuk 3.3 hebben wij gezien dat je als je een string in een door het algoritme van Ukkonen opgebouwde deelstring boom hebt gevonden, de positie kan bepalen waar een kopie van de tekst begint. Hier is de tekst waarin wij zoeken de tekst in het sliding window van positie 0 tot voor het teken dat wij zoeken. Daarbij wordt tegelijk gezocht en de deelstring boom verder opgebouwd: als de tekst $s_1, \ldots, s_n$ is en wij hebben het teken s_i in de deelstring boom voor $s_1, \ldots, s_{i-1}$ gevonden, dan maken wij de deelstring boom voor $s_1, \ldots, s_{i-1}, s_i$ en kijken wij of wij de al gevonden string nog met 1 kunnen verlengen. Deze manier van opbouwen waar altijd de strings met het laatste teken erbij worden toegevoegd is gelukkig hoe het algoritme van Ukkonen toch al werkt!
Door Stelling 8 weten wij dat alle updates van T samen kost $O(n)$ hebben – dus gemiddeld kost $O(1)$ per letterteken van de tekst. Het zoeken van de langste match heeft ook kost $O(1)$ per letterteken, omdat je voor elk letterteken alleen

maar moet kijken of er een tak is die met het letterteken begint als je in een top zit en of het volgende letterteken van de label het gezochte letterteken is als je in het midden van een label van een boog zit. Bovendien pas je deze zoekbewerking maar één keer per letterteken toe – zelfs als die niet successvol is, wordt het teken niet opnieuw gezocht, maar als *volgend teken* uitgevoerd. Alle bewerkingen zijn dus gemiddeld $O(1)$ per bewerking, dus $O(n)$ voor de hele tekst.
Maar let op: als je op deze manier een echt lange tekst wilt coderen, heb je extreem veel ruimte voor de deelstring boom nodig! Bovendien is *lineair* en *zeer snel in de praktijk* niet altijd hetzelfde. Je zou hier bijvoorbeeld de tekst ook in blokken kunnen opdelen en die apart comprimeren om geheugen te besparen. Je zou zelfs maar voor één blok de deelstring boom kunnen opbouwen en daarmee doorgaan zolang het nog lange matches oplevert. In teksten die niet sterk veranderen – waar dus het eerste blok al een geschikt woordenboek ter beschikking stelt – heb je dan een zeer snelle manier om te coderen.
In feite kan je – met dezelfde complexiteit als zonder sliding window – ook met sliding windows werken door ook het linkerteken van de string te verwijderen en de boom te updaten (Jesper Larsson 1996), maar dat zullen wij hier niet zien. In de literatuur zullen jullie extreem veel papers vinden over de beste manier om de langste match ook in de praktijk efficiënt te vinden.

4.4 LZW

(Abraham Lempel, Jacob Ziv en Terry Welch 1984)
LZW is gebaseerd op het LZ78 algoritme dat Lempel en Ziv in 1978 voorstelden. Het grote verschil met LZ77 is dat hier een echt woordenboek wordt gebruikt en niet – zoals in LZ77 – de tekst zelf als woordenboek. Dit maakt het bijzonder gemakkelijk – en snel – de woorden op te zoeken en daardoor kan je veel sneller coderen, wat vooral belangrijk is als je bv. wilt streamen.
Natuurlijk is het woordenboek ook hier gebaseerd op de actuele tekst om alleen maar *nuttige* woorden in het woordenboek te hebben. Hoe groter het aantal woorden, hoe langer de codes om de woorden te beschrijven – het is dus belangrijk zo weinig mogelijk overbodige woorden in het boek te hebben. De klemtoon ligt hier ook op de mogelijkheid efficiënt in het – eenvoudig en snel opgebouwde – woordenboek te kunnen zoeken.
Coderen: Eerst zullen wij het principe uitleggen voordat wij de details beschrijven:
Je wil teksten van bytes coderen in codewoorden van een op voorhand vastgelegde lengte – meestal 12 bits. Je kan dus getallen $0 \ldots 4095$ voorstellen. Het woordenboek zal maximaal 4096 woorden bevatten die door deze nummers worden voorgesteld. In het begin vul je de *gewone bytes* als woorden in. Zij krijgen als nummer het getal dat ze voorstellen – dat is dus $0, \ldots, 255$. De rest van het woordenboek zal nu dynamisch opgebouwd worden. Daarbij voeg je elke keer dat je een nieuw woord tegenkomt – dat is dan een woord dat 1 letterteken langer is dan de tot nu toe langste deelstring van dit woord in het woordenboek – dit woord toe aan jouw woordenboek. Dat blijf je doen totdat het woordenboek volzit. Je gebruikt de code van een woord dus pas de tweede keer dat je het woord tegenkomt – de eerste keer gebruik je het woord min het laatste teken en voeg je het woord toe aan het woordenboek.

Maar natuurlijk is dat alleen maar een informele beschrijving. De details zie je in de volgende pseudocode:

```
voeg eerst de lettertekens toe aan jouw woordenboek
en geef de nummers vanaf 0 in volgorde

//w is ons woord
w= de lege string

while (w!=EOF)
  {
   k= volgende teken
   if (wk al in woordenboek)
        w=wk
   else
        { output code van w
          voeg wk toe aan het woordenboek
          geef wk de volgende code
          w=''k''
        }
  }
```

Een voorbeeld toont hoe het algoritme werkt:

b a n a a n a a n v a l

↑ in boek

b a n a a n a a n v a l

↑ niet in boek output code(b)=98
voeg ba toe aan woordenboek met nummer 256

b a n a a n a a n v a l

↑ niet in boek output code(a)=97
voeg an toe aan woordenboek met nummer 257

b a n a a n a a n v a l

↑ niet in boek output code(n)=110

voeg na toe aan woordenboek met nummer 258

b a n a a n a a n v a l

↑ niet in boek output code(a)=97

voeg aa toe aan woordenboek met nummer 259

b a n a a n a a n v a l

in boek ↑

b a n a a n a a n v a l

niet in boek ↑ output code(an)=257

voeg ana toe aan woordenboek met nummer 260

b a n a a n a a n v a l

in boek ↑

b a n a a n a a n v a l

niet in boek ↑ output code(aa)=259

voeg aan toe aan woordenboek met nummer 261

b a n a a n a a n v a l

niet in boek ↑ etc.....

Als het woordenboek volzit kan je verschillende dingen doen:

- begin gewoon opnieuw met een nieuw woordenboek
- ga door met dit woordenboek tot het inefficiënt blijkt te zijn (weinig lange matches)
- verwijder het woord met meer dan 1 letterteken uit het woordenboek dat je het langst niet meer gebruikt hebt (als code of als prefix van een code).

Al deze manieren van werken worden toegepast (GIF, unix compress, British Telecom Standard) en jullie hebben zeker ook zelf interessante ideeën wat je zou kunnen doen...

Decoderen:
Tijdens het decoderen wordt hetzelfde woordenboek opgebouwd als tijdens het coderen. Ook hier worden eerst de lettertekens aan het woordenboek toegevoegd. Elke keer dat een code gelezen wordt, zit die al in het woordenboek. Om dat te garanderen wordt tijdens het coderen een woord pas door zijn code

beschreven als wij het woord voor de tweede keer tegenkomen. Dan wordt dit woord samen met het eerste letterteken van de volgende code aan het woordenboek toegevoegd. En hier hebben wij een probleem: Het volgende woord kan in feite de tweede keer zijn dat wij het woord tegenkomen dat wij net aan het woordenboek willen toevoegen.

Voorbeeld: Codeer **aaaaaaaa...**.
De code begint met 97 voor **a** en **aa** wordt aan het woordenboek toegevoegd (code 256). Dan wordt 256 als code geschreven en **aaa** toegevoegd (code 257) etc.

Als wij nu decoderen, schrijven wij eerst **a** en dan willen wij **a** gevolgd door het eerste letterteken van het woord met code 256 aan het woordenboek toevoegen – maar nummer 256 willen wij net op dit moment bepalen....

Er is dus een probleem als (en slechts als) een woord onmiddellijk nadat het aan het woordenboek wordt toegevoegd als code wordt geschreven. Dan wordt het tijdens het decoderen niet teruggevonden. Maar wij weten al hoe het woord begint: omdat in dit geval het vorige woord alle lettertekens behalve het laatste bevat, begint het nieuwe woord met hetzelfde teken. En wij hebben alleen het eerste teken nodig om het nieuwe woord te bepalen dat toegevoegd moet worden. Ook hier zie je de details het best in pseudocode waarbij `[woord,teken]` voor de string staat die met de string `woord` begint en daarop het teken `teken` volgt.

```
voeg eerst de lettertekens toe aan jouw woordenboek
en geef de nummers vanaf 0 in volgorde

// eerst de eerste code lezen

c=gelezen code // een getal
w=woord(c) // het woord met dit nummer in het woordenboek

output w

while (nog een code om te lezen)
   {
     c= volgende code
     if (c in woordenboek)
        volgend_woord=woord(c)
     else
        volgend_woord= [w,w[0]]
        // de string w met het teken w[0] achteraan

     voeg [w,volgend_woord[0]] toe aan het woordenboek
                               en geef het volgende nummer

     w=volgend_woord
     output w
   }
```

Ook hier helpt een voorbeeld om te zien hoe het werkt:

97 110 256 258 115

woord= a tekst=a | nog voor de lus

97 110 256 258 115

woord= a volgend woord= n
woordenboek: 256=an tekst=an

97 110 256 258 115

woord= n volgend woord= an
woordenboek: 257=na tekst=anan

97 110 256 258 115
woord= an volgend woord= ana (258 nog niet in woordenboek!)
woordenboek: 258=ana tekst=ananana

97 110 256 258 115

woord= ana volgend woord= s
woordenboek: 259=anas tekst=anananas

Over het feit of LZW of LZ77 beter comprimeren vind je verschillende uitspraken. Daarbij zijn zeker vooral de precieze implementaties belangrijk en natuurlijk de toepassingen. Maar één voordeel heeft LZW zeker: het coderen kan op een efficiëntere manier gebeuren dan voor LZ77 omdat het dure opzoeken van een langst mogelijke deelstring niet moet gebeuren. Het zoeken of een deelstring al in het woordenboek zit, kan op een veel efficiëntere manier gebeuren (bv. met tries).

Oefening 101 *Codeer de teksten*

- `swiss_miss_missing`
- `een_beenen_been` *(met spelfout werkt het iets beter ...)*
- `ananas-kanaal`

met LZW.

Oefening 102 *Decodeer de volgende met LZW gecodeerde tekst. De nieuwe woorden in het woordenboek beginnen met nummer 256. Geef ook de woorden*

met code ten minste 256 in het woordenboek. Wees niet verrast als de tekst niet echt zinnig is... Lettertekens worden hier als tekens gegeven en niet als hun code (om de ASCII code niet te hoeven opzoeken).

```
a 256 b c a b 257 260
```

Oefening 103 *Welke datastructuren zou je gebruiken om jouw woordenboek voor het LZW algoritme bij te houden? Bespreek het woordenboek dat je bij het coderen gebruikt en het woordenboek dat je bij het decoderen gebruikt apart.*

4.5 De Burrows-Wheeler transformatie.

De Burrows-Wheeler transformatie (Michael Burrows en David Wheeler) werd in een artikel van 1994 gepubliceerd, maar gaat terug op onderzoek dat Wheeler al in 1983 deed en toen (nog) niet publiceerde.
De Burrows-Wheeler transformatie is de basis van het programma *bzip2* dat in de meeste gevallen heel goede resultaten oplevert (beter dan bv. *gzip* of *zip*). Maar voor ons is het vooral interessant omdat het een fundamenteel nieuw idee bevat: waarom niet de tekst wijzigen (transformeren) zodat hij achteraf beter gecomprimeerd kan worden? Natuurlijk is het belangrijk dat de wijziging achteraf ongedaan gemaakt kan worden om de originele tekst terug te krijgen.
Wij zullen het algoritme uitleggen aan de voorbeeldtekst **geen_loon**. De Burrows-Wheeler transformatie gebruikt de tekst niet sequentieel, maar begint al met heel grote stukken van de tekst. Misschien gebruikt het de hele tekst, maar ten minste voldoende grote stukken (blokken). Hoe groter de blokken zijn hoe beter de compressie – maar vooral de compressie is ook duurder voor grotere blokken. In onze kleine voorbeelden zal de tekst natuurlijk helemaal in één blok getransformeerd kunnen worden.
Eerst zullen wij de tekst zo interpreteren alsof het geen string is, maar een cykel van karakters, dus alsof na het laatste letterteken terug het eerste letterteken komt. Stel dat de lengte van de tekst n is. Dan kan je van deze *cyclische tekst* n strings van lengte n krijgen door op de n verschillende posities te beginnen. In ons voorbeeld is $n = 9$ en wij krijgen

g	e	e	n	_	l	o	o	n
e	e	n	_	l	o	o	n	g
e	n	_	l	o	o	n	g	e
n	_	l	o	o	n	g	e	e
_	l	o	o	n	g	e	e	n
l	o	o	n	g	e	e	n	_
o	o	n	g	e	e	n	_	l
o	n	g	e	e	n	_	l	o
n	g	e	e	n	_	l	o	o

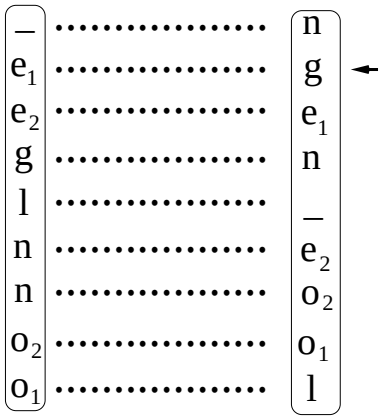
Hierbij toont een pijltje wat het eerste teken van de tekst in de laatste kolom staat. Wij zien dat in elke rij en elke kolom de hele tekst staat. Als jullie het echt willen implementeren zouden jullie natuurlijk het best niet echt n kopieën

van de tekst gebruiken, maar een enkele kopie en de verschillende strings gewoon voorstellen als pointers naar het beginkarakter!
Nu sorteren wij de rijen. Daarbij gebruiken wij de waarden van de bytes (een blank is kleiner dan de lettertekens), maar omdat onze voorbeelden voor compressie met gewone teksten met kleine lettertekens werken, komt dat hier overeen met de lexicografische volgorde. Dan krijgen wij

_	l	o	o	n	g	e	e	n
e_1	e	n	_	l	o	o	n	g
e_2	n	_	l	o	o	n	g	e_1
g	e	e	n	_	l	o	o	n
l	o	o	n	g	e	e	n	_
n	_	l	o	o	n	g	e	e_2
n	g	e	e	n	_	l	o	o_2
o_2	n	g	e	e	n	_	l	o_1
o_1	o	n	g	e	e	n	_	l

Daarbij hebben wij een index bij de o en bij de e in de eerste en laatste kolom geschreven om te kunnen zien *welk van de identieke lettertekens (bytes) het is*. Maar dat is alleen maar ter illustratie en niet deel van de methode.
Wat wij nu **eerst** stellen is dat alle rijen van deze matrix verschillend zijn. Als dat niet zo was, kan je dat gemakkelijk zien omdat dan in de gesorteerde matrix identieke rijen achter elkaar moeten staan. In dit geval is de hele tekst een herhaling van een deelstring en je kan het best de deelstring opslaan (misschien nog gecomprimeerd) en hoe vaak je hem moet herhalen. Een andere oplossing zou zijn dat je op het einde van de originele string een teken toevoegt dat nog niet in de string zit en na de inverse transformatie opnieuw verwijderd moet worden. Dat kan bv. een teken zijn dat kleiner is dan alle tekens in de string (als dat kan). Op deze manier kan je op voorhand garanderen dat alle rijen verschillend zijn!
De getransformeerde tekst is nu een kolom van deze matrix. Voor compressiedoeleinden zou het natuurlijk ideaal zijn de eerste kolom te gebruiken – in een lange tekst zouden daar in het begin alleen maar a's staan, dan alleen maar b's, etc – je zou dus alleen maar de hoeveelheden moeten bijhouden. Jammer genoeg is er geen manier om uit deze kolom de originele (cyclische) tekst te reconstrueren. Maar als het om een *echte* en niet om een toevallige tekst gaat, heeft ook de laatste kolom nog voordelen van het sorteren: ook hier is de verdeling van de lettertekens niet toevallig omdat de lettertekens de prefixen van de lettertekens in de eerste kolom zijn. En wat verrassend is: terwijl je uit de eerste kolom de cyclische tekst niet terug kan krijgen (zonder dure extra informatie op te slaan) kan je dat uit de laatste kolom wel! Je moet alleen nog bijhouden waar het eerste letterteken $s[0]$ van de string $s[]$ staat om ook de niet cyclische string te reconstrueren.
Een voordeel is dat als wij de laatse kolom hebben, wij de eerste kolom gemakkelijk kunnen verkrijgen: het zijn dezelfde lettertekens als in de laatste kolom en wij moeten die gewoon sorteren! Daarbij houden wij natuurlijk geen rekening

met de indices (tenslotte zijn die er niet echt), maar stel voor het moment dat je ook de indices kent. Wij hebben dus



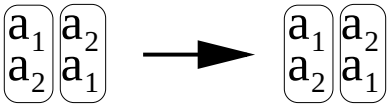
Het transformatiealgoritme kan dus als volgt geschetst worden:

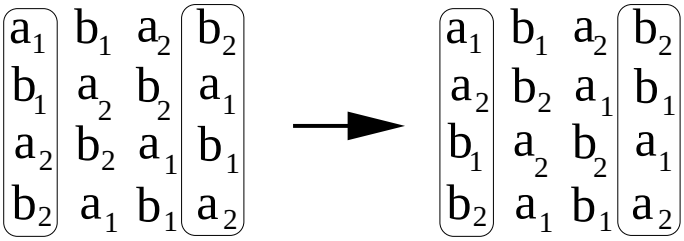
- interpreteer de tekst $s[0], s[1], \ldots, s[n-1]$ als cyclisch en vorm alle n strings $s_i[]$ met $0 \leq i < n$ die je krijgt door op positie i te beginnen.
- sorteer deze n strings in stijgende lexicografische volgorde. Wij schrijven $p_s(i)$ voor de string op positie i na het sorteren – $p_s(0)$ is dus de kleinste string.
- output het nummer i van de rij die met $s[0]$ eindigt – dus de rij i met $p_s(i)[n-1] = s[0]$. Dat is de rij waar het letterteken waarmee de originele string begint op de laatste positie staat. Output dan een blank.
- output de lettertekens $p_s(0)[n-1], \ldots, p_s(n-1)[n-1]$ op de laatste posities in deze volgorde.

Het eerste teken van de originele tekst staat in ons geval in rij 1 als wij met 0 beginnen te tellen. De Burrows-Wheeler getransformeerde van **geen_loon** zou dus bv. **(1_ngen_eool)** zijn als wij vastleggen dat het getal eerst komt en de tekst na de eerste blank staat. Dit is één mogelijke manier van doen, maar je zou ook eerst een integer van vaste lengte voor de positie kunnen gebruiken.
Maar nu is het gemakkelijk de cyclische tekst opnieuw op te bouwen. Wij moeten voor elk letterteken alleen het volgende letterteken kennen – maar na het letterteken dat in de laatste kolom in rij i staat komt het letterteken dat in de eerste kolom in rij i staat – en dat kennen wij!
Als wij dus met het eerste teken van de tekst beginnen – dus het teken nummer 1 van **ngen_eool** dan is dat **g** en in de eerste kolom staat op positie 1 $\mathbf{e}_1$ – het volgende teken is dus $\mathbf{e}_1$. In de laatste kolom staat $\mathbf{e}_1$ in rij 2 en in rij 2 in de eerste kolom staat $\mathbf{e}_2$. Dat vinden wij terug in rij 5 in de laatste kolom, etc... Als wij zo doorgaan krijgen wij de oorspronkelijke tekst **geen_loon**.

Er blijft maar één probleem: toen wij opgezocht hebben welk van de e's aan de rechterkant de **e** is die naar de **g** komt, hebben wij de indices gebruikt – maar die zijn er in het echt niet – daar zijn het alleen maar e's en o's!

Als jullie naar de volgorde kijken waarop de indices opduiken dan is het in de eerste kolom eerst $\mathbf{e}_1$ en dan $\mathbf{e}_2$ – en voor de tweede kolom ook. Voor de o's is het eerst $\mathbf{o}_2$ en dan $\mathbf{o}_1$ in de eerste kolom – en in de tweede kolom ook. En dat is in feite geen toeval: wij kunnen identieke lettertekens in de eerste en laatste kolom aan elkaar toekennen in de volgorde waarin ze opduiken!
Om dat te bewijzen stel dat de string $s[] = s[0] \dots s[n-1]$ is en dat $s[i] = s[j]$ waarbij $s[i]$ in de gesorteerde matrix in de eerste kolom voor $s[j]$ komt. Wij zouden nu graag bewijzen dat dan ook in de laatste kolom $s[i]$ voor $s[j]$ komt, maar jammer genoeg is dat niet altijd zo:





Maar als wij stellen dat de rijen allemaal verschillend zijn, kunnen wij gemakkelijk bewijzen dat de volgorden in de eerste en laatste kolom dezelfde zijn:
Stel dat $s[i] = s[j]$ en dat $s[i]$ in de gesorteerde matrix in de eerste kolom voor $s[j]$ komt – dus
$(s[i], s[i+1] \dots s[n-1]s[0] \dots s[i-1]) < (s[j], s[j+1] \dots s[n-1]s[0] \dots s[j-1])$.
Omdat $s[i] = s[j]$ geldt dus
$(s[i+1] \dots s[n-1]s[0] \dots s[i-1]) < (s[j+1] \dots s[n-1]s[0] \dots s[j-1])$ en dus ook
$(s[i+1] \dots s[n-1]s[0] \dots s[i-1]s[i]) < (s[j+1] \dots s[n-1]s[0] \dots s[j-1]s[j])$.
Maar dat betekent precies dat in de laatste kolom de rij waarin $s[i]$ staat voor de rij komt waarin $s[j]$ staat.
Maar precies dat wouden wij bewijzen en dus is onze methode om lettertekens met elkaar te identificeren (indices toe te kennen) onder deze omstandigheden juist.

Het terugtransformatiealgoritme kan dus als volgt geschetst worden:
Stel dat de code $(p, c[0], \dots, c[n-1])$ is.

- kopieer $c[0], \dots, c[n-1]$ en sorteer de tekens. Noem de nieuwe string $e[]$. In $e[i]$ staat dus het letterteken dat in de originele tekst na het teken $c[i]$ komt.
- voor elk letterteken x dat k keer in $c[]$ en dus ook k keer in $e[]$ zit, ken indices $1, \dots, k$ toe aan de posities van x in $c[]$ in de volgorde van de string. Doe hetzelfde voor $e[]$. Wij schrijven x_k voor het letterteken x met index k. Elk geïndexeerd letterteken zit dus één keer in $c[]$ en één keer in $e[]$.

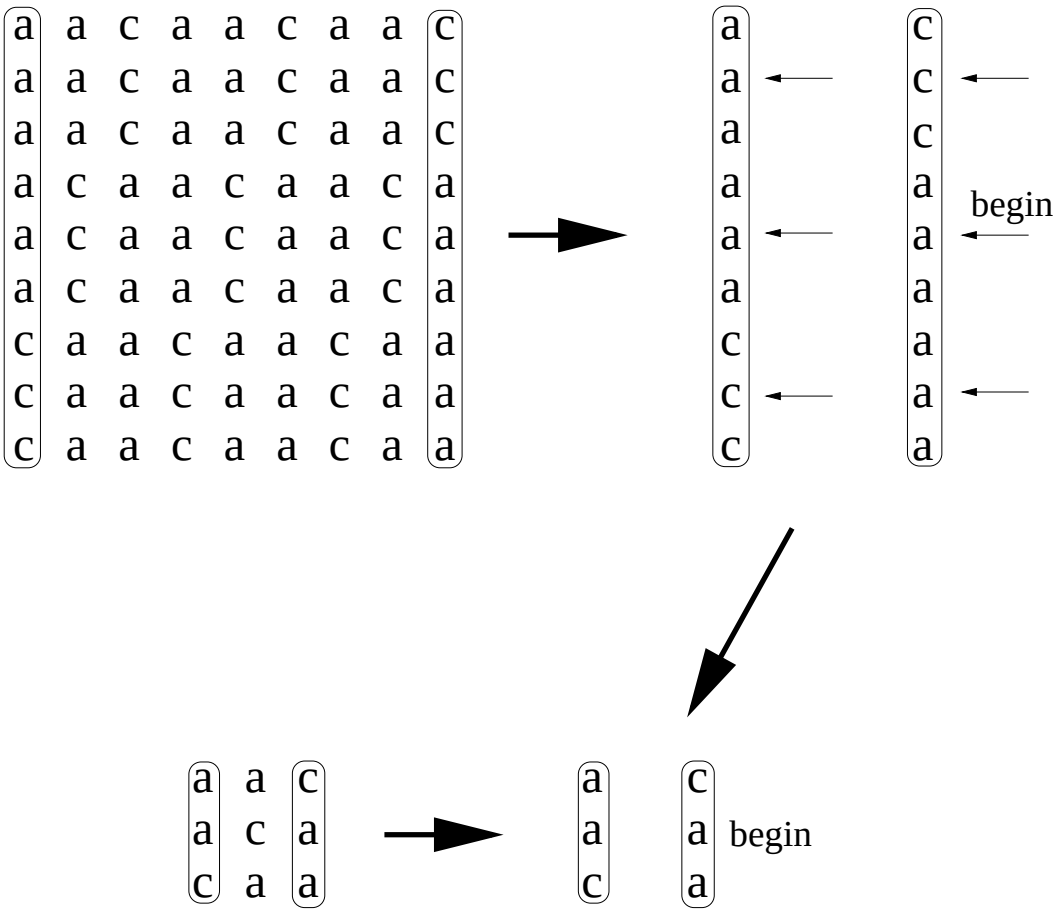
- begin met $x_j = c[p]$ dan herhaal n keer:
  - output het letterteken x (zonder index).
  - bepaal het nummer i zodat $c[i] = x_j$.
  - kies $x_j = e[i]$ als volgend geïndexeerd letterteken.

Jullie zullen op het internet ook beschrijvingen vinden waar jullie de lus niet n *keer* moeten herhalen, maar *totdat je terug bent bij het eerste letterteken*. Maar wij zullen zien dat het voordelen heeft het zo te doen...

Wat zou er gebeuren als wij het algoritme zouden toepassen op een string die een herhaling van deelstrings is – dus waar twee of meerdere rijen identiek zijn? Stel dat de string $t[0], \ldots, t[k-1]$ niet tot identieke rijen leidt maar de hele string $s[]$ wel, omdat hij uit l herhalingen van $t[]$ bestaat. Dus $s[] = t[0], \ldots, t[k-1], t[0], \ldots, t[k-1], \ldots, t[0], \ldots, t[k-1]$. Als wij op deze string het algoritme toepassen, hebben wij elke rij precies l keer achter elkaar. De hele matrix is dus een opeenvolging van k blokken waarvan elke blok l identieke rijen bevat. Rijen in verschillende blokken zijn verschillend. Maar voor de transformatie doet dat er natuurlijk niet toe – wij kunnen gewoon de getransformeerde tekst construeren.
Nu kijken wij naar het toekennen van indices in $c[]$ (en analoog in $e[]$). Tijdens de terugtransformatie zal een teken in $e[]$ of $c[]$ dat op positie $i < l$ in zijn eigen blok zit zeker een index j krijgen zodat $i = j (\mathrm{mod}\, l)$: in zijn blok is het zeker het i-de en in de blokken daarvoor was het identieke letterteken ofwel niet op de eerste (of laatste) positie aanwezig ofwel l keer – dus zal de index j in elk geval aan $i = j (\mathrm{mod}\, l)$ voldoen.
Maar als ons beginteken een index b heeft dan zien wij dat in het hele algoritme **nooit** een letterteken met een index i gebruikt wordt die niet aan $i (\mathrm{mod}\, l) = b (\mathrm{mod}\, l)$ voldoet. Het algoritme gebruikt dus alleen maar één rij van elk blok. Maar als wij naar de eerste en laatste lettertekens van alleen deze rijen kijken dan zien wij dat dit dezelfde zijn als bij het toepassen van het algoritme alleen op $t[0], \ldots, t[k-1]$. Een voorbeeld ervan zien wij in Figuur 26. In feite zullen wij dus na k stappen terug zijn met ons beginteken – maar omdat wij even veel stappen doen als er lettertekens in de getransformeerde tekst zitten – dus $l * k$, zullen wij doorgaan en de tekst $t[0], \ldots, t[k-1]$ l keer herhalen. De originele tekst wordt dus inderdaad ook in dit geval juist teruggetransformeerd.
Het lijkt natuurlijk niet echt efficiënt als wij het grootste deel van de getransformeerde tekst helemaal niet gebruiken – en ook niet nodig hebben. Maar wij zullen zien dat in dit geval de volgende stappen ook bijzonder goed comprimeren.

Wat hebben wij tot nu toe gewonnen? De getransformeerde tekst heeft even veel lettertekens. En als wij nu het Huffman algoritme zouden toepassen, hebben wij nog altijd niets gewonnen: de relatieve aantallen lettertekens zijn dezelfden als in de originele tekst – de code zou dus even lang zijn (in feite zelfs iets langer omdat het begin van de tekst ook opgeslaan moet worden).

Wat is dan het voordeel?
De volgorde van de strings is lexicografisch. De meeste invloed daarbij heeft natuurlijk het eerste letterteken in elke rij – daar hebben wij een mooie ordening.



Figuur 26:

Maar het laatste letterteken is een prefix van het eerste. In een toevallige tekst betekent dat niet veel omdat elk teken dezelfde kans heeft een prefix te zijn. Maar wij weten al dat je toevallige data niet kan comprimeren. In *echte* teksten (en ook andere bestanden) zijn dergelijke prefixen niet toevallig. De strings die bv. met *aa* beginnen, zullen allemaal achter elkaar staan. Maar het is zeker niet zo dat elk letterteken dezelfde kans heeft een prefix van *aa* te zijn. Als jullie bv. in het groene boekje kijken dan zijn er veel woorden die met **b**aa of **l**aa beginnen, duidelijk minder met **f**aa maar geen enkel woord begint met **c**aa. Dat zegt natuurlijk niet alles, omdat natuurlijk de prefixen niet altijd op het begin van een woord staan, maar het geeft al een idee. En als jullie naar langere deelstrings kijken, wordt dat nog duidelijker (bv. de prefixen van *aan_* of nog langere strings...). Wij kunnen dus hopen dat er in de rijen waar *aa* de volgorde bepaalt op het einde veel b's of l's bij elkaar staan. De letters in de laatste kolom zijn dus dezelfden als in de originele tekst – maar beter gegroepeerd. Wij hebben dus een compressiealgoritme nodig waarvoor dat een voordeel is – adaptive Huffman coding zou bv. al beter presteren!
Maar Burrows en Wheeler stellen voor deze eigenschap van de getransformeerde

tekst als volgt te gebruiken:

De *move to front* methode

Je legt vast hoe je jouw alfabet met getallen codeert – je houdt de tekens bij in een lijst en elk letterteken x krijgt als code $c(x)$ zijn positie in de lijst. In de realiteit zouden dat bv. alle ASCII codes zijn – omdat je niet op voorhand weet welke lettertekens opduiken en welke niet moet je een **vaste** lijst hebben met **alle** mogelijke lettertekens! In onze voorbeelden willen wij natuurlijk niet met zo'n lange lijst werken, wij kiezen dus een kortere, maar het is belangrijk om te onthouden dat dat alleen voor de voorbeelden is en in het algemeen niet kan! Hier staat het teken "␣" voor een blank, dat komt dus (code 32) voor de cijfers.
Voorbeeld: De lijst: (␣,1,a,b,e,f,g,n,o,l). Dus is $c(␣) = 0$, $c(a) = 2$,...,$c(l) = 9$ etc.
Als jullie nu een string coderen, dan schrijven jullie voor elk letterteken x zijn code $c(x)$ **en** plaatsen dit letterteken dan in het begin van de lijst – het heeft **achteraf** dus code 0.
Als jullie met de lijst hierboven dus **(1␣ngen␣eool)** willen coderen dan werkt dat als volgt:

de lijst is (␣,1,a,b,e,f,g,n,o,l): en **(1␣ngen␣eool)** moet gecodeerd worden.

lees:**1**: 1 schrijven, lijst nu (1,␣,a,b,e,f,g,n,o,l)
lees:␣: 1 schrijven, lijst nu (␣,1,a,b,e,f,g,n,o,l)
lees:**n**: 7 schrijven, lijst nu (n,␣,1,a,b,e,f,g,o,l)
lees:**g**: 7 schrijven, lijst nu (g,n,␣,1,a,b,e,f,o,l)
lees:**e**: 6 schrijven, lijst nu (e,g,n,␣,1,a,b,f,o,l)
lees:**n**: 2 schrijven, lijst nu (n,e,g,␣,1,a,b,f,o,l)
lees:␣: 3 schrijven, lijst nu (␣,n,e,g,1,a,b,f,o,l)
lees:**e**: 2 schrijven, lijst nu (e,␣,n,g,1,a,b,f,o,l)
lees:**o**: 8 schrijven, lijst nu (o,e,␣,n,g,1,a,b,f,l)
lees:**o**: 0 schrijven, lijst nu (o,e,␣,n,g,1,a,b,f,l)
lees:**l**: 9 schrijven, lijst nu (l,o,e,␣,n,g,1,a,b,f)

Dit toont wel het principe, maar het kleine voorbeeld is niet voldoende om aan te tonen dat het nuttig is. Maar omdat in de Burrows-Wheeler getransformeerde tekst heel vaak herhalingen van letters voorkomen – of ten hoogste met kleine onderbrekingen, zullen in de code zeer veel nullen en kleine getallen opduiken.
Als je het decodeert moet je natuurlijk met dezelfde lijst beginnen en die dan ook op dezelfde manier veranderen:

de lijst is:(␣,1,a,b,e,f,g,n,o,l) en **1,1,7,7,6,2,3,2,8,0,9** moet gedecodeerd worden.
lees:**1**: 1 schrijven, lijst nu (1,␣,a,b,e,f,g,n,o,l)
lees:**1**: ␣ schrijven, lijst nu (␣,1,a,b,e,f,g,n,o,l)
lees:**7**: n schrijven, lijst nu (n,␣,1,a,b,e,f,g,o,l)
lees:**7**: g schrijven, lijst nu (g,n,␣,1,a,b,e,f,o,l)
lees:**6**: e schrijven, lijst nu (e,g,n,␣,1,a,b,f,o,l)
lees:**2**: n schrijven, lijst nu (n,e,g,␣,1,a,b,f,o,l)
lees:**3**: ␣ schrijven, lijst nu (␣,n,e,g,1,a,b,f,o,l)
lees:**2**: e schrijven, lijst nu (e,␣,n,g,1,a,b,f,o,l)
lees:**8**: o schrijven, lijst nu (o,e,␣,n,g,1,a,b,f,l)

lees:**0**: o schrijven, lijst nu (o,e,_,n,g,1,a,b,f,l)
lees:**9**: l schrijven, lijst nu (l,o,e,_,n,g,1,a,b,f)

Na de move to front transformatie is de tekst nog altijd niet korter – je vervangt gewoon bytes met een teken door andere bytes die de positie beschrijven – maar die zijn even veel en even groot. De transformatie door de laatste kolom van de gesorteerde matrix te nemen en die dan move to front te coderen zijn dus alleen maar wijzigingen om een beter comprimeerbare tekst te maken – en dat is het volledig nieuwe aan deze aanzet!
Maar omdat er **heel veel** nullen en kleine getallen inzitten is de met de move to front methode gecodeerde Burrows-Wheeler getransformeerde tekst nu bijzonder geschikt om (statisch) Huffman gecodeerd te worden – en dat stellen Burrows en Wheeler inderdaad ook als één van de mogelijkheden voor. Daarbij kan je misschien ook korte strings van nullen als één teken zien.

Oefening 104 *Codeer de tekst* **toon_gaat** *door eerst de Burrows-Wheeler transformatie toe te passen en dan de move to front codering. De lijst voor de codering is (_,0,1,2,3,4,5,6,7,8,9,a,b,g,m,n,o,t).*

Oefening 105 *Decodeer de volgende code: (0,11,10,10,7,0,6,1,0,9,11,4,6,1)*
De code is de move to front code van een Burrows wheeler getransformeerde tekst waarbij het move to front algoritme met de lijst (7,a,e,i,o,u,b,c,n,t,w,_) begonnen is. De 7 is hier natuurlijk een gewoon letterteken en geen getal! Let erop dat de volgorde hier dus niet de ASCII volgorde is – maar als je voor het coderen en decoderen dezelfde volgorde gebruikt, doet dat er niet toe!

Oefening 106 *Als wij de eerste kolom als getransformeerde tekst nemen, kunnen wij de oorspronkelijke tekst niet reconstrueren. Maar hoe zit het met de tweede kolom. Werk een algoritme voor de tweede kolom uit of geef een redenering waarom de tweede kolom als getransformeerde tekst niet geschikt is.*

Oefening 107 *Zou het algoritme werken als je in plaats van de strings in stijgende volgorde te sorteren de strings in dalende volgorde zou sorteren? Zouden er wijzigingen nodig zijn? Kijk bv. naar de redenering dat de volgorde van lettertekens in de eerste en laatse kolom dezelfden zijn. Bewijs dat dat ook met dalende volgorde het geval is of geef een tegenvoorbeeld.*

Oefening 108 *Geef een string $x_t_0 \dots t_{n-1}$ waarbij x de voorstelling van een getal x' is met $0 \leq x' < n$ en $t_0, \dots, t_{n-1}$ lettertekens. Maar deze string mag niet het resultaat van een Burrows-Wheeler transformatie kunnen zijn. Geef uitleg.*

Oefening 109 *Bewijs dat het volgende algoritme dat ook het idee van de Burrows-Wheeler transformatie implementeert juist is.*

Neem de tekst $t = t_0, \dots, t_{n-1}$ en voeg het teken $t_n = \infty$ toe. Daarbij betekent ∞ dat het een teken is dat groter is dan welk teken ook in het alfabet. Schrijf dan s_i voor de string $t_i, \dots, t_n$ (dus de suffix die met teken i begint) en sorteer de strings $s_0, \dots, s_n$ in lexicografische volgorde. Als nu de string s_k op positie j

in deze volgorde staat dan is t_{k-1} het j-de teken van de getransformeerde tekst (met $t_{-1} = t_n$).
De terugtransformatie begint met het teken ∞ – maar zonder het te schrijven – en gaat dan door zoals in de les gezien totdat n tekens geschreven zijn.

Oefening 110 *Geef een algoritme dat de Burrows-Wheeler getransformeerde in lineaire tijd berekent – ook als je de hele tekst als één blok behandelt. Gebruik daarbij de aanzet uit oefening 109. Je bent suffix bomen hopelijk niet vergeten?*

5 Metaheuristieken

Veel problemen kunnen niet optimaal opgelost worden – b.v. problemen die verwant zijn aan NP-complete problemen en voor een grote input opgelost moeten worden. Jammer genoeg geldt dat ook voor veel problemen die in de praktijk belangrijk zijn! Wij moeten in zulke gevallen dus heuristieken toepassen om aanvaardbare oplossingen te vinden. De taak van metaheuristieken is ons te helpen dergelijke heuristieken te ontwikkelen.
Er bestaan ongelofelijk veel verschillende metaheuristieken. In deze les gaan wij een heel kort en oppervlakkig overzicht geven van verschillende metaheuristieken – maar er zijn er nog zeer veel meer! De reden dat wij maar een overzicht geven en bovendien niet echt alle details bespreken is dat als jullie eens in een situatie zouden zijn waar jullie zo'n metaheuristiek moeten toepassen, jullie er toch al heel veel tijd aan moeten besteden om te zien welke trucs voor deze metaheuristiek al gebruikt werden, hoe je hem het best op dit specifieke probleem kan toepassen, hoe je de parameters moet optimaliseren, etc. Dus: Je moet veel meer weten dan men in één enkele les kan vertellen. En als jullie nooit zelf een heuristiek moeten ontwikkelen, dan hebben jullie de details natuurlijk niet nodig. Maar in beide gevallen is het nuttig sommige ideeën te kennen en ongeveer te weten wat er bestaat.

Een metaheuristiek is een heuristiek om heuristieken te ontwikkelen. Dus een manier om voor sterk verschillende problemen heuristische algoritmen te ontwerpen. Maar over welk soort problemen hebben wij het?
In principe heb je altijd een (eindige of oneindige) verzameling M van punten en een doelfunctie (objective function) $f : M \to \mathbb{R}$. Je zoekt het punt x waarvoor $f(x)$ maximaal of minimaal is. In principe moet je er natuurlijk over nadenken, of zo'n punt bestaat, maar in de meeste gevallen (b.v. als de verzameling eindig is) is dat geen probleem.

Voorbeeld 1 *Situatie: Je bent op een vreemde planeet – noemen wij hem eens de aarde – waar er heel veel waas is. Je kan nauwelijks jouw voeten zien, maar je kan wel de hoogte meten van het punt waar je staat. Wat doe je om het hoogste punt van deze planeet te vinden?*
Of met andere woorden:
M is de verzameling van alle punten op de aarde en f is de hoogte van dit punt. Het doel is dus de top van de hoogste berg van de wereld te vinden.
Dit is helemaal geen slecht voorbeeld omdat het gebruikt kan worden (en later gebruikt zal worden) om sommige problemen en oplossingen beter te verstaan. Bovendien kan je andere problemen als analoog *interpreteren – het is gewoon een andere* aarde *en ander begrip van* hoogte.

Welke manieren om maxima van functies te berekenen kennen jullie?

Voorbeeld 2 $M = [-100, 100] \subset \mathbb{R}$, $f(x) = -x^2 + 10x$.
Hier zou je natuurlijk jouw kennis uit de analyse toepassen en f afleiden, bepalen waar $f'(x) = 0$ is en dan kijken of het een maximum of minimum is.
Dus $f'(x) = 10 - 2x$ – dat is 0 voor $x = 5$ en daar heb je ($f''(x) = -2$) een lokaal en ook globaal maximum. Heel gemakkelijk.
Maar hier zien wij meerdere dingen: Wij gebruiken niet alleen maar de verzameling van punten – wij gebruiken ook dat de verzameling een zekere structuur heeft – en dat deze structuur ook iets te maken heeft met de functie (b.v. dat voor punten die heel dicht bij elkaar liggen de waarden van de functie ook heel weinig verschillen. Inderdaad gebruiken wij zelfs dat ze continu en afleidbaar is om de technieken te kunnen toepassen!
Zo'n probleem is ongeveer het beste geval dat je kan tegenkomen.

Maar er zijn ook gevallen waar je niet zo veel structuur hebt:

Voorbeeld 3 $M = \{(x, y) \in \mathbb{N} \times \mathbb{N} | 0 \leq x, y \leq 1.000.000\}$ *en voor elk paar (x, y) kies je een toevallig getal $f(x, y) \in [0, 1 \ \subset \mathbb{R}$.*
Op het eerste gezicht lijkt dit voorbeeld misschien gemakkelijker omdat je maar eindig veel verschillende waarden hebt – maar hoe vind je hier het maximum?
De methoden uit het vorige voorbeeld kunnen wij hier zeker niet toepassen.
De enige manier om **zeker** *te zijn het maximum te hebben gevonden, is naar* **alle** *punten te kijken.*
Ook hier heeft M een zekere structuur – je kan b.v. zeggen dat (x, y) en (x', y') buren zijn als $|x - x'| + |y - y'| = 1$
maar jammer genoeg is er geen verband met $f()$, dus helpt de structuur niet met het vinden van het maximum. Als $f(x, y)$ een hoge waarde heeft, dan is de kans dat voor een buur *(x', y') de functie $f(x', y')$ ook een grote waarde heeft even groot en even klein als voor paren (x', y') die veraf liggen.*
Je hoeft niet naar ingewikkelde strategieën te kijken om dit probleem op te lossen. Elke verzameling van n punten heeft dezelfde kans het maximum te bevatten – als je dus **het** *maximum moet hebben, dan moet je toch al alle punten testen – en als er (b.v. omdat de tijd beperkt is) een bovengrens b voor het aantal punten is, dat je kan testen, kan je om het even welke b punten kiezen en het maximum nemen – de kans dat het een goede benadering is, is altijd dezelfde.*
Maar zo'n probleem is echt het slechtst mogelijke geval...

Algoritme 15 Toevallig zoeken:
Voorwaarde: gegeven een getal n en stel dat wij het maximum zoeken.

- *Kies toevallig (en onafhankelijk) n punten en kies het punt met de grootste waarde van $f()$.*

Het is vrij duidelijk wat je anders moet doen als je het minimum zoekt...
Voor Voorbeeld 2 zou toevallig zoeken zeker niet het optimum vinden. Of het een aanvaardbare oplossing vindt hangt ervan af hoe vaak je kiest en hoe goed de benadering moet zijn.... Het beste algoritme voor dit geval is het duidelijk niet.
Voor Voorbeeld 3 gaat het vrij zeker ook geen goede oplossing vinden, maar toch is het algoritme voor dit voorbeeld min of meer optimaal – er bestaan gewoon geen betere algoritmen....

Oefening 111 *Hoe denk je dat* toevallig zoeken voor het voorbeeld 1 zou presteren? Waarvan hangt het af hoe goed het algoritme presteert ?

Metaheuristieken steunen erop dat – hoewel wij niet in de ideale situatie zitten – het tenminste mogelijk is het zo te modelleren dat de structuur van de verzameling M waarop wij het optimum zoeken *een beetje vriendelijk* is en een beetje verband met de functie vertoont. De *structuur* van M is normaal geen deel van het probleem en ze moet nog vastgelegd worden als wij zo'n structuur nodig hebben.
Het is het gemakkelijkst om te begrijpen als wij gewoon beginnen naar sommige metaheuristieken te kijken.

5.1 Local search

Local search is ongeveer de eenvoudigste metaheuristiek. Het is in principe hetzelfde als een *gerandomiseerd gretig algoritme*. Stel dat er voor elk punt in de verzameling M zekere stappen (of operaties) bekend zijn om van dit punt naar een ander te gaan. Punten waar je naartoe kan gaan, noemen wij buren en voor de verzameling van alle buren van een punt a schrijven wij $N(a)$. Als je van punt a naar punt b in één stap kan komen – dus $b \in N(a)$, dan interpreteren wij dat zo, dat op onze kaart *a dicht bij b ligt.*

Algoritme 16 Local search:
Voorwaarde: Wij weten welke punten buren van elkaar zijn, dus hebben wij de functie $N()$. Stel dat wij het maximum zoeken.

a.) *begin met een toevallig gekozen punt x.*

b.) *kies een toevallige nog niet gekozen stap van jouw positie x naar een buur $y \in N(x)$. Als zo'n stap niet bestaat, stop dan.*

- *als $f(y) > f(x)$, kies dan y als jouw nieuwe positie x en herhaal b.)*
- *anders herhaal b.) met jouw oude positie x*

Soms zijn er variaties op dit algoritme nodig:

- Je moet in stap b.) niet altijd alle mogelijke stappen testen – het is b.v. mogelijk dat er oneindig veel zijn. Je kan ook een bovengrens B vastleggen en als je na B stappen nog geen betere positie hebt gevonden, stop je ook.
- Je kan in b.) ook maar $f(y) \geq f(x)$ in plaats van $f(y) > f(x)$ eisen als je op een andere manier garandeert dat je altijd gaat stoppen (b.v. stop altijd na een zeker aantal stappen of in het geval van een eindige verzameling hou de punten bij waar je al was, etc.)

Oefening 112 *Beschrijf* **precies** *een lokaal zoeken algoritme voor het probleem in Voorbeeld 2. Wat kies je als $N(x)$ voor een punt x? Met welke kans kies je een punt $y \in N(x)$? Denk je dat het algoritme goed presteert?*

In Voorbeeld 1 is het duidelijk wat je normaal als buren (dus $N()$) zou kunnen kiezen: b.v. alle punten met een afstand van ten hoogste 1 meter van een punt x zijn buren van x. De verdeling om toevallig te kiezen zou dan b.v. de uniforme verdeling van al deze punten zijn, maar je kan ook een voorkeur geven aan kleine of grote afstanden.

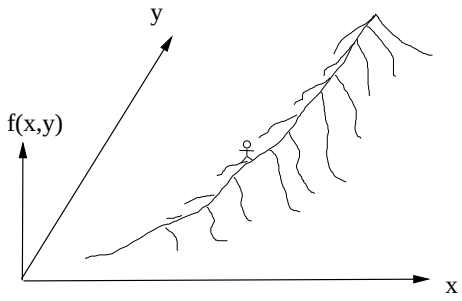
Oefening 113 *Wat zou gebeuren, als je b.v. in de leszaal zou starten en dit algoritme toepassen?*

Oefening 114 *Zou het niet beter zijn (het is maar theoretisch...) toe te laten dat de stappen t.e.m.* 20 000 *km lang mogen zijn? Dan kan je zeker zijn altijd kans te maken het optimum te vinden – om het even waar je start.*

Oefening 115 $M = \{(x,y)|x,y \in \mathbb{Z}\}$*, de doelfunctie* f *is gedefinieerd door* $f(x,y) = (1 + \frac{1}{1+x^2}) \cdot (1-(x-y)^2)$.
Wij definiëren voor elke $k \in \mathbb{N}$ *een buurfunctie:* $N_k(x,y) = \{(v,w) \in M||v-x| \leq k, |w-y| \leq k\}$.

- *Waar is het maximum van deze functie. Waar is zij positief, waar 0, waar negatief.*
- *Stel dat je lokaal zoeken in het punt* $x = (5,5)$ *begint.*
  - *Kan je door middel van lokaal zoeken het maximum vanuit* $x = (5,5)$ *voor elk* k *vinden?*
  - *Als je* N_1 *gebruikt: Wat is in het slechtste geval het aantal stappen dat je nodig hebt, om het maximum te vinden?*
  - *Als je* N_{20} *gebruikt: Hoeveel pogingen heb je ongeveer nodig tot de kans een enkel beter punt te hebben gevonden groter is dan* $1/2$ *?*
  - *Stel dat je ver van het optimum bent, maar wel in een punt* (x,y) *met* $x = y$ *zit. Hoeveel ben je na één stap gemiddeld dichter bij het optimum als je* $N_1()$ *gebruikt en hoeveel als je* $N_k()$ *voor* $k \to \infty$ *gebruikt? Zou je beter kleine of grote* k *gebruiken? Wat is het verschil als je op een vaste afstand van het optimum zit (b.v* $(x+3, y+3)$ *als* (x,y) *het optimum is)?*
  - *Stel dat* M *niet 2-dimensionaal, maar 3-dimensionaal is en* $f()$ *analoog (b.v.* $f(x,y,z) = (1+\frac{1}{1+x^2}) \cdot (1-((x-y)^2+(x-z)^2+(y-z)^2))$*). Zij* $N_k(x,y,z) = \{(v,w,t) \in M||v-x| \leq k, |w-y| \leq k, |t-z| \leq k\}$*. Zou je beter grote of kleine* k *kiezen voor* $N_k()$*? Argumenteer eerst zonder te rekenen en test dan of jouw vermoeden klopt.*

Hier kan je al zien, hoe belangrijk het is goed te kiezen welke stappen toegelaten zijn: Als je heel lange stappen toelaat, kan je zeker het maximum vinden – maar aan de andere kant heb je zoveel keuze dat het alleen maar toevallig zoeken is en je de structuur van het probleem helemaal niet benut.In de volgende situatie



zou je als je voor kleine stappen kiest vrij zeker de weg naar boven vinden. Maar hoe groter de stappen zijn hoe kleiner de kans wordt de top te vinden. Veel kleine stappen zijn hier beter dan weinig grote stappen.
In gevallen waar de structuur van sommige problemen een beetje op de structuur van deze figuur lijkt, is local search een goede oplossing.

Local search herhalen:
Om meer kans te maken niet alleen een lokaal maar een globaal optimum te vinden, wordt de lokale zoekmethode normaal niet één maar meerdere keren toegepast. Dat is inderdaad iets dat voor bijna elke metaheuristiek geldt: Soms is het beter de heuristiek meerdere keren toe te passen en de beste oplossing van de verschillende toepassingen te kiezen dan dure methoden te gebruiken om de resultaten van een enkele toepassing te verbeteren.

Maar wij moeten eens naar een voorbeeld kijken waar het niet zo duidelijk is op welke manier punten buren zijn en wat de stappen zijn:

Voorbeeld 4 *Voor het volgende voorbeeld zou je in de praktijk beter geen metaheuristiek toepassen (Prim, Kruskal), maar het is toch een goed voorbeeld omdat het helpt sommige belangrijke dingen te verstaan:*

Gegeven een graaf $G = (V, E)$ en gewichten $g(e) \geq 0$ voor de bogen van G. Wij zoeken een opspannende samenhangende deelgraaf B van G zodat de som van de gewichten van de bogen in B minimaal is.

Herinnering: een **opspannende** *deelgraaf is een deelgraaf die alle toppen bevat.*

Dus $M = \{B|B$is opspannende samenhangende deelgraaf van $G\}$ en voor $B = (V_B, E_B) \in M$ definiëren wij $g(B) = \sum_{E \in B} g(E)$
Maar wij hebben nog niet vastgelegd wat de structuur *van M is – dus welke elementen wij als* buren *willen beschouwen.*

Eén mogelijkheid:

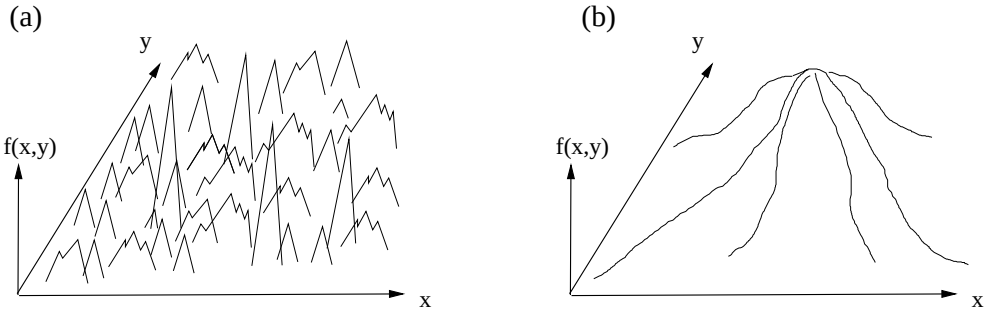
Definitie: (a) *Voor een opspannende deelgraaf $B = (V, E)$ definieren wij $N_a(B) = \{B' = (V', E')|B'$ is opspannende deelgraaf en $\#((E - E') \cup (E' - E)) \leq 1\}$.*

Twee grafen $B_1, B_2 \in M$ zijn dus buren, als ze in maar 1 boog verschillen. Dat betekent dat de ene graaf het resultaat van het toevoegen (of verwijderen) van een boog aan (van) de andere is.

Nu hebben wij alles om lokaal zoeken toe te passen. Wij beginnen met een toevallig gekozen startgraaf die samenhangend is en passen dan lokaal zoeken *toe. Werkt dat goed?*
Het is niet duidelijk hoe je op een toevallige manier een samenhangende deelgraaf moet kiezen – maar dat is een ander probleem.

Stel dat $\bar{B}$ een optimale oplossing is en B_0 de toevallig gekozen startgraaf. Kan het optimum vanuit B_0 worden bereikt? Als $\bar{B} \subset B_0$, kan het door het verwijderen van overbodige bogen zeker bereikt worden. Elke keer dat je een boog verwijdert wordt $g()$ kleiner. Maar als $\bar{B}$ ook maar één enkele boog bevat die niet in B_0 zit, dan is er ten minste één stap waar een boog toegevoegd moet worden – en dat zou leiden tot een graaf met een grotere waarde van $g()$ – dus zou in het geval $\bar{B} \not\subset B_0$ het optimum niet gevonden worden.
Definitie: (b)
$N_b(B) = \{B' = (V', E') | B'$ is opspannende deelgraaf en $\#((E - E)' \cup (E' - E)) \leq 2\}$. Twee grafen $B_1, B_2 \in M$ zijn dus buren, als ze in maar 1 of 2 bogen verschillen.
In het geval dat B_1, B_2 bomen zijn, is het gemakkelijk in te zien (maar ook een leuke oefening...) dat de twee bogen e_1, e_2 waarin ze verschillen op een cykel in $B_1 \cup B_2$ moeten liggen en de ene boog alleen maar in B_1 zit en de andere alleen maar in B_2.
Je kan aantonen (zie oefeningen) dat met definitie (b) vanuit elke samenhangende startgraaf B_0 het globale optimum gevonden kan worden.

Opmerking 25 *De verzameling van punten waarvan wij het optimum moeten vinden en de functie zijn deel van het probleem – daarop heb je geen invloed. De definitie van* buren *is deel van de methode die je toepast en kan je dus kiezen.*
Dit voorbeeld toont aan dat het buitengewoon belangrijk is, op een goede manier te kiezen wat buren zijn. Dat geldt niet alleen voor lokaal zoeken maar ook voor andere metaheuristieken die wij gaan zien. Soms is het helemaal niet gemakkelijk uit te vissen wat een goede manier is om $N()$ te definiëren – soms bestaan zo'n manieren misschien zelfs niet...
Het volgende beeld toont heel ongeveer het verschil tussen de twee definities: de ene keer ($N_a()$) heb je heel veel verschillende lokale maxima en je komt elke keer in een ander lokaal maximum terecht. De andere keer ($N_b()$) heb je maar één lokaal maximum dat ook het globale maximum is en waar je vanuit elk punt naartoe kan komen en waar je met elke stap vooruitgang boekt.
Als je het beeld van een landschap gebruikt, kan je dus zeggen dat de wereld er helemaal anders kan uitzien *als je een andere definitie van $N()$ gebruikt. Dat is natuurlijk niet echt onverwacht...*
In de beelden die wij tekenen, is M altijd 1- of 2-dimensionaal omdat je het anders niet kan tekenen. De werkelijke dimensie is in de meeste gevallen **veel** *groter...*



Oefening 116 *Stel dat G een complete graaf is met* 30 *toppen en precies één opspannende samenhangende deelgraaf D heeft minimaal gewicht. Je kiest een begingraaf door elke boog met kans 1/2 te kiezen. Als deze graaf niet samenhangend is, stop je. Anders pas je* lokaal zoeken *met de definitie $N_a()$ van* buren *toe.*

- *Hoe groot is de kans met 50 pogingen een startgraaf te kiezen die het toelaat het optimum te vinden?*
- *Hoe vaak moet je het algoritme herstarten om met kans tenminste 1/2 tenminste één keer van een startgraaf vertrokken te zijn waar het mogelijk is, D te vinden?*
- *Stel dat je een startgraaf met 100 bogen hebt die het toelaat D te vinden. Stel dat alle stappen gelijke kans maken en dat de graaf altijd samenhangend blijft als je maar 10 bogen verwijdert. Hoe groot is de kans dat je na 10 stappen nog altijd met een graaf zit die het toelaat D te vinden?*
- *Wat is de verwachtingswaarde van het aantal bogen in de startgraaf?*

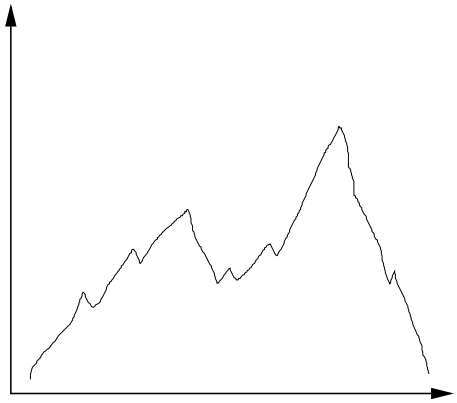
Oefening 117 *Toon aan, dat de definitie (b) van* buren *ervoor zorgt dat vanuit elke samenhangende startgraaf G een globaal optimum bereikt kan worden.*

Oefening 118 *Vertaal het probleem met de minimum opspannende deelgraaf naar een ander probleem (dus: waar een andere functie geoptimaliseerd moet worden), zodat het lokaal zoeken algoritme het toelaat met een toevallig gekozen graaf te beginnen (ook al is hij niet samenhangend) en waar de functie die geoptimaliseerd wordt ervoor zorgt dat altijd – om het even wat de startgraaf is – de juiste grafen als optimum gevonden worden.*

- *Geef een definitie van de functie die geoptimaliseerd moet worden, de nieuwe definitie van M en de nieuwe definitie van* buren.

5.2 Guided local search

Het probleem is dat lokaal zoeken gemakkelijk in lokale optima kan terechtkomen – zonder een kans te ontsnappen. Het onderwerp van de meeste zoekalgoritmen is vooral manieren te bedenken hoe je uit zo lokale optima kan ontsnappen – zonder echt op een toevallige manier opnieuw te beginnen. De hoop is dat dicht bij goede lokale optima er misschien zelfs nog betere zijn...



Stel dat een verzameling M en een doelfunctie $f()$ gegeven zijn en dat $N()$ ook al vastligt.
Het idee van geleid lokaal zoeken of guided local search is dat je als je in een lokaal optimum x vastzit de functie $f()$ vervangt door een functie $f'()$ waarvoor x geen lokaal optimum meer is en dan opnieuw lokaal zoeken toepast. Natuurlijk moet er wel een sterk verband tussen $f()$ en $f'()$ zijn. De gewone manier van doen is dat $f()$ een deel van $f'()$ is, maar f' veranderd wordt als je vastzit. Als je in een lokaal optimum zit, ken je een *straf* toe aan een eigenschap van het punt x en zo kan je soms ontsnappen.

Algoritme 17 Guided local search:
Voorwaarde: Gegeven M, $f()$ en $N()$. Stel dat wij het maximum zoeken.

a.) *$f'() = f()$ en kies een toevallig punt x en als beste oplossing b kies je $f(x)$.*

b.) *Herhaal de volgende stappen tot aan een zekere eindvoorwaarde is voldaan (b.v. een zekere tijd is verstreken, een zeker aantal stappen is gedaan, etc.):*

- *Pas lokaal zoeken toe om een lokaal optimum x van $f'()$ te vinden.*
- *Als $f(x) > b$ (**niet** $f'(x)$) dan zet je $b = f(x)$.*
- *Definieer een nieuwe functie $f'()$ die het (hopelijk) toelaat het lokale optimum x te verlaten. (Hoe je dat precies kan doen zien wij in het volgende deel.)*

c. *Geef b als resultaat terug.*

In principe klinkt dat heel gemakkelijk, maar de kunst is de functie $f'()$ op een manier te kiezen dat de nieuwe lokale optima ook echt goed zijn voor $f()$. Je kan zeker vele manieren bedenken om dit idee te verwezenlijken, maar in het volgende gaan wij één mogelijkheid zien:

Het principe is voor sommige eigenschappen van een punt een *straf* toe te kennen. Daardoor geef je voorkeur aan oplossingen met een andere structuur – oplossingen die deze eigenschap niet hebben. Zo'n eigenschap kan van alles zijn.

- In Voorbeeld 1 b.v. *je staat op een groene plek* of *het punt waar je staat is van steen* etc.

- In Voorbeeld 4 b.v. *de maximale graad van de graaf is kleiner dan* 4 of *de oplossing bevat geen cykel* of *de oplossing bevat een cykel* of *er is een partitie van de toppen van G in 2 verzamelingen M_1, M_2 zodat de oplossing niet de kleinste boog tussen M_1 en M_2 bevat* of ...

De laatste eigenschappen houden duidelijk verband met het probleem, maar de anderen kunnen ook gebruikt worden...
Natuurlijk mogen wij alleen maar eigenschappen gebruiken, die efficiënt berekend kunnen worden – ten slotte moet een heuristiek snel zijn.

Bovendien leggen wij voor elke eigenschap E_i een gewicht g_i vast. Als wij nu k eigenschappen $E_1, \ldots, E_k$ hebben, dan definiëren wij de functie $f'(x)$ als volgt. Wij gebruiken een getal λ dat wij op voorhand kiezen en een variabele a_i waarin wij bijhouden hoe vaak eigenschap i al in een lokaal optimum gebruikt werd om $f'()$ te wijzigen. Bovendien hebben wij

$$I_i(x) = \begin{cases} 1 & \text{als } x \text{ eigenschap } E_i \text{ heeft} \\ 0 & \text{anders} \end{cases}$$

Dan definiëren wij

$$f'(x) = f(x) - \lambda \cdot \sum_{i=1}^{k} (a_i \cdot I_i(x))$$

Behalve $N()$ en in dit geval de E_i hebben veel metaheuristieken nog één of meerdere parameters (in dit geval zijn dat λ, en de g_i) die je kan kiezen. Dan kan je testen met verschillende parameters draaien en bepalen welke parameters voor het specifieke probleem de beste zijn.
In het begin zijn dus alle a_i gelijk aan 0 en dus $f'(x) = f(x)$. Maar tijdens het algoritme worden de a_i gewijzigd, zodat $f'()$ later verschilt van $f()$.
Als wij een optimum bereiken – dat lokaal of globaal kan zijn – berekenen wij voor elke eigenschap E_i waaraan het lokale optimum voldoet, de waarde $q_i = g_i/(1 + a_i)$ en kiezen alle i waarvoor q_i het grootst is. Deze a_i's verhogen wij met 1, waardoor wij een nieuwe functie $f'()$ krijgen die het misschien toelaat dit optimum te verlaten en een nieuwe te vinden.

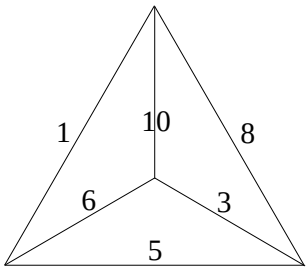
Voorbeeld 5 *Wij gebruiken Voorbeeld 4 met de definitie N_a van buren. Wij weten al dat je zo heel gemakkelijk in een lokaal optimum terecht kan komen. Wij gebruiken de eigenschappen*

E_1 *De maximale graad van de graaf is kleiner dan* 4.

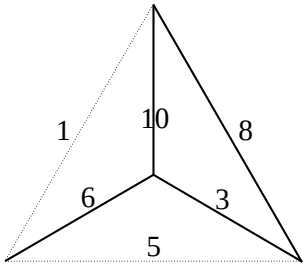
E_2 *De oplossing bevat geen cykel.*

E_3 *De oplossing bevat een cykel.*

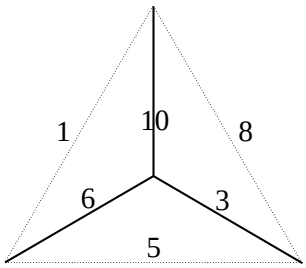
Wij kiezen $\lambda = 2$ en $g_1 = 2, g_2 = g_3 = 1$. $f(x)$ is de som van de gewichten van de bogen in de graaf x en wij zoeken het minimum. (Wij zouden natuurlijk even goed het maximum kunnen zoeken nadat we de functie met -1 vermenigvuldigd hebben.)
Stel dat de graaf G de K_4 is met gewichten als volgt:



en dat de random startgraaf x_0 *zoals volgt is:*



Dus $f(x_0) = 27$. *Als wij nu lokaal zoeken toepassen, dan zou b.v. het verwijderen van de boog met gewicht* 8 *een bewerking zijn, die aanvaard wordt en ons naar een lokaal optimum leidt waar lokaal zoeken stopt:*

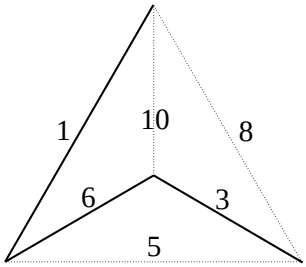


Voor dit optimum is $f(x) = 19$ – *het is dus een nieuwe beste oplossing. Deze graaf heeft eigenschappen* E_1 *en* E_2 *en wij berekenen* $q_1 = 2/(1+0) = 2$ *en* $q_2 = 1/(1+0) = 1$. *Dus wordt* E_1 *gekozen en wij hebben nu* $a_1 = 1$ *en (let op – wij zoeken hier een minimum in plaats van een maximum)*

$$f'(x) = \begin{cases} f(x)+2 & \textit{als alle graden kleiner dan 4 zijn} \\ f(x) & \textit{anders} \end{cases}$$

Als wij opnieuw lokaal zoeken toepassen, zien wij, dat wij het lokale minimum niet kunnen verlaten – het nieuwe lokale minimum is het oude.
Maar nu is $q_1 = 2/(1+1) = 1$ *en* $q_2 = 1/(1+0) = 1$. *Dus worden* E_1 *en* E_2 *gekozen en wij hebben nu* $a_1 = 2$ *en* $a_2 = 1$ *en*
$f'(x) = f(x) + e_1 + e_2$
waarbij $e_1 = 4$ *als* x *eigenschap* E_1 *heeft en anders* 0 *en* $e_2 = 2$ *als* x *eigenschap* E_2 *heeft en anders* 0.
Het lokale optimum x *heeft dus* $f'(x) = 19 + 4 + 2 = 25$. *Als wij opnieuw lokaal zoeken toepassen zien wij dat wij nu wel kunnen ontsnappen: Door de*

boog met gewicht 1 toe te voegen krijgen wij een cykel in de nieuwe graaf x' en dus $f'(x') = 20 + 4 = 24$. Daarna kan lokaal zoeken b.v. boog 10 verwijderen. dan hebben wij $f'(x'') = 10 + 4 + 2 = 16$ en wij zitten in een nieuw lokaal minimum waarvoor de functie met $f(x) = 10$ een nieuwe beste waarde oplevert.



Dat is nog altijd niet het globale optimum maar het is duidelijk hoe het werkt. En wij hebben ook gezien waarom het goed was niet gewoon van een nieuwe startgraaf te vertrekken: een beetje informatie over het globale optimum zit ook in de lokale optima – in dit geval bevatten ook lokale optima vaak veel bogen met een klein gewicht.

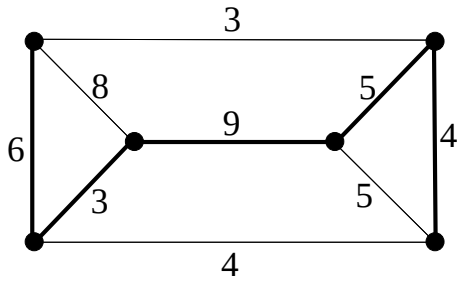
Oefening 119

- *Kan je als je met het algoritme doorgaat het globale optimum ook vinden?*
- *Zijn de waarden van λ en de g_i goed gekozen? Welke waarden zouden beter zijn?*

Natuurlijk is dit een klein en onnozel voorbeeld maar jammer genoeg zijn echte en interessante voorbeelden niet met de hand te doen ...

De ideeën van *guided local search* kunnen natuurlijk niet alleen op lokaal zoeken toegepast worden – ze werken met om het even welke metaheuristiek. Je moet gewoon de stap waar lokaal zoeken wordt gebruikt door een stap vervangen waar je een andere metaheuristiek gebruikt. Maar omdat lokaal zoeken gemakkelijk in slechte lokale optima terechtkomt terwijl de andere metaheuristieken zelf ideeën bevatten om dat te voorkomen, is het voor lokaal zoeken bijzonder belangrijk.

Oefening 120 *Pas het beschreven algoritme voor guided local search toe op de volgende graaf met de gegeven startdeelgraaf:*



Als je betere eigenschappen kan bedenken of betere parameters kan kiezen dan die in de les, gebruik die dan!

5.3 Variable neighbourhood search

Terwijl guided local search de definitie van de doelfunctie wijzigt om uit een lokaal optimum te ontsnappen, wijzigt – of beter vervangt – *variable neighbourhood search* de definitie van buur (dus $N()$) om te ontsnappen. Een globaal optimum is natuurlijk ook een lokaal optimum voor **elke** definitie van $N()$.
Als elk punt weinig buren heeft, kan je sneller zoeken en als er een buur is die naar een goede oplossing leidt, is de kans bovendien groter dat je hem ook kiest. Daarom is het goed als een punt niet te veel buren heeft. Jammer genoeg kan je dan snel in een lokaal optimum vastzitten.
Variable neighbourhood descent en variable neighbourhood search gebruiken een compromis: zij gebruiken meerdere definities van buren – maar niet tegelijk. Op deze manier heb je elke keer maar weinig buren die je moet onderzoeken en aan de andere kant als je naar alle definities kijkt toch veel richtingen die je in kan gaan om uit slechts lokale optima te ontsnappen.
Beide metaheuristieken gebruiken de ene definitie met niet te veel buren per punt om snel een lokaal optimum te vinden en dan een andere definitie om (hopelijk) uit dit lokale optimum te ontsnappen.

Algoritme 18 Variable neighbourhood descent
Gegeven een verzameling M, een doelfunctie $f : M \to \mathbb{R}$ waarvan wij het maximum zoeken en m buurfuncties $N_1(), \ldots, N_m()$. Bovendien hebben wij een voorwaarde om te stoppen (tijd, aantal stappen, etc.).

(a) *Kies een startpunt x – b.v. op een toevallige manier.*

(b) *Herhaal de volgende stappen tot aan de stopvoorwaarde is voldaan:*

- *begin met $k = 1$ en definitie $N_k() = N_1()$*
- *herhaal de volgende stappen tot en met $k = m$*

  Zoeken: *Zoek het beste punt $x' \in N_k(x)$.*

  Volgende stap vastleggen: – *Als $f(x') > f(x)$ neem dan x' als nieuw punt en begin opnieuw met $k = 1$.*
  – *Anders: $k = k + 1$*

Het is dus gewoon een gretige versie van lokaal zoeken met verschillende definities van buur en je stopt zodra geen enkele definitie van buur naar een beter punt leidt. Het feit dat je naar alle buren moet kijken zorgt er al voor dat elke functie $N()$ niet te veel buren per punt mag vastleggen om efficiënt te zijn.

Oefening 121 *Kan de prestatie van VND veranderen als de volgorde van de functies $N_1(), \ldots, N_m()$ verandert?*

Oefening 122 *Metaheuristieken zijn vooral belangrijk voor heel moeilijke problemen – b.v. NP-complete problemen. Stel dus dat $P \neq NP$ wat betekent dat voor deze algoritmen geen polynomiale algoritmen bestaan. Wie niet weet wat* NP-compleet *betekent, zou dat voor deze oefening beter nog eens opzoeken...*

Gegeven een graaf G_0. Zij dan $M = \{G | G \text{ is deelgraaf van } G_0\}$. Wij schrijven $h(G)$ voor de lengte van een kortste cykel in G. Als er geen cykels zijn, is $h(G) = 0$. De doelfunctie is $f(G) = 2 \cdot h(G) - |E(G)|$ en wij zoeken het maximum.

- *Kan je een deelgraaf beschrijven waarvoor deze functie maximaal is?*

Wij willen variable neighbourhood descent *toepassen voor een vast aantal k van buurfuncties $N_1(), \ldots, N_k()$. Stel dat de buurfuncties de buren in een zekere volgorde geven en niet als een verzameling zonder orde. Als er in de* zoek-*stap van VND meerdere even goede punten (grafen) zijn waar je naartoe kan gaan, neem je altijd de eerste van deze grafen in de volgorde van de buurfunctie die je gebruikt.*
Het is belangrijk voor de efficiëntie dat je niet te veel buren hebt en dat die ook niet te ingewikkeld berekend moeten worden. Stel dus dat voor elke graaf en elke $N_i()$ geldt $|N_i(G)| \leq |E(G_0)|$ en dat het ten hoogste lineaire tijd per buur vraagt om de buren te berekenen.
Natuurlijk proberen wij de $N()$'s op een manier te kiezen dat VND zo goed mogelijk presteert – maar hoe goed kan dat ?

- *Toon aan dat het (met de gegeven voorwaarden) niet mogelijk is $N_1(), \ldots, N_k()$ te vinden zodat het mogelijk is dat VND vanuit elke begingraaf het optimum kan bereiken.*
- *Is het mogelijk $N_1(), \ldots, N_k()$ te vinden zodat VND als het vanuit G_0 vertrekt het optimum kan vinden?*
- *Werken jouw argumenten ook als je b.v. in het geval van meerdere even goede punten voor de volgende stap de buren toevallig kiest?*
- *Kan je iets zeggen over hoe moeilijk het is voor een gegeven G_0 goede begingrafen te berekenen – dat zijn begingrafen zodat je met VND altijd het optimum vindt?*

Deze VND-heuristiek kan je alleenstaand gebruiken of als een deel van *variable neighbourhood search*:

Algoritme 19 Variable neighbourhood search
Gegeven een verzameling M, een doelfunctie $f()$ waarvan wij het maximum zoeken en m buurfuncties $N_1(), \ldots, N_m()$. Bovendien hebben wij een voorwaarde om te stoppen (tijd, aantal stappen, etc.).

(a) *Kies een startpunt x – b.v. op een toevallige manier.*

(b) *Herhaal de volgende stappen tot aan de stopvoorwaarde is voldaan:*

- *begin met $k = 1$ en buurfunctie $N_k = N_1$*
- *herhaal de volgende stappen tot en met $k = m$*

  Schudden: *Kies een toevallig punt $x' \in N_k(x)$. Houd er geen rekening mee of $f(x') \geq f(x)$.*

  Zoeken: *Zoek een lokaal optimum x'' door van het net gekozen punt x' te vertrekken. Dat kan je doen door lokaal zoeken waarbij definitie N_1 of N_k wordt toegepast, of beter door variable neighbourhood descent toe te passen.*

  Vergelijken: *– Als het nieuwe optimum x'' beter is dan het tot nu toe beste, kies $x = x''$ en $k = 1$.*

– *Anders verhoog k dus* $k \leftarrow k + 1$.

Oefening 123 *Geef 2 functies $N_1(), N_2()$ voor Voorbeeld 4 zodat voor geen van de twee functies lokaal zoeken altijd het optimum vindt – maar VND wel.*

Oefening 124 *Ontwerp een VNS algoritme voor het handelsreizigersprobleem. Gebruik ten minste 3 functies $N_1(), N_2(), N_3()$. Pas jouw algoritme toe op een klein voorbeeldje.*

Oefening 125 *Gegeven een graaf G. M is de verzameling van alle deelgrafen van G en voor $G \in M$ zij $f(G) = 1$ als G een opspannende cykel (dus een Hamiltoniaanse cykel) is en $f(G) = 0$ anders. Van welke van de tot nu toe gezien metaheuristieken denk je dat die het best presteert. Wat is het probleem met deze functie? Hoe kan je dat oplossen?*

Vaak worden de buurfuncties op een manier gekozen dat voor elk punt x geldt $N_1(x) \subset N_2(x) \subset \cdots \subset N_m(x)$. Omdat $N_1()$ het meest wordt gebruikt is het belangrijk dat die niet te groot is – dus dat er bijzonder snel mee gewerkt kan worden. $N_m(x)$ wordt duidelijk minder gebruikt, dus is het niet zo'n groot probleem als die duidelijk meer buren bevat.

5.4 Simulated annealing

Behalve in één stap in VNS zijn wij tot nu toe altijd naar betere punten gegaan. Wij hebben andere doelfuncties gekozen om uit lokale optima te kunnen ontsnappen, wij hebben de definities van buur gewijzigd, maar wij zijn nooit naar punten gegaan waar de op dat moment gebruikte doelfunctie slechter was.
Als je opnieuw het beeld van een landschap en het hoogste punt gebruikt, dan lijkt het vrij kunstmatig het landschap (de definitie van buur) of de interpretatie van *hoogte* (de doelfunctie) te wijzigen. De enige manier om uit een lokaal minimum te ontsnappen, is gewoon naar beneden te gaan – of met andere woorden: soms ook naar slechtere punten te gaan. De vraag is natuurlijk **wanneer** je naar slechtere punten gaat. Als je altijd naar punten in de buurt gaat – om het even of ze beter of slechter zijn – loop je gewoon toevallig rond. Dan kan je beter helemaal toevallige punten kiezen (dus *toevallig zoeken*).
In S.Kirkpatrick, C.D. Gelatt, M.P. Vecchi: *Optimization by Simulated Annealing*, Science 220, n. 4598, pp 671–680 (1983) werd *simulated annealing* voor de eerste keer voorgesteld.
De techniek van simulated annealing of gesimuleerd temperen is het nabootsen van wat mensen deden al lang voordat computers werden uitgevonden. Als ijzer vloeibaar is, hebben de atomen een heel grote energie – voldoende om niet op een manier gebonden te zijn die optimaal is qua energie. Ze kunnen dan ook naar slechtere posities gaan – gewoon omdat ze ervoor voldoende energie hebben. Tijdens het afkoelen wordt het dan altijd moeilijker energetisch slechte posities te bereiken en ten slotte zit elk atoom in een lokaal minimum waaruit het niet kan ontsnappen. Hoe dicht dat bij een globaal minimum zit, hangt van meerdere parameters af – b.v. hoe snel het materiaal afkoelt. Voor sommige materialen is het temperen ook nadat ze gelast zijn heel belangrijk om een stevig en duurzaam materiaal te hebben. De exacte parameters (hoe heet in het begin, hoe snel afkoelen, etc.) zijn voor elk materiaal verschillend. En inderdaad zijn de corresponderende parameters ook voor **gesimuleerd** temperen heel belangrijk.

Het idee is dus dat je in het begin een vrij hoge temperatuur hebt. Dat stelt je in staat ook naar punten te gaan die (qua energie – of in ons geval qua doelfunctie) veel slechter zijn dan waar je bent. Of precies: de **kans** is groot dat het lukt. Maar hoe meer tijd verstrijkt (hoe meer stappen je doet), hoe minder hoog de temperatuur is – dus hoe minder je naar slechtere punten kan gaan (of precies: hoe minder de **kans** is dat dat lukt). In feite is het in de realiteit zo (of tenminste in het model dat de fysica ervan heeft) dat het voor een zekere temperatuur niet zo is, dat sommige stappen naar een slechter punt altijd aanvaard worden en andere nooit, maar het is zo, dat alle met een zekere **kans** aanvaard worden. De kans van x naar x' te gaan daalt gewoon als $f(x) - f(x')$ groot is (als wij het maximum zoeken). En bovendien daalt de kans ook als de temperatuur kleiner is. Dus is het *in principe* nog altijd mogelijk ook naar een duidelijk slechter punt te gaan als de temperatuur klein is – de kans dat echt te doen, is gewoon heel klein.
Er zijn ook heuristieken, die geen kansen gebruiken, maar met drempels werken (thresholds) – dus tot een zekere drempel slechtere punten **altijd** aanvaarden en boven de drempel **nooit**. Ook dergelijke metaheuristieken (*threshold accepting* of *great deluge*) werken heel goed in sommige gevallen. Soms beter dan simulated annealing, soms slechter... Maar dat is iets voor een ander hoofdstuk (niet in deze les).
Dus: wat hebben wij nodig?

- Een temperatuurfunctie $t : \mathbb{N} \to \mathbb{R}_+ \setminus \{0\}$. De waarde $t(k)$ zegt hoe hoog de temperatuur in stap k is. Om goed te werken, zou $t()$ het best monotoon dalend zijn en $lim_{k\to\infty} t(k) = 0$.

- Een functie $p : \mathbb{R} \times \mathbb{R} \to [0, 1]$ die bepaalt hoe groot de kans is, een gekozen stap ook echt te doen. Als je in punt x bent en naar punt x' wilt gaan, waarbij $\Delta = f(x') - f(x)$ en de temperatuur is t, dan is $p(\Delta, t)$ de kans dat je dat ook echt doet. De kans hangt dus niet direct van de punten af, maar alleen van het verschil in de doelfuncties.

Een mogelijke functie $t()$ is bijvoorbeeld $t(n) = T_0 \cdot \lambda^{\lfloor n/L \rfloor}$ waarbij de begintemperatuur $T_0 \in \mathbb{R}$, $T_0 > 0$ en $L \in \mathbb{N}$, $L > 0$ en $\lambda \in \mathbb{R}$, $0 < \lambda < 1$ parameters zijn, die voor elke toepassing gekozen moeten worden – b.v. door testen te draaien met verschillende waarden en de resultaten te vergelijken. Eén manier om T_0 te kiezen, is b.v. een bovengrens te bepalen voor het maximale verschil $|f(x') - f(x)|$ als dat gemakkelijk is om te bepalen. Dat werkt vaak vrij goed, maar je kan T_0 ook anders kiezen.
Hier kan je *lambda* en L *in principe* ook tot één parameter samenvatten als je de $\lfloor . \rfloor$ weglaat, maar blijkbaar is het eenvoudiger als je die onafhankelijk van elkaar kan kiezen bij het optimaliseren van de parameters...
Als het maximum wordt gezocht – het dus in principe goed is, naar x' te gaan als $f(x') > f(x)$ wordt normaal als $p()$ de volgende functie gebruikt:

$$p(\Delta, t) = \begin{cases} 1 & \text{als } \Delta > 0 (\text{ dus } f(x') > f(x)) \\ e^{\Delta/t} & \text{anders. Omdat } \Delta \text{ negatief of nul is, is dat } \leq 1 \end{cases}$$

Deze functie beschrijft ongeveer de realiteit van het temperen van metalen – ze komt dus uit de fysica en kan met hulp van de Boltzmann verdeling berekend worden. Algoritmen die deze functie ook voor simulated annealing toepassen,

werken vaak vrij goed – maar *in principe* kan je natuurlijk ook andere functies gebruiken die ongeveer vergelijkbare eigenschappen hebben. Het feit dat ook algoritmen die de metaheuristiek *threshold accepting* toepassen goed presteren, tonen dat de details van deze functie niet altijd belangrijk zijn. Je kan threshold accepting interpreteren als simulated annealing met een heel gemakkelijke functie $p()$.

Oefening 126 *Wat moet je aan $p()$ wijzigen als je het minimum zoekt in plaats van het maximum?*

Oefening 127 *Welke eigenschappen zou jij van zo'n functie $p()$ eisen om te kunnen verwachten dat jouw algoritme goed werkt?*
Met welke eigenschappen zou jij zeker zijn dat hij niet goed werkt?

Algoritme 20 Simulated annealing
Gegeven een verzameling M, een doelfunctie $f()$ waarvan wij het maximum zoeken, een buurfunctie $N()$ en de functies $t()$ en $p()$. Bovendien hebben wij een voorwaarde om te stoppen (tijd, aantal stappen, etc.).

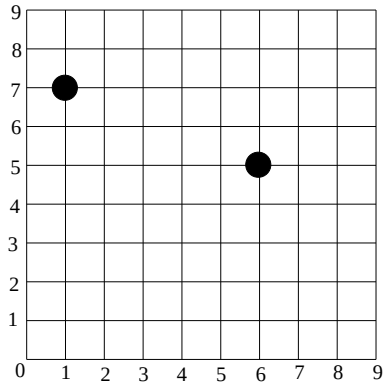
(a) *Kies een startpunt x – b.v. op een toevallige manier.*

(b) $s = 0$ *(s houdt het aantal stappen bij)*
Herhaal de volgende stappen tot aan de stopvoorwaarde is voldaan.

- *Kies een toevallig punt $x' \in N(x)$.*
- *Bereken $\Delta = f(x') - f(x)$.*
- *Kies $x = x'$ met kans $p(\Delta, t(s))$.*
- $s = s + 1$

Omdat simulated annealing een relatief oude techniek is, bestaan er natuurlijk al heel veel artikels over hoe je het basisalgoritme kan verbeteren, hoe je de parameters voor sommige problemen moet kiezen, etc.
Iets dat onmiddelijk een goed idee lijkt, is b.v. het beste gevonden punt altijd bij te houden. Er werd aangetoond dat simulated annealing (onder zekere voorwaarden) naar het optimum convergeert – maar dat vraagt in de praktijk meer tijd dan je ervoor kan/wil besteden.

Oefening 128 *De bedoeling is 2 verschillende toppen p_1, p_2 van een 10×10 tralie met een zekere eigenschap te vinden: Als de toppen de coordinaten (x, y) en (x', y') hebben dan is de afstand $d((x, y), (x', y')) = |x - x'| + |y - y'|$. Wij hebben $M = \{(x, y, x', y') | 0 \leq x, y, x', y' \leq 9, (x, y) \neq (x', y')\}$ dus de verzameling van alle mogelijke posities van de twee punten. De doelfunctie waarvoor je het maximum zoekt is gedefinieerd als $f(x, y, x', y') = |2 - d((4, 5), (x, y))| + |3 - d((x, y), (x', y'))| + |2 - d((5, 5), (x', y'))|$.*



Definieer functies $N()$, $t()$ en $p()$ en pas simulated annealing toe door sommige stappen met de hand uit te werken.

Oefening 129 *Gegeven k punten $p_1, \ldots, p_k$ in het vlak (dus in $\mathbb{R}^2$). Ze moeten op een manier geplaatst worden, dat voor elk paar geldt dat de afstand zo dicht mogelijk bij 1 is. Natuurlijk is het voor grote k niet mogelijk dat op een perfecte manier te doen.*
Wij definiëren de fout als

$$f(p_1, \ldots, p_k) = \sum_{1 \leq i < j \leq k} (d(p_i, p_k) - 1)^2$$

waarbij $d()$ de gewone Euclidische afstand is.

- *Wat is de grootste k waarvoor je een oplossing met fout 0 kan vinden? Toon aan dat jouw antwoord juist is.*
- *Ontwerp een simulated annealing algoritme, dat vertrekt van een toevallige plaatsing van de punten in het vierkant $[0, 10] \times [0, 10]$. Denk er goed over na wat je als mogelijke stappen wil kiezen en voorspel wat jouw algoritme gaat doen. Houd rekening met het aantal mogelijkheden voor elke stap (degrees of freedom).*
- *Implementeer jouw algoritme en vergelijk het met dat van jouw collega's. Kloppen jouw voorspellingen? Zijn er programma's die duidelijk beter/slechter presteren? Waarom?*

Oefening 130 *Gegeven n reële getallen $0 < a_1, \ldots, a_n < 1$. Gezocht is een deelverzameling $a_{i_1}, \ldots, a_{i_k}$ van getallen zodat*

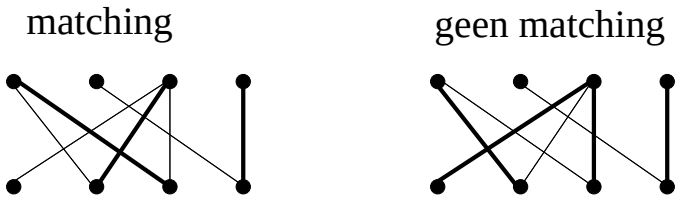
$$0 < \sum_{j=1}^{k} a_{i_j} \leq 1$$

en deze som maximaal is.
Ontwerp een simulated annealing algoritme voor dit probleem.
Geef alle definities en parameters die nodig zijn, om het algoritme volledig te beschrijven maar het is niet nodig dat ze ook zo gekozen zijn dat het algoritme goed werkt. Daarvoor zou je het echt moeten implementeren en testen laten draaien. Het algoritme moet alleen maar in principe *werken en het optimum kunnen vinden (ook al zou dat veel te lang duren en te veel pogingen vragen).*
De bedoeling is gewoon om aan te tonen dat het principe verstaan is.

5.5 Genetische algoritmen

Net zoals simulated annealing, proberen ook genetische algoritmen een optimalisatieprocess dat al lang gekend is, op de computer na te bootsen. In dit geval is dat de evolutie. De doelfunctie die door de evolutie geoptimaliseerd wordt is de fitness – dat is hoe goed een individu bij zijn omgeving past. De *fitness* is niet altijd **precies** de doelfunctie – soms is het noodzakelijk de doelfunctie een beetje te wijzigen (dat gaan wij ook in een voorbeeld zien), maar *in principe* bepaalt de doelfunctie de fitnessfunctie. Iets dat je genetische algoritmen zou kunnen noemen werd al in de jaren 60 voorgesteld (Rechenberg, Schwefel) toen de computers er nog helemaal niet klaar voor waren. Maar toen was het vooral het principe van *mutatie en selectie* van één individu dat werd gesimuleerd. Het resultaat is iets dat je goed met lokaal zoeken kan vergelijken: mutatie betekent een stap naar een buur en selectie betekent deze stap aanvaarden of niet aanvaarden.
Op dit moment wordt het begrip *genetische algoritmen* vooral gebruikt voor algoritmen van het soort dat wij nu zullen zien. Dergelijke algoritmen werden in de jaren 70 geïntroduceerd (Holland 1975) en ze simuleren de evolutie van een hele populatie.
Als je evolutie **heel** kort wil beschrijven, dan kan je dat als volgt: je begint met een populatie en elk individu heeft een zekere fitness. In elk stap worden kinderen verwekt en sommige individuen sterven. De kans kinderen te verwekken en de kans te sneuvelen hangen allebei af van de fitness van de individuen. Als er kinderen verwekt worden, dan zijn dat geen identieke kopieen van hun ouders, maar er gebeuren kleine veranderingen (mutaties) en de eigenschappen van de ouders worden ook vermengd (crossover). Na voldoende stappen is de fitness van de hele bevolking gestegen... Dit process zullen wij simuleren.
Voor genetische algoritmen gaan wij een nieuw voorbeeld gebruiken:

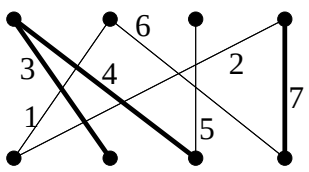
Voorbeeld 6 *Gegeven een graaf $G = (V, E)$. Een matching is een verzameling $E' \subseteq E$ zodat elke top in ten hoogste één boog uit E' zit. Ons doel is voor een gegeven graaf G de grootste matching te bepalen.*



Ook voor dit probleem zou je het best geen metaheuristiek toepassen – er bestaan polynomiale algoritmen – maar het is heel geschikt om de principes van genetische algoritmen te tonen.

Als verzameling M kiezen wij alle mogelijke verzamelingen van bogen van G. Dan kunnen wij als fitness natuurlijk niet het aantal bogen kiezen omdat wij dan altijd lokale extrema zouden vinden die geen matchings zijn. Voor een verzameling E' van bogen definiëren wij $f(E') = |E'| - 2 \cdot |E'_f|$ waarbij E'_f de deelverzameling van bogen van E' is, waarvan tenminste één eindpunt in tenminste 2 bogen zit. Het is gemakkelijk om in te zien dat het globale maximum

altijd een deelverzameling van bogen is die een maximale matching vormt. Als wij één van de andere metaheuristieken die met stappen werkt, zouden toepassen, zou het ervan afhangen hoe wij $N()$ definiëren of ook alle lokale maxima matchings zijn.
Voor genetische algoritmen (wij schrijven kort GA) wordt in de meeste gevallen een expliciete voorstelling van een punt in M als een string met tekens uit een eindig alfabet (b.v $\{0,1\}$ of zoals in de biologie $\{A,C,G,T\}$) geeist. Deze string heet het genotype en het punt dat het voorstelt het fenotype – zoals ook in de biologie. Het fenotype is dus waar wij echt in geïnteresseerd zijn en het genotype zoiets als het gen van een mens – een codering.
Wij hebben dus een injectieve functie c van de verzameling M naar een verzameling S van strings waarbij $c(x)$ de string is die de punt x voorstelt. Later zullen wij met de strings werken en dan hebben wij een ander probleem: Als wij de fitness van een string willen berekenen, dan moeten wij weten, welk punt een gegeven string voorstelt om de fitness van dit punt te berekenen. De fitness van een string s (die tot een punt in M behoort) is gedefinieerd als $f(c^{-1}(s))$. Wij moeten dus $c^{-1}()$ berekenen – maar het kan gebeuren, dat sommige strings s helemaal geen punten uit M voorstellen, dus dat $c^{-1}(s)$ niet gedefinieerd is. Als M bijvoorbeeld de verzameling van alle mensen is en S de verzameling van alle eindige strings uit de letters A,C,G,T (dus $S = \{A,C,G,T\}^*$). Dan heeft elke mens een voorstelling in S, maar de meeste strings in S behoren tot geen enkele mens (of dier of wat dan ook). Eén van de grootste problemen met het ontwerp van een GA is de codering en de veranderingen (mutatie, crossover) op een manier te kiezen dat als je iets aan een string verandert, het resultaat (tenminste meestal) ook tot een punt behoort. Maar dat is veel gemakkelijker gezegd dan gedaan...
In ons voorbeeld coderen wij de punten (verzamelingen van bogen) als volgt: wij geven nummers $1,\ldots,|E|$ aan de bogen in G. Als e_i de boog met nummer i is, dan is dus $E = \{e_1,\ldots,e_{|E|}\}$. Wij kiezen als alfabet $\{0,1\}$ en stellen de verzamelingen voor als 0,1-vectoren van lengte $|E|$. Als $E' = \{e_{i_1},\ldots,e_{i_m}\}$ dan is $c(E')$ de 0,1-vector $(j_1,\ldots,j_{|E|})$ met $j_k = 1$ als $j_k \in \{i_1,\ldots,i_m\}$ en $j_k = 0$ anders.



E' met dikke lijnen

c(E')=(0,0,1,1,0,0,1)

f(E')=3−2*2= −1

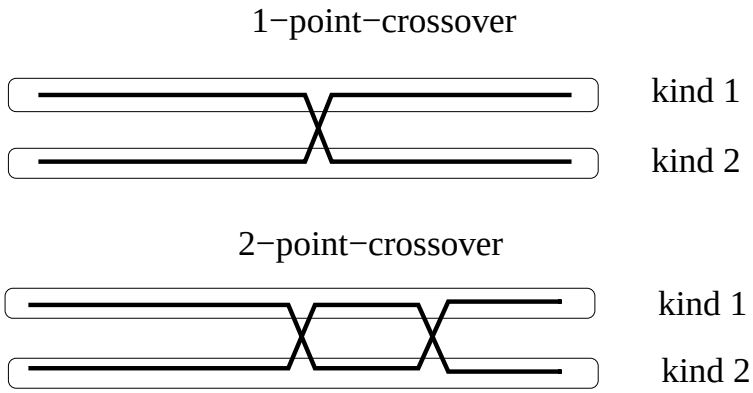
Dus stel dat wij een codering van de punten als een string hebben – dus een genotype. Terug naar het algemeen geval:

Mutatie: Een mutatie is in principe hetzelfde als een stap van een punt naar een buur. Wij beschrijven mutaties op het niveau van het genotype, dus de string: een mutatie is het vervangen van één of meerdere elementen in de string door een ander element uit het alfabet. Normaal wordt er op een gerandomiseerde manier over beslist, welke plaatsen vervangen worden en waardoor. Het probleem en de codering $c()$ bepalen of alle vervangingen toegelaten moeten worden of alleen maar sommige vervangingen zinvol en dus toegelaten zijn.

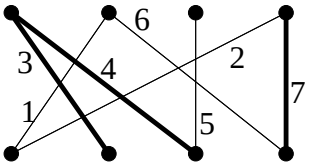
Hoe groot de kans gekozen moet worden om een element van de string door mutatie te veranderen, is voor elk probleem verschillend en kan het best bepaald worden door testen te draaien.

In ons voorbeeld zouden wij b.v. voor een string $(s_1, \ldots, s_7)$ voor elke $i \in \{1, \ldots, 7\}$ s_i met een kans van $1/5$ kunnen vervangen door $1 - s_i$.
Hier zien wij al dat het moeilijk geweest zou zijn als wij als M alleen de verzameling van matchings gekozen zouden hebben: als wij in onze codering een 0 door een 1 vervangen, is de kans groot dat het resultaat geen matching meer is – de code dus geen punt codeert.

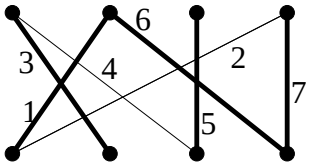
Crossover: Crossover gebeurt als 2 individuen één of meerdere kinderen verwekken. Het principe is dat het kind of de kinderen combinaties zijn van kopiën van de ouders. Er zijn meerdere manieren om dat te doen. De standaardmanieren zijn 1-point-crossover, 2-point-crossover, etc. Als wij b.v. 1-point-crossover willen toepassen, kunnen wij (b.v.) op een random manier een index i kiezen en het eerste kind krijgt de eerste $i - 1$ plaatsen van ouder 1 en de rest van ouder 2. Het tweede kind krijgt de eerste $i - 1$ plaatsen van ouder 2 en de rest van ouder 1. Dus: als de ouders $s_1, \ldots, s_k$ en $t_i, \ldots, t_k$ zijn, dan zijn de kinderen $s_1, \ldots, s_{i-1}, t_i, \ldots, t_k$ en $t_1, \ldots, t_{i-1}, s_i, \ldots, s_k$.
Zoals veel van de details, hangt de keuze of 1-point-crossover of multipoint-crossover beter is, af van het probleem. Ook de manier waarop je de crossover punten kiest, heeft een sterke invloed op de performantie en kan het best bepaald worden door testen te draaien.



In ons voorbeeld zou een 2-point-crossover met 2 kinderen er b.v. zo kunnen uitzien:

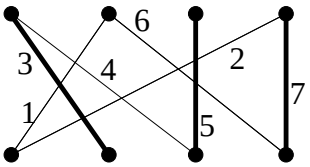


c(E')=(0,0,1,1,0,0,1)
f(E')= −1

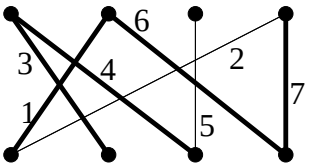


c(E'')=(1,0,1,0,1,1,1)
f(E'')= −1

2−point crossover met wisselposities 3 en 6:



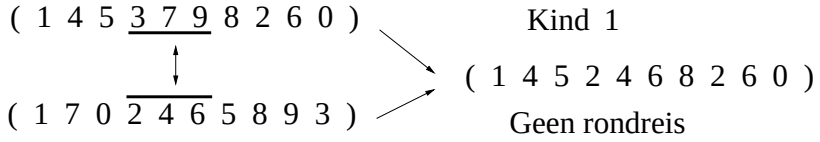
c(kind 1)=(0,0,1,0,1,0,1)
f(kind 1)= 3



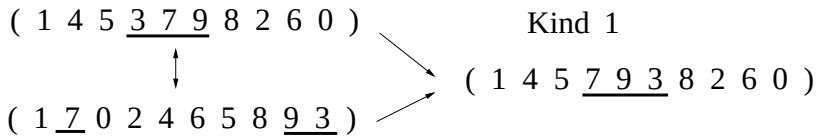
c(kind 2)=(1,0,1,1,0,1,1)
f(kind 2)= −5

Belangrijk: Het principe crossover te implementeren door het vervangen van 2 deelstrings op dezelfde positie van de twee ouders is misschien de gemakkelijkste en ook de gewone manier van doen, maar soms werkt het gewoon niet. Dat betekent dat met de gegeven codering het resultaat bijna nooit tot een punt behoort. Dan moeten wij crossover op een andere manier implementeren. Belangrijk is vooral dat *karakteristieken* van beide ouders in het kind aanwezig zijn. Een voorbeeld van *niet standaard crossover*:

Voorbeeld 7 *Wij zoeken een goedkoopste rondreis in een gewogen complete graaf. Wij coderen elke rondreis door een string met de nummers van de toppen. Als wij nu crossover op de gewone manier doen, zal het resultaat vaak geen rondreis zijn:*



Een andere manier een crossover te definiëren, is als volgt: Je kiest nog altijd een deelstring van de eerste ouder, maar vervangt hem niet door de deelstring op dezelfde posities van de andere ouder, maar herordent de deelstring zo dat de steden in deze deelstring in de volgorde van de tweede ouder worden bezocht. Op deze manier kan je garanderen dat elk kind ook tot een punt (in dit geval een rondreis) behoort.



Terug naar het algemeen geval:
Selectie:
Wij hebben twee stappen waar selectie gebeurt: één keer wanneer het algoritme moet kiezen welke individuen kinderen verwekken en één keer waar het beslist wie deel van de volgende populatie is (dus welk individu sterft). Zoals voor alles, bestaan ook voor deze selectiestappen meerdere mogelijkheden. Een gretige aanzet waar bijvoorbeeld altijd alleen maar de besten (die met de grootste fitness of met andere woorden *de fitste*) kinderen verwekken en overleven bleken vaak niet goed te werken. Het probleem is *inteelt* – de individuen lijken vaak te veel op elkaar en individuen met nieuwe eigenschappen hebben vaak geen kans, ook al kunnen net die eigenschappen later heel nuttig zijn. Het is beter, deze beslissingen ook gerandomiseerd te nemen: Hoe groter de *fitness* (waarde van de doelfunctie) van een individu is, hoe groter de kans dat hij kinderen verwekt en hoe groter de kans dat hij ook deel uitmaakt van de volgende populatie. Maar het is belangrijk dat ook individuen met een kleine fitness kans maken om lid van de volgende populatie te zijn en kinderen te verwekken.

Als wij al deze delen samenvatten, die wij net besproken hebben, dan hebben wij het volgende algoritme:

Algoritme 21 genetisch algoritme
Gegeven een verzameling M, een fitnessfunctie $f()$ waarvan wij het maximum zoeken en een codering $c : M \to A^*$ voor een alfabet A.
Wij hebben de parameters en methoden voor crossover, mutatie en selectie vastgelegd en beslist hoe groot de populatie moet zijn.

(a) *kies een beginpopulatie (b.v. op een toevallige manier).*

(b) *Herhaal de volgende stappen tot aan de stopvoorwaarde is voldaan.*

- *kies individuen om kinderen te verwekken*
- *vorm paren en verwek kinderen met hulp van crossover*
- *pas mutatie toe op de kinderen*
- *voeg de nieuwe kinderen toe tot de populatie*
- *selecteer de nieuwe populatie (dus beslis wie moet "sterven" zodat de populatie niet groeit).*

(c) *Het resultaat is het beste individu uit de laatste populatie.*

Geef voor de volgende oefeningen alle definities en parameters die nodig zijn om het algoritme volledig te beschrijven maar het is niet nodig dat ze ook zo gekozen zijn dat het algoritme echt goed werkt. Daarvoor zou je het moeten implementeren en testen laten draaien. Het algoritme moet alleen maar *in*

principe werken en het optimum kunnen vinden (ook al zou dat veel te lang duren en te veel pogingen vragen).
De bedoeling is gewoon om aan te tonen dat het principe verstaan is.

Oefening 131 *Ontwerp een genetisch algoritme voor het probleem een opspannende deelgraaf met minimaal gewicht van een gewogen complete graaf te zoeken, zodat elke top graad 2 in de deelgraaf heeft.*
Het handelsreizigersprobleem is er dus een speciaal geval van dit probleem – het geval waar de deelgraaf samenhangend is.

Oefening 132 *Ontwerp een genetisch algoritme voor het inpakprobleem uit AD2:*
Gegeven een rij $g_1, \ldots, g_n \leq C$ van gewichten van pakketten die vervoerd moeten worden. Je hebt identieke vrachtwagens met capaciteit C waarop je de pakketten kan plaatsen, maar je moet er wel op letten dat de capaciteit C van de vrachtwagens niet overschreden wordt. Het doel is natuurlijk de pakketten op een manier op de vrachtwagens te plaatsen dat zo weinig mogelijk vrachtwagens gebruikt worden.

5.6 Afsluitende opmerkingen

Hoewel wij er meerdere lessen aan hebben besteed, waren het alleen maar de oppervlakkige ideeën van de metaheuristieken die wij konden bespreken. En bovendien zijn er nog veel metaheuristieken waarvan wij zelfs de ideeën niet hebben besproken (ant colony optimization, particle swarm optimization, threshold accepting, tabu search, great deluge algorithm, asynchronous teams, etc.) Over elk van de metaheuristieken die wij hebben besproken, zijn er meer artikels gepubliceerd dan jullie ooit kunnen (of willen) lezen.
Als jullie een probleem moeten oplossen waarvoor een metaheuristiek een goede keuze is, dan is het vooral belangrijk het probleem goed te verstaan en de functies (b.v. $N()$) en parameters op een manier te kiezen die ideaal is voor dit **specifieke** probleem. Vooral in het geval van genetische algoritmen waren er ongelofelijk veel stappen die op verschillende manieren gedaan konden worden en parameters die gekozen moeten worden (het best door veel tests te draaien). Hoewel ook voor specifieke problemen heel goede en mooie ideeën ontwikkeld werden om een optimale prestatie van de metaheuristieken te garanderen, zijn deze ideeën vaak te specifiek om voor een les interessant te zijn.
Wat algemeen geldt, is dat de onderdelen van de metaheuristieken heel snel moeten zijn. In sommige gevallen is het beter een startpunt te kiezen in plaats van een toevallig startpunt te nemen. Maar het algoritme dat het startpunt kiest, moet wel snel zijn – anders kan je gewoon een ander algoritme toepassen om het probleem op te lossen. Maar vooral geldt dat natuurlijk voor onderdelen van het algoritme die heel vaak uitgevoerd moeten worden (b.v. de berekening van buren). Op deze plaatsen is het vooral belangrijk dat de berekeningen heel snel en efficiënt zijn. Daardoor zijn **te** ingewikkelde definities van b.v. *de buurt* een slechte keuze.

6 Parallelle algoritmen

Parallelle algoritmen zijn algoritmen die meerdere processoren gebruiken die samenwerken om één probleem op te lossen. Het voordeel is duidelijk en het principe van parallellisme wordt al lang gebruikt: als bv. een huis wordt gebouwd dan zou (bijna) niemand proberen dat helemaal alleen te doen – het aantal werkuren is zo groot dat de mens die het huis wil bouwen misschien al overleden is als het huis eindelijk klaar is... Daarom werken meerdere mensen tegelijk. Daardoor wordt het aantal werkuren niet kleiner (misschien zelfs groter) maar de tijd tussen begin van de bouw en het einde wordt veel kleiner. Dit is ook het doel van parallelle algoritmen. Het is duidelijk dat de totale verbruikte rekentijd niet kleiner kan worden (je kan de taken van de verschillende processoren ook na elkaar op één processor laten draaien) maar de *wachttijd* tussen het begin van de berekening en het einde kan wel duidelijk korter worden.
Maar het voorbeeld met ploegen die samen werken toont ook twee problemen: soms moet je goed op de volgorde letten (bv. de muren niet schilderen als er nog kabels in de muren gelegd moeten worden) en je kan ook niet arbitrair veel werkers gebruiken. Het helpt bv. niet als 10 personen proberen een enkele schakelaar voor het licht vast te maken. Het gemakkelijkst is het taken aan verschillende werknemers toe te kennen als de taken helemaal niets met elkaar te maken hebben (bv. de kelder schilderen en tegelijk pannen op het dak plaatsen).

Hetzelfde geldt voor parallelle algoritmen: als deeltaken niets met elkaar te maken hebben, is het veel gemakkelijker ze op verschillende processoren te laten draaien dan in gevallen waar de taken sterk van elkaar afhankelijk zijn.

Een probleem bij de beslissing welke onderwerpen voor dit deel van de les gekozen worden, was dat het heel moeilijk is om te voorspellen wat in de toekomst **echt** relevant zal zijn. En dat geldt vooral voor een les waar de klemtoon vooral op een realistische kijk op de zaak ligt. Rond 1980 leek het (voor een tijdje) dat de kloksnelheid van de computers niet meer zo snel groeide en in de tijd na 1980 werd verwacht dat *parallel computing* de toekomst zou zijn. Er werden veel verschillende modellen voor parallelle computers ontworpen. Verschillende architecturen bestonden zelfs als hardware – bv. de bekende Cray supercomputers (vanaf 1976). Maar de transputers waar interprocessorcommunicatie een belangrijk deel van het ontwerp was waren bijzonder interessant. Deze ontwerpen besteedden veel tijd en geld aan het ontwerp van een efficiënt netwerk voor de interprocessorcommunicatie.
Maar het "probleem" was vermoedelijk dat door de extreem grote aantallen van kleine computers er veel meer geld aan de ontwikkeling van CPU's besteed kon worden die geoptimaliseerd zijn om zelfstandig te werken. Daardoor verdwenen de *echte* parallelle computers langzaam en in plaats daarvan ontstonden clusters van stand-alone computers die door netwerken met elkaar verbonden waren die – in vergelijking met de kloksnelheid – zeer traag zijn. Deze ontwikkeling kan je – natuurlijk – ook in de opleiding terugvinden, waarin vroeger een hele les *parallelle algoritmen* gegeven werd die later gereduceerd werd op een relatief klein deel van DA 3.
Maar intussen is het opnieuw zo dat de kloksnelheden niet meer zo snel groeien. De processoren worden door het feit dat per clockcycle meer en ingewikkeldere bewerkingen plaats kunnen vinden wel nog altijd sneller, maar toch lijkt parallellisme opnieuw belangrijker te worden met als gevolg dat er nu zelfs één les over parallelle software en één les over parallelle hardware is. Bijzonder interessant zijn daarbij de multicore computers die het idee van een shared memory verwezenlijken en ook in grote hoeveelheden verkocht worden. Er gebeurt ook veel onderzoek over de mogelijkheid de grafische kaarten – die massief parallel zijn – voor berekeningen te gebruiken – trefwoord GPU-computing. De vraag is dus in welke richting de ontwikkeling deze keer gaat. Theoretische parallelle algoritmen uit het verleden hebben het normaal niet over 2, 4 of 8 processoren, maar over zoiets als bv. $O(n)$ als n de grootte van het probleem is. De vraag is dus of er een ontwikkeling in de richting van multicore processoren met duidelijk meer cores zal zijn of of de ontwikkeling zal stoppen en opnieuw in een andere richting gaan...
Misschien zeggen jullie nu dat er toch heel veel clusters zijn waarop *parallel computing* wordt toegepast – bv. voor voorspellingen van het weer en ook *scientific computing*. Maar in die gevallen wordt eerder iets toegepast dat *distributed computing* genoemd wordt. Het verschil is dat in *distributed computing* de communicatie tussen de verschillende delen *relatief* klein is. In een cluster gebeurt de communicatie via een intern netwerk, maar in sommige gevallen van distributed computing gebeurt de communicatie van de computers die de deeloplossingen berekenen zelfs over het internet. De communicatie tussen verschillende reken-nodes is dus **duidelijk** trager dan de communicatie van een CPU met zijn lokaal geheugen. Voor dergelijke algoritmen is de precieze vorm van het netwerk dan

ook minder belangrijk omdat de communicatie tussen de delen klein is.
De klemtoon van deze les ligt op realistische toepassingen eerder dan theoretische resultaten, dus zullen wij hier alleen één heel nuttige manier zien om branch and bound algoritmen te paralleliseren.

Notatie: Wij schrijven $t_p(n)$ voor de tijd die een gegeven parallel algoritme dat p processoren voor een taak met invoerlengte n gebruikt ten hoogste nodig heeft. Dat is dus niet de som van de tijden van de enkele processoren, maar gewoon de tijd die verstreken is tussen het begin van de berekeningen en het einde. Wij schrijven $t^s(n)$ voor de tijd die het snelste (gekende) sequentiële algoritme voor deze input nodig heeft. Het is onmiddellijk duidelijk dat als een parallel algoritme in tijd $O(t_p(n))$ draait dat dan $O(p \cdot t_p(n))$ een bovengrens is voor $t^s(n)$ omdat door achter elkaar uitvoeren van de stappen die de p processoren tegelijk uitvoeren het parallelle algoritme ook serieel gesimuleerd kan worden, maar de simulatie kan natuurlijk wel een zekere (constante) overhead vragen.
Het product van de tijd en het aantal processoren wordt soms ook de kost $K(n)$ van het algoritme genoemd. Wij weten door de mogelijkheid van simulatie dat dit product (op een constante na) optimaal is voor seriële algoritmen.
Een ander concept is de werk complexiteit van een algoritme. Als de scheduler op een parallelle computer of cluster slim genoeg is dan kan hij vaak herkennen dat sommige processoren na de eerste stappen niet meer gebruikt worden. Dat is niet in elk geval gemakkelijk, maar in principe kan de programmeur zelf in sommige gevallen misschien een `exit()` in de code voor een processor i schrijven. Dan kunnen deze processoren voor andere doeleinden gebruikt worden. Het is dus niet in elk geval zinvol zo te rekenen alsof alle ooit gebruikte processoren de hele tijd bezig waren als het om de kost van een algoritme gaat. De *kost* die de tijd van de processoren alleen telt als deze processoren ook stappen voor het algoritme uitvoeren wordt de *werk complexiteit* genoemd. Of precies: het werk $W(n)$ dat een parallel algoritme gebruikt voor een invoer met lengte n is gedefinieerd als

$W(n) = \sum_{i=1}^{p} t(i, n)$

waarbij processoren $1, \ldots, p$ aan de taak werken en $t(i, n)$ het aantal stappen (de tijd) is dat processor i maximaal doet voor een invoer met lengte n. Zoals het hier gedefinieerd is, is het niet noodzakelijk zo dat de tijd die voor verschillende processoren gerekend wordt altijd voor dezelfde invoer maximaal is.
Het is duidelijk dat een algoritme dat werk $W(n)$ vraagt op een seriële machine in tijd $O(W(n))$ gesimuleerd kan worden. Daarbij veronderstellen wij wel, dat de samenwerking van de verschillende delen goed bepaald is. Als verschillende delen soms op verschillende momenten afgewerkt worden, kan dat sneller of trager zijn omdat belangrijke informatie vroeger of later ter beschikking kan staan. Ons argument met de simulatie gebruikt in feite dat wij heel goed weten welke stappen wel dan niet tegelijk uitgevoerd worden.

6.1 Branch and bound met verdeelde processoren

Als het om problemen gaat met een heel goedkope asymptotische complexiteit (bv. $O(n)$ of $O(n \log n)$ met een niet te grote constante voor de $O()$) dan moet je al extreem grote hoeveelheden van data hebben om ervoor te zorgen dat meer dan één processor nodig is om het probleem in aanvaardbare tijd af te werken.

Bovendien is de bottleneck dan misschien zelfs eerder het lezen en schrijven van de data. Dat gebeurt dan het best ook verdeeld.
Maar de duurste problemen die jullie in Datastructuren en Algoritmen 2 hebben gezien, zijn NP-complete problemen waarvoor de voor de hand liggende oplossing vaak branch and bound algoritmen zijn (hoewel andere aanzetten – zoals bv. met integer programming – soms heel succesvol zijn). Hier zullen wij het erover hebben hoe je een branch and bound algoritme met een eenvoudig trucje efficiënt op verschillende machines kan verdelen. Ik heb deze manier om jobs te verdelen voor het eerst bij Brendan McKay gezien.
In een abstracte vorm kan je zo'n branch and bound algoritme ongeveer als volgt schrijven (waarbij in dit voorbeeld een minimum wordt gezocht):

Algoritme 22 *(Branch and bound)*

```
beste_waarde= +oneindig; // of een gemakkelijk berekenbare
                         // bovengrens

b_and_b(configuratie)
// test de volgende configuratie in de recursieboom
{

if (is_endconfiguratie(configuratie)) // alle keuzes gemaakt
{ if (waarde(configuratie)<beste_waarde)
                    beste_waarde=waarde(configuratie);
  return;
}

else
{
  benedengrens=bepaal_benedengrens(configuratie);
  // dat is de look-ahead van branch and bound die vaak moeilijk
  // is om te vinden maar belangrijk voor de performantie. Hier
  // wordt een benedengrens voor de waarde van een mogelijke
  // eindconfiguratie bepaald die uit deze configuratie voorkomt.

  if (benedengrens<beste_waarde)
  // verbetering nog mogelijk
  {
  for (elke mogelijke directe opvolgconfiguratie)
                      b_and_b(opvolgconfiguratie);
  }
}// end else
return;
}// end b_and_b
```

Algoritme 22 wordt dan opgestart met de beginconfiguratie (de wortel van de recursieboom) en op het einde staat de beste waarde in `beste_waarde`.
Om deze abstracte beschrijving iets beter te verstaan, kunnen wij bv. naar het inpakprobleem kijken, waar gewichten $g_1 \leq 1, \ldots, g_n \leq 1$ op een manier op

vrachtwagens met capaciteit 1 geplaatst moeten worden dat het totale aantal vrachtwagens minimaal is. De *beginconfiguratie* is dan bv. de situatie waar alleen gewicht g_1 op vrachtwagen 1 geplaatst is (dat mogen wij zeker stellen), een opvolgconfiguratie van een configuratie waar $g_1, \ldots, g_k$ geplaatst zijn is een configuratie waar $g_1, \ldots, g_{k+1}$ geplaatst zijn en een eindconfiguratie is een configuratie waar $g_1, \ldots, g_n$ geplaatst zijn. Een gemakkelijk berekenbare bovengrens is bv. n. De waarde van een configuratie is het aantal gebruikte vrachtwagens en `bepaal_benedengrens(configuratie)` is bv. de waarde van `configuratie` plus de som van de nog niet geplaatste gewichten die op geen vrachtwagen meer passen die al bestaat (een slechte benedengrens, maar het gaat hier alleen om het principe).
Als jullie naar de verschillende takken van de recursieboom kijken dan valt op dat het enige verband de waarde `beste_waarde` is.
Eén aanzet zou dus bv. zijn alle configuraties $c_1, \ldots, c_k$ op een zeker niveau – dat is de afstand x van de wortel – van de recursieboom op te slaan en die in parallel als beginconfiguraties te gebruiken. Elke keer dat een betere waarde voor `beste_waarde` wordt gevonden, wordt die aan alle computers doorgestuurd. In de meeste gevallen zal het zo zijn als bij het inpakprobleem – dus dat `beste_waarde` in vergelijking met het aantal toppen in de recursieboom niet vaak verbeterd kan worden. Bij het inpakprobleem kan `beste_waarde` ten hoogste $n-1$ keer verbeterd worden terwijl de recursieboom exponentieel groot kan zijn! De communicatie tussen computers die verschillende takken afwerken is dus **miniem** en het is zelfs geen probleem de betere waarden via het internet te sturen!
Dat ziet er al niet slecht uit – maar er zijn twee problemen:

a.) In de meeste gevallen zullen de takken allesbehalve even groot zijn. Sommige takken zullen zeer snel als slecht herkenbaar zijn (de bounding criteria zijn in deze delen heel efficiënt) en dus nauwelijks werk vragen en sommige takken zullen bijna de hele tijd vragen. Als wij bv. 100 computers ter beschikking hebben en de taak op deze manier in honderd delen splitsen dan is het mogelijk dat 99 delen na weinige seconden gedaan zijn en dat één van de taken weken duurt...

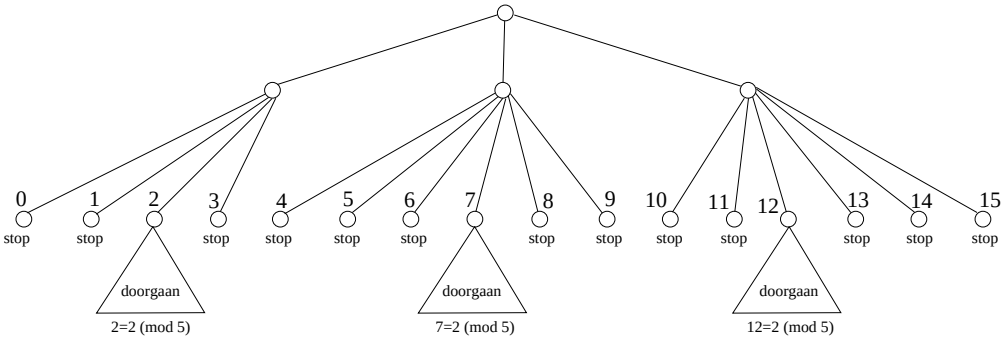
b.) De verschillende startconfiguraties moeten berekend en op een manier opgeslagen worden dat de programma's vanuit dit punt kunnen beginnen. Dat is niet echt moeilijk, maar wij zullen zien dat het vaak overbodig werk is.

Probleem a.) kan gedeeltelijk opgelost worden door de afstand van de wortel van de recursieboom duidelijk groter te kiezen. Daardoor krijgen wij veel meer delen dan wij computers hebben en hoewel ook deze delen zeker verschillend veel tijd nodig hebben, kunnen wij hopen dat door het feit dat elke computer meerdere delen moet afwerken het verschil tussen de tijd die de computer nodig heeft die het langst bezig is (dat is onze wachttijd) en die het minst bezig is niet te groot wordt. Ten slotte kunnen computers die heel goedkope delen opstarten ook vroeger met het volgende deel beginnen en werken daardoor misschien meer delen af. Een probleem is natuurlijk dat wij misschien extreem veel delen hebben...
Branch and bound algoritmen zijn natuurlijk alleen duur als de bomen heel groot zijn. Dat betekent meestal niet dat de bomen heel diep zijn, maar dat er een grote vertakking is. Of met andere woorden: dat er op de lage niveaus (dus

dicht bij de wortel) in vergelijking met de hele boom verwaarloosbaar weinig toppen zijn.
Om probleem b.) op te lossen zullen wij het gedeelte tot het gekozen niveau gewoon elke keer opnieuw doorlopen. Omdat het relatief klein is, verliezen wij op deze manier niet veel tijd. Het aantal gevallen mag natuurlijk niet te groot zijn! Om het aantal gevallen klein te houden en aan de andere kant een goede verdeling te krijgen, kiezen wij het level waar wij splitsen relatief diep in de boom (dus veel verschillende toppen om te splitsen), maar vatten wij toppen uit verschillende delen van de zoekboom samen. Of precies:

- Kies een diepte d om te splitsen en een aantal k van delen waarin het probleem gesplitst moet worden. Het aantal toppen van de recursieboom op diepte d moet zeer veel groter zijn dan k. De verschillende delen waarin de recursieboom opgesplitst wordt, kunnen nu beschreven worden door de indices $0, 1, \ldots, k-1$.
- Houd een teller bij die – beginnend met 0 – nummers toekent aan de toppen op diepte d in de volgorde waarop ze bezocht worden.
- Als deel nummer i moet afgewerkt worden en je hebt een top met nummer m op diepte d bereikt dan ga door naar de volgende diepte als en slechts als $i = m \pmod{k}$. Anders backtrack.
- Als de delen $0, 1, \ldots, k-1$ afgewerkt zijn (dat kan in parallel gebeuren) is de hele recursieboom doorzocht. Communicatie tussen de delen is alleen het mededelen van nieuwe bovengrenzen.

Deze manier om het doorzoeken van een recursieboom op te splitsen in deeltaken zien jullie in Figuur 27. Diepte 2 en maar 16 toppen op de diepte is niet echt realistisch (het moeten er **veel** meer zijn), maar realistische gevallen kan je lang niet meer tekenen...
Maar: je moet erop letten dat in het gedeelte dat in elk deel wordt doorlopen – dus de niveaus ten hoogste de diepte waar gesplitst wordt – niet gesnoeid wordt of altijd met dezelfde waarde voor de grens gesnoeid wordt – anders is de nummering van de toppen op het splitlevel verschillend in delen die snel klaar zijn en delen die laat klaar zijn en misschien al met een betere grens kunnen werken.
Maar let op: het argument dat computers die gemakkelijke delen draaien ook sneller een nieuw deel opstarten geldt hier niet meer – wij moeten gewoon hopen dat door het feit dat elk deel onderdelen bevat ook de moeilijke gevallen (de takken met grote complexiteit) goed verdeeld zijn.
In feite is het niet echt nodig dat je atijd op dezelfde diepte splitst. Soms is het beter in sommige takken op een grotere diepte te splitsen dan in andere. De bedoeling is gewoon dat je een verzameling van toppen in de boom hebt die een relatief klein gedeelte bij de wortel splitst van een groot gedeelte en waar je elke van deze splitstoppen vanuit het gedeelte dat altijd doorlopen wordt kan bereiken. Altijd op dezelfde diepte te splitsen is gewoon één van heel veel mogelijkheden die hier als voorbeeld werd gebruikt.
Over de tijd die nodig is om alle delen af te werken in vergelijking met de tijd als je het in één deel laat draaien, kan niet veel bewezen worden. Omdat de volgorde waarin de boom wordt doorlopen sterk verschilt van de volgorde waarin het in



Figuur 27: De recursieboom voor het geval dat op diepte $d = 2$ gesplitst wordt en deel nummer 2 voor $k = 5$ afgewerkt wordt. Voor toppen met een nummer dat gelijk is aan 2 (mod 5) wordt doorgegaan en voor de anderen gestopt.

één deel wordt doorlopen – en dus de verschillende takken met andere waarden voor de bovengrens worden doorlopen, kan de som van de tijden theoretisch zelfs arbitrair veel kleiner zijn dan de tijd als je het in één deel laat draaien. Dat is geen strijdigheid met de observatie dat $t^s(n)$ in feite $O(p \cdot t_p(n))$ is omdat $t^s(n)$ betrekking heeft op een optimaal algoritme – en in dit geval zou een optimaal algoritme de takken in een andere volgorde doorlopen.
Aan de andere kant kan het theoretisch ook gebeuren dat elk deel bijna even veel tijd vraagt als de hele taak...
In de praktijk zal deze manier van opsplitsen een goede manier zijn om dingen in het parallel op te starten waarbij de som van de tijden ongeveer even groot is als de tijd als het serieel wordt opgestart.

Oefening 133 *Ons argument dat een serieel algoritme altijd ten hoogste p keer trager is dan een parallel algoritme was dat een serieel algoritme de parallelle berekeningen kan simuleren. Beschrijf* **expliciet** *wat dat voor dit verdeeld algoritme betekent. Hoe werkt dit serieel algoritme voor een gegeven p? Je mag veronderstellen dat p ook het aantal delen is – dus alle delen parallel worden opgestart.*

Oefening 134 *Voor het volgende mag je stellen dat als een optimum gevonden wordt het als resultaat heeft dat alle nog lopende delen onmiddellijk kunnen stoppen omdat de snoeiecriteria merken dat deze waarde niet verbeterd kan worden. Deze veronderstelling is in veel gevallen inderdaad realistisch (bv. als een langste cykel in een graaf gezocht wordt en een Hamiltoniaanse cykel wordt gevonden). Stel bovendien dat alle delen tegelijk worden opgestart.*

- *Gegeven een constante $C \geq 1$. Schets een scenario (precies: de structuur van een recursieboom) waar voor een branch and bound algoritme dat op de geziene manier in delen wordt opgesplitst de som van de tijden C keer kleiner is dan de tijd als het in één deel wordt opgestart.*
- *Gegeven een constante $C \geq 1$. Schets een scenario (precies: de structuur van een recursieboom) waar voor een branch and bound algoritme dat op de geziene manier in delen wordt opgesplitst de som van de tijden C keer groter is dan de tijd als het in één deel wordt opgestart.*

Oefening 135 *Herschrijf de pseudocode in Algoritme 22 zo dat het branch and bound algoritme op de net geziene manier opgesplitst wordt.*

Oefening 136 *Gegeven gewichten $g_1 \leq 1, \ldots, g_n \leq 1$. Gezocht is het minimale aantal vrachtwagens met capaciteit 1 dat nodig is om deze gewichten te vervoeren.*

- *Schrijf de pseudocode – met de nodige delen om het algoritme met de modulo optie te kunnen verdelen – voor een branch and bound algoritme dat de gewichten $g_1, \ldots, g_n$ in deze volgorde op elke mogelijke manier plaatst en de waarde b van een beste oplossing tot nu toe altijd bijhoudt.*

  Als bounding criterium gebruik het volgende: als de som k van alle gewichten die nog niet geplaatst zijn en op geen van de al gebruikte a vrachtwagens past, voldoet aan $a+k > b-1$, kan je snoeien. Je mag veronderstellen dat een functie `get_k()` *al bestaat die de waarde van k teruggeeft.*

- *Geef ook een andere manier* splitstoppen *te bepalen dan altijd op dezelfde diepte te splitsen. Houdt er bv. rekening mee hoeveel vrachtwagens al gebruikt zijn.*

Oefening 137 *Stel dat je pas nadat al een groot deel van de taken afgewerkt is, merkt dat één deel toch veel langer draait dan alle anderen. Geef een oplossing hoe je deze taak verder kan opsplitsen zonder de andere delen die al afgewerkt zijn opnieuw op te starten.*

Oefening 138 *Hetzelfde principe kan ook gebruikt worden voor het doorzoeken van recursiebomen waar geen optimum gezocht wordt, maar waar output op verschillende levels van de zoekboom gebeurt. Waarop moet je hier letten om nog altijd dezelfde verzameling van outputs te hebben en hoe los je het probleem op?*

Index

$K(n)$, 148
$N(a)$, 125
$S(1, i)$, 35
$S(1, i)_j$, 36
$W()$, 148
$\sigma_{j,i}$, 36
$a(t)$, 40
$b(i)$, 36
$l(t)$, 40

adaptieve Huffman codering, 87
adaptive Huffman coding, 89
afstand
    Levenshtein, 72
algoritme
    Boyer-Moore-Horspool, 63
    genetisch, 140, 144
    Knuth-Morris-Pratt, 55
    parallel, 146
    Rabin-Karp, 52
    shift-AND, 66
    shift-AND voor matches met fouten, 70
arithmetic coding, 98
arithmetisch coderen, 98

b'(i), 37
Baeza-Yates, R., 66
Bloom filter, 15
boom
    deelstring, 35
    string, 35
    suffix, 32
Boyer, 63
Boyer-Moore-Horspool, 63
buren, 125
Burrows-Wheeler transformatie, 115
Burton Bloom, 15
bzip2, 115

coderen
    arithmetisch, 98
codering
    Huffman, 84
coding
    Huffman, 84
    Huffman, adaptive, 89
computing
    distributed, 147
    parallel, 146
crossover, 140–142

deelstring boom, 35
deletion, 71
distributed computing, 147
doelfunctie, 123
dynamic Huffman coding, 89
dynamisch programmeren, 74

editeerafstand, 70–72
extendible hashing, 7

false positives, 15
fenotype, 141
filter
    Bloom, 15
fitness, 140

GA, 141, 144
geleid lokaal zoeken, 130
genetisch algoritme, 144
genetische algoritmen, 140
genotype, 141
gesimuleerd temperen, 136
Gonnet, G., 66
guided local search, 130

Horspool, 63
Huffman algoritme
    two pass, 89
Huffman codering, 84
Huffman coding
    adaptive, dynamic, 89
Huffman coding, adaptive, dynamic, 89
Huffman-boom, 85

interne top, 35

karakteristieke vector, 67
Karp, 52
Knuth, D., 55
kost, 148

laadfactor, 11
Levenshtein, 72

Local search, 125
local search, 125
lokaal zoeken, 125

Manber, U., 70
Mc Creight, 34
metaheuristiek, 123
model, 98
Moore, 63
Morris, J.H., 55
move to front, 121
mutatie, 141
mutaties, 140

niveau, 150

objective function, 123
overstroomemmers, 10

parallel computing, 146
parallelle algoritmen, 146
Patricia trees, 28
Patricia tries, 28
PPM, 103
Pratt, V., 55
Prediction by Partial Matching, 103
prefix tree, 25

Rabin, 52
run length encoding, 104

SA, 138
selectie, 144
shift-AND, 66
    matches met fouten, 70
simulated annealing, 136, 138
string boom, 35
suffix arrays, 50
suffix boom, 32
suffix boom
    veralgemeend, 49
suffix links, 40
suffix trees, 32

ternary trees, 30
ternary tries, 30
toevallig zoeken, 124
top
    intern, 35
transformatie
    Burrows-Wheeler, 115
transputers, 147
tree
    Patricia, 28
    prefix, 25
    suffix, 32
    ternary, 30
trie, 25, 26
trie
    Patricia, 28
    ternary, 30
two-pass Huffman algoritme, 89

Ukkonen, 34

variable neighbourhood descent, 134
variable neighbourhood search, 134, 135
vector
    karakteristiek, 67
veralgemeende suffix boom, 49
verschuivingstabel, 56
vnd, 134
vns, 135

Weiner, 34
werk complexiteit, 148
Wu, S., 70